

SOCIALLY RESPONSIBLE SOFTWARE DEVELOPMENT

mit: 10/7

mit: 10/7

SOCIALLY RESPONSIBLE SOFTWARE DEVELOPMENT

Matthias Felleisen

mit: 10/7

©2024–2026 Matthias Felleisen

This work is licensed to the public under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 license (international):

<http://creativecommons.org/licenses/by-nc-nd/4.0/>

All rights reserved except as licensed pursuant to the Creative Commons license identified above. Any reproduction or other use not licensed as above, by any electronic or mechanical means (including but not limited to photocopying, public distribution, online display, and digital information storage and retrieval) requires permission in writing from the publisher.

This book was set in Scribble and LaTeX by the author.

Library of Congress Cataloging-in-Publication Data

Names: Felleisen, Matthias.

Title: Socially responsible software development
/ Matthias Felleisen.

Identifiers: LCCN 2017018384 | ISBN 9780262534802 (pbk. : alk. paper)

Subjects: LCSH: Computer programming. | Electronic data processing.

Classification: LCC QA76.6 .H697 2017 | DDC 005.1/2–dc23

LC record available at <https://lcn.loc.gov/2017018384>

10 9 8 7 6 5 4 3 2 1

Thou shalt love thy neighbor as thyself. – Matthew 22:35–40

mit: 10/7

mit: 10/7

The Chapters

Prelude	xiii
I Perspective	1
II Principles	7
III Project	23
IV Programming Interfaces	69
V Programming Basics	141
Postlude	cc

The Software Development Questions

1. What does the life of a software system look like?
2. What is software to its developer?
3. What is a software system? What goes into its production?
4. What is a software component? How are they developed? Used?
5. What is programming to a software developer?

Detailed Contents

Prelude	xiii
Experiences for Junior Developers	xvi
Developing a Mindset in the Age of AI	xvii
This Book, This Course	xviii
Dear Instructor: Some Context	xx
I Perspective	1
1 The Death of Software, The Life of Software	2
2 Are You Socially Responsible?	4
II Principles	7
3 The Nature of Software Development	8
4 Egoless Programming	10
5 Pair Programming, With AI	12
6 Inspections; of AI and by AI	16
III Project	25
7 Running a Software Systems	26
8 Developing a Complete Software System	30
8.1 Ways to Proceed	30
8.2 Analysis: Understanding the Project's Purpose and Context	33
8.3 Synthesis: Discovering Components	36
8.4 Planning: How the Construction Proceeds	41
8.5 With AI	43
9 Software Architecture	45
9.1 Pipelines	46
9.2 Model, View, Controller	47
9.3 Client, Server	48

10 A Sample Project: Analysis, Discovery, Planning	50
10.1 Core Idea	52
10.2 Context Analysis	52
10.3 Component Discovery via Use Cases	53
10.4 Construction Plan	58
11 Inspecting the Plan and its Inputs	65
12 Inspections Matter, Especially in the Age of AI	71
12.1 Documents: Start the README, Before the Coding Begins .	72
IV Programming Interfaces	75
13 The Nature of Interfaces	77
13.1 An Interface is the Common Ontology for Clients and Server	79
14 Encapsulation via Interfaces	83
14.1 Interface Inspections for Encapsulation	84
14.2 Encapsulating Properly	88
15 Designing an Interface	90
15.1 Content	91
15.2 Process	93
15.3 A Simple Interface Design Process	96
15.4 An Interface for a Component	105
15.5 A Component as an Interface; Evaluating Alternatives	112
16 Expressing an Interface	123
16.1 Existing Practice: Types, Subtypes	124
16.2 Contracts for Individual Methods	126
16.3 Contracts for Individual Objects	134
16.4 Beyond Individual Methods and Objects	140
17 Interfaces and AI	148
17.1 Why Contracts? Why Separate?	149
17.2 Contract Verification via AI	150
V Programming Basics	151

18 Clarity of Code	152
19 Comments are Okay in a Small Number of Places	153
20 The Very Basics	157
20.1 Names Matter	157
20.2 The Size of Functions and Methods	160
20.3 A Short Note on Conditionals	162
20.4 And an Even Shorter One on Loops	165
21 Code Inspections, the Basics	166
21.1 Inspecting Code for Coding Flaws	168
21.2 A Code Inspection Memo	171
21.3 Reacting to a Code Inspection	173
21.4 Inspecting Code for Errors and Test Coverage Problems	174
21.5 Pushing Back During a Code Inspection	178
21.6 Inspecting Code for Design Mistakes	179
22 Systematic Design: Data	184
22.1 Developing Data Representations for Information	185
Information that is Considered Atomic	186
Structured Information: Don't Use Strings to Represent It	186
Structured Information: Use Structs, Records, Objects	187
From Information to Data Representations, And Back	188
From Information to Classes via Doodling	192
22.2 Choosing from Several Alternatives	198
22.3 Developing Data Representations for Composition	200
23 Systematic Design: Functionality	203
23.1 Designing Functions, Systematically	205
23.2 Atomic vs Composite: A Tiny Case Study	208
23.3 Designing Methods, Systematically	212
Favor Functions, Favor Immutability	213
23.4 A Second Look at the Tiny Case Study	216
23.5 Why Systematic Design is Socially Responsible	218
24 Systematic Design: Abstraction aka DRY	222
24.1 The Systematic Abstraction Process	224
24.2 Abstraction Mechanisms	226
Abstracting with Functions	226
Abstracting with Classes	228

Abstracting with Types	228
24.3 Using Abstractions	228
24.4 Abstracting is Socially Responsible	228
24.5 Abstraction Beyond Functions and Classes	228
24.6 Abstracting with Functions	228
24.7 Abstracting with Classes	228
25 Code Inspections, Systematically	232
25.1 The Labyrinth Game: A Problem Statement	232
25.2 Presenting Systematically Designed Code	233
25.3 Inspecting Systematically Designed Code	243
25.4 A General Guide to Code Inspections	252
README Files	253
Presenting	256
Inspecting	258
Postlude	cclxi
How to Continue the Learning Process, with AI	cclxi
How to Identify Socially Responsible Candidates	cclxii
Onward!	cclxiii
Acknowledgments	cclxv

Prelude

Artificial Intelligence (AI) has changed the lives of working software developers in a dramatic fashion. For academics, this change poses the challenge of how to educate future developers: which parts of the existing education program to retain, which parts to drop, and which new elements to add. One way to answer these questions is to rephrase them in terms how AI affects the marketplace for professionals:

Does AI render software developers superfluous?	Absolutely not.
What makes you so sure?	As of summer 2026, the number of job ads for software developers has risen by 15% over one year.
Should this growth inspire current students?	Absolutely not.
What?	Most of these ads demand several years of experience.
Why is that?	Companies have learned that junior developers use AI to produce “vibe slob.”
How does experience help?	Experienced developers use AI to produce secure and maintainable code.
How can inexperienced students become experienced developers?	They need a new mindset about software, starting in college.

This book is one answer to the question. In order to understand the challenges, we need to agree on what AI knows and how it got to know what it knows:



So what is AI?	AI is the result of a learning process that used a vast pool of publicly available code.
Isn't the use of publicly available code a good thing?	The code that creates wealth is rarely open source. It's usually a trade secret.
But doesn't the use of a large pool of code include enough example?	A large pool contains it all: the good, the bad, and the ugly.
Are you saying "trade secret" code is always good?	Not always, but if it isn't, it kills the business eventually.
Okay. It is in the interest of a business to continuously improve code.	Yes.
Can't the AI just learn from the secure and maintainable code?	It is an experienced developer who can tell an AI what is good code and what is bad code.
Is AI like a junior developer, just out of an ordinary undergraduate program?	Yes! It is best to think of AI as an apprentice programmer, still figuring out how to live up to a developer's and a team's standards.

In short, AI's learning averages over code that is rarely the source of profits and wealth, including a large amount of code published by novice programmers. What it infers from the acquired parameters suffers from the usual safety and security problems, not to speak of the lack of organization that beginner code tends to exhibit.

Given this insight, the next question is how junior developers become senior developers. And how do they and AIs gain experiences? One of the many experienced developers I worked with once put it rather succinctly as "we have the scars to show for," meaning experience is the result

of reflecting on failures and the associated negative feedback. Becoming an experienced developer is a socialization process, and that applies to AI agents, too:

What does “socialization” mean here?	Real software systems aren’t little vibe-coded apps.
What are they?	They consist of many interacting components and millions of lines of code.
So is it just a matter of scale?	And they are never finished.
How come?	They come with bugs, they suffer performance problems, and they lack features.
And how does all this demand a “socialization process” for developers? Or AI agents?	Software is not just a bunch of instructions that computers execute. It communicates ideas among developers.
Is that because developers have to revisit existing code all the time to fix bugs, improve its performance, or add features?	Correct, the creator of code sends it to another person who must reconstruct the planning and thinking that went into the creation.
Does “socialization” mean “thinking of those who must read the code” in the future?	Yes, and it happens when junior developers go beyond plain programming.
Does this apply to AI?	Yes, every AI starts as an apprentice.

In short, we train AI agents as if they were human partners, and in this sense, we have to make them understand their responsibility to the future of the software system and their maintainers.

Experiences for Junior Developers

After a couple of semesters, a student of software knows how to program but cannot develop software. To start bridging the gap, a student needs to take a course that teaches the following: (1) to turn the wish for some software product into a plan; (2) to execute this plan over the course of a semester and across different code bases; (3) to provide, receive, and react to (negative) feedback; and (4) to transform this feedback into lessons for the future. It is the combination of these steps that starts the process of “gaining experience” and becoming a socially responsible developer. Furthermore, if students truly wrap their heads around these lessons, they also learn how to instruct an AI so that it becomes a productive partner.

In first-year *programming* courses, students solve small problems and usually do not reuse code. A small problem does not require much planning; instead students can keep it all in their heads while they work out a solution. Worse, once students turn in this code, it often gets auto-graded, meaning nobody else inspects it and points out flaws that do not affect functionality. After they get their grade back, students rarely look at the code again, not to speak of fixing bugs or adding features.

A course on *software development* confronts students with a complex problem, meaning a problem they cannot solve in a short amount of time, keeping all aspects in their head. The course thus demands that students analyze the problem and try to spell out a solution path. Since students lack experience, the instructor will actually provide a actionable sequence of milestones. For every milestone, the students will need to react to test cases from quality-assurance engineers, i.e, their peers; to keep up, they will need to fix the bugs that these tests expose. At the same time, the milestone specification will request the addition of new pieces of functionality, which in turn may force students to revisit and modify existing code.

Warning *Soft skills are much harder to teach than hard skills.*

Critically, a software development course must also teach so-called soft skills that relate to code maintenance and the interaction with AI. The key soft skill is the ability to conduct technical conversations about all aspects of the development of software: the analysis of a wish for a software product; an attempt to turn the analysis into a specification and a plan; their code and their tests. Each such conversation has two sides: inspecting a development product and critiquing it versus receiving critique and reacting to it. By the end of the course, students may understand what a socially responsible developer is, though they may not yet be one. Key is that the course changes the mindset about the nature of software and informs students how they can properly train a junior partner called AI.

Developing a Mindset in the Age of AI

A mindset-changing course on software development focuses on two imperatives, starting with this:

- First, learn to develop software with, *and for*, other developers.

While it is easy and good to act responsibly with immediate colleagues—to be nice—socially responsible software development really means far more than that, and it might just mean not being “nice” (in the superficial sense of the word) for a moment. It is about treating the colleagues of the near and distant future as equal and respected partners in this conversation across time. They depend on the creators of code to express the “software message” as clearly as possible: so that the former don’t waste massive amounts of time—or expensive AI tokens—of the latter. The key is a bit of investment on the side of the creator:

- Second, develop software in a systematic fashion.

Clearly the two imperatives are intertwined. Systematic software development means a lot of hard thinking up front and expressing the results of this thinking as clearly as possible. When it comes to thinking, two partners are better than one, even if one of them is an AI. But, at a minimum, one of the two has to be a human being who checks the thinking and the articulation of the thinking as plans, code, tests, and so on. Without continuous checking, the results are likely not a message that is easily understood when it arrives at its destination: developers who must modify the code.

On the flip side is the developer who is tasked with fixing a bug in old code or with adding a new piece of functionality. Nobody can fix code without some minimal comprehension of the software system’s organization, the purpose of each of its pieces, and the code in the relevant pieces. It takes time to read and comprehend everything, be that a series of prompts or actual code, often much more time than the fix or the addition itself. Although AI can assist with this task, the human partner is ultimately responsible. Systematically developed software systems—i.e. thinking properly expressed as English or code—obviously has an advantage here. It reduces the cost (time, AI tokens) needed for this comprehension phase, and it simultaneously reduces the time it takes to check whether the fix or added feature mesh with the existing code.

Students must experience both sides of this conversation. On the *producer side*, they must develop code with human partners at first. Those must learn to critique each other’s work and to learn from this critique. As

students gain experience over the course of the semester, pairing them with just an AI is a feasible alternative. It works well if they can express specifications properly, follow a partner's chain of reasoning, and inspect the resulting code quickly. Pairs tend to develop a form of group think, however, and group think can mislead as much as solo thinking. To combat this phenomenon and advance students' ability to express their rationale, they must also present their products for detailed inspection in class.

On the *consumer side*, inspections play a central role and serve several purposes. First, the students' peers represent message recipients. Their feedback is critical to clarifying the "software message." Second, these peers learn to read and comprehend it in an effective manner. Third, they learn to articulate constructive critique. Finally, the presentations prepare students for the necessary switching of code bases, so that they can on-board a new partner and act as on-boarded partners during a hand-over.

In short, changing students' mindset takes place across multiple levels of technical conversations that generate critique. Constructive work is performed in pairs; inspections are public solo-performances. To motivate the latter, an inspection starts with "assume your partner has left the team" or "assume you lost connectivity to your AI." Solo presentations check whether a student completely understands the product (plan, code, tests), so that if one of them leaves, the other one isn't lost. Conversely, students also play the role of message recipients, colleagues who take over the code. These peers represent the future, unmet developers. It is therefore their task to ask penetrating and potentially unpleasant questions about every aspect of the design and the code. Without exception, such interactions can lead to unhappy conversations. Yet, they are for the benefit of the students and their preparation for jobs that need experience.

Being socially responsible is hard. Practice helps, and it is the instructor's job is to make sure that the practice is done right. To assist the instructor with this difficult task, I present this book. It distills three decades of experience into five workable chapters.

This Book, This Course

Instructors should think of this book as a guide to a course on *software development*. Its central premises are as follows:

1. software is a message from one developer to another across time;
2. the students' work must account for this "messaging;"

3. an instructor must run a semester-long project that involves the rotation of partners and of code bases so that students begin to appreciate what “software as a message” implies for their work.

Since a message turns thoughts into software products and receiving the message demands a reconstruction of the original thoughts, the course must put “thinking” into the center of the syllabus. Students must recognize that the best thinking happens in a social context. They must

- pair-program to continuously subject their thinking to critique;
- submit their work to public inspections to avoid “pair think;”
- learn to ask hard questions about other students’ code; and
- figure out whether such questions truly expose a problem.

Understood this way, the goal is to develop a mindset that coding is a social process: the immediate pair-programming partner; the review panel; the teammate who takes over the module; the peer who in a few weeks must fix bugs and add features.

It is the instructor’s task to coach students. Coaching involves feedback for everything students deliver: their problem analysis, their execution plans, their code and their tests, but also their ability to read code and spot essential problems, their ability to recognize relevant from irrelevant critique, and their positive attitude to negative feedback.

A Note on Software Engineering Courses A software engineering course goes beyond a software development course in several ways:

1. Students will work in teams, i.e. the course should cover some basic team organization techniques.
2. An instructor may even choose to use an extremely large code base, i.e. a code base that students cannot produce in a single semester.
3. The project demands trade-offs in many dimensions, e.g. a trade-off between the pieces of functionality and the given development time.
4. It may even demand that students find and eliminate performance bottlenecks involving space, time, or energy.
5. Lastly, a software engineering course may involve special domain-specific aspects, forcing students to wrap their head around a completely unfamiliar application area.

Simply put, software engineering is much more than just software development, which is much more than programming.

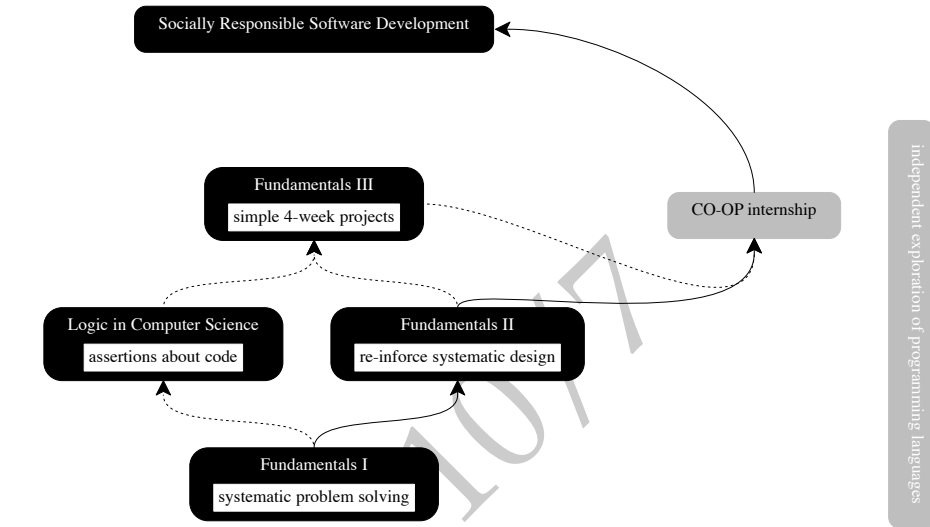


Figure 1: A core programming curriculum

Dear Instructor: Some Context

The course has evolved over three decades and across two rather different institutions. I spent these decades trying a number of ideas. Quite a few of them failed. This book collects the successful ones. Several former PhD students and friends have successfully test-driven this version of the course at other institutions.

Ideally the course sits atop two to three programming courses and one summer internship. A course on *Logic* is helpful, if it teaches how to express specifications of units of code as assertions in interfaces, i.e. software contracts. See figure 1 for a dependency diagram.

The first two courses, *Fundamentals I* and *II*, should introduce students to the idea that good programming calls for a systematic problem-solving approach. Focusing on a systematic approach, with testable intermediate

products for even small functions, is the key. It is far more important than the choice of programming language. Text books such as *How to Design Programs* and *How to Design Classes* are illustrative instances.

This book covers the essential ideas of *Socially Responsible Software Development (SRSD)*. While teaching this material isn't easy, it is also the most rewarding material I have ever taught.

Teaching *SRSD* poses a number of challenges. Each instance should use a distinct 12-week project. After several years of trial and error in the 1990s, I have settled on board games for 10-year-olds, because (1) all students have experience with those, (2) their implementation naturally suggests a series of milestones, and (3) it is easy to scale these projects (up or down) so that they match the students' experience and the available time. The latter means the end product could run one game via a monolithic implementation or it could run a tournament via a distributed (hub and spoke) system.

Delivering lectures based on the first three chapters for the first couple of weeks is straightforward. An instructor with experience in software development for actual users should illustrate each topic with personal stories. If the courses on programming failed to focus on systematic design, I recommend covering it with a few additional lectures based on the last two chapters; otherwise assign those chapters as required reading.

The true challenges show up when the instructor has to manage inspections. Doing so requires serious multi-tasking:

- The instructor must observe each presenter in order to provide feedback on how to improve the *presentation* of content, a possibly AI-generated artifact.
- The instructor must assess the quality of the panelists' challenges. If some critique is justified, the presenter's task is to show that it is understood; conversely, if it is wrong, the presenter must explain why the artifact already resolves the issue.
- The instructor must manage the interaction between the presenter and the panelists to make sure the conversation focuses on the relevant parts of the artifacts.
- The instructor has to step in when the presented artifact suffers from a major, undiscovered flaw. After all, these flaws yield "teachable moments." Recognizing them and turning them into lessons is the point of inspections.

Having students articulate these lessons is the best approach. Students who can do so, can instruct AI agents in a similar way and thus get them to produce secure and maintainable code.

Once the inspections are done, an instructor must sit down, reflect, and deliver to each presenter and each panelist concise, actionable feedback that enables them to improve.

The rewards come to an instructor in the form of student feedback once they enter the workforce, as interns or as professionals. Time and again, students shared with me that *SRSD* was the one course that prepared them for their professional lives, even students who did not do well in the course.

Employers, too, consider graduates of *SRSD* as the best-prepared products of colleges. Since the production of well-qualified professionals is one of the most advertised promises of the University, receiving this kind of feedback is the proper confirmation of success.

For the last few years, graduates have also confirmed that the course prepares them to properly deal with AI. They especially appreciate their ability to conduct technical planning and reflective conversations with the agents, so that they acquire the skills to proceed systematically. So, even though *SRSD* does not teach *how* to exploit AIs, it turns out the resulting soft skills transfer to the new world of professional software development.

mit: 10/7

mit: 10/7

What does the life of a software system look like?

The shortest lie in software is “it works,” where “it” refers to code. Although the point of this first sentence is that real software is hard to get really right, the implicit message is that “even if it works, you’re not done yet.” Software systems, once deployed, need a lot of attention and a lot of additional work. Software engineering researchers label this second phase *maintenance*. For some reasons, this word has acquired a negative connotation in the developer community. In the end, though, only permanently maintained software is truly relevant and important.

Maintenance is hard. People dislike maintaining software because they have to read and comprehend old code before they get to do anything. Sometimes this reading-and-comprehending phase confronts them with the results of their own inadequate programming skills from their early days on the job. Sometimes they don’t even know who created this code decades ago, and they just wish they could rewrite it all from scratch.

All of this brings us to the crux of the matter, namely, our perception of software as “something that works” but also “something that communicates complex thoughts across potentially long periods of time.” From this formulation, it also becomes clear that social responsibility has two sides: on one hand there is the end-user of the software; on the other, we find the software developers themselves and how they relate to each other. While both are important, this book focuses on the second aspect, the software developers’ responsibility to each other. In the long run, properly produced code also tends to serve end-users better than the code “that works.”

Let’s investigate these arguments slowly and carefully.

1 The Death of Software, The Life of Software

Most software dies an early death, without anyone ever laying eyes on it.

College majors in computer science know this fact well, too well. In most courses with a programming component, they spend hours writing programs, struggle to get them right, turn them in on time, and then get back the output of an auto-grading system. The professor certainly didn't look at their programs; the teaching assistants probably didn't. And when the task is done, the students won't ever look at their code again.

Professional software developers have similar experiences throughout their career. Someone has a great idea for a new product or feature. The team toils day and night to get it finished. Until eventually some "guy in a suit," also known as a manager, tells them that the marketing department no longer believes in the idea. And then it's all over.

Even after the launch of some software, chances are that the project fails. It just doesn't attract enough users. It doesn't make enough money. It's a failure, so there is no need to keep it around and invest in its maintenance.

When software survives, it is likely to survive for a very long time. In all likelihood, it will survive its creator. Consider the simple example of a credit card. Even though you may pay for your espresso with a mobile app that combines Objective C with HTML and CSS, the transaction is probably going to invoke some COBOL or RPG code from the 1960s. On its way, it may trigger some JavaScript scripts; run Java on the server; poke a C/C++ routine or even some old Pascal; and then land on the old COBOL code base—a code base that is 50+ years old.

In the 1990s, when your parents first started paying for things on the web with credit cards, the banks were panicking. Everyone knew that the year 2000 was coming and that the COBOL developers in the 1960s and 1970s had never imagined their software to be in service beyond 1999. They represented every date with two digits per day, per month, and per year. Memory was in scarce supply and therefore expensive. But what does 11 mean in the real world? 1911? 2011? This choice of representing dates as two-digit data became known as the "Year 2000" problem.

Dates aren't the only one form of information that poses such problems. In 2021, the stock price of Berkshire Hathaway reached a price of close to \$429,496.7295, which as an integer (price in 1/1000th of a dollar) is the largest integer certain languages can represent. At that point, the software developers got worried and rushed to fix the problem so consumers would be protected—which ironically caused a serious problem the next trading day. Indeed, there are many other glitches that have made it into software

If you have never heard of these programming languages, look them up. It's educational.



For details, see this link.

systems and that some developers had to fix or will have to fix. So imagine yourself be tasked with reading such badly aged software and spending just enough effort to eliminate such problems—because your company’s well-being and your high salary depend on it.

So what is software?

Surviving software systems are messages across time—weeks, months, years, decades—from one software developer to another. Initially, the message may go from one software developer to the same one, just an older version. Or one team member may hand it over to another member of the team. As time goes by, the software may need less and less attention. Everyone involved in its initial creation may move on. Yet, on occasion, someone needs to take a look at it. And every so often, an entire team may have to comb through all its details because the software needs a major overhaul.

Why does software need attention?

During the “birth phase” of a software system, developers may focus on creating features, fixing basic bugs, and making the user interface convenient for customers. After the launch, customers may use the software in unforeseen ways and discover problems; the user interface may need tune-ups for people with less exposure to modern software; some may wish for an additional feature; or they may request a fast response in scenarios that the original developers didn’t anticipate. Even after a product has been stable for a while, it is never truly finished. Software is the most malleable material with which humanity has built amazing artifacts, and we all—end-users, managers, developers—think that because of this malleability, we can turn software into something new and better at any point in time.

In summary, a surviving software system is a continuously evolving product. At different points in time, the evolution speeds up or slows down. But the system evolves. Evolution means changes, and changes affect both end-users and the system’s developers. The burden on the latter is immense: they must ensure that end-users are safe and secure, all while helping future developers understand how the product functions. In other words, software developers have an obligation to many parties—that is, an obligation of social responsibility.

2 Are You Socially Responsible?

What is social responsibility?

One interpretation of “social responsibility” focuses on the users and their satisfaction. End-users of software deserve (at least) the same quality service as the users of grocery bags, bridges, coffee cups, elevators, and so on. After all, our software ends up in mobile games, nuclear reactors, point-of-sales systems, X-ray machines, or airplanes. Some of those systems are life-critical; some are irrelevant to life but a good source of amusement.

Think of the pilots of a Boeing 737; they want the control software to work *correctly*. Consider “friends” on social media; they might be less concerned with correctness than *privacy*. In both cases, the *convenience* of the graphical user interface plays a large role. So making software work properly and well is a many-faceted problem, and it definitely connects developers to other people—end-users as they are called here.

Exercise 1. Can you think of other properties an end-user may expect from software? For example, we could weaken “correct” to “fail-safe,” which means “the code should never perform any harmful actions.”

Although there is no denying the importance of this form of social responsibility, software developers rarely get directly involved with these concerns about end-users. In many companies, managers develop functionality criteria, perhaps in conjunction with customer and/or end-user representatives but usually without much input from developers. The marketing guys may make promises about how safe user data is, and developers have to live up to those as well as they can. Finally, companies also often employ people who study the interactions between end-users and the software, and these people then turn those studies into feedback for developer teams. The point is, social responsibility to end-users is always expressed in terms of specifications that software developers must simply accept.

Which brings us to a first critical point:

While software developers bear responsibility for their product’s interactions with end-users, software developers are usually insulated from end-users. The people between them and end-users must accept the largest degree of responsibility.

Another interpretation of “social responsibility” concerns a developer’s responsibility to other developers. If developers are somewhat insulated from end-users, they are closely connected to their teammates. A common and good practice is to rotate pieces of a code base among the members

of a team, meaning one close colleague receives code from another one. Consider the reaction of the second if the code from the first is difficult or impossible to understand. “This colleague just robbed me of hours of my time, my evening with my wife/boyfriend/son with this bad code.” That’s the best reaction to expect. Even if a team doesn’t rotate responsibilities, a new colleague may encounter this hard-to-grok code and have this reaction. Or, months and years later, a maintainer newly assigned to this old code may get a shock looking at it—and chase down the original creator to share some thoughts.

Look up the term “spaghetti code” to find out how old this problem is

Having said that, we can now articulate the second insight:

Software developers are responsible to each other. Only socially responsible software development will save colleagues—near and far, in space and time—time and energy to decipher the thinking that the code expresses.

In other words, this second interpretation is the most relevant one for budding programmers. After all, they must internalize this interpretation before they become productive team members.

At this point, everyone will want to respond with a resounding “yes” to the titular question. Most of you might think “socially responsible” means being nice to a pair programming partner and getting along with the team. But that’s easy. You see those people every day, they see you, and a violation of a social protocol has immediate, unpleasant repercussions. In the software business “being nice” to teammates isn’t even remotely enough; indeed, it might be counter-productive.

True social responsibility includes caring about the people who take on the team’s code base down the line, whether we know them or not. For the sake of these future colleagues, it is socially responsible to critique what our immediate colleagues produce and to ask them to critique our work. This mutual critiquing must substitute for our interaction with maintainer when they take on our code base, in a year or ten years from now.

The goal of this book is to make you aware of this form of responsibility, both in terms of working with others and working on software. The expectation is that this training prepares you to understand the immensely difficult trade-off involved in developing software. It may mean being a little bit “less nice” to people you meet on a daily basis and to trade their feelings for the welfare of yourself (taking over their code in a few weeks); for the welfare of other team members; and for an uncountable number of

others in the future. This trade-off isn't easy, because we all prefer to "get along." Getting this trade-off right requires following basic development and interaction techniques. Practicing these techniques on a daily basis turns them into second nature, which makes them useful and acceptable in the long run.

Reading

Kent Beck. *Extreme Programming Explained*. Addison-Wesley. 1999.— In this book, Beck repeatedly argues for eight-hour days, 40-hour weeks, vacation time, and time for family. His key argument is that a software developer needs time to recover from thinking intensively. It is occasionally worth re-reading such passages, such as the one near the end of section 10 in his book.

mit: 10/7

What is software to its developer?

This question is at the heart of the conversation we must have about the software development process and its social context. Even if developers claim they are coding “for just money,” they still identify with their software to some degree. On one hand, this identification is important. A developer who identifies with his work is going to strive for high quality. On the other hand, identification with the product is an obstacle to a positive feedback cycle. A developer who identifies with his software may feel hurt when this product is subjected to intense criticism.

Modern software shops recognize this dilemma and put an impersonal intermediary between the developers and the reviewers, the github comment section and the like. It removes the necessity that the two people look at each other—look each other in the eye—and this makes the process less personal, less social, and less responsible. By contrast, this chapter argues that, while this reduction in direct communication may result in eliminating some amount of conflict from developer teams, it does so by imposing a significant cost on the people who work with the software now and even more so on those who work on it in the future.

You cannot look an AI partner in the eye. See below.

Instead software developers must learn to remove their ego from the process and turn negative feedback into positive actions. While nobody is born ready for negative feedback, learning from it is possible with explicit practice. Ideally, a developer can become so comfortable with criticism that it is solicited explicitly. Internalizing this attitude is ideal for a socially responsible software development process.

The following sections spell out how a developer might acquire a positive attitude toward negative feedback. The first step is to remove one’s ego from the product. The second is to embrace working in pairs. The final one is to solicit team-wide criticism via probing inspections.

3 The Nature of Software Development

Software developers are artists, creators, designers, and inventors. The software systems they create are like books, compositions, paintings, poems, or sculptures. Like other creative people, developers labor and struggle to produce these artifacts.

When developers see other people successfully and easily use their software, they are, and should be, happy about their achievement. One developer may point out to her father which piece of an app's interface is hers; and another may explain to his mother how his machine-learning software discovers suspicious patterns in a patient's MRI scan.

Similarly, developers are always happy when another developer remarks on the clarity of their code, its efficiency, its low memory footprint, and so on. If the other developer is a respected and experienced member of the team, the praise is particularly meaningful.

Like all creators, software developers inevitably identify themselves with their product and love to receive praise for it or enjoy watching others use it. In other words, they invest a part of their ego in their creation. This investment—this identification—is neither surprising nor bad. Whenever people are deeply invested in a task, they are motivated to give their best, to make a respected product.

The problem is that all creative people must stomach criticism, too. Authors hate to read bad book reviews; composers don't want the audience to leave the premiere early; painters prefer good-willed, "nice" people at their vernissages; and poets might be reluctant to read their works in public. Why? All creators fear negative criticism, because of their investment in the product, because of their identification with it. Deep down criticism of a creative product is considered criticism of the creator's ego.

Developers are no different in this respect. End-users may criticize their software for being clunky, slow, or buggy. Worse, their colleagues may criticize them for style-guide violations, lack of elegance, a badly chosen data representation, and so on. While the former may hurt a bit, the latter stings really hard because these colleagues are professionals whose opinions count a lot, even if it is just within the developer's team.

Poets, painters, and other artists can ignore criticism and hide from their critics. Society will simply ignore a new musical composition if newspapers pan it. Readers don't buy a novel that consistently receives bad reviews. In short, the economic damage of bad artistic creations is relatively

small. None of this is true for the product of software developers; the people who have to use these bad software systems suffer, and the developers themselves suffer, too, from their own creations of those of their peers.

Bad software can inflict serious harm on end-users. It may shut down an electricity grid or a phone switch, leaving millions stranded. It may cause financial harm. It may down an airplane. You name it, software can do it. The worst aspect is that software libraries and frameworks are migrated from one code base to another, and the original creator may never find out where code ends up.

Badly written software also affects current and future colleagues. One colleague may have left yesterday for a better-paying job at a competing company, and another one is assigned to his portion of the code base—and it turns out to be incomprehensible. The newly assigned colleague will have to work extra hours to catch up with the rest of the team—hours that, as mentioned in the preceding section, come at the expense of spouses, children etc. Similarly, imagine an intern who has just signed up for a six-months stint. She is pointed to a code base of some 100 files/modules and is told to resolve some error tickets. The code repository lacks any documentation of how to navigate the modules, which one serves which purpose, or how they are related. All design decisions are buried in commit messages, including AI prompts, and there are tens of thousands of small commits. How is she going to live up to the expectations of her mentor?

Think of the “Year 2000” problem already mentioned in section 1. Numerous programmers had to dig through huge volumes of decades-old software in the late 1990s to find all instances of two-digit date declarations and all their uses. They had to repair those systematically; they had to adjust how this repaired code connected to old databases; and they had to make sure that the code’s functionality remained the same. And sadly, there were few people left who actually knew the programming languages of the 1960s, which they had to teach others all while working on revisions.

In contrast to most artistic creators, though, developers are consistently paid a significant salary almost immediately after graduation. Furthermore, their software often has a potentially large value. So unlike artists, software developers “owe” their end-users and their colleagues. In particular, they owe them an attitude that welcomes criticism, welcomes negative feedback and turns those into positive changes to their products. Doing so means letting go of some ego.

The concept of pull requests (PRs) has leaked into performance evaluation, causing a pay per PR problem.



4 Egoless Programming

In 2018, the Wall Street Journal's Sue Shellenbarger wrote

Many people hate getting negative feedback and avoid dishing it out too, fearing it will spark anger, dejection or even tears.

But some people actually want and even seek out criticism at work. Who are these people?

Shellenbarger at WSJ

in a column on workplace and family affairs. Further along she reveals that

[e]xperts in their fields tend to be motivated by criticism.

They solicit criticism because they have come to understand it as a way to improve themselves.

Decades before, in the 1970s, Gerald Weinberg noted how programmers benefited tremendously from honest and frank feedback—if they could let go of their ego in their code. He coined the slogan *egoless programming*. The idea is easily expressed with three bullets:

- Show your code to other developers and ask for feedback.
- Expect, embrace, and evaluate criticism. If necessary, fix things.
- Reflect on what happened.

Rinse, repeat, and never stop.

In essence, both of these authors just focus our mind on the essence of learning outside of school. Without sharing our work, without exposing ourselves to others, we cannot hope to discover problems with our thinking. This is especially true when this thinking turns into a message for current and future colleagues.

Implementing egoless programming is challenging:

Egoless programming requires a significant change in mind set. It must start with yourself and your desire to become an expert developer.

One consequence is that it can only be accomplished in a bottom-up manner, not via commands from managers, which might merely antagonize the team. Setting an example works well, and encouraging others to emulate the example helps spread the idea.

So, first you invest energy into programming but at the same time separate yourself from the resulting code. Then you need to find like-minded

others who are willing to listen and reciprocate. In some teams, doing so might be straightforward. Your colleagues may already practice egoless programming and the critiquing techniques described here. In others, you may have to bring up the idea with your team, because nobody has thought of just doing it. You may find open minds, and everything will go well.

If nothing works, there are some old standbys of the olden days: “water coolers” and “vending machines.” Now these words are just metaphors for “a place where developers meet” during a break, to chat, to exchange gossip, and so on. When programming meant exploring the limits of computers, operating systems, and languages, people congregated around card readers and printers. There, while they waited, they discussed their programs and exchanged hints on basic programming problems.

A historically minded reader may wish to search for images of old card readers and line printers.

In short, make a point of finding contemporary “water coolers” and “printers.” Or, if needed, create such places—real ones, not virtual ones. Then use these gatherings to learn from experienced developers, to show people what you’re working on and ask for criticism.

Let’s say you have found a partner and a review uncovers flaws. Your reaction is critical. Clarify to people that discovering flaws is *good for you and good for the whole team*. You can assess the flaw, evaluate your options, and improve your code. Doing so will help others, and your positive attitude about finding flaws in time may infect others.

Also take time to reflect and draw general conclusions for such mistakes. Sometimes it is possible to change habits based on a single example. Other times it may take several conversations to figure out the patterns of one’s problems. In all cases, you will improve your skills and your product—something that will make you happy.

Of course, you will also need to balance criticism with the ingrained perception of others that criticism reflects badly on you and your skills. It will take time to convince those skeptics, but if they notice improvements in your skill set, they will see how it improves at a *faster rate* than theirs. Then the time has come to suggest that they too can benefit from egoless programming, negative feedback, and turning it into positive energy.

Reading

Gerald M. Weinberg. *The Psychology of Programming*. Van Nostrand Reinhold Company. 1971.—The entire book is an excellent introduction to the topic, but if you don’t have time for the whole book, focus on chapter 4. It directly addresses the idea of programming in a group, the principle of egoless programming, and the importance of “water cooler” conversations.

5 Pair Programming, With AI

What happens when software developers program?

Obviously they write code or they write prompts that generate code. Before that, they need to understand the purpose of their new code; the context in which it exists and how it interacts with this context; its basic performance characteristics; and much more. “Understanding” means *thinking*, as in, thinking through someone’s requests and about existing code. It also means thinking while coding and reflecting on the code’s context.

Coding doesn’t always start from a blank page though. It is equally common that developers must modify existing code. They must read and comprehend code. They have to reconstruct the design rationale from comments and commit messages, which means understanding the thinking of their predecessors. When they finally edit the code, they must not break the existing functionality. They have to formulate new comments and new commit messages. And, if they wish to help their successors, they should leave behind documents about what they found out from reading the code. All of this just means more thinking.

Yet, coding isn’t just thinking either. It requires a bunch of mechanical tasks: keyboarding text, picking imports, selecting identifiers from menus, compiling, testing, and so on. For a long time, a programmer had to perform these tasks by hand, then integrated development environments (IDEs) took over some of them. More recently, AI agents have taken on the burden. Over the time the community, collectively and individually, gained experience with the generation of code—and students are well advised to follow the same path, though at a faster pace. At the same time, mechanical work—regardless of how they are performed—means that the developers’ thinking about code gets constantly interrupted, also because working developers spend half their time in meetings.

We all know that thinking is hard and interrupted thinking is harder. Contrary to appearances, AI agents do not help (much) with the thinking of a junior developer. The only way to get *immediate* feedback on *thoughts* is to work with a partner that proposes ideas, specifications, plans, code, tests, and debugging approaches or, conversely, checks them. And this brings us to pair programming with another person.

Ideally, *pair programming* works like this:

- two partners look at the same text;
- one of them acts as designated driver, the other as co-pilot;

- the co-pilot requests products; the driver *proposes* results;
- the co-pilot checks them and demands explanations or revisions.

After completing a unit of work, the human partners switch roles. Once a developer has sufficient experience with this process, an AI agent can replace a human partner. When this happens, the human partner should ask the AI agent to reflect on what happened during the interaction and to even write the reflection down as a rule to be observed in the future.

In a team setting with just human beings, it is common that every member pairs up with everyone else on the team over the course of a project. This way the team members get to know each other, their thinking, and their coding habits. Additionally, the code-base knowledge gets smeared over more than two members.

The immediate purpose of a co-pilot is obvious. It is about checking the thinking of the driver to provide immediate and direct feedback. But, the co-pilot is also a representative of the rest of the team. In this light, it's the co-pilot's job to wonder "whether my colleagues will understand it given that I don't." And if those colleagues don't understand it, they cannot fix code or add features without major effort.

By extension, the co-pilot also represents those colleagues that aren't a part of the team yet. Those colleagues may still be in high school. They may not be born yet. In short, the co-pilot makes sure that the driver acts in a socially responsible manner, toward team members, new hires, and people who will maintain the code in five, ten, or twenty years from now.

Pair programming has several benefits for developers, their teams, and their company. Most importantly, one developer's thinking, including an AI's, is immediately checked by another person. Thinking happens in our mind, and our mind doesn't cross-check ideas easily. Hence, when we think just by ourselves for a long time, we may build up a complex construction of ideas on top of some misconception—bluntly put, we may get "off the rails." Forcing ourselves to explain an idea to someone else may reveal such misconceptions directly, or the partner may discover them. Either way early discovery of thinking in the wrong direction is beneficial.

*Look up the term
"rubber duck
debugging."*

Joint work also has psychological benefits. While some complex mathematical problems may require long, hard solo work, software development rarely calls for this kind of deep thinking, a search for the creative spark. Yes, it requires thinking but some of the thinking is just tedious and error-prone for various reasons; it is not creative at the level of individual lines of

code. Sharing makes it somewhat easier to perform such work, especially when a modern AI agent can get most of this labor out of people's way.

Shared work also means shared knowledge. If a team member has to stay home one day due to sickness or a family emergency, past pair programming comes in handy. The other team members have partnered with the absent one and have thus acquired some understanding of his part of the code base. Even when AI generates the code base, rotation and explicit plan inclusion in the repository preserves enough context so that nobody has to feel stuck due to some absence.

This benefit of shared knowledge of the entire project is even more important when a team member quits. From the company's perspective, this benefit might be the most important one, directly affecting the bottom line of this cost center. All the money "sunk" into the member who's leaving doesn't translate into lost knowledge that someone else has to figure out. It is distributed over the entire team and the AI agents' reasoning documents. A new hire can be assigned to the team, and she can pair-program with others to acquire this shared understanding of the code base.

After reading all these positives, you would think that companies have been racing to implement human-based pair programming as quickly as possible. Sadly, too many managers completely failed to understand the nature of programming and the benefits of pairing up. The most common objection has been that "two people are doing the work of one."

To counter this nonsense takes careful and patient "managing up" with four effective arguments deployed and re-deployed time and again:

- The software is going to run faster, use less space, and consume less energy. Managers love performance arguments.
- Shared knowledge of the code base reduces inefficiencies when someone leaves. It makes programmers somewhat replaceable, not unique gems. Managers like this idea as much as performance arguments.
- Continuous partnering ensures that the code will be accessible three or five years down the line after the product has become a profit center. The company will be able to add extensions quickly and cheaply. Cost reduction arguments almost always work, too.
- Pair programming is also likely to get the software right faster than solo work. Since bugs often cost money, bugs mean loss of profit. If the cost motive doesn't work, the profit idea will.

The key is to figure out what matters to managers, to learn to speak their language, and then to articulate advantages in those terms.

Fortunately, the emergence of AI has changed this picture to some extent. While managers have been unwilling to spend money on pairing up humans, they do spend on AI partners. Even though this development is welcome progress, it needs experienced programmers to make it work and to find a proper balance.

Reading

Laurie Williams, Robert Kesler. *Pair Programming Illuminated*. Addison-Wesley. 2002.—This short book collects in one place all that you need to know about pair programming: what works, what doesn't, and how to "sell" it to your team and manager.

mit: 10/7

6 Inspections; of AI and by AI



Technical
Conversations

When two developers stare at, and discuss, the same piece of a project for long enough, they will convince each other that “it’s done.” This trap exists for pairs of human developers as well as human-AI partnerships. It is a form of unintentional collusion. One way to overcome this trap is for teams to conduct *inspections* of requirements, specifications, designs, code, tests, and more. Colleagues with only peripheral involvement *inspect* the software and critique it. Most developers get to know these meetings as *design reviews* and *code walks*, even though they often cover much more than just design or just code.

The idea of software inspection dates back to the 1970s, a time when software companies began to understand that programming was more than just creating code, that it was actually akin to an engineering activity. While design reviews had been used in many engineering disciplines, adapting them to the engineering of software took some time. All forms of review meant face-to-face meetings, and people followed a variety of processes: code browsing, desk checking, peer review, walk-through, and more.

IBM’s *formal code inspections* came in the form of a rigorously defined process, based on scientific investigations. The goal was to make the results of these inspections measurable, with a focus on the rate of error discovery. IBM and others who adopted this form of formal code inspection reported rather high discovery rates, though everyone also admitted that following the strict protocol was quite costly.

Over the past couple of decades, especially with the emergence of commenting features as part of on-line code repository software, the review of code has become common. These reviews rely on one set of developers to leave comments for another set of developers for a specific unit of code, often just a pull request. Not all, but many companies practice such platform-mediated reviews. Sadly though, these practices leave behind many of the qualities of in-person inspections and conversations.

Since these various forms of inspection differ so much—and don’t aim for the same outcomes—this book uses the phrases *code inspection* for the informal in-person version and *code review* for the on-line version. Keep in mind that “inspection” as used here is *not* IBM’s formal inspection process, though it takes its inspiration from this work.

Let’s take a close look at the basic ideas behind in-person inspections and on-line reviews. Inspections make reviewers face developers, that is, the latter look at the former and vice versa as they discuss code. Having to

face each other during the inspection and conversing in real time has some obvious advantages. When a reviewer expresses a criticism, a presenter can re-articulate the comment to clarify its meaning and a back and forth is likely to eliminate misunderstandings. These real-time clarifications also become the basis for improvements to the software, including its planning and component documents, its specifications, its code, etc. A face-to-face conversation is typically friendlier than other forms of exchanges; just think how asynchronous conversations on modern social networks can deteriorate rather quickly. Furthermore, a conversation naturally generates teaching situations in which reviewers can learn from presenters and vice versa.

Facing someone to express criticism can trigger stress in both parties, however. To reduce this stress, the inspection process needs to be properly managed. Here, “properly managed” means developers must know how to conduct inspections both as presenters and as reviewers. They must be trained and prepared. According to several studies, the training’s pay-off seems to be rather large. These studies show that this low-tech technique discovers many communication problems quickly and, as a result, saves a good amount of development time.

	platform-mediated reviews	in-person inspection
<i>cost</i>	a moderator may be needed	preparing the reviewer
<i>efficiency</i>	time for analysis & reflection	real-time conversations
<i>stress</i>	from quasi-anonymous posting	from face-to-face criticism
<i>benefit</i>	finds some bugs	finds bugs & design flaws, improves readability, and develops human resources
<i>it's like handing code over to a maintainer who ...</i>	... joins in 10 years from now	... is an immediate colleague
<i>teaching situations arise ...</i>	N/A	... naturally
<i>conversations develop the skill ...</i>	rarely take place/work	... to reason about code, which helps with coding

Figure 2: A comparison of the two dominant code inspection modes

By contrast, platform-mediated code reviews eliminate direct interactions. Depending on the setting, an assigned developer, or any developer in the open-source world, reads portions of the software and leaves com-

ments. Reading means a reviewer can take time to sit back to analyze and reflect. It can thus produce well-thought and thorough comments. Spatial and temporal distance between the creator of the software and the reviewer simulates the situation of a developer who months or years later is asked to read, comprehend, and modify the code; this particular aspect directly contributes to the goal of social responsibility to strangers. By removing the potentially awkward personal encounter during critical sessions, on-line reviews seem to eliminate some stress. Experience with such platforms suggests, however, that inviting quasi-anonymous reviewers can encourage rather aggressive commenting. Such aggression has shown up in both private and public forums, leading to new forms of harassment. In short, the cost isn't as low as it seems in some situations.

Figure 2 displays a three-column table that concisely summarizes this comparison. As even a cursory look suggests, there is no clear winner. But, in the spirit of putting the developer into the focus of the software development process, this book focuses on software inspections. After all, the goal of the book (and the course) is to grow developers so that they can cooperate with AI agents, and to this end the emphasis has to be on interactions. Once growing developers understand how to conduct inspections, they can also perform those with AI agents and grow them, too.

Running software inspections requires preparing and training developers for this task. This training must clarify the goals of inspections, the role of panelists, and some mechanics. Not all setups work well; the one described in the remainder of this section has proven its worth over a couple of decades. When the prose speaks of “students,” the reader may wish to think of “trainee” and “AI agent.”

Goals Inspections have two goals:

1. understanding whether and how the presented part of the software project meets the expectations, and
2. understanding whether it is systematically designed and organized in a way that facilitates comprehending and modifying it.

Concisely put, inspections aim to find bugs and design flaws.

The first goal, meeting expectations, depends on what is presented. In the initial stages of a software development project, the requirements are typically spelled out, meaning the focus rests on documents such as the specifications of components and their interfaces, production plans for these components, etc. For these documents, the expectations concern their

organization and their clarity. For developers working with AI agents, both of these attributes matter a lot, because they determine how well the agents can translate ideas into properly working code. Once the developers start generating code, expectations shift to functionality, performance, robustness, security, and similar code characteristics. Usually the reviewers and the developers can easily agree that some code will not behave in the specified manner, and they can also easily settle whether an algorithm performs the desired task in quadratic time ($O(n^2)$ where n is the size of the input) when it could easily be linear ($O(n)$, with n again the size of the input). Less obviously, but equally important, panelists must check for corner cases that can bring down a running system or for leakage of protected information.

The second goal, a comprehensible organization, again applies to all aspects of software development. Concerning English-language documents, this goal is easy to achieve via clarifying conversations. Concerning code, the idea raises somewhat amorphous questions. Here are some examples:

- Is the code organized into functionally coherent and clearly delineated units (modules, classes)? Are those reasonably small?
- Does each basic building block (function, method) have a clear purpose? A purposeful name? A reasonable size?
- Are the expressions and statements clear?

For programming principles and sample inspections, see chapter III.

Each of these questions has an obvious negative variant, and those are the ones that reviewers must have in mind. Accordingly, the feedback will be about convoluted units of code, over-sized functions, and too-clever-by-half expressions.

Critique and Criticism Inspections are about delivering critique, that is, *constructive yet negative feedback*. At some point in your life, you might have been told to deliver all negative feedback in a sandwiched manner: some nice feedback, the bad part, some more nice words. *Don't!*

Sandwiching negative feedback consumes too much time and obscures the critique. Meeting time is precious, so taking up time with obvious compliments destroys a precious good. Worse, the criticism gets lost in a sea of forced praise. You will find a lot of mistakes in any piece worth inspecting. Wrapping every one of these criticisms with nice words requires an effort and form of communication that drowns the important parts. Don't do it.

At the same time running a panel demands a laser-sharp focus on the habits of egoless programming. The reviewers must keep in mind that the presenter may be stuck in ego-focused programming. It is thus imperative

to ask *questions about the software* **not** *questions about “you, the programmer.”*
Here are some examples:

<i>good</i>	<i>bad</i>
Does this production plan allows us to demo a minimal valuable product by year end?	Do you realize that we need to demo a minimal valuable product by year end?
How does this function account for the special case of 0?	How do you deal with 0 in this function?
Where does the exception get caught?	Where do you catch the exception?

Note how most questions make more sense when stated in terms of code rather than its creators.

To summarize, panelists bring out the best in presenters by emphasizing egoless programming as much as presenters can do so by separating their ego from the presented code.

The Panel An inspection confronts a pair of developers with a panel. The pair is jointly responsible for a particular piece of functionality. The panelists are typically members of the team, know the presenters, and have a basic understanding of the presenter’s task in the context of their project.

A panel has three members:

1. a *head reader*, who assumes the dual role of reader and moderator of the review effort;
2. an *assistant reader*, who may exclusively focus on reading the software and posing questions about it; and
3. a *secretary*, who is primarily charged with taking notes that describe the problems. These notes must enable the presenters to solve the problems. They do *not* record the exchanges between the presenter and the panel. A panelist may ask questions as long as doing so does not distract from note taking.

Experience with this arrangement and IBM’s stringent protocol suggests that more than three people do *not* improve the quality of the code walk.

Preparation In general, the panelists should prepare themselves for an inspection session in two ways. First, they should study the problem in

depth and the relevant context. Second, they should familiarize themselves with the inspection protocol. This second point is particularly important for panelists with little or no experience with inspections. Surprisingly, studies of code inspection have repeatedly found that reading the documents or the code in question ahead of an inspection session does *not* affect the outcome of reviews.

In a *course context*, the panelists are practicing code inspections after having been told about the principles. So their preparation should focus on how to run panels. By contrast, they do *not* need to prepare themselves otherwise. Since all students work on the same problem (per milestone), they are intimately familiar with its nature and its context. Indeed, because all students have worked out their own solutions by the time a panel meets, the real danger is that panelists try to superimpose their own solution instead of comprehending the presenters'. When this happens, instructors must redirect the panel's inquiry.

In a *professional context*, the head reader should be an experienced panelist who can implicitly teach the assistant reader. Both may understand the role of panels, but they may not be as familiar with the problem itself. So their preparation might be more about reading up on the problem and the requirements rather than refreshing their knowledge about panels.

Mechanics During a panel presentation, the head and assistant readers need to imagine themselves as developers who take over the presented software. When they do not understand any aspect of the software, they must question it. Some of those questions may merely clarify superficial aspects and receive immediate answers; others will uncover problems.

Although both readers can work with the presenter to figure out something, the head reader must assume the role of moderator. Specifically, the head reader must stop conversations that

- cause cross-talk or pursue conflicting goals;
- seem to go nowhere promising;
- turn into defenses of existing documents and code that both readers consider wrong;
- discuss *solutions*. Inspections should be mostly about *problems*; if presenters request a discussion of *solutions*, it is up to the head to decide whether doing so is fruitful.

In those cases, the head reader is responsible for getting the inspection back on track.



Evaluating Panels

If a solution is obvious, the secretary adds it to the notes and mark it as such.

When students become defensive, the instructor must step in.

The third item is particularly important. Defensiveness means a person has invested too much “ego” into the product. In such cases, it is the head’s task to remind everyone that questions are about the software product and that critiques are also about the software product—not the person.

Presenters While the presenters created the artifact as a pair, the advantages of pair programming are realized only when both partners can explain any piece of the joint work. To check on this state of knowledge about the code base, only one partner should present at any point in time. An instructor or technical manager should switch from one presenter to another at arbitrary points to validate that both are sufficiently knowledgeable. The quiet partner may navigate if it facilitates the discussion and should take notes as needed. These notes can supplement what the pair receives from the secretary.

Examples The remaining parts of the book include sections on how to review planning documents, specifications, interfaces, code, and so on. While all inspections follow the same general principles, they differ in how presenters and panelists interact due to the nature of the presented artifacts.

Artificial Intelligence At this point, it is obvious that inspections are far more intense than platform-based code reviews. Most importantly, the back-and-forth gives rise to teachable moments and thus contribute to the education of junior developers. Which brings us back to AI and its role in this process.

Concretely, we need to address the following questions about AI:

- Should AI-generated software be reviewed? If so, how?
- Should AI review software that a human or an AI created?
- How should the AI learn from the feedback?

The answers are straightforward. When a developer starts working with an AI agent, the latter is at best a junior developer, learning the ropes. Just like a student in a software development course, its output has to be inspected, because we know that AI code suffers from all kinds of weaknesses: bugs, performance problems, security holes, design flaws, and so on. Most importantly, the code often violates a team’s agreed-upon standards. And just like in a human-only pair programming setting, it is the partner who is in charge of the initial review. If this person has any doubts about the outcome of such inspections, the code should be subjected to the

panel-style review, using the same process the team uses to inspect human-generated code.

Such a panel could consist of human developers or, if a developer is sufficiently experienced, it may include—or consist of just—a properly prepared AI. The emphasis is on “experienced” here, and it mostly means the developer has trained the AI as a reader with guidelines on how to analyze code and how to provide feedback via a problem-focused memo. It remains the task of the original creator to solve the problems.

Finally, as the AI receives feedback, the human developer must ensure that some learning takes place, just like with a human junior developer. One successful approach has the AI re-formulate the feedback as a lesson about a specific piece of the code base. It is the human developer’s task to ensure that the AI has drawn the proper lesson, which is then deposited at the appropriate place in the code base. Like an instructor, a human developer may then request that the AI compares this new lesson with existing ones. If it finds similarities or overlaps, the AI should promote such lessons into a general skill, to be used for all future software development tasks.

An instructor should ask students to follow the same pattern of reflecting on feedback.

Once again, management may object to the cost of spending a good portion of a team’s time on mutual code inspections, just like with pair programming. The discussion with management has to address the same aspects as the ones for pair programming and must use the same managerial language. Unlike pair programming, formal software inspections come with peer-reviewed measurements that show just how much inspections save in the long run. Note however, that the informal process presented here—and similar processes used in industry—have not been subjected to such rigorous analyses. Indeed, even if such results existed, it is also well-known that management rarely respects such experimental data. The excuse is almost always that “we are different.” Hence the goal has to be to “sneak” mutual code inspections into the process, to point to successes, and to get them adapted across teams and AIs.

Readings

The following articles are about *formal inspections*, which differ in some ways from the code inspection described here. They are even more labor-intensive than the ones presented here. In turn, they come with a high payoff in terms of error detection. Reading some of the original literature may help clarify how much in-person inspections can accomplish and how to inject these processes into existing organizations.

Michael E. Fagan. Advances in software inspection. *IEEE Transactions on Software Engineering*. **12**(7). 1986.—Code inspections are the brain child of Michael Fagan from IBM. He started investigating the idea in the 1970s and reported on his progress in several papers. This one is the most substantial report from this period.

Glen W. Russel. Experience with inspection in ultralarge-scale development. In *IEEE Software* **8**(1). 1991.—This magazine article presents the interesting claims that (1) code inspections may save up to 33 hours of developer time per bug found and (2) code inspections find twice as many bugs as integration testing. Both claims have to be understood in the specific context of this experience (telecom switches, understanding of software development processes). The article also comes with extensive hints on how to introduce formal code inspections into a reluctant organization, ideas that probably apply to all forms of code inspection.

Edward F. Weller. Lessons from three years of inspection data. In *IEEE Software*. **10**(9).—Weller's article reports on experiences similar to those from Russel, but for a different context. The goal is again to inform readers how to manage their managers.

NASA Office of Safety and Mission Assurance. *Software Formal Inspections Guidebook*. Technical Report OFFICE NASA-GB-A302.—NASA has a large stake in getting all software correct: unmanned missions cost billions of dollars and for manned missions, lives depend on software as much as the surrounding hardware. This report is one of their first comprehensive guides on code inspections, though the agency and its contractors has had a long history of working in groups on mission software.

What is a software system? What goes into its production?

A software system is many lines of code in the same way as a house is just a large number of bricks. Everyone knows, though, that there is much more to a house than a large number of bricks combined with mortar. It consists of walls and roofs and doors and many more distinct pieces. In this spirit, a software system is more than just 10,000 lines of code or 1,000,000 lines or even more. It consists of major and distinct building blocks, often called components; and yes, deep down these components consist of code, but as we shall see, “lines” is the wrong idea even at that level.

Identifying and delineating these major pieces is the first goal of any software development project. To find the pieces, it is necessary to study the various ways how the system interacts with its context, and how the components inside of the system interact with each other. Once the major pieces and their interactions are identified, someone can make a construction plan and assign the construction of the pieces to developer teams.

The following sections provide a detailed look at the nature of software systems and the first phase of the production process. It starts with a close look at how software systems function, because understanding this much is a prerequisite to recognizing what their major pieces look like. How these pieces are identified, differs from era to era and from team to team. But, at this point in time, we understand the basic steps of getting from a problem statement to a software-construction plan. Understanding the general process will also help developers who join projects long after the initial product has been constructed, especially if the developing team has taken the time to create documents that provide an overview of the system: its purpose, its major components, their interactions with the system’s context, non-functional constraints, and even a construction plan for the code base. A team acting in such a responsible manner will greatly help developers who join the project late and are tasked with modifying the system.



*Getting Started,
Getting Stressed*

7 Running a Software Systems

How does a software system function?

Before we tackle the question of how to construct a software system, we need to look at how a system is run. Reflecting on how software systems work will help understand the analysis step, the component identification, the planning phase, and more. Clearly, an operator must launch a software; when it runs, the system will respond to requests; and eventually the operator commands the system to shut itself down gracefully, which may include preparation for the next run. But, people have also come to expect that running software systems adapt to new contexts and end-users. That is, the system adjusts itself to various settings: it configures and re-configure itself; it loads and unloads pieces dynamically; and it cooperates with external systems as needed. Figure 3 illustrates a run with an abstract “plot sketch.” Keep it in mind as you read the remainder of this section.

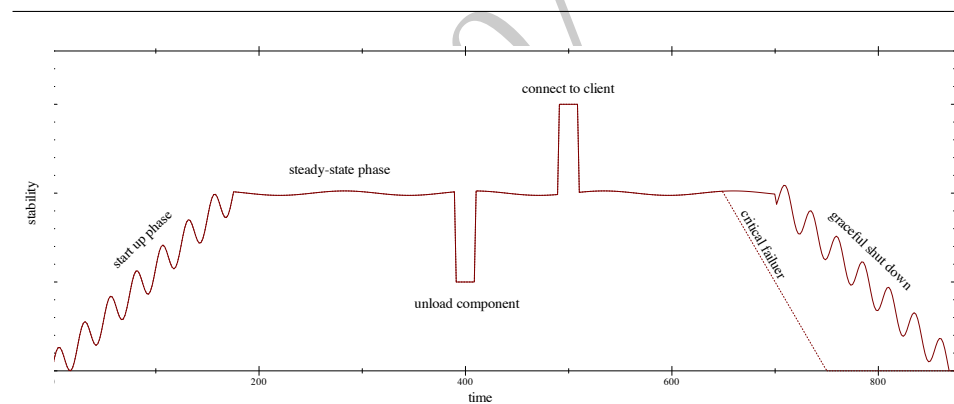


Figure 3: A typical run of a software system

The Abstract View Let’s take a look at each phase of a typical software system run. You may wish to consider how each of these ideas applies to some of the systems you have already used: a web browser, an IDE, or perhaps a networked game. The second part of the section shows how to map the workings of an IDE onto this abstract picture.

When launched, a system configures itself according to instructions from the command-line and/or a configuration file. During this configuration process, the system loads, or connects to, components; components

initialize themselves according to configuration parameters; and one component will pass configuration information to others.

During this initialization phase, the system goes through a period of instability. Components may become available in a concurrent manner, meaning the order in which they are loaded or connected is unpredictable. Hence, any individual component cannot assume that the services of some other component are available when it starts up. Once the configuration is completed, the system transitions to steady-state processing.

In this steady state, it responds to requests from the outside. A request typically triggers the execution of some service—runs some pieces of functionality, one after another or perhaps in parallel—and in the end delivers a response. An external requests may trigger the loading or unloading of components. Such run-time extensions dynamically integrate modules or classes into the running code. Others execute components as separate processes—to isolate the main system from any effects of the subsystem—and communicate inputs and outputs via operating systems facilities (files, pipes, databases, and so on). Yet others connect to completely external components for a time and then disconnect again.

In such a multi-component system, some services are critical and some are optional. A component is critical if a failure in the component implies that the system can't deliver a response to an external request that is considered absolutely necessary. A component is optional if its externally visible effects are of a low-priority nature.

Since all software comes with flaws and flaws may cause failures, a system must be constructed so that it deals differently with these failures in critical and optional components:

- when a critical component fails, a software system should shut down, ideally gracefully, and perhaps even restart itself;
- when an optional component fails, the system may recover by unloading this component or working around it.

This observation has direct consequences for the developers of the software system. They must not only think about the proper functionality but also about failure discovery and dealing with failures. To deal with failures in optional pieces, they must include recovery capabilities. To deal with failures in critical pieces, they must harden the system to go down in a manner that does not damage its context and, ideally, logs some error message that gives the system's operators a hint as to what went wrong.

Exercise 2. People have come to expect the fault-free, continuous execution of some software systems, for example, the phone network or the internet. But even these systems suffer from problems on occasion. Their reliability is thus measured in “the number of nines.” For example, “five nines” means the system is up and running 99.999% of the time, and that it may be down for less than 0.001% of the time. Calculate what these percentages means in terms of down time for one month and one year expressed in minutes.

Finally, sometimes operators must shut down a software system. When a software system receives the request to shut down, it must do so in an orderly manner. It should save any data it may need for a re-start. It should disconnect from external systems properly, which may mean disconnecting databases, closing network connections, or unloading components. It should terminate any external processes that it launched. Generally speaking, it should release its resources. Like the start-up phase, the shut-down phase may induce some instability, and it is thus imperative that the system no longer responds to requests during this phase.

Let’s summarize this description:

1. Any interesting software system comes with a start-up configuration.
2. During the start-up phase, a software system loads and configures components according to the given information. It may be unstable during this phase.
3. Once the execution of a system enters its steady-state phase, external requests call for the execution of services and responses. A request may also result in the loading or unloading of components.
4. A system must be prepared to shut down in an orderly fashion in response to specific external requests.
5. Due to design and coding errors, systems may experience failures, some partial and some total. A system’s construction must take into account failure and options for how to react to them:
 - The failure of an optional component must not tear down the entire system.
 - In the case of a failure of a critical component, a software system should shut down in the same orderly fashion as if an operator had requested it.

Modern operating systems manage such resources, but some software systems must run on impoverished platforms.

While a system deals with failures, it should not respond to external requests; it would be too easy for things to go wrong.

A responsible software developer must plan for every one of these aspects of software systems during all phases of the construction process.

A Concrete Example The preceding overview is quite abstract. It demands a concrete example and an illustrations of these points.

Every reader is familiar with IDEs, interactive development environments. These IDEs are software systems in the sense of the five bullets. When a typical IDE starts up, it consults its configuration file or directory, where it stores information such as language modes, recently visited files, positions within files, and much more. A high-end AI-connected IDE may also have to restore the context and read “directives” from files before it can accept input from a developer.

If the IDE opens a file for which a language mode isn’t available yet, it cannot parse the code, paint it with appropriate colors, or check its types. But, this lack of tools must not prevent the IDE from starting up. Once it has reached a stable point during start-up, it can ask the programmer for information on how to configure the relevant pieces of functionality.

A developer can edit the code, query it, refactor pieces, run tests, and so on when the IDE is up and running. During this stable phase, a bug in the language server will not tear down the IDE; instead the IDE will signal that there is a problem and ask the developer how to proceed. On the other hand, if the IDE code itself has a bug, the developer may have to “kill” the IDE process and restart it. But even in this bad scenario, the IDE won’t let down its user. It will have stored away enough backup and recovery information so that the loss will be tolerable. Finally, a developer is likely to shut down an IDE in the evening and restart it in the morning.

Now that we have a common idea of what a software system is and how it runs, we can turn to the next part, namely, the construction of systems.

Exercise 3. How could an IDE become unstable while it is looking to load a language mode (parser, type checker) while a developer is creating code in this language new to the IDE? What if the IDE replaces a language mode with a revised version? Look up the term “hot swapping.”

Exercise 4. Pick another software system that you have used in the past and analyze it in terms of the preceding abstract explanation.

8 Developing a Complete Software System

How is a software system constructed?

A modern software system consists of many interacting pieces of software. Over time the pieces have been called by different names: component, module, package, plug-in, (micro)service, and more. These names reflect our growing understanding of the nature of software systems plus the advantages and disadvantages of these organizations.

Early on, programmers thought that these components are designed, implemented, linked, and deployed for good, like any other large engineering product, say, a bridge or an airplane. Once a product was deployed, it just needed occasional maintenance action, perhaps a bug fix or a port to a some new hardware.

Now we understand the nature of software differently. People view software as the extremely malleable medium that it is. They want bugs fixed immediately; they want new features added *asap*; and they don't have time to wait for the results. All of this calls for highly responsive developer teams and instantaneous delivery. In short, software construction has become maintenance and maintenance is continuous construction. One key consequence is that developers have to look at existing code quite frequently and reconstruct the thoughts of those who constructed it.

To understand what the construction of a software system looks like, this section presents the imaginary scenario of being in charge of a brand-new project. Some call this situation a “green field” project. Although this situation is rare, it is a great way for getting a sense of how the system's purpose is determined; how its components are discovered; how these components are arranged into a whole; and how developers describe the results of this work. After that, it's onward to programming—but all of this work is software development, and the word “software” refers to the collection of the results of these efforts, not just the code that comes next.

8.1 Ways to Proceed

Let's look at two different, historically important ways of approaching the development of a software system: the “waterfall” model from the 1980s and the agile approach from the 2000s. Modified version of both have a place in industry. Neither is superior to the other, though the agile approach still dominates most industrial work.

The Waterfall Approach In the early 1970s, Barry Boehm, one of the first software engineering researchers, wanted to explain the cost of bugs. More

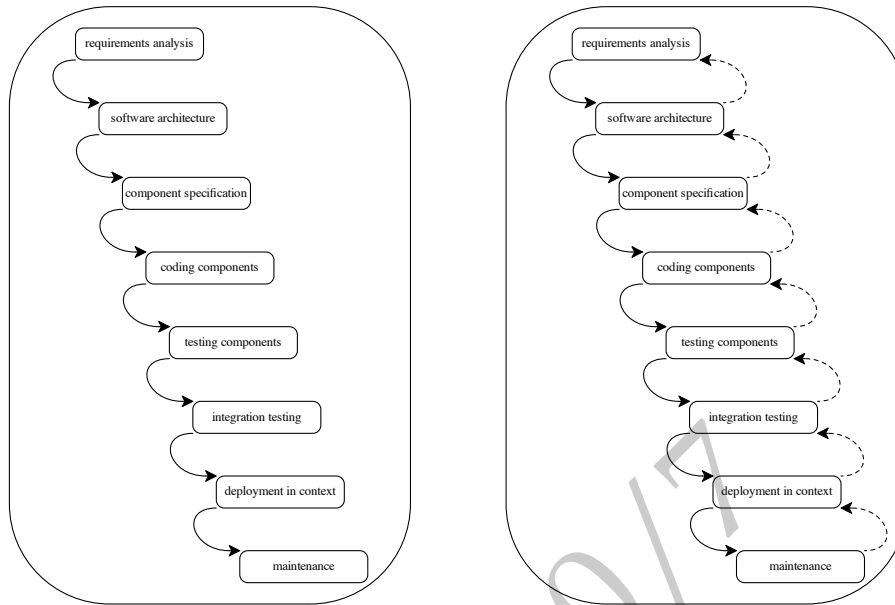


Figure 4: Software development processes: waterfall and spiral

specifically, he wanted to understand the relationship between the cost and the time of bug discovery during development. To do so, he first needed a model of development, for which he borrowed and simplified a model from ordinary engineering projects.

Figure 4 (left) shows the essence of Boehm's process idea, the *waterfall model*. The engineer starts with a thorough analysis of what the software system is to accomplish, its context, its performance characteristics, and so on. The result is a document of requirements, and it is "thrown over the wall" to specification specialists. Their task is to turn the requirements document into a system specification, often called a *software architecture*. Once again, the architects hand this document to people who can write down concrete component specifications: interfaces that spell out which functionality is needed and purpose statements for each piece of functionality. From here, the workflow goes to coding, testing, and integration testing until the system is ready to be deployed. The rest is simply called "maintenance."

Boehm used the diagram to state a simple-minded exponential law of bug cost. If a bug is discovered n phases after its introduction into the de-

velopment process, its cost is $O(10^n)$ (measured in time, money, or other resources), an amount that covers both the direct damage caused by the bug, the effort of finding the bug's source, and the labor of eliminating it. Boehm's paper implies that the model is a strawman, and the number is just a "guestimate." Nevertheless people tried to follow this waterfall process of developing software, and in principle, its sheer existence is critical because it identifies important intermediate products: requirements; architecture and its components; unit tests and integration tests; deployment documentation; etc.

In 1976, when Boehm consolidated his presentations into a book, he presented a refined variant of this waterfall model of software development: a *spiral* version. Here "spiral" means that each phase in the workflow diagram of figure 4 (right) is equipped with a backarrow to its predecessor. These backarrows indicate critical feedback, which demands a conversation between the people working on phase $n+1$ and those working on phase n . In the terminology of Boehm's book, a component specifier would conduct a thorough review of the software architecture and thus provide feedback for improvements. This conversation continues until the component specifier is satisfied with the architectural specification. In turn, a demand for revisions to the architecture may raise questions about the analysis of the purpose and context of the software system. Generally speaking, the document of phase n is no longer chiseled in stone but a negotiable product, based on software inspections. Boehm explicitly states that such negotiations may end up going all the way up the chain to the original starting point, the demand for the software system itself.

The Agile Approach By 2000, the ideas of agile development and extreme programming changed the landscape of software development even more. Coding is no longer seen as a minor step in the middle but as the beginning-and-end. Testing is tightly integrated with, and drives, the development of code. Due to internet-based delivery of software, deployment and maintenance aren't separate phases anymore but simply become another element of a tightly knit development-and-feedback cycle.

At first glance, the agile approach skips many steps: the analysis of requirements, the design of an architecture, a construction plan, and so on. But, these appearances are deceiving. They are still needed, though they may not play the same role; and they may not be performed as sequential steps in an orderly process. Without a goal statement and an analysis of this statement, developers don't know where they are going with their software. So formulating the goal, and analyzing how to get there, *is* a form

of requirements analysis. Similarly, other parts of the old processes can be identified via a close inspection, and the addition of AI to the process has placed a renewed emphasis on the corresponding documents.

More generally, a significant software system cannot exist without some architectural plans for a long time. Like a plan for a home, a software architecture is a blue print and a guide to the numerous interacting pieces. While followers of a strict process, such as Boehm's spiral approach, develop a draft architecture explicitly and up front, practitioners of the agile approach come to this understanding over the course of the creation phase. It may be implicit at first, but when the software development process is successful, someone will have to describe the architecture explicitly if the "owner" wishes to maintain the product for the long run—or suffer the consequences of a socially irresponsible development process.



*What works in a
course setting*

Readings

Barry W. Boehm. Software engineering. *IEEE Transactions on Computers*. 25(12). 1976.—On occasion, it is worth our while to read original literature, especially when follow-up characterization of this work are rather murky.

Kent Beck. *Extreme Programming Explained*. Addison-Wesley. 1999.—In this short book, Beck argues for "programming" as the central, creative activity and explains how a "programming first" approach to software development may work in practice.



The Agile Manifesto

Kent Beck, et al. *Manifesto for Agile Software Development*. 2001.—Beck and many other signatories published a dozen principles describing a flexible approach to software development. They fit on a laptop screen, and decades later it is an interesting read, if only to understand how this short piece affected industry at many levels.

8.2 Analysis: Understanding the Project's Purpose and Context

Although this book is *not* about software engineering, it is still important for a student of software development to see how an experienced developer might conduct a rudimentary analysis—because it helps explain to an onboarding developer the state of the software. A budding developer must understand how software systems start with an idea; that they always exist in a context; and that their operation is about interacting with this context. Once this much is understood, it is possible to create a rough, high-level sketch of the components needed to implement the desired functionality and how these components depend on each other. The goal is to put the

development team, including AI agents, into a position where it can begin to work out a construction plan.

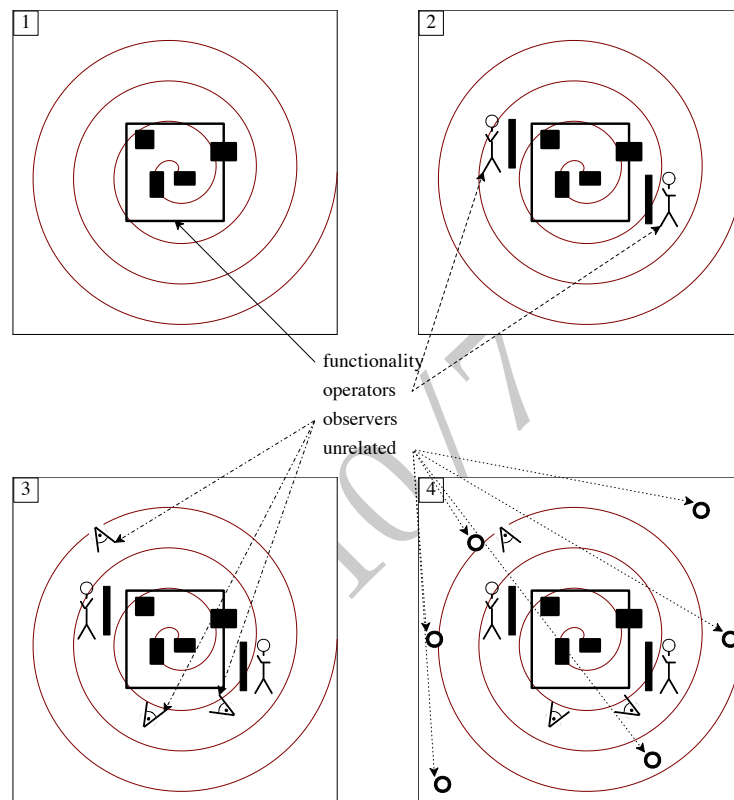


Figure 5: A spiral analysis of an imagined software system and its context

Every project starts with an idea about some novel functionality that people would like to have implemented as code. Recognizing this need, and articulating it in the form of a “system purpose” statement, is nothing that a human can delegate to an AI. The history of computing demonstrates this clearly.

In the beginning, military men needed weapons-related calculations done. Engineers aimed to help with the construction and control of airplanes and rocket. Scientists wished to model the weather. Business people wanted to replace the tedious paper-based bookkeeping with account-

ing software. Doctors asked for assistance with diagnosing illnesses. Once the world-wide web came about, most travel agents lost their jobs to dot coms. Drivers ditched their city maps and used web-based navigation systems for trips around town. People also imagined how others might like to share experiences via stories, links, pictures, videos, and so on. The addition of the cell phone brought all these things and many more to our finger tips wherever we went, whenever we requested them.

This short history points to a key insight about software: it never exists in a vacuum. Software entrepreneurs and their developers must understand and keep in mind the context in which the software lives. As a matter of fact, basic development practices alone demand that they investigate this context *before* they even start and that they document their insights.

Diagram 1 in figure 5 illustrates this insight. When someone wishes to build some software, it is common to have an idea that any non-trivial system (the square frame) will consist of a bunch of components (the black rectangles inside and at the boundary) and that it connects to some context, which actually determines how the software will be used. It is natural to proceed in a spiral-like fashion, outwards from the imagined software system, to explore this context.

As an analyst investigates the very first layer around the software, *operators* come up; see diagram 2 in figure 5. An operator could be a soldier, a construction worker, an accountant, an engineer, or just someone looking up a friend's latest dance video. It may not even be a person, but a thermostat, a camera, a light switch, or even some other software system. Whatever it is, it must be able to *stimulate* the imagined software system so that it *responds*. This response could affect a person, or it could turn on/off a mechanical device such as a cannon, the injection part of a car engine, or a phone's speaker. Indeed, it may even be a message to some other software system. Let's call this the *stimulus-response* layer.

Next an analyst is likely to find *passive observers*; see diagram 3 in figure 5. These context elements may perceive the existence of the software; they may experience the effects of the system; but they will not directly interact with the software. If the software malfunctions or disappears, some may notice, some may not; some might become unhappy about it; some may welcome it. Consider a software system for designing aircraft wings. The engineers at Boeing will interact with it. By contrast, the pilots know about it and may notice that their new airplane consumes less fuel than the previous model. Similarly, the guests of a restaurant may face TV screens that show some political conversation, and most would likely be happy if the delivery software malfunctioned.

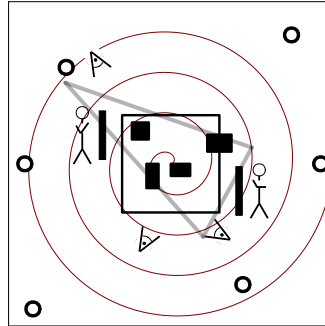


Figure 6: Drawing the boundary between a software system and its context

Finally, when the analysts have explored the context to a sufficiently large distance, they will notice that more and more unrelated concepts show up; see diagram 4 in figure 5. Think of the passengers in the airplane or the air-traffic controllers, none of whom will interact or even consider the wing-design program mentioned above. These concepts just don't matter for the product development.

At this point the analysis is concluded and yields two critical pieces: a description of the relevant context and the boundary between the software and its context. Figure 6 makes this point with the triangle that separates the core functionality and its operators-and-observers context.

Now the time has come to study the results of the analysis so that a developer understands the uses. This step focuses on the operators and how they might interact with the system, that is, the entire path from a stimulus to a response. It also includes the observers and what they may wish (or not) to experience when the software runs. A consultant may also make a plan on how to deploy the finished system and how it will affect the people in its context.

8.3 Synthesis: Discovering Components

Drawing the boundary between an imagined software system and its context is key to figuring out the system's major components and the role they play. The starting point of this step is the set of operators in the context of the system. After identifying them, the team needs to determine how these

operators stimulate the system. Concretely, the developers need to imagine how each operator stimulus triggers communications among the system's components. And once again, a developer team acting in a socially responsible manner needs to describe the responses and which pieces of the context they affects in which manner. Software engineers refer to this step as a use-case analysis.

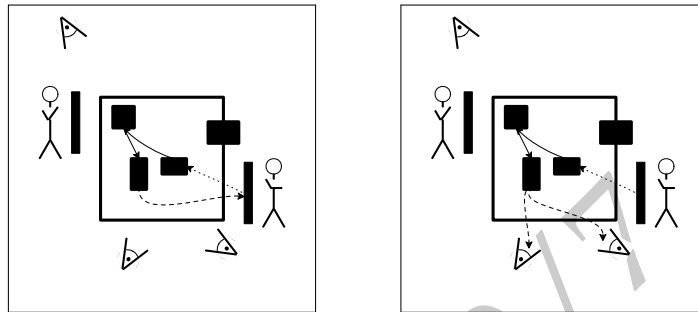


Figure 7: Understanding use cases

Figure 7 displays two diagrams that illustrate a use-case analysis. The diagram on the left shows how an operator stimulates the system. The stimulus affects one of the system's components. In turn, stimulating this component triggers an internal event with another component. One way to understand this second step is to think of a shift of the boundary. The active component has become something like an operator, and to get its computation performed, it stimulates another component. From the perspective of a stimulus-response chain, this second stimulus is the response of the first component to the operator's stimulus. This process continues until the software system is ready to issue a response to its context. In the case of the diagram on the left this response goes back to the original operator. As the diagram on the right side shows, the response does not have to go back to the operator; instead, it could be a signal to passive observers.

Keep in mind that operators and observers are not necessarily people. An operator could be a sensor, such as a thermometer, combined with an actuator, say, for an air condition unit (A/C). A thermometer sends measurements to a software component that checks the measured temperature relative to some preset value. If the measured temperature is above the present value, the software system may request cold air from the A/C. Or,

Experienced developers know that the stimulus-response idea generalizes from system to component creation.

think of a smart phone. It is essentially a large software system that may receive messages from other software systems: a ping from a cell tower, an old-fashioned text messages, notifications from social media, a signal for its clock, and many more. While a ping may demand an acknowledgment from the phone, signals to the clock do not require a response though an owner will eventually observe the change.

The purpose of a use-case analysis is to discover a system's essential components. A component typically represents information with data and implements a complete piece of functionality. In a smart home, the software system may come with a control component for configuring the various devices in the home. And, this control component is likely to connect with communication components (drivers), which translates operations from the occupants and sensors into actions. A communication component will interact with actuators in the home, say, music players that send the sound to the speakers in the room where people are or light switches that turn off all light bulbs when everybody has left a room.

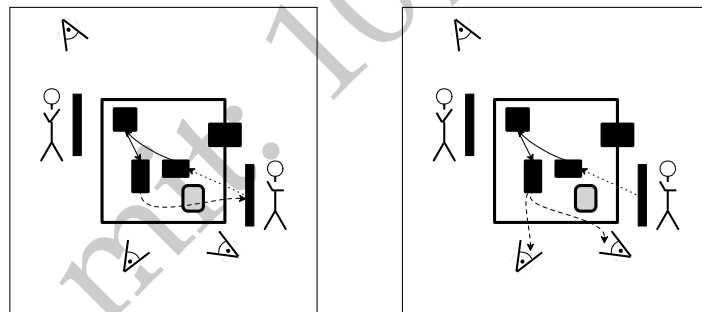


Figure 8: From use cases to component identification

Figure 8 displays diagrams similar to those in figure 7 but with an additional, gray-shaded component. The diagram on the left indicates how this component was missing from the stimulus-response path that starts at the operator icon on the right and whose response goes to this very operator. Think of a component that renders an internal data structure into a visual presentation that a person can understand; or imagine the translation of an internal piece of data into a piece of information for an actuator. The diagram on the right of figure 8 shows a use case that shares a prefix of the stimulus-response path with the one on the left. But, the newly discovered

component does not become a part of this use case. Generally speaking, every use case deserves its own careful analysis.

Moving from the abstract to the concrete, the question really is what kind of components are being discussed. Here are some examples, inspired by our understanding of what software systems are (section 7):

- *the central data structures*

Software must represent domain-relevant information with data and (possibly sophisticated) algorithms on this data. And, because of the latter, people have identified programming—and even software development—with algorithm design for a long time. While this confusion is mostly gone from industry, it is still the case that almost every software project has at its heart a data-structure component that encodes and manages complex information. This data-structure component plays a critical role during the steady phase of an execution, though it may also require special attention during the start-up phase.

Some of this confusion is still on display during job interviews, though companies are learning that algorithm design skills do not predict socially responsible and cost-effective software development skills.

- *knowledge about external actors*

In addition to representing domain-specific information, a software system must represent information about operators and observers in its context. Since this data representation is unrelated to the central data structure, separating the two into different components satisfies general software development guidelines.

- *communication with people*

All software systems communicate with operators and observers. If those are people, the communication proceeds via graphical user interfaces (GUIs). For a long time, the construction of GUIs called for three different, independent components: the model, which relies on the chosen data-representation(s); the view, which presents data to end-users and provides mechanisms for operators to stimulate the system; and the controller, which connects the two pieces. This arrangement comes with an obvious name: a Model-View-Control architecture; you can research this arrangement and find lots of information about it.

- *communication with devices and separately operating software systems*

If the interaction involves other software systems or devices, the construction depends on two levels of protocols:

- The “logical” communication between the system and external components or other software systems concerns the timing and the kind of data that needs to be exchanged. It also refers to the correctness of the exchange. This logic should exist in one component.
- Another component should be dedicated to the “physical” communication, that is, a component for receiving data from, and sending it to, other actors. It relies on existing protocols, say, the device manufacturer’s specifications or an internet protocol (UDP, TCP, HTTPS, and so on). In the second case, the developers of the software systems may even build a protocol on top of the existing ones if they have some control over the sending and receiving parties.

Keeping the two forms of communication separate is important and therefore justifies the separation into (at least) two components. Even if the manufacturers of devices update their protocols, the communication *logic* can then stay the same for the existing functionality. In other words, repairing the “physical” component should suffice. Similarly, external software systems may undergo maintenance and react to different protocols. Or, a company may decide that switching from TCP to UDP gains enough speed to justify the effort. In both cases, the changes are contained to the physical communication component, not its logic-oriented one.

- *configuration interpretation*

The start-up phase of a software system is specified in a configuration, data that may live in a file, at a URI on the internet, and/or come in via the command-line. A software system needs a component that interprets this data, where “interpretation” means initializing components, connecting them, and getting the system ready for its steady-state processing phase.

- *re-configuration*

Software systems are dynamic. When an inessential component goes wrong, it is disconnected. Some systems allow the loading of external components during execution so that new pieces of functionality become available; these pieces may not have existed when the system was launched. To monitor this dynamic reconfiguration, a software system may come with a supervisor component, whose task it

is to manage loading, connecting, disconnecting, and even unloading components.

This list of examples is by no means exhaustive, and yet, it already suggests the importance of

separation of concerns as separation of components.

By keeping key pieces of functionality separate, developers provide direct help with the future maintenance. The key to this separation is the development of interfaces, which divide externally useful pieces from internal design decisions. Additionally, it requires a strict enforcement of programming to these interfaces instead of internal details, all of which is the topic of chapter IV. If it then turns out that a central data structure is a performance bottleneck, a developer can probably change its internals without changing other parts of the system. Other kinds of changes to functionality can also be isolated to one component or two if different concerns of functionality are allocated to different components.

8.4 Planning: How the Construction Proceeds

The analysis of a software system's context develops the big picture idea. Here "big picture" means seeing the system in context—the idea, the operators, the observers, and their general relationships—as well as drilling down to discover key details, say, the exact relationship between the system and the external actors. In computer science this way of proceeding is called *top-down*.

Proceeding in a top-down fashion is still appropriate when developers perform a use-case analysis to discover the distinct components of the system. The big picture here consists of the essential data representations and the key pieces of functionality. The "drilling down" discovers such details as to the existence of one component may depend on the existence of (functionality in) others. Discovering the major dependencies is important at this stage, understanding their exact nature is not yet critical.

Understanding this high-level view of components and their dependencies is critical for the next step, namely, turning the component analysis into a development plan. In contrast to the context analysis and component discovery, the development of the code base proceeds in a *bottom-up* fashion. To discover the bottom, it is necessary to analyze the dependencies among the identified components and to find those components that stand alone. That is, they obviously do not depend on any of the other major components. Once these foundations have been identified, the components can

be arranged in layers. The first layer depends on just the components in the foundation. The layer on top of this one may depend on both, and so on. In short, a plan is really a topological sort of the component-nodes in an acyclic graph of dependencies.

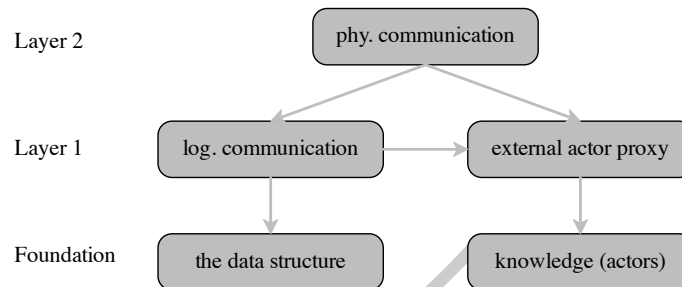


Figure 9: Layering the identified components

Figure 9 displays a suggestive layering diagram of some of the sample components mentioned in the preceding section. An arrow from *A* to *B* should be read as “component *A* depends on component *B*.” In this sample arrangement, the foundation comprises the central data structure component and the component that records knowledge about external actors. Layer 1 consists of two components: a component that represents the external actors and another one that implements how to communicate with them. The top layer implements the actual, physical communication.

Exercise 5. Organize the five components in figure 9 into a linear plan sequence so that a component listed at position $n+1$ depends only on components 1 through n .

In addition to inter-component dependencies, external constraints also play a role in the composition of a construction plan. For example, the ability to demonstrate a working prototype or a minimal valuable product might dominate other concerns. A client, an investor, or a marketing person may wish to see a working code interacting with people or external software components.

Consider the development of a software system that may have to rely on interactions with many external software systems. A demo may rely on code that merely implements the logical communication between the

core system and locally running mocks of external pieces on just a single computer. Depending on the various desired features, this stage can be broken into several smaller ones, too, as long as each of them comes with observably distinct functionalities.

A development plan is like a battle plan. Once the “battle” starts, the plan quickly becomes irrelevant because reality diverges from the plan. Similarly, once a team has built a working prototype or a minimal valuable product, the client, the investor, or the marketing manager are likely to ask for a change in the development plan, demand changes to existing features, come up with ideas for new ones. The key is to have a plan and to adjust it in response to such feedback.

8.5 With AI

Learning to analyze, synthesize, and plan is a critical element of the path from inexperienced programmer to experienced (junior) developer. Students of software development must work through the first (few) processes on their own before they can collaborate with an AI on these steps. If they don’t, they won’t acquire the vocabulary, the articulation skills, and the judgment to interact with AI agents, to inspect their work, and to assess whether it is on the right track.

An experienced student wishing to use AI has to follow the same steps as if he were working with a human partner. For the analysis of the context and the articulation of a “systems purpose” statement, the human partner should take the lead and request feedback from the AI. This feedback exists in a loop, which the human partners and AIs have to traverse many times, each iteration informed by through inspections.

When the two agree on the essential elements, especially the boundary between the system and its context, they can proceed to work on the description of the possible system stimuli. Each of these stimuli gives rise to a use-case analysis, which leads to the identification of components. As the next section explains, this discovery step is often a mix between picking an architectural blueprint and matching it to the concrete parts of the imagined system. The AI can assist the human partners with an informal but structured description of the components they discovered.

Once the key components are identified, the human partner must decide what a minimal valuable product is or what kind of prototype could be used for a demo. This choice depends on the human partners of the project team: investors, clients, marketing people, and so on. It may also depend on resource constraints such as time, money, hardware availability,

and so on. But, assuming these factors are known, the human can direct an AI to create a production plan using the analysis and synthesis documents. Just like all other AI-generated documents, this plan needs to be inspected and revised in response to inspections.

mit: 10/7

9 Software Architecture

When developers identify the components of a software system, they often already have an idea how to arrange these components into a coherent whole. Conversely, existing ways of arranging components tend to determine how developers identify components and their relationships. Only on some rare occasions, do developers come up with new kinds of arrangements or variations of existing ones. Before we continue, we must therefore take a look at these preconceptions.

Researchers refer to these established and widely used arrangement of components as *software architectures*. By now, this phrase describes an entire field of study. Practicing software architects view the use of (variants of) existing architectures as a way to communicate the overall organization of a system so that it is easily understood in the future. In other words, choosing an architecture and documenting it is yet another small act of socially responsible development.

Any developer team that grows to a certain size has a dedicated architect, and many developers consider this aspect of software development the most interesting part of their work. With AI agents taking over the routine part of coding, the importance of software architects and their work is only going to grow. Becoming a software architect requires experience and reflection. The former starts with knowing what basic coding means and how to recognize the architectures of a system; the latter means having an eye for proper relationships among components.

Developers who turn themselves into solid architects will survive in the emergent world of agentic AI and may even thrive. While recently developed AI models can clearly take on some of the mundane coding tasks, it is also becoming clear that they will need guidance from a human being. The latter will remain in charge of articulating the “system ideas,” performing the analysis, picking an architecture, and creating an executable plan. The AI agents can then fill in the details, just like junior software developers on a team.

The following three subsections present three architectures that every developer should know. There are many more, and descriptions can be found in the research literature. The notion is also somewhat related to the field of object-oriented design patterns, though as the name says, the latter is merely focused on a specific class of programming languages and how to circumnavigate some of their deficiencies, whereas architectures are language-agnostic concepts for creating systems and communicating basic ideas about them.

Though be careful, this very literature can lead people to turn into “architecture astronauts.”

9.1 Pipelines

The Unix operating system introduced the *pipe* as a mechanism for composing programs that read their arguments from a standard input device and write their results to a standard output device. If you are mathematically inclined, pipes are just a form of function composition, a concept that students first encounter in pre-algebra courses in high school.

Operationally, the pipe mechanism enables a programmer to turn the standard output device of one program into the input device for a second program. This way the second program can process the output of the first and compute its results from those. A *pipeline* is a composition of several programs with pipes between them. While the idea sounds simple, it has consistently shown surprising value and has been ported to almost all operating systems.

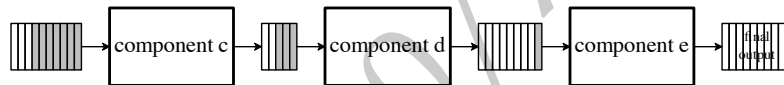


Figure 10: The pipeline architecture

Figure 10 shows a diagram that illustrates the idea geometrically. Each component consumes an input *stream*, which contains a number of elements. It also produces an output stream, again a sequence of elements. The shaded elements of the input streams are those that the component has consumed. As the diagram implicitly points out, different components process their input streams at different speeds.

Depending on the relative speeds, one component may have to wait for its predecessor. The pipes-as-streams mechanism smoothly accommodates such differences, because it clearly distinguishes the end of the stream from the situation when the next element is not ready for processing yet.

Consider the following example, where \$ indicates the system prompt:

```
$ ls -l | grep rkt | wc -l
15
```

The `ls -l` command consists a program name, `ls`, and a command-line argument, `-l`. It displays the files in the current directory to the standard output device, one per line. This output device becomes the input for the following command. The `grep` program searches its input stream for the

word given on the commandline (`rkt`) and displays only those lines that contain it. Finally, `wc -l` counts the lines of its input stream. In short, the pipeline counts the files in some directory whose name contains the word `rkt`. The result of the entire pipeline is 15.

Here is a second example:

```
$ cat I.scrbl | tr -s '[:space:]' | sort | uniq | head -4
A
ASCII
Absolutely
Addison
```

This pipeline uses `cat` to display `I.scrbl`, the source of part I of this book, line by line; the second command breaks it into words; the third one sorts those; the fifth eliminates duplicates; and the last one finally displays the first four words.

Note: AI agents In many cases, AI agents are pipeline arrangements of models, memories, tasks, and so on. The key difference is that they are often hooked together in a looping arrangement.

9.2 Model, View, Controller

The introduction of graphical user interfaces in the 1970s pointed out the essential difference between the data that a program manages, the way the data is presented to customers, and how the latter component interacts with this presentation component. To realize this separation, a developer at Xerox Parc proposed a three-part architecture: model-view-controller.

Using standard science terminology, the *model* is the essence, the data set that the software system manages. A *view*—notice the indeterminate article—is a way of displaying the data; one data set may have many views, with none being necessarily superior to any other. If a user wishes to interact with a particular view, the software system needs a component that translates the user's gestures into updates to the model. Since such changes to the model should affect the view, the model component must have some way of informing its views of such changes, as needed.

Figure 11 presents the idea as a diagram. A model-view-controller architecture consists of one model component and $2 * n$ controllers and views. The latter come in pairs; each view may need its own way of allowing users to interact with the model. The end-user understands the data through a view presentation, and his gestures are conveyed to the model via a controller for the user's view. In turn, the model component informs the view of changes to the data display.

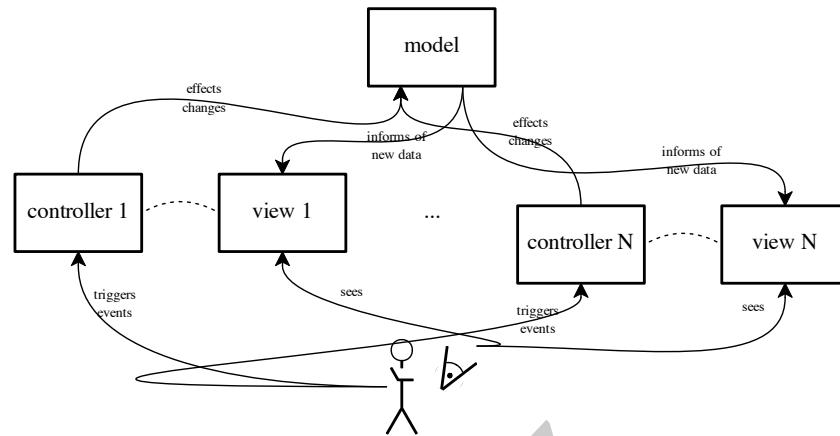


Figure 11: The model-view-controller architecture

The ubiquitous spreadsheet is an early and well-known system that employs the model-view-controller architecture. An end-user enters data and can then request different views of the same data with various gestures: tables, bar charts, pie charts, etc.

Similarly, many modern IDEs make use of the model-view-controller architecture. An IDE may display the full program text or an outline view; it may show a hierarchical project view; and it may allow programmers to interact with the code via a console.

9.3 Client, Server

Xerox Parc’s networked and distributed file system led to the client-server architecture. It classifies some computers as providers of services and others as clients of these services. The architecture has found many uses since.

A classic client-server arrangement consists of one server program running on a computer and an arbitrary number of client programs, usually but not always running on different computers. Each client interacts only with the server. It sends a request message to the server, asking for some data or for some work to be performed on its behalf. The server sends back a response or just an acknowledgment that the work has been performed.

Figure 12 sketches a client-server architecture with six different clients, each of which is interacting with the server. Here the server sits in a “cen-

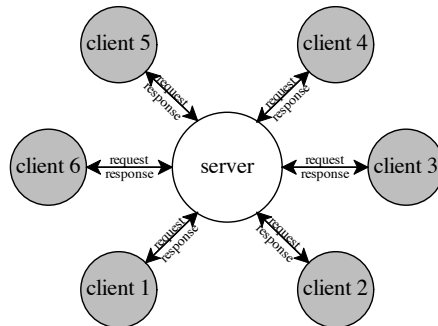


Figure 12: The client-server architecture

tral” location, indicating that it should be easily reachable for all expected clients. The double arrows between the clients and the server explain how the components communicate; the absence of any other arrows and lines implies that those are the only forms of communication that take place.

Every computer user is implicitly familiar with the client-server architecture. The web browser on laptops sends requests to a web server. Simple requests ask for the content of a file, usually with some HTML content, which the browser then renders on the user’s monitor. The response may contain JavaScript programs, which turn the browser window into an interactive graphical user interface. Complex requests demand that the server runs a script that composes the response.

While the classic client-server architecture typically replies with one response per request, alternative variants exist. For example, a request from a client to a server may initiate an entire conversation, consisting of many back-and-forth messages. Indeed, the server may engage with other clients at the same time, concurrently or sequentially, and the server’s responses may even depend on how other clients react.

Reading

Mary Shaw and David Garlan. *Software Architecture*. Prentice Hall. 1996. —The book covers the key issues of the software architecture field: basic arrangements, style guidelines, linguistics, formal methods, and tools.



Planning a Project

10 A Sample Project: Analysis, Discovery, Planning

Imagine a startup that wants to engage hackers. The founders noticed that hackers are mostly bored on their jobs and complain about the bad habits of their teammates. So, the founders want to run servers that host tournaments of board games, and hackers from all over the world will produce automated players that connect to the servers. These hackers will pay a small fee to have their automated players participate in tournaments, and the winning player (and its owner) will win a significant prize. The founders are also considering broadcasting the games to subscribers, and like in all unicorn dreams, these broadcasts could become channels for delivering advertisements, though this really is just a dream for now.

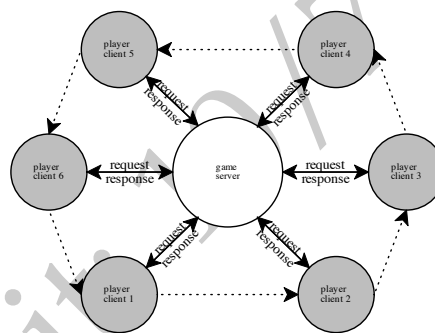


Figure 13: The player-client/game-server architecture

Starting with a description of the idea, a software developer no longer sees a blank slate but the idea of the client-server architecture in figure 13. The clients are the automated players; the server is a game server. Player-clients request to participate in the game, and the game-server arranges through its responses for the execution of a game. The architecture differs from the classic one, because they conduct a complete game and games tend to require many turns. Specifically, they differ in that (1) the server and the client exchange a series of messages and (2) depending on the nature of the game, the server arranges these conversations in a particular order, as indicated by the dashed arrows in the figure.

Let's make all this a bit more concrete via a game that this company may pick to explore the market:

Ticket to Ride is a board game about the railroad barons in the 1800s and how they built train networks. It accommodates four to six players. The central game piece is a map of cities and direct train connections between them. Each connection has a distinct color: white, red, blue, green. A connection consists of 3, 4, or 5 segments.

At the beginning of the game, each player receives some colored cards, which it keeps hidden from the others, and some rails. In addition, a player must choose some destinations (from a small set). A destination is a pair of cities on the game map that must be connected via some path of connections.

Once all players are set up, the game begins. At each turn a player may choose one of three actions: (1) request additional cards; (2) acquire and occupy a connection; or (3) pass. To acquire a connection of color c with n segments, a player must hand over n cards of color c . To occupy it, the player must place a rail on each segment of the connection. Once a player has acquired and occupied a connection, no other players can acquire or occupy it.

When one of the player's number of rails drops below the smallest number of connection segments at the end of its turn, every other player gets to take one more turn. The game also ends if every player has had an opportunity to play a turn and all of them passed.

The player with the most points wins. Each acquired rail segment is worth one point. Each player-chosen city destination is worth 20 points if a player manages to connect the specified places via a path; if a player fails to connect the cities of a chosen destination, 20 points are subtracted from the player's total. Lastly, the player with the longest continuous path of rail connections receives 20 extra points.

The game is simple; the details are manageable; and it also seems to mix some strategic thinking with luck. The imagined bored hackers may just try to win again and again and pay a fee each time.

Note: Focus To simplify the analysis, discovery, and planning, let's ignore the financial side of the software system. This side of the business is mostly orthogonal to its core functionality: running tournaments.

Note: AI You might now think that we should discuss what kind of AI hackers should us. Although this idea is a natural one, it is unrelated to our considerations of AI and the role it plays with respect to the objective of this book: as an assistant to software developers who wish to act in a socially responsible manner.

10.1 Core Idea

The starting point should be clear. When friends play a board game, they tend to run it jointly: managing common game pieces; agreeing to, and interpreting, the rules; and deciding whether someone cheated. They rarely put a neutral person in charge. None of these social conventions apply to our imagined setting, however. If hackers are to participate, the company's software has to be the neutral and trustworthy party.

Following tradition, let's use *referee* for the component that manages a game. It will have to rely on a data representation of the game pieces: the *map*, the *colored cards*, the *rails*, etc. Furthermore, it needs to *represent* what it *knows* about the *participating players* so that it can enforce the rules.

Beyond individual games, the company's software is also in charge of running tournaments. Since this task is completely separate from refereeing a game, a separate *tournament manager* component is called for.

While the identification of these two components (referee, tournament manager) and some others (game pieces, knowledge about players) is a straightforward step, let's see what we can discover via a context analysis.

10.2 Context Analysis

The operators are hackers who run *automated players*. Where these operators live seems to not matter, other than that the company's interaction with the players must allow for long distances. After all, if the company runs its servers in Seattle and the automated player executes on a computer in Capetown, communication between these two is going to take longer than a method call from the referee to a "house player" in the same language.

Concerning observers, the company must certainly accommodate its investors. Most *investors* do not care about the code—although they probably should—but a company can impress them with visual effects. Additionally, the human *operators* of the server software may wish to have some visible means of checking on the progress of individual games or entire tournaments. Finally, the "business plan" calls for *subscribers* to be able to watch games, possibly for a fee or in exchange of watching advertisements.

Outside this circle of operators and observers, an analyst may see sports bars, the parents of a hacker, server-machine farms, and many more things. In reality, though, the concepts identified so far describe the software system's components and its context as much as needed for now. Put differently, the description clearly tells us where to place the boundary between the system and unrelated pieces of contexts. We can thus proceed with the use case analysis to identify the core components of the software system.

10.3 Component Discovery via Use Cases

The context analysis yields a description of the system's environment with operators and observers. Imagining these external actors, an analyst plays through key use cases and looks for the components of the clients and server in the imagined architecture of figure 13.

For *Ticket to Ride*, the analysis of the basic idea has unearthed six core components: a tournament manager, a referee plus data representations for the map, the cards, the rails, and the destinations. There are two operators: hackers and automated players. At least two kinds of observers are essential: the people who monitor the running system and those who invest in the company before it goes unicorn.

Let's look at how these external actors interact with the game system, keeping in mind that, like all software systems, the game server starts up, runs tournaments and games, communicates with players, informs observers, deals with failing components, and eventually shuts down.

Launch The hackers want to launch their automated players and connect them to the company's server software. Just stating this much of a purpose raises two questions. The first one is to which component they connect their players. The tournament manager is the only identified component that could even remotely play this role. But then the tournament manager would take on *two distinct* tasks: managing a tournament of hundreds or thousands or more automated players *and* signing them up. If the idea takes off, the two tasks might have to be performed concurrently: while a tournament is going on, the next wave of automated players is signing up. What this statement exposes, is that the use-case analysis points to a gap in the component identification. The game system needs one more component, one that signs up automated players and launches tournament managers.

The second question is how the automated player contacts the sign-up component. Since the automated players and the game system will run on

separate computers, choosing a physical, or remote, communication protocol is necessary. The choice will determine how a player connects and how a game server interacts with the players.

A derivative question concerns the timing of launches. The company may announce “the sign-up phase starts at 9am Eastern,” but its operators and the hackers cannot possibly synchronize their launches exactly. So, when the company operators start the system, they can’t know whether the automated players are already up and running. The converse is also open: the hackers can’t know whether the server is ready for sign ups when they launch their players. This timing problem is one small symptom of the start-up instability mentioned at the beginning of this chapter.

Running a Tournament Once enough players are signed up for a tournament, the sign-up component can hand them over to a tournament manager. In turn, this manager informs the players to get ready for their first game. When the tournament ends, the manager component should let the players know. At a minimum, the winner(s) of the tournament should find out that they won a lot of money.

What this use case immediately points out is the need for a *logical* communication protocol between server and player. This protocol specifies how the game server and its components contact the players, which information is revealed and as what kind of data it flows from the tournament manager to the player, and what comes back.

Recall that this concern is separate from the choice of a physical communication protocol. “Separate” implies that these two concerns are distinct, and as such, their implementation ought to exist in distinct components.

Running a Game Just like in a real-world tournament, the manager will put instances of the referee component in charge of running individual games. When handed the players for a game, this referee should inform the players about the start of the game and its end, at least the winners of the game. Naturally, in a tournament setting, the referee is going to inform the tournament manager of the outcome of the game.

During the game, the referee should grant players turns. In the context of *Ticket* the players get the right to take a turn in a round-robin fashion, typical for most board games.

On reflection, this use case suggests a number of extensions of the logical communication protocol. Before you read on, make sure to note what kind of interactions will take place among the pieces of the overall system, which comprises both the server and the remote (AI) players.

Communication Concerns The preceding two use cases allude to the ways the company's software and the players interact: the logic of interaction and the mechanism remote interactions. Both forms of interaction need rigorous testing before the company runs its first tournament. Testing the logical interactions differs from testing remote interactions.

- In the case of the *logical* protocol, it suffices to implement player components in the same programming language and to have them interact according to the desired protocol. Any errors in the specification can thus be exposed without going through any remote communication protocol. The key is that the company itself must implement players.
- In the case of the *remote* protocol, the server and the player must exist in distinct processes, possibly on the same machine or on a local network. As the team separates the server from the player(s), it must create a bridge between the two pieces. This bridge is of course yet another software component that must eventually be implemented.

Observing Any observer will want to watch how games progress and how the tournament progresses, two rather different forms of observation.

Watching a tournament is about observing the progress of individual players. Imagine a knock-out tournament in which only the winner(s) of a game may move to the next round. An observer could bet on the tournament rounds, just like people bet on the NCAA March madness tournament or the European Champions League play-offs.

Watching a game publicly calls for visualizing the map and the acquisitions that players make during their turns. Looking over the shoulders' of a player should be like watching a game but might perhaps reveal the player's private knowledge. These ideas point to yet another concern: privacy. To keep things simple, we ignore the privacy concern as far as observers are concerned.

Notice how these observer use cases call for different observer components and a communication protocol that sends information from the manager and the referees to the respective observers. The observer components differ from each other, because the visualization and the intended audience differ. Just as with operator interactions, the server-observer communication protocol calls for two layers: a logical layer and a remote layer, if the company wishes to enable observers at distant locations.

-
- | | |
|----------------------|--|
| • game server | • game pieces
(cards, destinations,
connection map, rails) |
| • tournament manager | |
| • player | • game observer |
| • player interface: | • observer interface: |
| – logical | – logical |
| – remote | – remote |
| • referee | • failure discovery |
-

Figure 14: The discovered components

Figure 14 presents the identified components. Note that the entry for game pieces summarizes the need for four components, each representing game pieces. Also, keep in mind that both the player and the observer interact with the server-side system on a logical level and a physical one.

Failure Before we can proceed to the planning stage, though, we need to consider the failure possibilities. Following section 7, a use case analysis must consider two kinds of failures: critical vs non-critical.

From the overall system perspective, software failures inside of the sign-up component, the tournament manager, or the referee are obviously critical. In this case, the game system should shut down as gracefully as possible. Here “gracefully” includes closing connections to the (remote) players and leaving behind sufficient information about the current state. Fairness calls for the former, while the latter is needed for debugging.

From the perspective of an individual hacker, a software failure in the player is critical. Again, if a hacker wishes to repair such players, a graceful shutdown is needed.

Concerning failures in a system such as the one proposed here, we should also consider where failures may occur. While they can obviously show up *inside* of components, a system that consists of coordinated and communicating pieces may also suffer from communication failures:

- A remote interaction can fail for a number of reasons. For example, a political regime may decided to disconnect a nation’s network from the global internet. More relevant here, when a player component

crashes, the symptom is a lack of a response to a request from the server. Alternatively, a player may misuse the physical communication layer. More generally, remote players may fail to communicate in three different ways: a lack of a timely response, an ill-formed response, or a well-formed response that is invalid with respect to the context.

- A logical interaction is going to fail for different reasons. The response from a player may be ill-formed or invalid. In the first case, the player's response is an entirely inappropriate piece of data. In the second case, the player's response is a valid piece of data, but the game action it represents is illegal. In human terms, the second case corresponds to a cheating attempt.

In any and all of these cases, the server-side components must discover the communication problem and react. To keep things simple, the best reaction is to disconnect from the player, because it is no longer trustworthy and communicating with it may corrupt the overall system.

Likewise, failures in observers should be considered non-critical from the overall system perspective. While observers typically do not respond—meaning the communication is basically one-way—it is still possible for observers to misbehave and affect the performance of the system. Whenever the server-side components notice any bad behaviors in observers, they should disconnect too.

Three server-side components communicate with automated players: the sign-up component, the tournament manager, and the referee. Since it is mostly the manager and the referee that communicate with the automated players and observes, it is also up to those two to discover failures and to react. The reaction may differ from component to component, from protocol to protocol. Given that failing player components—external or internal—are potentially dangerous to the functioning of the system, one policy could be to ignore them. By contrast, a failure in a non-critical component such as an observer can be left to a human operator. If needed, an operator could just re-launch an observer component in some sports bar after it has crashed and the game server could accept the registration of this new remote observer.

Depending on the context, the discovery of failures may call for a component in its own right or an enrichment of the communication components. Planning the construction and programming them typically points in the right direction.

10.4 Construction Plan

The process of making a construction plan starts from a component listing such as the one in figure 14. Next the team works out how these components might depend on each other. For this part, it is best to draw a graph of dependencies. This graph suggests layers of the software system and implicitly a preliminary construction plan. Turning these layers into an actual plan must take into account external constraints, including intuitions about interactions with potential investors and customers. Note how the first part suggests an AI agent can perform the planning for you, except that it still needs your judgment about the external constraints. If you have an AI do the work, don't forget to have it write down the plan and to iterate until it satisfies your constraints.

Even a cursory understanding of the component list tells us that the game-piece components do not depend on anything else in this list, while the game server depends on almost every other component, directly or indirectly. Similarly, the tournament manager must rely on the referee component to run each of the games in a tournament, meaning the former depends on the latter. Working through the whole list in this manner yields the graph of figure 15.

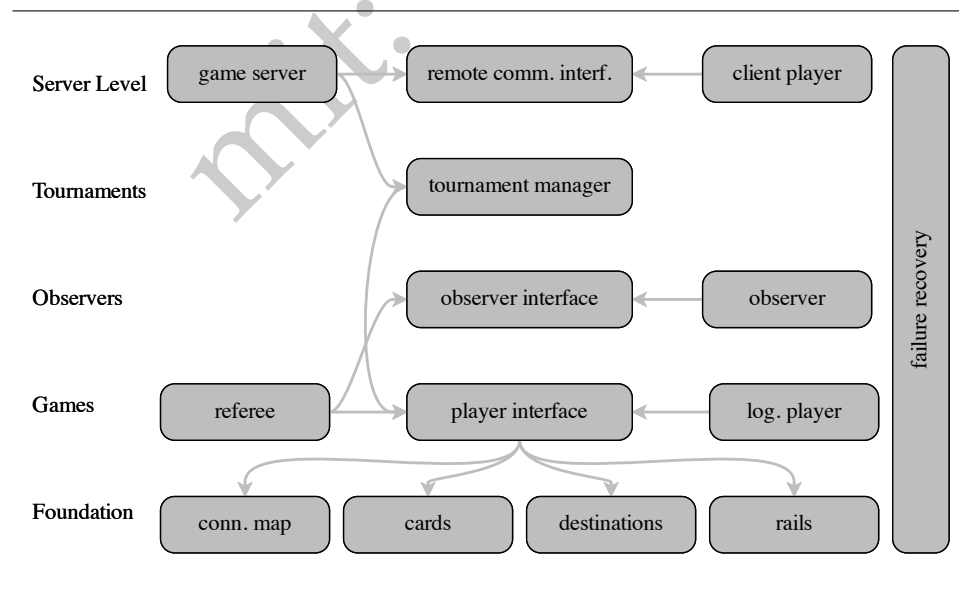


Figure 15: Planning the game server; its dependencies

Figure 15 shows how this dependency graph naturally yields five layers. Each layer's components depend on components in layers below but not on components in layers above the current one. An arrow points from component A to component B means A can be developed and compiled only if B exists—or in terms of modules, if A imports B. Note that even though none of the components directly depend on the “failure recovery” component, its vertical positioning implies that it might be omnipresent in many of them; as indicated already, its actual nature and dependencies are likely to be discovered during the development of the code.

Simply put, a construction plan arranges this graph in a linear order; technically, any topologically sorted arrangement might do. If a component A depends on B, the latter appears first in the list. Following this plan ensures that each component is properly tested before a team moves on to components that depend on it. If a bug appears during the construction of a dependent component, the likelihood is high that it is in the component under construction, not those that already exist and are tested. Additionally, by building the system in a bottom-up manner, the developers can formulate and run preliminary integration tests at each level. Integration tests exercise the flow of data along use cases from the highest layer in the graph to the bottom. Unsurprisingly, such integration tests tend to reveal new kinds of bugs in the various layers, because they reach pieces of the code that plain unit tests overlook.

One key improvement over such a simple plan is to take into account external constraints. Examples of important constraints are delivery speed or demo sessions with investors, potential customers, focus groups, and so on. To pick an example for this particular case, an investor might wish to watch an individual game on a single computer to make sure the team is making progress. The next logical step is to run a demo for an entire tournament, again on a single computer. Finally, the investors will wish to see how tournaments are run on a bunch of computers, perhaps all in a single room. Eventually the company must also ascertain that a country-wide or world-wide distribution satisfies basic performance criteria.

In order to reflect these external constraints, the construction plan for *Ticket to Ride* specifies three major phases. Each phase yields a prototype that illustrates progress on the basic ideas: local games, local tournaments, and remote tournaments. Once these goals are accomplished, the product will need performance tuning—something beyond the scope of a course on software development. Note how the first two phases are all about the logic of games and tournaments, while the last one deals with the concerns of the remote-communication layer.

Phase 1: Game Logic The goal of the first phase is to build a complete, observable game implementation:

- the basic game pieces

Of those, the game map is clearly the most complex piece that the referee depends on. All player implementations will have to use a map component, too. The running example demands the representation of a bidirectional graph and certain traversal operations on this graph. Working on the remaining components, especially the player and the referee, is likely to reveal specifics and call for changes.

In all likelihood, a developer will have to revisit the game-map component several times during the development of the rest of the system. Expressed in terms of our social-responsibility terminology, an older version of the developer is going to read the code of a younger self. A “design-it-well” approach is going to pay off for just one developer alone.

Let’s go further still. Imagine the departure of the original game-map developer during this first phase. What will it take for a different developer to modify the implementation when it becomes clear that, for example, a player implementation calls for many more operations on the map than those created for the referee?

- the logical player interface

The player interface is one of two central pieces for the company’s goal. It describes how the referee *logically* interacts with the automated player, meaning it is an interface description. As such it consists of descriptions of data that the referee delivers to players and vice versa. Since the logical interface exists in the same programming language as the referee and player components, it is acceptable to express this information in terms of method or function signatures for classes or modules, respectively.

Before this interface becomes truly useful, though, it might be best to translate it into plain English. After all, the hacker customers are likely expecting such a document, even if they might be happy to see it expressed as code, too.

Inspecting the player interface in detail is critical. The clients of the company must accept this specification as the logical interface of automated player, and this means the clients must understand what this data means in terms of information about the game. If there is not

enough information in the description, the clients will fail to complete games, which will obstruct the company's goals. Clearly, all of this says that a thorough inspection of this design—its formal variant as well as the natural-language document—is called for, and the following sections address this issue in depth.

- the logical player and the referee

The referee and player components are independent of, and symmetric to, the player-interface component. In principle, different developers or even developer teams could implement them. The player-interface component dictates how the referee component connects with the player and how the player responds.

- the observer interface and an observer

During phase 1, the observer and its interface exist so that the team can demo the software system to people who do not appreciate a code base. Like the player-interface component, the observer-interface dictates how the referee component connects to observers and what the data denotes in terms of the on-going game. It is up to the implementer of the observer component to render this information in a meaningful and quickly comprehensible manner.

In the long term, the observer interface may play the same kind of role as the player interface. The company may allow outsiders to implement observers, perhaps for a fee.

Keep in mind that all of these components are constructed within the same programming language. Sticking to one language enforces a logic-of-code perspective, ensures that the logic is correct before the challenging remote communication is implemented, and thus facilitates the construction of the complete system.

Phase 2: Tournament Logic The goal of the second phase is to construct the tournament management system. Since this construction also involves some amount of sophisticated logic, the development continues under the assumption that all code is in one and the same language. In contrast to the first phase, this one seems to involve just a single component:

- The tournament manager implements the functionality that allocates players to games, runs games via referees, informs some players that they will participate in the next round, closes connections to losers, and more.

This description suggests that the team or the company must settle on the form of tournament they wish to organize. In the real world, tournaments often use one of two modes: round robin, pitching every player against every other player in some form, or a play-off format, filtering out winners in one round and moving them to the next one. Some tournaments, like FIFA's World Cup, mix the two, starting with a round-robin format and switching to play-off style later.

In some way, this refinement implies that the analysis of this system layer is superficial. Perhaps the tournament manager should employ auxiliary components for allocating players and referees to games and for managing the games from the beginning to the end.

Beyond these considerations, thinking about the tasks of the manager also clarifies that it will have to communicate with the players directly. It may, for example, inform the participating players that a tournament has started, that they will participate in a particular game, or that they won the big prize. In short, the development of the tournament manager may require changes to the existing player interface description. As always, a well-designed interface is easier to change than one that is ad hoc.

As at the end of phase 2, the company might wish to demonstrate the product again. While the tournament still runs on a single computer, an investor could observe it—if the team had thought of a tournament-observer interface and a component that renders tournament-relevant information. Note how thinking about an external constraints at this stage reveals another gap in the analysis and the construction plan. See the next section for how to inspect analyses and plans.

Phase 3: The Game Server The goal of the third phase is to break up the monolithic prototype into two pieces: a game server and game-playing clients. Figure 16 contrasts the two view. While the left side shows the entire logical system running on a single computer the right side puts the clients outside of the game-server's machine box. In both diagrams, the bold-boundary box represents the server machine(s): on the left, this machine also runs the players; on the right, it host nothing but the server.

Now consider that the goal of this last phase is to cut the tie between the logical player and the game-server component and to replace it with remote-communication capabilities. To avoid the injection of bugs, the team will proceed systematically. At this point, experienced developers may reach for a handbook on design patterns that solve specific problems,

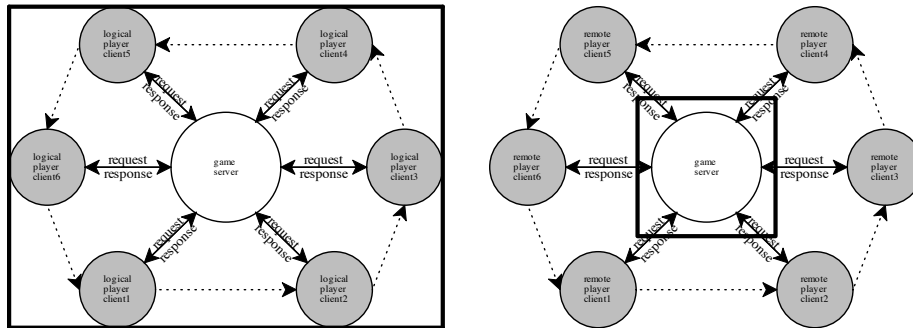


Figure 16: The logical and remote client-server architectures

or they may just recall relevant patterns. Once they have chosen a suitable design pattern, they can probably ask an AI agent to modify the existing code base accordingly.

In the concrete case of turning a monolithic system into a distributed one, the developers will reach for the *remote proxy pattern*. In essence, the pattern explains how to “splice” remote communication components between two logical components that already exist. By *adding* code to the existing base and not touching any of the existing one, the developers know that bugs are likely to exist in the new code not in the old one. The following lists sketches the key steps:

- the remote communication interface

The team must equip the existing logical player interface with a description of the remote-communication interface. While the player interface dictates the logic, the remote-communication interface fixes two different aspects:

- the choice of data-exchange language, say JSON or XML;
- the format of the data that is going to be exchanged, which should be a straightforward serialization of the arguments to player-interface method calls and returns.

Naturally, it also specifies the transport protocol (TCP, HTTPS, etc).

- the game server and the client player

Like the referee and the player, these two pieces are independent and symmetric. Once the remote-communication interface describes how the function and method calls of the player interface become long-distance data exchanges, different developers should be able to create those components separately.

The purpose of the server is to sign up clients. It turns every single client into a player-like component, often called a proxy-player. The latter live up to the same interface specifications as logical players, but turn method calls into the transmission of data to their corresponding clients and turn their response data into return values. When the server has signed up enough clients, it can hand over the proxy-players to the tournament manager, because running tournaments is distinct from signing up players.

The purpose of a client is to connect the hacker's player to the server and to isolate the logical player from the details of the remote communications. By symmetry, people often refer to the component that realizes this isolation as a proxy-server. A proxy-server turns transmissions from the server into method calls, and it translates return values from the encapsulated player into remote messages. When the connection is established, the client code hands control to the proxy-server player.

Note how these descriptions imply that the remote-communication interface must specify how the sign-up step functions.

This last phase once again explains why it is so important to develop and test the logic well. The developers who add the remote-communication layer do not wish to have to "fight" bugs in the layers below. When bugs show up now, they are mostly likely in the communication components, which narrows the debugging space a lot.

At this point, the team has a plan, and can ask

whether it is possible to find problems at this stage?

This question and related ones are the topic of the next section, which brings us back to the socially responsible aspects of planning.

11 Inspecting the Plan and its Inputs

Between each of the three steps—context analysis, use-case analysis, and construction plan—developers should take a close look at the results. Although there is no code yet, it is still good practice to have a second pair of eyes—and even a few more—inspect how the developers’ thinking about the project is expressed. And just because there is no code, we can still speak of “software inspection” because the results of these three steps *are* a part of the software product.

Let’s take a look at what could happen when a developer presents the diagram of figure 15 and the construction plan to the team. The presenter would probably give a brief summary of the context analysis and then move to the diagram as the result of the component discovery. Most of the time would be spent on the construction plan, simply because the team is about to embark on the code development phase of the project.

The presenter would explain the plan as is, proceeding bottom up in terms of the dependency diagram. While the first layer of game pieces looks uncontroversial, inspecting the second layer—the player interface and the two interacting components (referee, player)—might trigger something like the following dialog:

This brings us to the second layer from the bottom in our diagram (figure 15), the first interesting one.	The one that contains the referee, the player interface, and the player?
Yes, the interface describes how the referee interacts with a player.	Is it correct to assume that the referee instructs the player to get ready for the game?
We imagine that the interface will specify a method signature that corresponds to setting up the game.	Does this mean the interface will also specify how the referee grants the player a turn?
Correct, the referee tells a player that it is its turn and expects data that describes what the player wants to do.	Here is what I understand now. The “referee” is the component that manages a game and enforces the rules.

Absolutely.	Doesn't this co-mingle two pieces of functionality in one component?
What do you mean?	The referee seems to be in charge of both managing the mechanics of running a game and making sure the players play by the rules.
Oh.	These could be two major, complex pieces of functionality.
Now that you say so I think this is right. Even if the rules are easy to check for this particular game, in the future we may deal with games that come with truly complex rules.	Let's note this as a possible gap in the list of identified components and the construction plan.
Agreed. Ready to move on?	...

Let's take a break here and reflect on the style and content of this dialog.

The style of the dialog illustrates egoless reviewing. Find all references to the artifacts (plan, components) and re-read the passage. None of them use "how do you check the rules" (or similar language) to discuss what the intended software is supposed to do, what attributes it should possess. Otherwise the participants would identify the creator and the creation, which injects unnecessary personal attributes into the conversation. Inspections of code or designs are *not* about their creators, only their products. Language reinforces this idea.

The content of this dialog is about a first planning issue. While it is clear that the presenters and the reviewers have discovered a potential problem, think about what kind of problem it is. For code, we think of design flaws and bugs. For pre-code designs, it is better to think in terms of "looking ahead" vs. "needed now" or about a component's size and functionality

versus functionality that is not at all allocated to a specific component. The discovery of a possibly overly complex component—one that performs too many tasks or represents too many kinds of information—is probably a design flaw. While addressing such an issue at this stage might be premature, the discovery of a missing piece of functionality allocation is definitely something that the team must incorporate into its work schedule as soon as possible.

The next part of the dialog is going to discover such a gap:

...	Can we get a better sense of the player interface?
As I said, it specifies the method signatures that a player must implement.	One of the methods should be about taking a turn.
It is.	A player that takes a turn must know something about what other players have done so far. How does this work?
The referee sends the data a player is allowed to know about the action of others, when it's the player's turn.	What is this data called? Where is the corresponding component?

Note how the creator of the construction plan has something specific in mind already plus a supportive rationale. Still, this design choice deserves an additional discussion at some point; see section 15.5. For now, let's resume this inspection.

We could call it <i>public game state</i> .	If it is the information that all players can know, it is the perfect name.
Yes, it is precisely the information that describes the visible game pieces and their relationships.	Isn't there invisible information that the referee needs to know to make sense of the players' response?

Correct. The referee must also have information about each individual player that the others can't know.	This sounds like a need for a <i>private game state component</i> , too.
Indeed!	Let's take a note that the game states are components for recording the referee's knowledge about the current state of the game. Some is public and some is hidden.
We do need that component. But the referee also need to hand players the public information.	The game-state component can come with a method that extracts what a player should know for a turn.
I am glad we discussed this point. We will record this relationship in the planning document.	...

Once again, we must discuss both the style and the content of this inspection. Concerning the style, notice that the conversation between the panel and the presenter is not just about a *problem* but also about a *solution*. But, the participants don't go into details, because the developers can design the split on their own. After all, it isn't too different from sports games where some organization comes up with rules and the referee applies these rules to the current state of the game.

Concerning the content of the conversation, it is about a second problem with the existing plan: the referee as both maintainer of the game state and rule checker. It is worth, though, to consider how this discovery differs from the one that exposed the lack of game-state components. If the panel had not pointed out that the referee's purpose statement comprises two tasks, the developers assigned to the referee would discover that the component is just too complex to construct, to read, and to maintain. They could then alert the rest of the team about the need for a separate component, especially with an eye toward the future. By contrast, the game-state components hadn't been identified at all; they aren't even in the graph. As a

result, the developers of the player interface may end up writing signatures in terms of the game pieces instead of a comprehensive understanding of the game state. In short, the discoveries differ in nature but both save the developers from future comprehension problems.

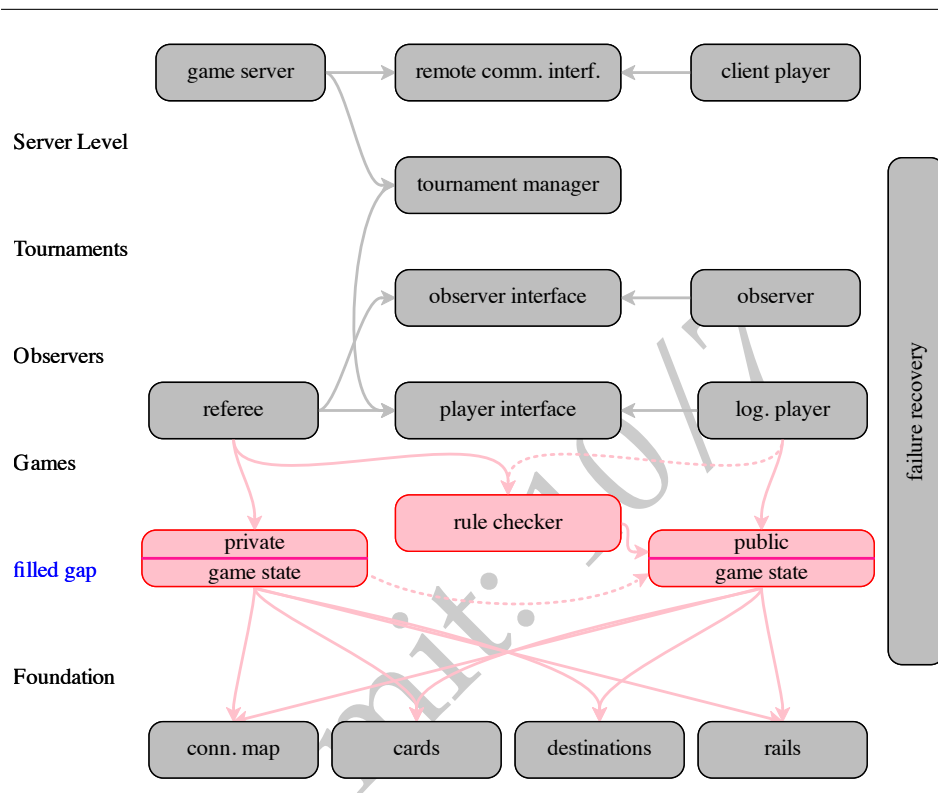


Figure 17: Fixing the construction plan for the game server

Figure 17 displays a revision of the dependency graph in figure 15, based on the feedback from the plan inspection. The diagram comes with a new layer, indicated in the left column with “filled gap.” This new layer contains three components: the private game state; the rule checker in the middle; and the public game state. As implied by the conversation, a public game state is derived from the private one (see dotted arrow between the two), the “ground truth” of the game that may represent “secret” information on a per player basis. Furthermore, while the two game-state components directly depend on the data representation of the basic game

pieces in the “Foundation” layer, the rule-checking component depends only on the public game state and indirectly on the representation of the game pieces.

This last dependency—of the rule checker on the public game state—deserves an explanation. As the conversation also suggests, the referee derives the public game state from the private one and then hands it to the player whose turn it is. In response, the player computes its next action, which should be legal according to the rules. Naturally, the creator of a player is going to write the code such that these actions are legal. To accomplish this goal, the creator must understand the rules of the game. Consulting the natural-language description is what people normally do; in fairness, though, the creator of the server should just publish the Foundation layer and the rule checker so that the creator of a player component can study what is implemented and base a player-implementation on this understanding (see dotted arrow from player to rule checker).

Exercise 6. Describe at least two methods that the private game-state component should implement: their signatures and their purpose statements. Also describe informally how they implement the functionality.

Exercise 7. Read up on the remote-proxy design pattern. Then imagine and write down a short inspection conversation for the layer at the top of figure 17. The goal is to discover at least one missing component.

Reading Erich Gamma, Richard Helm, Ralph Johnson, John Vlissides. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley. 1994.—Over many years, this “gang of four” performed a form of “software archaeology” across many code bases. The result was a language of patterns for describing common program design problems and general solutions in object-oriented languages.

12 Inspections Matter, Especially in the Age of AI

Software development denotes the series of steps from the initial conception of the (change to the) system's purpose to its realization as code. Its outcomes are not just the code but the documents that result from the analysis of the context, the specification plan, and other documents. This applies equally whether a team works on a brand-new system, the repair of an old legacy system, or just the addition of a new feature. It is also true of whether the team uses AI agents to create the system, whether it uses an engineering approach, or whether it insists on agile sprints. Sooner or later a surviving code base needs a proper organization. So let

Most of the book is agnostic when it comes to these aspects.

The Purpose In all likelihood, the purpose statement for a system or a feature and a contextual analysis is the result of a negotiation between “marketing” and the leader(s) of the development team. The latter translates the agreement into actionable tasks, that is, something developers can actually implement. A purpose statement should be written down and turned into a one-line slogan, especially in the age of AI agents.

A clear purpose statement for software systems and their features, like the one in section 10, helps the developers and their AI agents to focus. The leader and the team members can evaluate every new feature request with respect to the overarching purpose. If it doesn't (seem to) fit, the statement should help the leader start a renegotiation process and articulate arguments in support of the changes.

Writing down the results of a contextual analysis—with sketches like those in figure 6 and component lists like in figure 14—helps the current team, the AI, and the future maintainers. The discovered constraints and contextual peculiarities (may) help explain the organization of the system and particularly intricate pieces of code. The description is also going to help with planning future extensions and adaptations of the system.

The Plan A description of component dependencies is equally likely to be needed for any moderately complex system. If it isn't created up front, the team may wish to document it during the coding process, when a first draft of the (essential) software has been developed. No matter, it needs adaptation as inspections uncover flaws and additional coding change the relationships or create new ones—a task that AI agents can definitely take on in this age. If a team dives into coding without a dependency description, it must develop one as soon as any of its members expresses concerns about any relationship between components during an inspection.

Every new team member is going to welcome a dependency description. Imagine yourself joining a team, being assigned a feature ticket, and staring at dozens of files and modules in the code base. Where to begin. A dependency diagram, such as the one in figure 15, will greatly facilitate the navigation of the code. If the components come with short purpose statements, a new team member can quickly hone in on the places where code has to be adapted or added. By using the relationships, a maintainer can also inspect functionality in use contexts, which may explain oddities in the implementations of some piece of functionality.

12.1 Documents: Start the README, Before the Coding Begins

At this point of the development process, the developer team has a good understanding of the basic project parameters:

- its purpose and its context;
- its major components plus their dependencies; and
- a plan for building these components and its major stages.

These steps may have taken place over a number of conversations: in a meeting room near white boards; at a coffee shop using napkins; via a laptop's keyboard with an AI; or some other form. In some settings—say a safety-critical project concerning a nuclear reactor—someone will be asked to turn those discussions into formal documents. In other settings—a start-up whose founders are impatiently waiting for code so that they can run demos—nobody may wish to take the time for such a mundane task. But, the task isn't mundane, and the resulting documents will matter if the start-up wishes to survive. In other words, no matter the setting, someone should preserve whatever was discussed and reasoned through. At a minimum, someone should take pictures of the white boards and collect those in a project repository.

Once the repository is created, it makes sense to add a document that guides future readers through its files. In many cases, people call this document a

README file

and the joke is that this file is the best place to hide secrets. The problem with such read-me files is that they are often an after-thought. Someone may remember late in the creation process that such a document should

exist, and it is compiled in a rushed manner, yielding an incomplete and possibly uninformative content. If such a file is created at the very beginning, is thought of as a guide to the project, and is updated regularly, it may just help future maintainers understand the big picture behind the project, its pieces, and their relationships.

AI agents know to create these README files. They typically don't wait to be prompted, because even average projects have those. It is the human developer's task, however, to ensure that these generated README files—and all the auxiliary documents that they point to—make sense to a human reader. The “making sense” part is easy to spot for an experienced developer; junior developers must create them and get feedback on their creations to understand what a good set of such files brings across and what a useless one is missing.

Once someone has record the results of the context analysis, identified components, created a construction plan, it is time to tackle what most developers think is the key task—coding—but which we now understand is just a part of the overall software product and the development process.

Exercise 8. Distill the sample project from the preceding section into a README file. It is okay to take pictures of the diagram in this book, but use your own words to describe what's going on.

mit: 10/7

CHAPTER IV PROGRAMMING INTERFACES

What is a software component? How are they developed? Used?

As the preceding chapter suggests, the first step of any software development process identifies all the major components of the system, at least as much as possible. “Identifying a component” means picking a title or name; writing down a purpose statement; and perhaps formulating an informal description of the functionality and other attributes. Once this much is worked out—with formal documents or an informal README file or as a rigorous AI prompt or as a verbal expression from a team member—the developers switch roles: from architects to component creators.

In essence, a component is the location of a major design decision, concerning either a piece of software that faces the external context or one needed for internal aspects of some use-cases. A design decision means choosing one of several alternatives: the data chosen to represent some particular information; the data chosen to represent intermediate values; or the interpretation mapping data to information and vice versa.

Creating a single container for a design decision should mean that any necessary revisions of the decision affect only this one place. That is, if unforeseen circumstances reveal the need to implement an alternative choice, a developer can go to this container and make appropriate changes there—and only there. And ideally, such changes should not affect the functionality of the rest of the system. Software developers speak of *encapsulating* a design decision or of just *encapsulation*.

Encapsulation of a design decision demands two steps. First, the developers must separate the implementation details of the component from how others perceive and use the component, that is,

- the *implementation*, i.e., how a developer implements the design;
- the *interface*, i.e., how others view the component.

The software engineering literature often says an interface describes the *server component* and those that use it are called *client components*. The cre-

ators of the client components are clients of the server component's interface. In this chapter, we call them consuming developers or consumers.

Figure 18 displays a schematic diagram of the arrangement. A component's interface shields the internal details—hidden code of the server component—from the rest of the system and especially all those client components that need to use the functionality of the component.

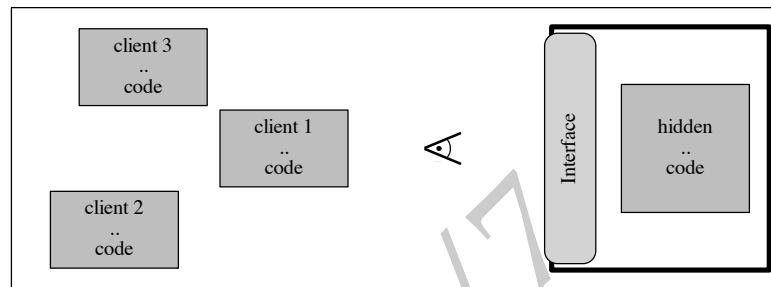


Figure 18: An interface for hiding a component's internals

Second, everyone else working on the system must develop to the interface of the component—not its internals—whether the project is written in a typed language with some support for interfaces or in an untyped language without any idea of an interface. This understanding of interfaces—as a policy of visibility and interaction between pieces of code—makes up half of the design of interfaces. The other half concerns the expressiveness of the interface, its convenient use, and controlling the integrity of its use.

This chapter focuses on the design of component interfaces and its consequences; the next chapter is about implementing the hidden details with accessible code that people can understand. Designing a component interface is key to designing and implementing the component. It is thus often equated with programming components, hence the title of this chapter. The following sections introduce the proper meaning of encapsulation, an approach to designing interfaces, and ways of expressing complete interfaces. In parallel, the sections illustrate how to inspect interface designs and what such inspections reveal about system-level decisions. The combination—practicing interface design and receiving feedback via inspections—is essential to turning an inexperienced developer or student into someone who can guide an AI agent to make reasonable-in-context decisions.

13 The Nature of Interfaces

What are interfaces? What are they to developers?

While a component consists of code, an interface is a document that explains how this code should interact with the code of other components. As such interfaces inform developers of client code about pieces of the data representations in the component (types, constructors, factories, data interpretations, and even examples); about the functionality that can create, instantiate, inspect, and manipulate these data representations; about conditions that these operations must obey; and perhaps even about non-functional aspects, such as resource consumption and management. What is mentioned in the interface, remains hidden from consumers.

These documents come in many different forms and shapes. In some cases, the document is just an informal description. Some research languages are so expressive that an interface is just code. For typed mainstream languages, an interface document tends to mix language constructs with informal descriptions. When code exists in interfaces, the language should perform some amount of compile-time checking.

Java serves as a good example of this common mixture. Its `interface` construct allows developers to write down method signatures and even default implementations. A Java `interface` does not suffice, however, to describe the interfaces between components in our sense. For example, if a method's input signature is `int x` but a small natural number is required, a developer has to add an informal comment—using, say, JavaDoc—that `x` must be greater or equal to 0 and smaller than some other number `n`; the actual run-time check (`0 <= input && input < n`) is usually found inside the method itself. Similarly, consider an interface that specifies three methods—`start`, `extract`, and `end`—but also requires that `start` is called at most once and before `extract` and `end` are called. The developer is again forced to describe this relationship in informal prose. In short,

Technically speaking, Java lacks a contract system.

a component interface is more than a Java `interface`.

To avoid confusion, we occasionally capitalize the word (“Interface”), emphasizing with this hint the general nature of the word’s meaning instead of the narrow meaning of Java’s (and other languages’) `interface`.

In order to understand how interfaces relate components, we need to take a close look at components. For a homework in an introductory course, a component may consist of a single class. A follow-up course may ask students to combine a class with a Java-style interface—for practice. In a professional setting, a developer may have to read an interface document

of many pages to understand a component that comes with a dozen classes or then some. On occasion, a component may itself play the role of an interface, a situation that figure 19 captures.

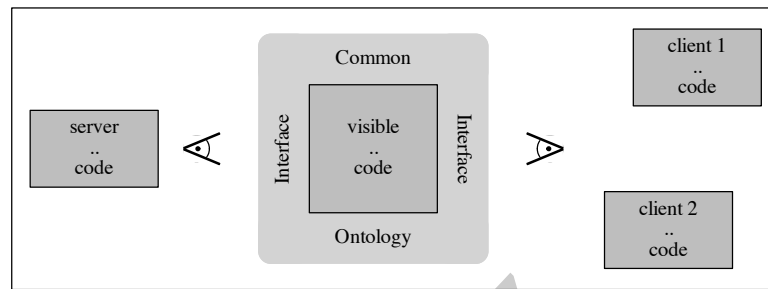


Figure 19: A component as an interface between two system layers

When a component is just a single class, its code *is* the interface for all the *objects* created from this class. For example, in Java all `public` pieces of an object are available to developers of client code in different packages; all `private` pieces remain hidden; the class encapsulates these pieces. And still, a client programmer may have to read the code to understand how the object functions.

A (class) framework—or in ancient days: library—tends to come with an interface document that reveals nothing about the code *per se* and that assumes little to nothing about the client code. Such frameworks tend to consist of many classes or modules. In Java, it is also going to include *interfaces*, though as mentioned, they do not describe the component's interface, neither individually nor collectively. Introductory courses tend to use such frameworks from the standard library of the chosen language.

Finally, some bespoke systems may contain components that play the role of an interface. In this case, these interface components sit between two layers of components, and making their code available is needed to completely clarify some rules of interaction. This kind of component is typically beyond lower-curriculum courses and deserves an illustrative explanation.

So let's return to figure 19, which illustrates this idea with a diagram. Concretely, the diagram can be understood as summarizing how the game server for *Ticket to Ride*—the sample project from the preceding chapter—and its clients interact. Recall the architecture from the right side of fig-

ure 16. It shows how the game server and the hacker's game players live on distinct computers, interacting via means of remote communication. Figure 17 displays a construction plan. Focus on just the interface component in the third layer from the bottom; it is supposed to specify how the referee from the game server interacts with the logical player(s). As the dependency arrows in the diagram imply, designing and expressing this interface relies on a thorough understanding of several other components: the rule checker, the game state, and those representing the game pieces. In this sense, the player-referee interfaces consist of not just something like a Java `interface` but of knowledge about several specific components.

13.1 An Interface is the Common Ontology for Clients and Server

In some language contexts, developers think of component interfaces as essential linguistic constructs for separating the specification of functionality from its implementation; in others, it is just a separation of such pieces in someone's mind. While this separation is definitely an important aspect for all kinds of reasons—from clarity of purpose to separate compilation and variability of implementation—this view undercuts that every interface—not just external ones—is primarily for human consumption. The true purpose of an interface is to establish a common ground for conversations between the producing and consuming developers of a component.

A consumer must completely understand the meaning of the pieces of an interface to exploit the functionality of the server component properly. Typical uses compose the pieces of functionality from an interface where “composition” may mean a series of statements, including loops and conditionals, nested expressions formed from the exported methods and functions; or any other form of a composite phrase. No matter what, from a language perspective, an interface provides a vocabulary for creating “sentences” and “paragraphs” from the visible parts of a component. This perspective suggests the following re-statement of the definition of Interface, using the terminology of linguists:

a component interface provides a *common ontology* for the producers and consumers of a server component.

“Common ontology” is a concise phrase for saying that an interface determines the precise meaning of the specified vocabulary. Here “vocabulary” denotes the words (names of types; function and methods names; the pieces of their signatures; and so on) mentioned in the interface. And “determines the meaning” says that an interface must make sure a reader

understands how to use these words properly, especially their proper use in context, their relationships, and anything else that may affect communication between components.

To make this concrete, consider a software system that controls an autonomous vehicle. In this context you may encounter the following interface element during a design inspection:

```
interface Vehicle:
...
int moveRight(int d)
...
```

You must insist on a clarification from its creator. Here are some questions you could ask:

1. is `d` short for distance?
2. if so, distance from where to where?
3. if so, distance measured in terms of which units?

Question 1 clarifies a somewhat obvious aspect of such a software system, namely, that it must deal with distances. The very word—or an allusion to the concept with a parameter name such as `d`—will show up. The developers have to agree on its exact meaning. Thus, question 2 addresses the difference between theory and practice of mechanics. In a theoretical course on mechanical physics, calculations tend to assume an object is a point. A software developer on an autonomous vehicle team cannot make such an assumption; the vehicle has an extent and that extent must be taken into account. So the answer to the question clarifies on which two points the distance measurement is based. Question 3 is especially critical in a country such as the United States, which has not accepted the metric system but whose engineers often work with colleagues in countries that have. Even if the teams have agreed to use the metric system for all distances, it still matters whether the methods work on `m`, `cm`, or `mm`.

Some languages include notations for specifying units, but not for the meaning of measurements.

While a presenter may verbally clarify the interpretation, you must insist on written clarifications. Specifically, these clarification must become a part of the method's interface, i.e., the purpose statement. Conversely, without a written clarification, things can go horribly wrong when someone is assigned to the code and has to program to this interface.

A common ontology may require a lot more than a simple comment that explains the meaning of a parameter. Let's once again take a close look at what a "hacker" needs to understand in order to participate in the

board-game tournaments of our running example. It is best to start with a close look at figure 17, focusing on the `player interface` component. As the inspection of the construction plan revealed, the interface is best understood in terms of the (visible part of the) game state to which a `Player` component has access. In turn, this data structure refers to the map, the colored cards, the destinations, and the rails. A “hacker” can implement a player only with a complete understanding of these pieces. But this is not all. Additionally, a “hacker” must also know the protocol of how the referee interacts with the player.

Here then is a list of all the pieces that belong to the common ontology of our running case study, starting with the least important but not to be overlooked piece:

- the rails
- the cards
- the destinations
- the map
- the game state
- the protocol: how the methods are called and in what order.

To drive home how all this matters, consider the methods for starting a railroad game. A refined design may consist of two methods, whose plain signatures in a Java-like programming notation are like this:

```
void mapForTheGame(GameMap m)

Set[Destinations] choose(Set[Destinations] from)
```

Now consider the following questions about this variant:

- Which set of `Destinations` should `choose` return? The ones that the player picked or the ones the player rejected?
- Which call order makes the most sense for these methods?
Should the player know that `mapForTheGame` is called only once?

These simple and obvious questions illustrate just how important it is to establish a comprehensive understanding for the vocabulary of an interface, to nail down the meaning of all functions, methods, and other pieces

of an interface. Put differently, the answers to such questions make up an interface; otherwise it does not provide a common ontology.

Exercise 9. Provide answers for the above two questions. Justify your answers. Work through these points with a partner.

Different kinds of interfaces call for different kinds of common ontologies. For library interfaces and for the interfaces of plug-in frameworks, the common ontology is often the responsibility of its developer, and the consumer is typically an anonymous developer. Ensuring that the interface is properly explained calls for thorough in-person inspections and reactions to on-line reviews. For bespoke interfaces specifically created for a software system, the producer of the server module and its consumers may jointly develop the common ontology; for an inspection, they may wish to get a third party involved to overcome “group think.” When the consumer of a bespoke interface is unknown, the authors of the interface document should revisit and inspect it with the help of actual end-users once the product has been soft-launched.

Reading

Ajay Harish. When NASA Lost a Spacecraft Due to a Metric Math Mistake. 2021.—Harish summarizes a NASA crash story from the late 1990s, when the agency launched the Mars Climate Orbiter. The Orbiter, a space probe, was to explore the Martian atmosphere and surface changes. It burned up and broke into pieces, because its software developers had not developed a common ontology. The story is widely studied in engineering and software circles; also research the story on-line. The only way a “hacker” can truly comprehend this interface requires an inspection of all classes in the common-ontology component.



Harish at Simscale

14 Encapsulation via Interfaces

How do interfaces relate components? How do they separate them?

While equipping a component with an interface has the primary objective to create a common ontology for producers and consumers, it also means that the existence of an interface relieves developers from reading the entire code of a component just because they need its services. Conversely, a component producer can rely on the existence of an interface in two ways. First, if the interface is written down up front, it can guide the implementation effort of a junior developer or an AI agent. See the next chapter for this idea. Second, the producer can use the interface to isolate design decisions from the rest of the system and thus keep the options open to change them in the future.

This last point is about *encapsulation*. Specifically, developers speak of encapsulating a design decision. Language designers recognized the importance of encapsulation a long time ago. For example, the SmallTalk language from the early 1970s hides everything but the methods of a class so that consumers are forced to use methods for any manipulation of an object. The creators of Java help developers with another form of *syntactic* (that is, notational) support to express encapsulation constraints: `private`, `protected`, and `public` class annotations. The Java compiler enforces these annotations so that consumers do not accidentally use features of an object that are supposed to be hidden. By the 1980s, typed functional languages offered linguistic constructs for specifying the interface of a component separately from the implementing code. Developers working with untyped programming languages must establish and follow some (unchecked) discipline to express the same kind of separation and to enforce encapsulation.

Encapsulation is far more, however, than annotating the members of a class or writing an interface for a module. If encapsulation means “housing a design decision in a single hiding place,” then mere syntactic annotations simply do not suffice. To capture this difference, researchers recommend the simple policy of

encapsulating those pieces of knowledge of a component that restrict future changes to a data representation to this one component.

This kind of knowledge encompasses many aspects, including (pieces of) the chosen data representation, its interpretation, its composition, some of the functionality for operating on it, and much more.

14.1 Interface Inspections for Encapsulation

An illustrative example of encapsulation easily beats any abstract explanation. So let's study yet another game that could be used in the context of the running game-server example (see Section 10):

Qwirkle is a game that challenges players to construct a contiguous arrangement of square tiles on some flat surface, such as a kitchen table. The size of the arrangement is bounded only by the number of the available tiles. A player may add a tile as long as one of its sides is aligned with the side of an already-present tile. Furthermore, each tile displays a shape in one of six colors. Newly added tiles must satisfy certain constraints regarding the shapes and colors of all neighboring tiles.

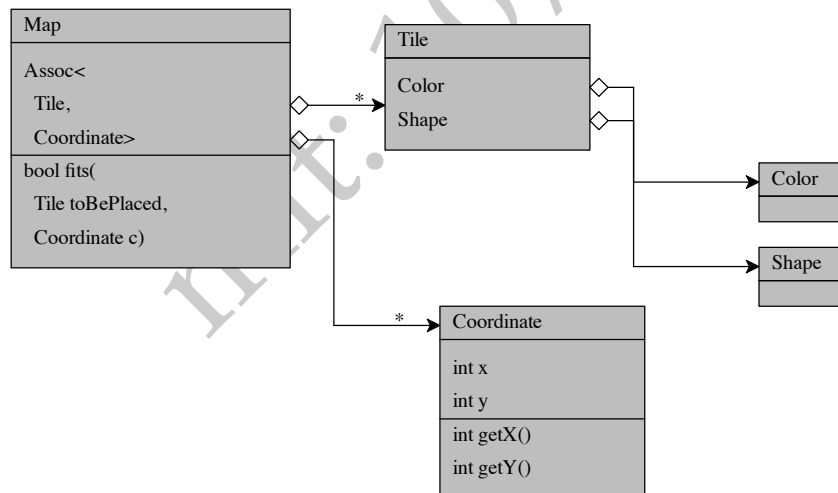


Figure 20: Encapsulating knowledge about a Qwirkle arrangement)

This description clearly identifies four pieces of information: the *arrangement*, which consists of *tiles* plus their *shapes* and *colors*. Less obviously, it includes the information concept of *neighbor*, that is, that each tile has at least one and up to four neighbors. One straightforward way to rep-

resent this neighbor concept uses a notion of *coordinate*, associating each tile in the arrangement with one place.

Figure 20 presents a plan in the form of a doodle diagram. It sketches the four concepts and their relationship. Each box denotes a data representation of an information concept. The `Map` box represents the arrangement of tiles; it suggests the use of a collection to create an association between `Tiles` and `Coordinates`. The former combines an instance of `Color` with an instance of `Shape`, just as the informal description indicates.

The intention behind such a diagram is to convey several important ideas to the future maintainer of this component

1. `Map` is the key class, and all knowledge about the tile arrangement is found there;
2. similarly, `Coordinate`, `Tile`, `Color`, and `Shape` are the classes that collect the knowledge about the respective information concepts; and
3. if any client module needs a service about any of these five concepts, it must consult the respective class.

Instead of discussing encapsulation abstractly, let's see how a code inspection might deal with the concept:

Hi. I am presenting the design of the central data representation for the Qwirkle game. Here is the class diagram (figure 20).	Could you explain the <code>Coordinate</code> class first? It looks suspicious because it comes with getters for the <code>X</code> and <code>Y</code> fields?
Sure. Here is the code for the coordinate class (figure 21). It comes with a purpose statement.	This is a comprehensive purpose statement with the inclusion of the second line. But what is the interpretation of a <code>Coordinate</code> object?

We stop here and look back. The panel takes control of the code inspection because the diagram alone may suggest a problem. While the presenter and the panel could discuss this potential problem abstractly, working with the actual code is best in this situation.

```

// represents either the location of tiles in the map
// or: places where a player may wish to place a tile
class Coordinate {
    private int x;
    private int y;

    ...
    public int getX() { return x; }
    public int getY() { return y; }
    ...
}

```

Figure 21: A Java-style class for representing coordinates

Interpretation?	Well, an X-Y pair may denote Cartesian or computer coordinates.
Our implementation aligns with the standard computer-graphics meaning, just in case we want to add some GUI.	Should this statement be added to the implementation so that a reader of the code doesn't have to dig through the entire class to find out?
I will make sure an interpretation gets added.	We will need to consider whether this statement is made public or kept hidden.
Yes.	Why does the class include plain getters for the fields?
The two integers are private. But the Map class needs access to their values.	What for? And how are they used?
Here is the code for Map (figure 22).	Where do the getters used?

All the computations matching a tile to its potential neighbor use the getters.	Let's take a look.
---	--------------------

Stop! Can you already guess what the panelist is getting at?

```
class Map {
    private Assoc<Tile, Coordinate> assoc = ...
    ...
    public Optional<Tile> above(Coordinate atC) {
        // extract the tile above coordinate 'atC'
        Coordinate up = new Coordinate(atC.getX(), atC.getY()-1);
        return this.assoc.retrieve(up);
    }
    ...
}
```

Figure 22: A Java-style method in the Map class for Qwirkle

It's probably best to look at the computation that makes sure tiles match an up neighbor. These three lines are the key.	This comment cleanly separates one direction from the other. Can you explain the three lines?
--	--

The first line computes the place that is exactly above the given coordinate.	How does this computation work?
---	------------------------------------

It retrieves X and Y parts, subtracts 1 from the latter, ...	Stop. Why does it subtract 1?
---	-------------------------------

Isn't it the natural way to determine the Y part of the place above the given c?	Only if the person who reads the code of Map also knows the interpretation of the Coordinate class
--	---

Oh.	.. which isn't even written down.
-----	-----------------------------------

At this point, the panelists have clarified that the `Coordinate` class does *not* encapsulate the coordinate information concept. Even though the `x` and `y` fields are private and thus hidden, exposing plain getters implicitly leaks knowledge about the data interpretation. Here the presenters exploit this knowledge for the construction of a client class, specifically both that coordinates are integers and that if $y_1 < y_2$, then y_1 is *above* y_2 . Thus computing `atC.getY() - 1` gets the correct neighbor in this case. If, for some reason however, the instances of the `Coordinate` class should be interpreted as ordinary Cartesian coordinates, changing just `Coordinate` will not work. Encapsulation failed, because the students confused `private` with the idea.

14.2 Encapsulating Properly

The inspection's focus on encapsulation leads to a general lesson. The developer of a component who wishes to encapsulate knowledge must ask

whether a change to the data interpretation or any internal code can possibly affect client code.

If so, we say that the interface is too wide. Positively put, a component's interface should be sufficiently narrow, where "narrow" means "do not expose functions or methods that enable client programmers to exploit the internals of the component." While fields and some methods are obviously internals, the informal data interpretation may have to count, too, especially if the interface includes plain getters.

In Java, and many object-oriented languages, the most basic meaning of "interface" is the collection of fields, methods, and other pieces of an object design that are accessible to a client programmer. This collection may or may not come with a Java `interface` or an Interface document, though, a Java component usually needs both.

Let's take a look at how a Java developer could react to the code inspection. One reaction is to replace the getters from the `Coordinate` class with methods that determine the coordinates of neighboring places: `above`, `below`, and so on. See the left-hand side of figure 23 for the result of such a revision.

```

class Coordinate {
    // INTERPRETATION
    // a data representation
    // of computer-graphics
    // integer coordinates
    ...
    private int x;
    private int y;
    ...
    public Coordinate above() {
        int newX = this.x;
        int newY = this.y-1;
        return
            new Coordinate(newX, newY)
    }
    ...
}

interface ICoordinate {
    // INTERPRETATION
    // a data representation
    // for neighbor relations
    // on the game's Map
    ...
    // the place logically
    // above 'this' coordinate
    Coordinate above();
    ...
    Coordinate below();
    ...
    Coordinate left();
    ...
    Coordinate right();
}

```

Figure 23: A Java-style class for an *encapsulated* coordinate representation

Once this revision of `Coordinate` is available, the `Map` client class in figure 22 could be revised too. It would use an `above` method to compute the value of `matchesUp` instead of exposing any implementation knowledge about the `Coordinate` class.

If the coordinate class should be considered a component all by itself, it should come with an interface like the one on the right-hand side of figure 23. The display hints at what an Interface document might look like. Here it states a simple data interpretation and adds signatures equipped with purpose statements of methods that compute the four coordinates of immediate neighbors. That is, the document consists of a mix of actual code—a Java *interface*—and plain English—the data interpretation and purpose statements.

Taken this way, the figure's left-hand side is an implementation of the right-hand interface. The implementation refines the data interpretation with more information about the design and supplies full method definitions. In this sense encapsulation is subtraction of knowledge that is present in an implementation so that the producers are enabled to come up with alternative ways of satisfying the Interface.

An instructor of a course on object-oriented programming may ask students to design an interface per class as just such an exercise.

15 Designing an Interface

How do component interfaces get designed? What do they describe?

Figure 18 shows two ways of looking at an interface: as the producing developer and as the consuming one. From the perspective of the first, it is the document that guides the implementation. The component must live up to this interface; it must implement the promised functionality and everything else that the document says it delivers. From the perspective of the second, it is the document that tells everyone how to use the component.

As such, the figure suggests that an interface is the result of a “negotiation” between the two parties. The word “negotiation” implies that the producer of a component interface knows the consumer, and that the two parties communicate. On occasion, this is indeed the case. Consider a one-of-a-kind component, say a part of a large bespoke system; in this case, the two parties are likely to know each other and can actually negotiate. But, on many other occasions, the word “component” refers to libraries and object-oriented frameworks that are created in the abstract. Consider the game-server interface presented in figure 19. In this particular case, there is no actual negotiation partner. Hence, the team must pick a representative, a substitute partner, who via conversations and inspections critiques the interface design in a feedback loop. There is no other way to create interfaces. And there are a variety of situation between these. Given this variety, interface design is clearly not going to follow one and only way either.

One part of the negotiation concerns encapsulation. Since an interface is to some large extent about hiding design decisions, understanding its design must also answer the question of how much it can change over time. The answers differ a lot for the two ends of the spectrum. When a change of a bespoke component affects its interface—the result of an actual negotiation between two parties—the producer can alert the consumer, who, in turn, can search the code base for potential problems. For example, if some function must consume an additional argument, the creator of clients just looks for applications of this function and fixes those call sites. Indeed, modern IDEs support such changes with project-wide refactoring tools.

When a change concerns a library or framework, the situation is entirely different. The producers of a library or framework do not know their consumers, and they must therefore be much more careful about changes. In many cases, changing the interface is nearly impossible. External developers may have created extensive bodies of code relying on the published interface. Hence, removing functionality is essentially impossible, which imposes constraints on changes to internals of the component. Adding named

functionality to the interface may interfere with names that client programmers chose for something. Finally, even changing a “bug”—meaning a contradiction between elements of the Interface and implementation—may affect the behavior of client code; consumers may have developed elaborate work-arounds for such short-comings, and they might just be unhappy to find out that a function’s behavior changed.

Exercise 10. Research how the designers of Java introduced modified libraries when they added generics—also called parametric-polymorphic types—to the type system.

Given an understanding of these constraints, let’s consider what goes into the design of an interface: the content and the process.

15.1 Content

The preceding sections implicitly suggest a number of items that show up in basically every interface. Furthermore, they also clarify that the content is based on a focused and clear purpose statement for the component; an experienced user of AI might say a “concisely articulated prompt.”

A component interface must obviously state and explain the types of data that it makes available. This part must include a sufficiently precise data interpretation, so that consumers can make sense of these types of data. At the same time, it should not give away any representation details.

Next, an Interface must specify the pieces of functionality that the component makes available. These specifications typically contains the following kinds of functions and methods: those that

- create or initialize (data from) the component;
- observe properties of the data originating from the component;
- extract properties from the data of this component; and
- construct updated data or modify it.

An interface may not include all of these kinds of functionality, but for each specified piece of functionality it must state its purpose and its type signature

If the chosen language is untyped, the team members should use an agreed-upon form of comment to simulate signatures; see next chapter. If the language is typed, it often comes with a type notation for signatures that helps express the essence but lacks power to express it completely.

Hence, an Interface for such languages also specifies any inexpressible constraints as comments. Such constraints come in different flavors:

- restrictions on the values of individual arguments or the result;

Consider a method `m` that is expected to be given a natural number in the context of a type system that supports only the integer type. The Interface should add this constraint so that consumers don't accidentally call the method with a negative number.

Conversely, a developer of method `k` may wish to promise that its result is always going to be a natural number so that it be fed into a method like `m`: `m(k (...))`.

- restrictions on the relationships among several arguments;

A function may expect pairs of numbers, `x` and `y`, such that `x < y` is always true. The Interface must express this constraint. Alternatively, the component should provide a pair type that enforces this constraint at construction time.

- restrictions between the values of an argument, or several arguments, and the result value.

If a method computes the square root of a non-negative real number `a`, its developer may add a promise about the result `r`

```
// promised: r * r is close to the value of a
```

An actual mathematics library promises a tighter bound than this.

Developers using an untyped programming language may occasionally express some of these constraints directly with the informal signatures they write down as comments, though in general, they will also need to write a proper Interface document.

Finally, an Interface often comes with constraints on the relationship among pieces of functionality. While most of these constraints are purely temporal, some also involve the values of results and arguments. In general these restrictions concern

- the order in which the various pieces of functionality are used;

The example from the preceding chapter—that `start` is called before `extract` and `end`—illustrates this point well.

- the results that several calls to the same function produce;

The run-time system of a programming language may provide a function for checking on the memory usage of a running system. If this function ignores garbage collection, the calls to this function should produce a sequence of increasing natural numbers (representing the number of bytes allocated).

- the result of one method call and calls to the next method(s).

Suppose we wish to equip the game servers from the preceding chapter with a tournament manager that realizes knock-out tournaments. If a player object has a method for initializing its state for a game and another one for being told whether it won, then the tournament manager may promise that the first method is called after the first game only if the second one was called to inform the player of a win.

15.2 Process

Given the nature of an interface as the result of a negotiation, the design process should involve both parties: the producer and a representative for the consumers or possibly all potential consumers. The producer is clearly in charge of proposing drafts of an interface, starting from the focused purpose statement that the project analysis articulated. The representative consumer must often—or continuously—check whether it is possible to get the work done with the proposed interface. If the consumer representative determines that the interface lacks functionality or is too inconvenient to use, it is back to the drawing board for the component producer. In short, the process is iterative.

As this iterative process unfolds, this producer may also have to take into account implementation constraints. Some experienced developers can imagine an implementation that meets all of the constraints of an Interface. Inexperienced developers and students may have to sketch out an implementation with actual code. Sketching is especially important if the Interface imposes performance constraints, in which case the sketch can clarify that such constraints can be met in principle.

Figure 24 shows the essential elements of this interface-design feedback loop. The starting points are the two “clouds” on the left: an idea of the component’s purpose (on paper or in someone’s mind) and an idea of what the implementation may look like. The goal is the rounded rectangle in the middle, which at first is a draft of the interface, and after several review steps, becomes the actual interface. Critically, the process separates the

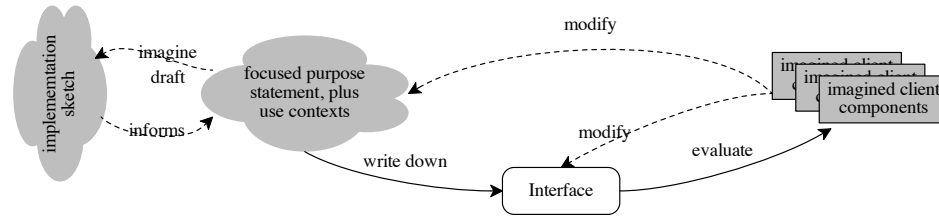


Figure 24: Feedback loop for the design of interfaces

implementation idea and any refinements from the use cases. After all the point of the interface is to isolate one from the other.

Like for any building block in the world of software development, the articulation of a focused purpose statement is the key to getting the design process started and re-started. Without such a statement, it is too easy to pack too much or too little functionality into an interface. That is, when the negotiation of the interface questions arise as to whether a certain piece of functionality should be exposed, a consideration of the purpose statement will answer them.

Recall the sample problem from the preceding section, and let's use it to illustrate this point. The panelists could have asked the creators of the `Coordinate` class what the purpose statement exactly means. Concretely, they could have asked

if coordinates exist only for the purpose of representing the four cardinal neighbor relations on `Maps`, why are there numeric computations on coordinates in the `Map` class?

With this question the inspection clarifies that the purpose of the `Coordinate` class is to determine the neighbors of some given place. Hence, the interface should expose these pieces of functionality and should hide the necessary numeric computations in the implementation.

Generally speaking, a focused purpose statement helps with the balancing act of deciding

- which parts of the component must remain hidden; and
- which parts to make public and what explanations are needed so that client programmers can get their work done.

A good starting point for getting this separation correct is the interpretation of the data representations that a component contains. It is usually a restriction of this interpretation that identifies the public ones and thus separates the hidden aspect. Take the `Coordinate` class as an example again. Its general purpose statement would be something like “represents places on the `Map` so that the referee and the players can determine neighbor relationships.” While the data interpretation inside the class should inform future readers whether the implementation uses coordinates of the Cartesian or computer-graphics or a third kind, a client programmer has no need to know this much. It suffices if the purpose statement for the component says that it determines whether two places on the `Map` are neighbors.

The purpose statement serves as a guideline for a two-faced investigation: whether the public pieces are useful for potential use cases and how to make the implementation idea concrete. But, imagination can go only so far. Sooner rather than later, the developer should write down a draft of the interface and present it to potential consumers and the producers of the component. If the producers include the people who design the interface, it might be best to separate roles. An inspection via both consumers and producers will yield valuable suggestions on how to modify either the starting point or the written document or both.

When the interaction has converged on a reasonably complete interface document, the producing developer can finalize the actual implementation of the component’s internals and the consumers can write additional experimental code to the interface.

In sum then, the process works as follows:

1. write down a focused purpose statement;
2. imagine an implementation and some client use cases;
3. refine the purpose statement and identify the public information:
 - the public parts of the data interpretations;
 - the pieces of functionality and their purpose statement;
 - signatures for these pieces, formal ones in a typed language; and
 - any additional constraints on these pieces of functionality.
4. request an inspection of this draft; and
5. re-start at step 2, until the process converges. If your junior partner is an AI, re-start at step 1.

The next three sections illustrate this abstract description with examples.

15.3 A Simple Interface Design Process

While the inspection of the simple `Coordinate` component in section 14 illustrates the basic idea of encapsulation, it does not explain the transition from the identification of a component to the design of its interface. But, given an understanding of the `Coordinate` class and its interface, the `Map` component is a good candidate for illustrating this step of the process.

The description of Qwirkle suggests that, from a “physical” perspective, the game is about “a contiguous arrangement of square tiles on a flat surface.” We can take this sentence about the *information* in the game description and turn it into a purpose statement for the `Map` component. See figure 25.

```
// represents a contiguous, extensible arrangement of square tiles
interface Map = ...
```

Figure 25: A purpose statement for the `Map` component and interface

Next a developer should consider whether the interface needs to expose functionality for creating or initializing the component, for observing or extracting properties, and for updating the component’s internal data. Since the data representation is about the creation of a contiguous arrangement of tiles, it makes sense to create a `Map` from at least one, given tile:

```
// creates a Map by placing the given tile
create :: Tile -> Map
```

The question is where this first tile should be placed. Conversely, the placement of the tile could determine the “origin” of the `Coordinate` system. If a developer chooses the second option, `Coordinate` must provide one fixed instance: `origin`.

Adding a tile is subject to the contiguity constraint. One side of the added tile must touch the side of tile that belongs to the `Map`. This implies a need for two pieces of functionality beyond a simple `add`: (1) one for checking whether a spot is available; and (2) another one for checking whether this spot is adjacent to an existing tile. On the implementation side, the developer can use these two to ensure the integrity of the `Map` according to the purpose statement. Here is an implementation sketch:

```
// inside of Map:
//   a HashMap hm associates Coordinates with Tiles
Map add(Map m, Tile t, Coordinate c) {
    if (!available(m, c))
        throw Exception("the desired spot is occupied");
    if (!adjacent(m,c))
        throw Exception("no adjacent coordinate is occupied");
    addToHash(m.hm, c, t);
    return m;
}
```

On the client side, these pieces of functionality can be used to avoid errors:

```
// inside of Player
Pair<Tile,Coordinate>findPlace(Map m) {
    ...
    Coordinate c = .. choose a coordinate from m ..;
    Tile t       = .. choose a tile in this player's possession;
    if (available(m,c) && adjacent(m,c))
        return pair(t,c);
    ...
}
```

This implementation sketch confirms that the developer of `Map` does not impose constraints on, say, the producer of a `Player` component that are impossible to satisfy.

Figure 26 shows an intermediate draft of the Interface. The draft specifies four pieces of functionality: one for creating a map from a tile; two more for checking properties; and a last one for extending an existing map. Each specification consists of a purpose statement and a functional signature. Notice the two **bold-faced** lines of the purpose statement for `add`; they tell any consumer that the method enforces constraints that the type system cannot express. The next chapter expands on this idea.

At this point, the interface designer should stop and consider what other properties or observations a consumer may need to produce client components. This concern requires the consideration of at least two different components:

- the automated player itself, which during its turn must decide where to add tiles to the existing `Map`
- the rule checker, which enforces the laws of the game, for example those that govern a player's additions to a `Map`.

Let's deal with one concern at a time.

```

interface Map = {
  uses Coordinate, Tile

  // creates a Map by placing the given tile at the origin
  create :: Tile -> Map

  // is the given coordinate adjacent to an occupied one?
  adjacent :: Map Coordinate -> Boolean

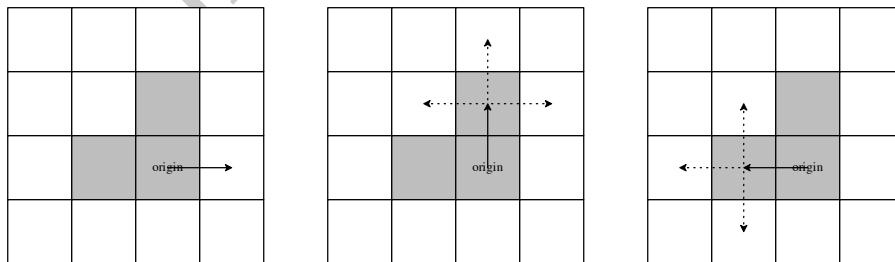
  // is the given coordinate a available spot on this map?
  available :: Map Coordinate -> Boolean

  // add the given tile at the specified coordinate to this map
  // ASSUME when add(m,t,c) is called, adjacent(m,c) must hold
  // ASSUME and available(m,c) must hold
  add :: Map Tile Coordinate -> Map
}

```

Figure 26: Draft 0, focusing on building up the tile arrangement

The sketch of the player functionality comes with two gaps: choosing an available coordinate and choosing a tile to place there. Of these, the first one poses an interesting choice. Using the functionality of `Coordinate`, a reasonably standard graph search finds all coordinates where a player can add tiles:



Concretely, the search starts from the origin tile and checks all four immediate neighbors, using the four appropriate methods from `Coordinate`. When the first step (solid arrow) to a neighbor reaches an available spot (white), the algorithm adds the corresponding `Coordinate` to the set of available ones (e.g. left-most diagram). When such a first step (dotted arrow) reaches a spot that is not available (gray), the search recursively

continues from these spots (dotted arrows) to the neighbors (e.g. the two right-most diagrams).

Exercise 11. Sketch the algorithm, assuming the functionality of `Map` and `Coordinate` is available.

A comparison would quickly point out that adding the same functionality to the `Map` component is much more straightforward than a search:

```
// inside of Map:
//   a HashMap hm associates Coordinates with Tiles
Set[Coordinate] availableSpots(Map m) {
  Set[Coordinate] result = EmptySet();
  forEach(Coordinate c in keys(hm)) {
    result = setUnion(result, freeSpotsAround(m, c));
  }
  return result;
}

// computes the subset of the four neighbors that are free
Set[Coordinate] freeSpotsAround(Map m, Coordinate c) {
  ... available(m, c) ...
}
```

This algorithm is applicative, that is, several instances of `Map` may exist simultaneously. A `Player` may thus easily compare and evaluate various tile placements. If efficiency matters more, an implementation could track the set of available spots when a tile is added and cache it internally.

The comparison clearly favors widening the interface of `Map` with a piece of functionality that delivers the set of available spots:

```
// computes the set of coordinates where a tile can be added
availableSpots :: Map -> Set[Coordinate]
```

It is time to turn to a second client component of `Map`: the rule-checker. As the summary description of `Qwirkle` states,

a newly added tile must satisfy certain constraints regarding the shapes and colors of all neighboring tiles.

For simplicity, let's say that some property `good` represents this unspecified constraint of the newly added tile and its up to four neighbors. In turn, this suggests that the `Map` component comes with functions for extracting the four neighbors. Assuming these functions exist, sketching the rule-checker functionality is easy:

```
// inside of RuleChecker:
// is placing t at c on this Map m legal?
Boolean legalPlacement(Map m, Tile t, Coordinate c) {
    Optional[Tile] a = above(m, c);
    Optional[Tile] b = below(m, c);
    Optional[Tile] l = left(m, c);
    Optional[Tile] r = right(m, c);
    return good(t, a, b, l, r);
}
```

At this point, a Map developer may wish to sketch one of these four functions just to make sure they are a feasible addition:

```
// inside of Map:
// a HashMap hm associates Coordinates with Tiles
Optional[Tile] above(Map m, Coordinate c) {
    Coordinate a = Coordinate.above(c);
    return hm[ a #:whenUndefined Optional.none ];
}
```

Since Map composes the HashMap data structure with the custom-defined Coordinates, the function merely composes the above method for the latter with looking up the coordinate in the hash map. If there is no tile associated with the coordinate, the lookup returns the none value.

Figure 27 collects all the ideas about Map's interface in once place: its initialization functionality (create); several functions for extracting properties (available, ..., right), and a function for updating its state (add). Based on a quick scan of the Qwirkle description, this first draft seems to serve its potential clients well. Hence it is time for presenting it to a panel of potential producers and consumers of Map.

The producers of the draft have assembled their team, including an implementer and a potential consumer:

Welcome to this session. Here is our first draft for the interface of the Map component (figure 27).	Does the class diagram (figure 20) from the component analysis review still apply?
Yes, it does and you should keep the Coordinates, Tiles, Colors, and Shapes representations in mind.	Okay. Let's work through the interface then.

```
// a data representation of a contiguous and
// extensible arrangement of square tiles
interface Map = {
  uses Coordinate, Tile

  // creates a Map by placing the given tile at the origin
  create :: Tile -> Map

  // is the given coordinate an available spot on this map?
  available :: Map Coordinate -> Boolean

  // computes the set of coordinates where a tile can be added
  availableSpots :: Map -> Set[Coordinate]

  // retrieves the tile that is above, below, to the left, or
  // to the right of the given coordinate in this map, if any
  above :: Map Coordinate -> Optional[Tile]
  below :: Map Coordinate -> Optional[Tile]
  left  :: Map Coordinate -> Optional[Tile]
  right :: Map Coordinate -> Optional[Tile]

  // add the given tile at the specified coordinate to this map
  // ASSUME when add(m,t,c) is called, c must be in availableSpots(m)
  add :: Map Tile Coordinate -> Map
}
```

Figure 27: Draft 1, focusing on building the arrangement and observing it

For this session, the presenter immediately sets up an expectation, namely, that the assembled team members recall the component analysis and its essential pieces. Demanding this much preparation is fair game.

We decided that the purpose of Map is to represent an arrangement of square tiles.	Meaning?
--	----------

It represents tiles and how they share sides and nothing else.	Does this mean the representation ignores all other properties of the tiles, say their colors?
--	--

This is precisely what our purpose statement is supposed to express.	Why? Why not ensure that the colors match?
Remember that the overall analysis of the project also identified a rule-checker component (figure 17).	You want to separate the functionality for placing tiles from ruling on the legality of the placement.
Precisely. Representing the physical arrangement is one thing. Representing the rule-book is another one.	Yes. One could make a game with slightly different rules but the same physical arrangement of tiles.
And anyone who is charged with doing so can then navigate to the rule-checker component and modify it or make a new component.	This separation is good. Arranging the tiles and ruling on the legality of the arrangement really are two different concerns.

This part of the inspection clarifies how the purpose statement focuses the designers attention and why this is important for anyone who will work on the software in the future. If a bug or a feature request concerns a rule change, the developer will find it in the rule-checker component; if it is about the physical arrangement of tiles, the `Map` component has to change.

Also notice how the presenter responds to an implied critique from the panel. Instead of agreeing to the request of incorporating the color constraints, he pushes back by pointing to result of the project analysis. So keep in mind that presenters don't have to accept all criticisms, and that pushing back needs a foundation, like a pointer to the presentation's project context.

So far, the conversation addresses general concerns, mostly from the side of client consumers, that is, developers who must use the interface to implement client components such as the player. Let's see what the implementer of `Map` has to add:

Any other questions?	The pre-condition of <code>add</code> looks expensive. Why does it not use <code>available</code>
This points out an oversight. We didn't want the <code>available</code> functionality in the interface.	Is it correct that <code>available(m, c)</code> holds precisely when <code>c</code> is an element of the set returned by <code>availableSpots(m)</code> ?
It is.	So one of these two is superfluous?
<code>available</code> is from a prior draft. It should remain hidden.	Then I'd like to return to my initial point.
Which was?	Isn't it expensive to compute the set of all available spots when all we need is to check whether a tile can be added at a particular spot?
That's why <code>available</code> might have to exist as a hidden piece of functionality.	I see. The document expresses the constraint with the functionality that is useful to the player but checks it with an inexpensive function internally?
We also considered an alternative. The availability functionality could be computed incrementally and cached inside the implementation.	Oh, this is true. I will use this idea when we implement the interface.

Note how this conversation is between the designers of `Map`'s interface and the developer who is going to implement the functionality. It points out the redundancy of some visible functionality as well as an apparent tension

between the contractual obligation for `add` and its actual implementation. The presenters clarify this second point with a hint to the implementer.

An attentive developer of an test player may notice that this exchange has implications for the implementation of a consuming component:

Should we cover anything else?	I would just like to add that this idea would help me a lot with the task of implementing the player.
--------------------------------	---

Yes, that was our intention.	My player code clearly needs to call <code>availableSpots</code> first and then decide which of the coordinates to use for tile placements.
------------------------------	---

But, there is also the team member responsible for the rule checker, who has to program to this interface, too:

This functionality should also help with the implementation of the rule checker.	It will! Speaking of which, I have read the rules of the game in some depth. I think I will need to look inside of <code>Coordinates</code> .
--	---

How come?	A player may place several tiles during a turn, as long as all of them are in one row or column.
-----------	--

This seems to be a question about the interface for the entire component rather than just <code>Map</code> .	This is true. But can you speak to it briefly?
--	--

Our purpose statement for <code>Coordinate</code> says it represents “neighborhood” relationships. So, perhaps we need functionality to check whether a set of coordinates is in a row or in a column.	That’s a good start, and I can wait until we have a concrete draft for the component interface.
--	---

The presenters demonstrate that they did their work; their draft is clearly based on imaginary developments of the component itself and the clients. Also, the presenters pushed back against a request with hints at the purpose statement. Formulating this focused statement first gives them an easy way to decide which requests are valid (remove `available`) and invalid (legality checks).

Exercise 12. Design a class in your favorite object-oriented language that implements the interface of figure 26 using an imperative style. Re-implement the interface in the same language in a functional style.

This section illustrates the design process. While the chosen example is relatively simple and the result mostly predictable, the point is that the process scales up to complex interfaces. The result of this process differs, however, in both size and complexity.

15.4 An Interface for a Component

Some software components consist of many `interfaces`, classes, methods, and so on. The Interface for such large and complex components consists of the collection of documents that explain the computations that the exposed pieces of functionality perform; usage patterns for those and constraints on them; and some guarantees that come with these computations.

Each externally visible piece comes with a description in an Interface. This document may, for example, restate the purpose of a class and the types of its exposed methods. Additionally, such an Interface may state constraints on the use of methods that the type system cannot express. Some of these constraints are enforced inside of methods and signal an error when a misuse occurs; alternatively, the Interface comes with phrases such as “may cause unpredictable behavior.” But, the constraints stated in the individual sections of an Interface are relatively simple ones.

Beyond these simple constraints, large and complex components tend to impose cross-cutting constraints. For example, they may require a method in one class to respect changes to instances of a completely different class. It is the existence of such constraints that differentiate the Interface for the entire component from the mere collection of documents for all pieces of the component. This section illustrates this idea with a real-world example and sketches how code inspections can lead to the discovery of such constraints and discussions on how to document them.

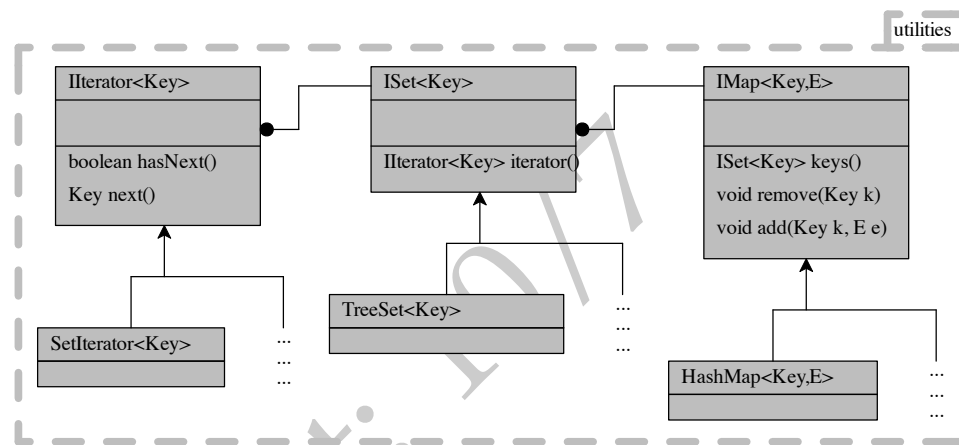


Figure 28: A sketch of a full-fledged component

The example is inspired by Java's utility package, and it is chosen for three reasons:

1. a good example of a large component, containing many interfaces and classes;
2. a component whose interface is not negotiated within some company's production team or between two such teams; and
3. an Interface that imposes cross-cutting constraints.

Our focus is the third aspect.

Figure 28 displays the relevant class diagram, which depicts the interfaces and classes involved plus their relationships. From right to left, the figure depicts the interfaces for three class-based data representations:

finite maps, which associate a finite number of keys with values on a one-to-one basis; sets of any type of element; and iterators over sets. Each of these interfaces comes with many implementing classes, which is indicated with one example followed by dots. The dashed gray line around the three class diagrams denotes that these three interface-and-class arrangements belong to a single component.

The diagram indicates the relationship among the three clusters with a bullet-anchored connection between the interfaces. Consider two specific connections. Every implementation of `IMap` must come with a method that delivers the domain of the finite map as a set, that is, a set of keys in active use. In turn, every implementation of `ISet` has a method for generating an iterator for the elements of the set.

A connection like this one clearly must raise a number of questions during an inspection. Here is how the Java team might have conducted such an inspection, starting after the presenting implementer has wrapped up the overview:

Thanks for the overview. How does an iterator pick elements from the set?	The choice depends on the concrete implementation of the set.
Can you give examples?	A set implementation based on a tree representation can create an iterator that delivers the elements in some guaranteed order.
And?	If a set implementation uses hashing, the order is not guaranteed.
I think this explanation of what is guaranteed and what isn't should make it into the interface description.	Agreed. Whoever maintains this package in the future needs to know whether an iterator promises such a guarantee.

The conversation once again clarifies that an Interface is a contract between the producer and, in this case, an imagined, external consumer. On the production side, a future maintainer must understand the Interface well

enough so that its promises will be kept. And on the consumption side, a developer must understand what an Interface does and doesn't promise.

In response to this inspection, the interface designer may add something like the following statement to the component's Interface document:

An iterator for a set may or may not guarantee the order in which the elements are presented. Concerning these guarantees, see the documentation of the concrete implementations.

The chosen component supplies a second example of a cross-class connection that is even more complex than the first one. Concretely, the Interface must relate all three classes to state when an iterator works properly and when it might try to signal an exception. This part of a design inspection takes a bit longer:

I have another question about iterators. Each iterator seems to be tied to the collection that creates it.	This is correct.
And a collection might belong to a class such as Map.	Indeed, Map comes with a method for generating an instance of Set for its keys.
This is precisely what I am getting at. I am imagining code that mutates the instance of Map after the iterator is created but before its hasNext method is called.	I don't follow this. Can you sketch an example on the white board or on your laptop?

Note how an inspection might benefit from written-down examples, which suggests that future readers should see such examples.

Suppose the panelist may show the presenters this code snippet:

```
HashMap<String, Integer> map = createSomeHashMap("a", 1, ...);
Set<String> keys = map.keySet();
Iterator<String> it = keys.iterator();
map.remove("a");
while(it.hasNext()) {
    String key = it.next();
    ...
}
```

The bold-faced line is what the panelist is going to point at:

The first three lines set up some <code>Map</code> , generate the underlying set of keys, and then derive an iterator.	Understood. It is clearly the fourth line that causes a problem.
It should. If "a" is the only key, is the generated set empty or does it still contain this one key?	Since the point of the iteration is to go over all keys that are associated with a value in <code>Map</code> , the set should be empty.
But what if there are other keys? Should <code>next</code> succeed?	It shouldn't. Once the underlying <code>Map</code> is mutated, we cannot guarantee that a derived iterator works properly.
Does this mean all derived iterators are invalid after a <code>Map</code> mutation?	Not quite. Let's change the example a little bit.

It is back to the white board, where the interface designer swaps two lines:

```
HashMap<String, Integer> map = createSomeHashMap("a", 1, ...);
Set<String> keys = map.keySet();
map.remove("a");
Iterator<String> it = keys.iterator();
while(it.hasNext()) {
    String key = it.next();
    ...
}
```

Oh, I see. The iterator is generated after the key is removed.	It is, and now the program has a well-defined behavior.
Why? Didn't the set get generated before the key is removed from the set.	No, the word "generate" is wrong here.

In what way?	The set literally <i>is</i> the collection of all keys in the instance of <code>Map</code> .
And if a key gets removed?	The set changes.
So the iterator relies on the actual instance of <code>Set</code> ?	It does.
Then I have one more question. But I need to change the example again.	Go ahead.

This time the panelist adds a line of code:

```
HashMap<String, Integer> map = createSomeHashMap("a", 1, ...);
Set<String> keys = map.keySet();
Iterator<String> it = keys.iterator();
map.remove("a");
Iterator<String> it2 = keys.iterator();
while(it2.hasNext()) {
    String key = it2.next();
    ...
}
```

And the inspection can be resumed:

If the client code derives two iterators, one before the key gets removed ...	... and one after. I see. This makes sense.
How?	The second iterator consults the set of keys after the removal happened.
Meaning?	We know exactly how to make it iterate over each element in the set.
This is complicated.	The Interface clearly needs to explain all this.

While this inspection is entirely imagined, several such conversations should have taken place as the developers of the `utilities` component reviewed their design. One conclusion from this last conversation is that the Interface document needs a clarification about these relationships:

A collection generated from an instance of `Map` is a live image of the underlying keys. When the `Map`'s keys change, the collection changes. Hence, the methods of an iterator work properly only as long as no code mutates the underlying map. Once this map is modified, the behavior of the derived iterator is undefined.

As an alternative to the “undefined behavior” language, the specification might require an implementation of the iterator to signal an error, like Java's `ConcurrentModificationException`.

The big question is where such specifications belong. That is, the team should ponder

- whether there is a Interface for the entire component, which can then state such constraints relating to three distinct elements; or
- whether the developers put such an explanation into the Interface of the most appropriate individual piece inside the component.

For library components such as `utilities`, developers seem to use the latter approach. In this particular case, the Interface of `Map` may alert readers that the result of the `keys` method is a live image and that this affects iterations over this set. By contrast, for a bespoke component such as those discussed in the following section, it is best to write an Interface for the entire component that explains cross-cutting constraints (plus promises) and with Interfaces for its pieces, which explains the pieces of functionality and local constraints (plus promises). The kinds of constraints imposed are just too complex to show up in some subordinate document.



Take a look at the media player interface for Androids.

Reading

Joshua Bloch. *Effective Java*. Pearson Education, Limited. 2001.—Bloch designed Java's API. His book and many lectures based on this book explain his experience and the lessons he has drawn from this experience. Chapters with titles such as “Favor Composition Over Inheritance” and “Favor Functions, Favor Immutability” clearly indicate how Bloch views interface, component, and method design in hindsight.

15.5 A Component as an Interface; Evaluating Alternatives

On some occasions, an entire component serves as an Interface between two other components. Section 10.4 mentions the idea in the context of the game-server project and the interactions between the referee on the server side and the player on the client side. The interface must be an entire component because a full understanding of the interaction between the two parties requires an understanding of several data representations and their relationships: the game state and the rule checker plus the underlying data representations of the game pieces. Studying a component-as-an-Interface also provides an opportunity to see how designers choose from alternatives. That is, which alternatives they consider and which rationales they offer for their decisions.

Stop! Re-read the inspection of the propoos plan in section 11 before you continue reading the rest of this section.

```
// how the referee interacts with the player
interface Player = {
  type TurnAction = ...

  // the player receives the map and some destinations &
  // returns the destinations it will not try to connect
  startGame :: GameMap Destinations -> Destinations

  // the player is informed of the current
  // publicly visible state of the game and
  // returns a description of the action it wishes to take
  takeTurn :: TurnState -> TurnAction

  // the player is given additional cards
  // if the call to takeTurn requested them
  moreCards :: Cards -> Void

  // the player is told whether it is a winner
  endGame :: Boolean -> Void }
```

Figure 29: A *restful* interface for the player-client

Considering the critical role of the `Player` interface, its pieces deserve a thorough inspection. Figure 29 displays a first draft with four named pieces of functionality. Each comes with a signature that lists the input types,

followed by `->`, and a result type. Each of them is also accompanied by a purpose statement, a comment that explains *what* (not *how*) it computes. Let's see how the designers present it to a panel:

The interface requests that our hackers and clients implement four methods.	How did you decide on these four methods?
Each method belongs to one of the basic phases of a board game.	Obviously, <code>startGame</code> is about setting up the game. How about the others?
The <code>takeTurn</code> method hands the player component the publicly visible game state. We decided to call it <code>TurnState</code> , because it is the state of the game relevant to taking a turn.	And what is <code>TurnAction</code> ?
By returning a <code>TurnAction</code> , a player requests that the referee component perform certain actions on its behalf.	This is good. It means the referee can ensure that these actions are performed only if they are legal.
Correct.	But does <code>TurnAction</code> represent all the information that matters?
It does. We decided that a player gets to know everything it needs when it is its time to take a turn.	Why did you go with this kind of arrangement?
What we have in mind is a RESTful protocol. That way we can easily test our own players and hackers can test theirs.	How?

A RESTful protocol is a protocol for plain function calls, and functions are much easier to test than imperative methods.	This seems to make sense. Let's look at the rest of the methods for now and return to this point later.
---	---

Notice how this inspection forces the presenter to clarify the timing of player notifications relative to game-state changes. The method documentation explicitly states that

the player is informed of the current publicly visible state of the game

which differs quite a bit from everyone's personal experience with playing board games. Based on this experience, the panelist may have expected to see somewhere in the interface that

the player is informed of any change to the publicly visible state of the game as soon as it happens.

and that's why a panelist might have had reservations about. Another one is that a panelist may have imagined a serious hacker-client. Such a participant in the imagined tournaments might want to exploit "immediate notifications" for some form of speculative computation of future turn actions. And it is the panel's task to represent all kinds of developers of the client-side components.

Exercise 13. Explain the roles of `moreCards` and `win` in the interface of figure 29 as if you were a presenter.

Since the remaining two pieces of functionality in this interface don't deserve much consideration, let's skip their inspection and focus on the panel's imagined reservation instead. Discussing alternatives at this point is quite natural because the team is about to settle a critical, architectural-level element. So, let's see how the members of this well-practiced panel return to this point in the inspection and revisit the architects' decision:

...	Can we return to the <code>takeTurn</code> method and your decision to go with a RESTful protocol?
-----	--

We considered two extreme alternatives before we made the decision to go with a RESTful protocol.	One is clearly the functional one. What else did you consider? What do you mean by “extreme” here?
Let’s start with the two dimensions of the decision: time and data.	Does “time” refer to <i>when</i> a player is informed?
Yes, and “data” means what the server-side component sends to the client.	I can see where this is going.
Our functional variant combines two extremes: it delays informing a player about the changes to the state of the game until it is its turn again; and when it is its turn, it sends over the entire new game state.	You mean the public game state?
Correct. And that’s why it is a “functional” protocol. The <code>takeTurn</code> method can compute the desired action from just the given information.	Doesn’t it need the <code>GameMap</code> and the <code>Destinations</code> ?
The referee could include those in the <code>TurnState</code> , or we could agree that these two pieces of information are essentially global constants.	Okay. Sending them over with the first method suffices. Thanks for clarifying why this proposed <code>takeTurn</code> method is functional and why you like it. It is easily testable.
Good. Ready to move on to the alternative?	Yes.

The rest of the inspection might proceed like this:

```

interface player =
{ type TurnAction = ...

    // the player receives the map and some destinations &
    // returns the destinations it will not try to connect
    Destinations startGame(GameMap gm, Destinations avail)

    // the player is informed of any change to the publicly
    // visible state of the game as soon as it happens
    Void inform(TurnState gs)

    // the player is told to take a turn
    // returns a description of the action it wishes to take
    TurnAction takeTurn()

    // the player is given additional cards
    // if the call to takeTurn requested them
    Void moreCards(Cards c)

    // the player is told whether it is a winner
    Void endGame(Boolean winOrLose) }

```

Figure 30: A *stateful* interface for the player-client

Here it is (shows figure 30).	Could you summarize it in terms of your “extremes?”
The server-side component informs all players right after one player has taken a turn, and it does so by sending the entire TurnState.	Why does it send the entire new state and not just the action that the player wishes to take?
There are several kinds of changes that can happen.	True. A player may choose different kinds of actions.
Even worse, a player may fail to respond, in which case it is going to be eliminated.	This makes a lot of sense. Sending the entire state covers all possibilities.

I have bold-faced the two critical differences to the first alternative so we can discuss them.	One of them is the new function for informing the players about other players' actions. But why does <code>takeTurn</code> no longer consume a <code>TurnState</code> ?
Right before a player is granted a turn, it has received the most recent <code>TurnState</code> via a call to <code>inform</code> .	Yes, but isn't this kind of subtle? How should an implementer of the interface know?

The panel's questions point to an issue that no previous inspection exposed. While the alternative `takeTurn` does not consume the current `TurnState`, the player will get the relevant information. As the presenter states, a call to `inform` with the relevant `TurnState` data immediately precedes the call to `takeTurn`. In sum, the temporal relationship among method calls plays a critical role—and the textual representation does not clarify it. A design inspection must pick up on this concern:

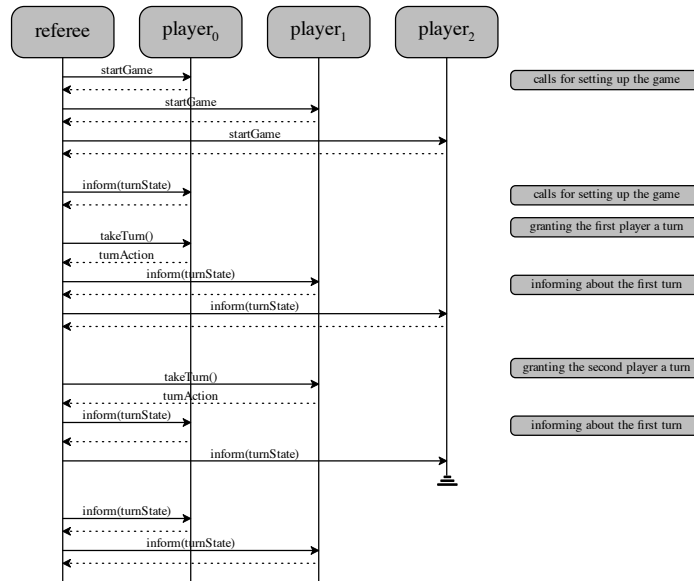


Figure 31: A diagram illustrating stateful interactions from figure 30

We figured we'd need to state this relationship among these two pieces of functionality differently, so we doodled a diagram (shows figure 31).

Can you walk us through this diagram?

The components are listed at the top.

Do the vertical lines indicate execution time?

They do. And calls go from one life line to another.

According to your notes on the right, the first three arrows show how the referee sets up the game.

They do, and the dotted lines show that they return `Void`.

So the next arrow indicates that the referee informs the first player of the game state.

Yes.

But why?

During the setup phase, a player might not respond in a timely fashion and has to be eliminated.	So this call shares the state of the game with the first player so that it can decide which action to take.
That's exactly right.	Isn't it true that the server-side component could inform the players after each call to <code>startGame</code> ?
It could and it would increase the regularity of the interactions.	So why not?
This would come at the cost of a bit more network traffic.	OK. The first player must know whether the other players are in the game, so what you have is probably fine.
Now the next three arrows represent a single turn.	The first arrow is the call to the first player's <code>takeTurn</code> method.
And the remaining two <code>inform</code> the other participants.	Okay, this all makes sense now.

As this inspection dialog shows, doodling such diagrams allows the presenters to interact with the panelists in an effective manner. Since panelists represent potential “clients” and future readers of the code, this suggests that such diagrams—inspired by the UML sequence notation—are better at bringing across temporal relationships among calls than plain text. Given how common such relationships are, it is good practice to supplement the textual specifications with such “doodle diagrams.” At a minimum, a photo of such doodle art should accommodate the README file as the project evolves. Or have your AI agent draw a diagram and deposit it in your code repository.

It is unnecessary to follow a particular standard, such as UML.

Let's move on to another interesting part of the diagram:

One more thing. Check the bottom of the <code>player₂</code> lifeline	Right, what does the triangle mean?
It indicates that the client component terminates, possibly in an irregular fashion.	That seems to be a special case.
We wanted your opinion about it.	Does it mean <code>player₂</code> does not respond during a call to <code>inform</code> ?
Precisely.	And do the next two arrows mean <code>player₀</code> and <code>player₁</code> find out about <code>player₂</code> and its unresponsiveness?
As we said before, we were thinking that such a player should be eliminated. The question we have is whether the others should find out immediately.	Yes like we already agreed for the setup phase of the game, this seems to be a reasonable decision.
Good.	So let's confirm. The key invariant is that at the bottom of the sketch of a full turn, both players have the same information.

This last part of the interaction also shows how they can help with disambiguating potentially ambiguous situations. Even if all components exist within a single program in the same programming language, a bug in the `Player` component could bring down the entire system. Similarly, a remote `Player` component may become unresponsive due to other reasons and cause problems to the overall system. Assuming the team has decided that such single-component failures should not terminate the game server,

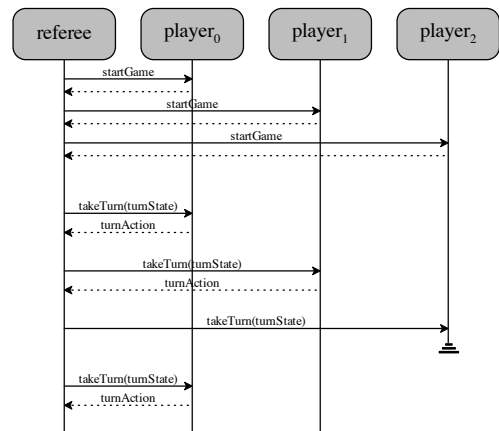


Figure 32: A diagram illustrating RESTful interactions (figure 29)

it should now inspect the specification of the `Referee` component to ensure the developers know how to deal with such situations.

With these sequence diagrams, the designers can also contrast the two alternatives in some detail:

	Do you have a doodle diagram for the RESTful alternative?
Yes, we have such a sequence diagram for the functional alternative (shows figure 32).	Is it worth comparing the two?
It is quite instructive to compare them, especially the failure scenarios.	What can fail here? The <code>Player</code> doesn't have an <code>inform</code> functionality.

True, but any method can fail.	Of course. It looks like you worked through a failure of the <code>takeTurn</code> method to <code>player₂</code> this time.
Exactly. Notice how the lifeline of <code>player₂</code> ends during the call, before it returns a <code>turnAction</code> .	Is it correct that the next call of the <code>takeTurn</code> method to <code>player₀</code> delivers this knowledge about the elimination of <code>player₂</code> as part of <code>gameState</code> ?
It does. And you probably guessed that <code>player₁</code> has not found out yet.	Yes, that's clear. You should add these diagrams to the repository and point to them from the README file.
That's a good idea. We will add a section that discusses the design alternatives, and the diagrams will help people understand why we chose the RESTful one.	And add an explicit rationale section for the design decision. People still don't get that functional interactions are so much easier to test than stateful ones.

Stop! Reflect on the *style* of the design inspection dialog and its content.

Stop! Even if you work with AI, it is a good idea to consider and document alternative designs and document the rationale behind the decision.

Exercise 14. Consider the following scenario. The `referee` successfully calls a method of `player1` but the player fails after it returns a `takeAction` result. When do `player0` and `player2` find out according to the two alternative interfaces? Sketch sequence diagrams.

Exercise 15. The design inspection has demonstrated that the interface of figure 29 differs from the one in figure 30 in significant ways. Write down a concise explanation of an advantage or disadvantage to the developer of the game-server software, and similarly, articulate an advantage or disadvantage to the developer of the player clients.

16 Expressing an Interface

How do developers write down interfaces?

The practice of writing down Interface documents has remained the same for decades. Most of the time developers use a mix of programming-language elements and (usually) rigorous but informal write-ups in natural languages. Developers borrow these formal elements from the chosen implementation, most often the syntax of interfaces, method headers, and types—depending on what is available. The write-ups are needed for two different reasons. First, such write-ups help consuming developers with comprehension, because they communicate ideas in human terms. Second, most programming languages simply lack expressive power when it comes to Interfaces, meaning many constraints and promises from the informal write-ups cannot be expressed in the chosen programming language.

When it comes to the informal write-ups, the only way to assist server-component developers with their articulation is to frequently inspect and review these products. For external interfaces, it is best for the developers to write them or to closely work with the person in charge of writing the Interface. Indeed, once a team considers an Interface ready for release, it should ask an outside developer—someone who is unfamiliar with the component—to program some or all of the intended use cases. This exercise is going to provide essential feedback on the Interface’s (lack of) clarity and completeness.

As for formal elements, developers have to accept what the chosen language offers. Some mainstream languages, such as JavaScript and Python, come without any support for expressing interfaces. Others, such as C# and Java, supply `interface` constructs, which can express some basic type and subtype properties. As this chapter illustrates, however, Interfaces tend to state many more critical properties than types: *obligations* that clients must meet and *promises* that servers realize.

Expressing properties beyond types and subtypes is a topic that has received a lot of attention in the design of alternative programming languages, such as Eiffel, Racket, and Spec#. In these language communities, the topic is known as interfaces with *contracts*. Contracts come in many flavors: enhancements for typed method signatures; constraints on the objects a method consumes; constraints on multiple method calls; and even constraints on method calls across several classes.

This section focuses on the formal elements of Interfaces. It first briefly presents the conventional type-based view before moving on to method-level contracts—with a heavy emphasis on how the producing and con-

suming developers benefit from them. The remainder of the section covers some advanced contract system that the author of the book has worked on.

Reading

A. Beugnard, J.-M. Jézéquel, N. Plouzeau and D. Watkins. Making components contract aware. *IEEE Software*, June 1999.—This somewhat academic paper presents the classification of contracts used in this section. Beyond the functional properties mentioned here, the paper also mentions ideas concerning performance and security properties.

16.1 Existing Practice: Types, Subtypes

Well-trained or experienced developers use as much as possible from the chosen programming language to express the interface of a component. And, to this day, they supplement it with an informal document that describes the purpose of the functionality (not its mechanism), constraints on the functionality, and any promises it makes. If these interfaces are visible to external, unknown programmers—such as the Interfaces of Java’s utility class—developers work with technical authors to polish these documents.

The creators of the component and the client programmers mostly rely on the formal parts, though. The former tends to write code that realizes the promises of the interface and often exploits the constraints on the inputs. The latter ensures that their client code meets the type obligations of the interface to which they program. Developers like these formal elements of interfaces because compilers and IDEs check their code’s consistency with them. AI agents even make coding suggestions based on these specs.

For simplicity, let’s make this description concrete with examples from the domain of coordinates. Consider the following simple Java interface:

```
// a data representation of points
// in the 2-dimensional Cartesian plane
interface IPoint2D {
    // computes the distance of 'this'
    // coordinate to the given one
    // the distance is positive
    double distanceTo(IPoint2D other);
    ...
}
```

While the input type, `IPoint2D`, describes the natural possibilities, the result type of `double` is necessarily imprecise. Since the language does not offer a non-negative numeric type, it is the best the creator can do, even though everyone knows that the method cannot return a negative number.

Both the producer and consumer of the interface benefit from stating these formal elements. When a developer creates a class that implements `IPoint2D`, the IDE's type checker enables it to suggest appropriate methods on `other` during the implementation of the method. Similarly, when a developer creates a client component, the IDE checks whether the input to a call to `distanceTo` implements the `ICoordinate` interface. Furthermore it informs the developer that the result is a `double` so that it isn't accidentally used in an integer context.

In addition to types, mainstream object-oriented languages also enable interface developers to state sub-typing relationships, which languages and IDEs can check. The benefits are similar for all developers. Sadly, the *ideal* sub-typing relationship is different from the advertised language mechanisms (e.g., Java's `extends`). While a developer could specify that any interface extends any other interface, say `IApple` extends `ITree`, creating such arbitrary relationships is meaningless.

Behavioral sub-typing is the technical name for the ideal relationship. Its definition is reasonably straightforward, consider an object `O` that implements an interface `I`. Furthermore assume that `I` extends `J`. In this context, `O` should behave exactly like any object that merely implements `J`. Surprisingly many coding examples violate this condition and thus undermine responsible coding practices.

Let's take yet another look at the `ICoordinate` interface from section 14 whose purpose is to represent neighbor relations on a game's map:

```
interface ICoordinate {
    Coordinate above();
    Coordinate below();
    Coordinate left();
    Coordinate right();
}

interface ICoordinateDebug
    extends ICoordinate {
    // draw 'this' coordinate
    // into the given drawing context
    void paint(IDrawingContext dc);
}
```

The original interface is on the left. Since a developer often wishes to visualize points, adding a method for drawing an object of type `ICoordinate` is a behavioral sub-type of this original interface. Using `extends` for an appropriate interface, as shown on the right, is appropriate.

Here is a counter-example, an interface for points in a 3-dimension Cartesian space that mistakenly extends the one for 2-dimensional points:

```
// a data representation of points in the 3D Cartesian space
interface IPoint3D extends IPoint2D {
    ...
}
```

By the semantics of Java, inherits the `distanceTo` method, which ignores the third dimension and is thus not what is expected.

Pretending that `ICoordinate3D` is a behavioral sub-type of `ICoordinate2D` misleads both the developer of any class that implements the interface and the developer who programs to this interface. The former has to re-implement the `distanceTo` method, even though it seemingly could be inherited from a class that implements `IPoint2D`; a 3-dimensional point's distance must take into account all three coordinate numbers, not just two. The latter may accidentally send an object of type `IPoint3D` into a context of type `IPoint2D`, but `distanceTo(Origin)` is going to produce an unexpected, indeed a wrong, result, because of the third coordinate.

Generally speaking, abusing sub-type relationships may cause methods to break their implicit promises in silent, and thus insidious ways. Part of the problem is of course that these implicit promises—in the name and/or the purpose statement of a method—aren't stated in a formal way and thus aren't checked. How developers may overcome this problem is the topic of the next section.

Note Programmers often create such mistaken sub-typing relationships to re-use pieces of functionality, especially when classes are involved. A better way to achieve this goal, without misleading the software's maintainers, is to use class composition.

Reading

Joshua Bloch. *Effective Java*. Pearson Education, Limited. 2001.—As mentioned in section 15.4, Bloch urges developers to “Favor Composition Over Inheritance,” i.e. implement code re-use via class composition not class inheritance.

16.2 Contracts for Individual Methods

Some languages come with specification constructs that go beyond types. Bertrand Meyer's Eiffel was the first of these languages to make it into the commercial mainstream. The language allows developers to equip each method with pre-conditions and post-conditions. Both are Boolean expressions involving the parameters and the results of the method. The former ensure that any caller respects publicly stated obligations; the latter enforce that the method's return values lives up to the also explicitly and publicly stated promises. Microsoft's Spec# language and the Racket programming language also come with extensive contract systems; while Microsoft's de-

sign pushes partial static verification of contracts, Racket follows the Eiffel example and checks contracts during execution.

The `distanceTo` example from the preceding section makes for an illustrative example. Its creator states, informally, that the method always returns a non-negative number. In Eiffel, this promise is added to the method signature:

```
class IPoint2D
  double: distanceTo(IPoint2D other)
  do
    ... this.x ... other.x ...
    ... intentionally elided ...
  ensure
    Result >= 0
  end
```

When the program is run, Eiffel will evaluate the `ensure` condition every time this `distanceTo` method is about to return a result. If the condition evaluates to `false`, Eiffel signals an error with a message that informs the developer about `distanceTo`'s failure.

The Eiffel community uses the word “contract” for the combination of pre-conditions and post-conditions around a method. Indeed, the community insists on developing methods by first writing down just these conditions and to fill in the method body afterwards. Unsurprisingly, this approach to software development is dubbed “design by contract.”

Behind the “design by contract” approach lies the recognition that Interface specifications matter to the creator of an implementation, the future reader of this code, and the developer that programs to an interface—not to speak of the people whom the software should benefit:

- Any developer who tests the software benefits. While Interfaces will always consist of a combination of formal code with informal comments, only the formal parts can be checked. The informal comments don't undergo any mechanical validation.
- The creator of an Interface implementation can assume that the pre-conditions are satisfied during the development of the method body. Similarly, the developer can check the post-condition for hints on how the method body may compute the result.
- A future reader of such an implementation will encounter a clean separation of concerns between the method's body plus the assumed obligations and the promised properties of the result. This separation is impossible in a language that does not support contracts.

End-users may also suffer when an Eiffel program signals a contract violation, which is why this last issue isn't clear-cut.

- Developers of client code may not recognize that calls to certain methods must meet certain preconditions, because they are buried inside of native-language comments.
- So-called end-users benefit less directly than the various developers from the first four items, but they do. Imagine a piece of software that violates any of these pre-conditions or post-conditions and, as a result, computes a type-correct but wrong answer. The end-user may suffer quite some harm due to such results.

Let's look at these points using concrete examples, item by item.

A well-trained Java programmer would use javadoc strings not just a line comment.

The first example makes an argument for the first two points. Suppose a team's task includes the development of `Timer` classes, including the design of a method for setting the seconds of an instance. Here is a fragment of the appropriate Java interface:

```
interface ITimer {
    // sets the timer to run for `s` seconds
    // ASSUME `s` is in [0, 60)
    void setSeconds(int s);

    // starts the time to count down from `seconds` to 0
    void startTimer();
    ...
}
```

Following convention, this interface mixes code with informal comments.

Now consider an implementation that does not check the informally stated assumption:

```
class Timer implements ITimer {
    private int seconds;
    public void setSeconds(int s) {
        this.seconds = s;
    }
    ...
    ...
}
```

If some client code were to call `setSeconds` on 61, followed by a call to `startTimer`, a developer might not notice the problem at all, because the timer will expire, just not in the promised time. Similarly, setting seconds to -1 may cause strange problems. Due to Java's modular integer arithmetic, a test will eventually terminate but take much longer than expected. An end-user might also suffer harm in this case, due to the extra-long wait

time. Still, the computation proceeds in a type-correct manner, because the type system can't express this constraint on the method's single argument.

An experienced Java developer is likely to add a run-time check to the implementation of the `setSeconds` method:

```
class Timer implements ITimer {
    private int seconds;
    public void setSeconds(int s) {
        if (! (0 <= s && s < 59)) {
            throw new IllegalArgumentException("...");
        }
        this.seconds = s;
    }
    ...
}
```

In turn, a one-line method turns into a three-line method, burying the one line that matters. Due to Java's nature, though, a developer can't do anything else.

Software engineers refer to this injection of run-time checks as *defensive programming*. Since all mainstream languages have limited ways of expressing Interfaces (of methods, classes, modules), instructors have taught this form of coding for a long time. Defensive programming imposes extra cognitive work on all readers, however. While the `setSeconds` method remains readable, imagine encountering a method that consists of forty lines of code, co-mingling twenty lines of defensive checks and twenty lines of computational code. Eiffel's contracts are a response to just this kind of observation.

The Eiffel language aims to empower developers to cleanly separate the "workhorse" code from the run-time checking code in method definitions:

```
set_seconds (s: INTEGER)
  require
    valid_argument_for_seconds: 0 <= s and s <= 59
  do
    seconds := s
  end
```

When developers read this code in the future, this separation will help them hone in on the relevant parts. The keywords (`require`, `do`, `ensure`) and indentation clearly indicate which part performs which computation.

While Eiffel is a step in the right direction, the lack of a distinct interface construct is still an obstacle for some readers. Specifically, a developer of client code must find the actual method implementation to read up on its pre-conditions and post-conditions.

Class inheritance makes it even harder to find the exact set of conditions associated with a method.

```

#; {PositiveInt PositiveInt Real Real -> Image}
#; {(create width height channels colorspace)}
;; creates a `width` x `height` image with ...
;; ASSUME .. `width` .. `height` .. `channels` .. `colorspace` ..
(define (create width height channels colorspace)
  (unless (and (integer? width) (> width 0))
    (error 'create "expected width > 0: ~a" width))
  (unless (and (integer? height) (> height 0))
    (error 'create "expected height > 0: ~a" height))
  (define size (* width height))
  (unless (<= size P-MAX)
    (error 'create "expected width * height <= ~a: ~a" P-MAX size))
  (unless (and (real? channels) (<= 3 channels 4))
    (error 'create "expected channels in [3,4): ~a" channels))
  (unless (and (real? channels) (<= 0 colorspace 1))
    (error 'create "expected colorspace in [0,1): ~a" colorspace))
  (define load (make-bytes (* 4 size)))
  (private-image width height channels colorspace load))

```

Figure 33: Defensive Racket code

The Racket language supports a complete separation of a module's Interface and the classes and functions that it supplies, and the rest of this section uses Racket to illustrate how Interfaces can express many informal elements with code. To begin with, take a look at the code in Figure 33, which displays a simple piece of functionality from an image library.

Stop! Can you spot the computation that the method actually performs?

An inspection shows that the `create` function consists of eight lines of basic checks, one more check that require a tiny bit of calculation, and three lines of actual code, all of them co-mingled into one big collage.

Without question, such defensive code is difficult to read for a maintenance developer. If a module consists of dozens of such function definitions, it is easy to imagine how a developer might spend a large amount of time teasing the defensive checks out from the “workhorse” code.

A Racket programmer can move these defensive run-time checks into a module interface. Figure 34 shows how to impose the basic checks via a module-level contract:

- The `provide` form exports a function, and `contract-out` attaches a contract to the `create` functions.

*The Racket keywords
provide and
require specify
what a module exports
and imports,
respectively.*

```
#lang racket ; server module, version 1

(provide
 (contract-out
  (create
   (-> (and/c (>/c 0) integer?)
        (and/c (>/c 0) integer?)
        (and/c (between/c 3 4) real?)
        (and/c (between/c 0 1) real?)
        image?))))

(define (create width height channels colorspace)
  (define size (* width height))
  (unless (<= size P-MAX)
    (error 'create "expected width * height <= ~a: ~a" P-MAX size))
  (define load (make-bytes (* 4 size)))
  (private-image width height channels colorspace load))
```

Figure 34: A Racket Interface

- A function's contract has the shape

```
(-> domain_1? ... domain_n? range?),
```

where `domain_1? ... domain_n?` are predicates that state and enforce a function's obligations and `range?` is a predicate that expresses a function's promises and checks them as the function returns a result.

- Racket supplies a number of easy ways to formulate predicates for contracts: `(>/c 0)` checks that an argument is a (real) number and positive; `(between/c 3 4)` checks that a (real) number between 3 and 4; and `(and/c c1 c2)` combines two contracts into one such that both must hold.

Stop! Read the contract in figure 34 carefully and compare it with the checks in figure 33.

The bottom of the figure shows the contract-protected `create` function, which consists of just five lines of code. Of these lines, two are run-time checks that really belong into a contract. Over the years, Racket's contract system has evolved to accommodate such checks, too. Take a look at figure 35, which displays a second version of the contract for `create`.

```
#lang racket ; server module, version 2

(provide
 (contract-out
  (create
   (->i ([width      (and/c integer? (>/c 0))]
         [height     (and/c integer? (>/c 0))]
         [channels    (between/c 3 4)]
         [colorspace (between/c 0 1)])
   #:pre/name (width height) "area too large"
   (<= (* width height) P-MAX)
   (result image?))))))

(define (create width height channels colorspace)
  (define load (make-bytes (* 4 width height)))
  (private-image width height channels colorspace load))
```

Figure 35: A Racket Interface, with named pre-conditions

This second contract for `create` names each parameter of the function: `width`, `height`, `channels`, and `colorspace`. On the same line, it associates each function parameter with the same contract predicate as the first version. Critically, using these names, the contract also states a named pre-condition: "area too large". This pre-condition ensures that the product of `width` and `height` is less than some (presumably publicly visible) constant `P-MAX`. If the condition yields `false` during execution, the contract system uses the names of the function and the contract to report the contract violation; the error message includes the given argument values. As the bottom of the figure shows, the second version of the `create` function now consists of two lines.

Any reader of the contract-equipped versions of this module is better off than a reader of the module in figure 33. The contract cleanly states the non-trivial obligations that a caller must meet; hence, a client programmer doesn't need to consult the function definition at all. Similarly, the client programmer finds out what the function promises, here, that the result satisfies the `image?` predicate. Finally, a maintenance developer looking for a bug in the image-creation code can focus on the two lines in the function.

Let's make the issue of a contract violation concrete, to illustrate how a maintenance developer can start the debugging process. Here is the error

message when a client module calls `create` with bad arguments:

```
area too large
  width: 11
  height: 21
in: (->i
      ((width (and/c integer? (>/c 0)))
       (height (and/c integer? (>/c 0)))
       (channels (between/c 3 4))
       (colorspace (between/c 0 1)))
      #:pre/name
      (width height)
      "area too large"
      (<= (* width height) P-MAX)
      (r natural?))
contract from:
  (server2.rkt)
blaming: client.rkt
  (assuming the contract is correct)
at: server2.rkt:39:5
```

The message uses the name of the failed pre-condition to clarify what went wrong; it displays the broken contract, including the parties that agreed to the contract; and it blames the module that sent a bad value into the contract. Every tester can easily formulate a bug report based on this message or even start the search for the broken code in `client.rkt`.

In sum, even contract systems that merely separate pre-conditions and post-conditions from function definitions assist developers with the task of sending code from one person to another across time.

Note Here is what mainstream programmers can do to separate the defensive part of a method from the workhorse:

- write a public method that performs all checks;
- write a private method that performs the actual work;
- have the first method call the second, after all argument checks pass;
- check the result before returning it.

This arrangement comes close to the Eiffel approach in cleanly separating pre-conditions and post-conditions from the code that computes results, at the cost of making the developer implement a pattern. It does not have the advantages of Racket's contract system, which puts the contracts into a module's interface, thus separating the interface itself from the function definitions.

Exercise 16. Which parts of the contracts in figure 35 can be expressed as types and thus checked at compile time (in a mainstream language), and which parts are predicates that must be evaluated when the program runs?

Reading

Bertrand Meyer. *Object-Oriented Software Construction*. Prentice Hall. 1988.—This book is an expansive presentation of Eiffel contracts, one of the earliest object-oriented programming languages. It explains why separating contracts matter, how to write them, how to understand them, and how they benefit working developers. Although Meyer’s contracts are somewhat simplistic from the perspective of the 2020s, this presentation remains one of the easiest introductions to this topic.

16.3 Contracts for Individual Objects

While Eiffel’s contracts allow developers to state the pre-conditions and post-conditions of individual methods, they are only about simple values. By contrast, Racket’s contract system can express and check conditions on higher-order values. Roughly, a higher-order value comes with behavior. Objects and functions are concrete and common examples of higher-order values. Since most contemporary programming languages come with higher-order values, and since the construction of software systems involves such values, it pays off to take a look at what a contract system can accomplish in such contexts.

We will use the sample project from section 10, *Ticket to Ride*, to develop a basic understanding of object contracts: their role in a system; their essential pieces and the nature of these pieces; and their effectiveness in uncovering bugs. Recall that the game is played as follows:

- The setup phase consists of two steps. First, the players are handed a map and other basic game pieces. Second, the referee hands each player a set of destinations—places on the map—and requests that it picks a subset of those.
- Once all players have chosen destinations, the referee grants players turns until some end-of-game condition holds. In the case of *Ticket to Ride*, a turn may consist of two steps: one for dealing with rails and one for receiving additional cards.
- When the game ends, the referee informs each player whether it won and returns the winning players to the tournament manager.

The software system is supposed to supply a server that waits for the players of “bored hackers” to connect and runs tournaments of *Ticket to Ride* with those players plus house players.

```
#lang racket
```

Common/player-interface.rkt

```

(provide player/c ...)

(require Common/maps) ;; imports map? and destination?
(require Common/cards) ;; imports cards?
(require Common/state) ;; imports turn-state?

(define player/c
  (object/c
    ;; hand the player the map for the game,
    ;; a number of rails, and some cards
    [setup (->m map? (and/c natural? (>/c 0)) cards? void?)]

    [pick (->dm [[given (set/c destination?)]]
                [result (set/c destination?)]
                #:post-cond (subset? result given))]]

    ;; grant the player the right to take a turn
    [play (->m turn-state? (or/c more? action?))]

    ;; if the preceding call to 'play' returned 'more?',
    ;; call 'more' to hand out more cards
    [more (->m cards? void?)]

    ;; inform the player whether it won (#t)/lost (#f) the game
    [win (->m boolean? void?)]))

```

Figure 36: A Racket player interface for the *Ticket to Ride* game

This sketch suggests that a player is an object that implements at least five methods: one for setting up the game, one for picking destination, two for taking turns, and one for finding out the outcome of a game. A translation of this informal description yields a player interface, that is, a specification-level description of the player object. Figure 36 displays this player interface in the form of a Racket object contract. Together with the game-piece modules, this player interface represents the common ontology

that connects the server and the player components. It also *is* the interface to the common-ontology component.

In Racket contracts are ordinary values, like numbers and strings, so giving this contract the name `player/c` is just fine. An `object/c` contract consists of contracts for methods, which roughly look like function contracts. Instead of `->` or `->i`, method contracts use the notation `->m` or `->dm`. The method contracts in `player/c` codify the informal descriptions in a straightforward manner. Consider the first method contract. It says that the name of the method is `setup`. Like the function contracts from the preceding section, it imposes an obligation on any callers, specifically, that they must pass this method a map, a positive natural number, and a bunch of cards. In turn, it promises to return `void`.

For the purpose of this section, the boxed contract for `pick` is the most interesting one. Like an `->i` contract, it names the parameter, `given`, and associates it with a predicate. The latter tells us that the referee must call this method with a set of destinations. Furthermore, the contract promises that the method returns a set of destinations, and, via the post-condition, that this result is a subset of the `given` destinations—just as the informal game description says. Its purpose statement informs us that the result denotes those destinations that the player will not use.

Based on this player interface, the work can proceed in parallel. One team can focus on the server-side component and another one on the client-side component so that the entire system can be tested. For testing purposes, this second team will produce players that do not live up to the agreed-upon interface. After all, the participating hackers may make mistakes or may intentionally probe ways of bringing the server down. Such tests explore whether the server-side component can be brought down by ill-behaved clients, a highly undesirable situation as chapter III points out. Conversely, it is critical that the server can *impose* the player-interface contract on player objects, independently of whether the creators of these objects attach this contract or not.

Which brings us to the first key difference to Eiffel's contract system:

In the context of higher-order values, a contract system must support the capability of attaching contracts to objects retroactively.

The subset condition is part of the rules of the game, and it should be checked by a rule-checking module. We use it here for illustrative purposes.

Racket's contract system supports several such capabilities. To understand how such a capability works, take a look at figure 37, in particular the boxed server-export declaration. The server is expected to consume a list of player objects. By specifying that this must be a list of `player/c` objects, Racket's contract system attaches this contract to each element of the list

```

#lang racket
Player/player.rkt

(provide player)

(define player ...)

```

```

#lang racket
Server/server.rkt

(provide
 (contract-out
  [server
   ;; determine the names of the winners
   ;; of a Ticket to Ride tournament
   (-> (listof player/c) (listof string?))]))

(require Server/referee)

(define (server house-players)
  ... (referee (append house-players sign-ups)) ...)

```

```

#lang racket
Server/referee.rkt

(provide
 #; [Listof Player] -> [Listof Player]
 referee)

(define (referee lop)
  ... (send a-player pick some-destinations) ...)

```

```

#lang racket
Tests/main.rkt

(provide main)

(require Server/server)
(require Player/player)

(define (main)
  (server (list .. player ..)))

```

Figure 37: A system sketch for a *Ticket to Ride* game

when the `server` function is called. Looking back at the explanation of the `pick/c` contract, this attachment of the contract obliges any caller of the player's `pick` method in the server component to supply a set of objects that satisfy the predicate `destination?` and to expect such a set in return.

The server itself will not run games, however. Its task is to accept sign-ups from external players and to organize a tournament of games once the sign-up period has expired. The participating player objects comprise both external player and "house players," that is, players constructed by the second team. When it comes to games, the `server` component is going to delegate this task to `tournament-managers` and `referees`. Figure 37 sketches this arrangement, including a `main` module that composes the player component with the server component.

Notice how the export declaration in the `referee` module does *not* impose a contract on the list of players it receives. Instead it comes with a type-like comment. The sketch also indicates that the referee function picks players (from the given list) and asks them to choose destinations. Now imagine a concrete method call like this one:

```
(send some-player pick {set "Boston" "Houston" "Tucson"})
```

This method call to `some-player`'s `pick` method uses a concrete set of destinations, the names of cities in the United States. The question is what should happen if the method returns `(set "Boston" "LA")`, which is clearly *not* a subset of the given destinations.

Once again, the error message from the contract system is informative but also surprising:

```
pick: broke its own contract
#:post violation, arguments were:
(object:player% ...)
(set 'Boston 'Tucson 'Houston)
result was: (set 'Boston 'LA)
in: the pick method in
    an element of
    the 1st argument of
(->
  (listof player/c)
  (listof string?))
contract from:
  (player-interactions.rkt)
contract on: server.rkt
blaming: (main.rkt)
  (assuming the contract is correct)
at: player-interactions.rkt:81:5
```

The first surprise is that the contract system catches the player's mistake even though the `referee` module does not impose a contract. And this is a second difference between Eiffel's contract system and one that deals with higher-order values:

In the context of higher-order values, a contract system must check conditions in modules different from those that attached them.

In this specific example, `server` imposes the contract on "house player" objects as they flow into the function from the `main` function in the test module. Put into the terminology of contracts, it is the `server` module and the `main` module that agree to the contract. And this perspective explains the second surprise, namely that it is the `main` module that gets blamed, not `player`. Generalized, this is the third major difference to Eiffel:

In the context of higher-order values, a contract system must support the capability of blaming a remote module boundary, i.e., a module that is not involved in the call-and-return interaction.

At this point, the test engineer must debug the system, starting with a close look at `main`, because it is blamed for the problem. We know that the module merely imports `player` objects and passes them on to `server`. By taking a look at this component's interface, it becomes clear that it protects itself against misbehaving `players`. In turn, this protection results in blaming whichever module uses the `server` function—the `main` module in this case. Furthermore, a look at the `player` component confirms that it does *not* claim to live up to the agreed-upon interface—something also to be expected by hackers who sign up remote players.

The test engineer can shift the blame by protecting the `main` module against ill-behaved `player` object. Since `main` is neither exported nor applied outside the module, the protective contract must be attached to `players` differently. To this end, Racket's module system enables developers to impose an interface on a module import, like this:

```
(require Server/server))
(require Common/player-interface)
(require (contract-in Player/player [player player/c]))
```

In other words, `contract-in` is yet another facility when contracts don't just check simple values but objects and functions. Once this change is implemented and the test is re-run, the contract system assigns the blame to the `player` module, which this situation calls for.

Exercise 17. The final system consists of clients spread across the globe, each of which registers a player or several players with the server. The latter creates local representations of these players dynamically and eventually hands them over to the tournament manager. Which component should impose the `player/c` contract on these dynamically created players and how?

Reading

Robert B. Findler and Matthias Felleisen. Contracts for Higher-Order Functions. In the *Proceedings of the International Conference on Functional Programming*, July 2002.—Findler and the author of this book were the first to research a contract system that overcomes the limitations of Eiffel. This research paper presents their results. They turned this research into a fundamental part of the Racket language. Many contributors have enhanced it since. Even more researchers have used this system to deepen the community’s understanding of how developers can benefit from contracts.

16.4 Beyond Individual Methods and Objects

Beyond pre-conditions and post-conditions for individual methods, Interfaces state many other interesting properties. Take a second look at figure 36 and notice the bold-faced word “preceding” in the method signature of `more`. This word clearly alludes to a temporal relationship among the methods within a single class. The example in section 15.4 goes even further and relates method calls among instances of distinct classes.

Trace contracts generalize object contracts for exactly the purpose of turning such informally stated properties into code within Interfaces. The word “trace” refers to a sequence of calls to methods of one object or several. A *trace contract* is a property of such a trace, roughly speaking, a function from traces to `Boolean`. This section sketches how trace contracts can express the two examples to give readers an idea of how developers can benefit from turning these informal constraints into code.

Figure 38 displays a so-called state machine. This state-machine graphically expresses the complete, though mostly implicit, temporal aspect of the Interface for `player` objects from the preceding section. The chosen presentation of a state machine consists of (1) a directed graph with labeled nodes and optionally labeled edges; (2) some nodes labeled as start states (bold border); and (3) some nodes labeled as final states (painted gray).

The idea is that the object is initialized to a start state and that a method call transitions the object to a connected state. The name of each node rep-

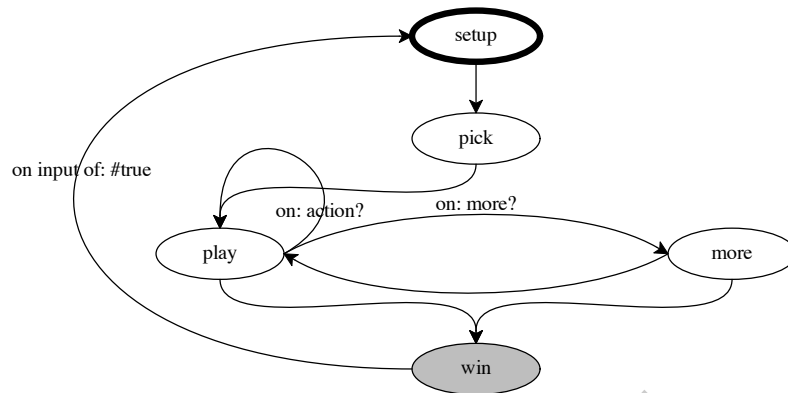


Figure 38: A state-machine contract diagram for figure 36

resents the method that must be called if the object is in this state. If such a state has a single outgoing edge, a call to the named method causes a transition to the connected node; otherwise, a method call may cause a transition to either of the connected states. Edges without label represent unconditional transitions; thus, a call to `setup` in the initial state takes the object into the “pick” state. Labeled edges denote conditional transitions, meaning a call to the named method must also satisfy the condition named by the label on the edge for the transition to take place. For instance, a call to `play` causes the object to transition to the “more” state if its result satisfies the `more?` predicate. Similarly, the label on the edge from `win` to `setup` specifies that this edge is valid only when the input was `#true`, reflecting the nature of a knock-out tournament in which all losers are eliminated from the next round.

Given this explanation, it should not come as a surprise that many Interfaces come with graphs to illustrate constraints on the order of calls to methods of their classes. Legal call sequences make up graph traversals; illegal call sequences fail to describe a path. The contract system needs textual representation of a graph traversal, which is what we really call a trace. A *potentially legal* trace consists of a path through the state-machine graph, beginning at a start state. Such a trace is *legal* if it ends in a final state. All other traces are illegal.



See the interface for the Android media player, which comes with similar graphs.

Let's take a look at three concrete traces:

(1) legal	(2) illegal	(3) potentially legal
(setup ...)	(setup ...)	(setup ...)
(pick ...)	(pick ...)	(pick ...)
(play ...): action?	(play ...): action?	(play ...): action?
(play ...): more?	(play ...): more?	(play ...): action?
(more ...)	(more ...)	(play ...): action?
(win #false)	(win #false)	(win #true)
	(setup ...)	(setup ...)

Trace (1) is the simplest. It represents a player object whose methods are called in the expected order. Notice how the first call to `play` returns an action, while the second call to the method returns the request for more cards and thus the object's `more` method must be called. Finally, the object is informed that it lost the game and receives no further calls,

Except for the last, bold-faced line, trace (2) looks like trace (1), and this last line invalidates the trace. The condition on the edge connecting `win` to `setup` specifies that the input to `win` must have been `#true`. As indicated, though, the input is `#false`, meaning the player lost and cannot move on to the next round. What this last line of the trace exposes then, is that the server-side of the system is ill-behaved.

Finally, trace (3) depicts a player that is granted three turns, returns an action for each of them, and eventually wins the game. Hence, the second call to `setup` is legitimate. This trace is incomplete, however, and it is impossible to know whether the server side will interact in a legal manner with the object until the program run ends.

A trace contract can check whether a sequence of method calls into a object fails to live up to the specified or implied sequencing constraints. Roughly speaking, a trace contract consists of two parts. The first part specifies how the trace of method calls is to be collected on a per object basis. The second part is a predicate that checks the trace whenever it is extended. In case the predicate returns false, the trace-contract system raises an exception and blames the parties involved in the illegal trace.

In this particular scenario, such a trace contract could be employed in two different ways:

- The company could equip the player interface with a trace contract, promising to live up to these temporal constraints. The hackers par-

ticipating in the *Ticket to Ride* tournaments could then rely on this promise and could possibly take advantage of it.

- If the player interface does not include such a trace contract, the participating hackers may wish to protect themselves against problems with the server. By attaching a trace contract to their player implementation, they force the server-side of the system to live up to the natural sequence of interactions based on the rules of the game. In case of a contract failure, the trace-contract system would blame the proxy of the server in the client component, which in turn, would enable the hackers to complain to the company or even sue it in court for supplying faulty software.

Critically, a run-time checked trace contract cannot discover whether a potentially legal trace represents an illegal interaction until the server shuts down gracefully. A trace that does not end in a “win” state could always be completed after a significant delay. Researchers are therefore working on ways to check such trace properties with (software-supported) mathematical approaches when possible.

Exercise 18. Pick a data representation for traces and write a predicate that confirms the potential legality of traces based on in figure 38.

Exercise 19. A utilities component for an object-oriented language, as sketched in figure 28, comes with an `IIterator` interface, which, at a minimum, supports a constructor and two methods: `hasNext` and `next`. Clearly these methods are expected to be called in a particular order. Draw a state-machine that represents this order. Make examples of a legal trace, a potentially legal trace, and an illegal trace through this state machine.

Section 15.4 explains how an Interface for a multi-class component imposes constraints across multiple instances of distinct classes, and briefly discusses where the document could mention this constraint so that client programmers become aware of it. Here is the constraint again:

A collection generated from an instance of `Map` is a live image of the underlying keys. When the `Map`'s keys change, the collection changes. Hence, the methods of an iterator work properly only as long as no code mutates the underlying map. Once this map is modified, the behavior of the derived iterator is undefined.

A simple defensive-programming strategy can enforce this property in Java, or any contemporary programming language that supports nested

```

class Map implements IMap {

    private int counter = 0;

    public void remove(Key d) {
        this.counter = this.counter + 1;
        ..;
    }

    public ISet keys() {
        return new Set(.. domain of map ..);
    }

    // -----
    class Set implements ISet {
        Key elements = ..;

        public IIterator iterator() {
            return new Iterator(elements);
        }
    }

    // -----
    class Iterator implements IIterator {
        private int myCounter = 0;

        Iterator(.. domain ..) {
            this.myCounter = counter;
            ..;
        }

        public Key next() {
            if ( this.myCounter < counter )
                throw new CCMExn(...);
            else
                return return thereAreMoreElements;
        }
    }
}

```

Figure 39: A sketch of an implementation of IMap, ISet, and IIterator

classes. Figure 39 displays a complete sketch of the arrangement. The key idea is to implement the three related classes via a single class with

two sub-classes. The outer class implements the `IMap` interface and comes with a version `counter`. As the figure indicates, each method that modifies the `Map`'s domain increases this `counter`. The inner classes implement the `ISet` and `IIterator` interfaces, respectively. When the `iterator` method instantiates an `Iterator` object, the latter initializes its own private version `counter` to the value of `Map`'s `counter`. Finally, whenever some client code calls the `next` method of this `Iterator`, the two `counter` values are compared. If the `counter` value of the `Iterator` instance is below the `iterator` value of the instance of `Map`, the latter's domain has been modified, invalidating the former.

This arrangement realizes the enforcement of the invariant in an effective and efficient manner, but it also illustrates once again the problems of defensive programming. First, the arrangement cleverly exploits a linguistic capability, which not all languages may support. Second, it requires the implementation of a programming pattern, meaning every implementer of these three interfaces must adapt it appropriately. Since this pattern is hidden in a component whose source code is (may) not (be) available, consuming software developers may not be aware of this idea. Put differently, enforcing the constraint in this way is well and good but it is not a consumer-oriented, informative way of accomplishing this goal.

By contrast, an Interface contract is visible and is typically available for re-use for external implementations of the interface. For this particular case, a contract must know the ordering of method calls into three distinct objects, meaning this constraint enforcement is a task for trace contracts. To construct the appropriate trace, the contract is attached to every instance of `Iterator`, and it references the trace contract attached to the underlying instances of `Map` and `Set`. Concretely, the type of `iterator` in `Set` is enriched with this trace contract so that every call attaches its own instance of the trace collector to the `Iterator` object.

As a trace is constructed, it is relatively straightforward to recognize whether it is legal or not. Here are three examples:

(1) legal	(2) illegal	(3) legal
<code>m = new Map(...)</code>	<code>m = new Map(...)</code>	<code>m = new Map(...)</code>
<code>m.add(...)</code>	<code>m.remove(...)</code>	<code>m.remove(...)</code>
<code>m.remove(...)</code>	<code>s = m.keys(...)</code>	<code>s = m.keys(...)</code>
<code>s = m.keys(...)</code>	<code>i = s.iterator(...)</code>	<code>m.add(...)</code>
<code>i = s.iterator(...)</code>	<code>m.add(...)</code>	<code>t = m.keys(...)</code>
<code>i.hasNext() : true</code>	<code>i.hasNext() : true</code>	<code>i = t.iterator(...)</code>
<code>i.next()</code>	<code>i.next()</code>	<code>i.hasNext() : true</code>
		<code>i.next()</code>

Trace (1) satisfies the constraint because `i.next()` uses an iterator created after the last visible modification of the underlying set of keys. It is equally clear why trace (2) is illegal; a key is added to the domain of the `Map` *after* the iterator is created. Although a similar form of reasoning seems to apply to trace (3), there is one critical difference, namely, the second instance of `Set` is created *after* the addition of the key. Hence, iterating over the elements of `t` is acceptable, while iterating over `s` would be illegal.

The Trade Off Compared to the defensive-programming checks of figure 39, a trace contract arrangement appears to impose a rather large cost. While the former is a simple, constant-time check per method call, the latter seemingly imposes a quadratic time overhead (in the number of elements of the trace). After all, every time a method call is added to a trace, the associated predicate traverses the entire trace.

In order to make trace contracts affordable, Racket’s trace contract system supports an alternative to the naive predicate-based checking scheme. Concretely, the `Interface` specifies a compact data representation of the trace plus an update function instead of the predicate. This update function consumes the current data representation of a trace and a representation of the method call; its return value is a revised trace representation or a signal about a contract violation. The trace contract system tracks the data representation of the trace and interprets the failure signal.

For the running example, it suffices to represent a trace for an iterator as a single bit:

- `#false` is the initial data representation, indicating that the underlying key `Set` of the `Map` hasn’t been modified;
- `#true` is the data representation that indicates that the instance of `Map` has been modified.

The key is that the contract system redirects *all* method calls to the underlying `Map`, its domain `Set`, and its `Iterator` into the update function. So, when a client class calls `remove` on the instance of `Map`, for example, the bit is flipped to `true`. As long as no client code calls `next` on this instance of `Iterator`, the contract isn’t violated; but, if such a method call takes place, the update function notices that the trace is represented as `true`, and it therefore signals a contract failure.

A quick calculation explains that the computational cost of this arrangement is similar to the one based on defensive programming though with a larger constant than the latter. In both cases, each method call comes with

a small constant-cost operation to maintain or check the specified property. While the defensive programming arrangement creates, updates, and compares fields of two classes, the contract system calls the update method, which also creates, updates, and compares two values.

To sum up, turning informal Interface prose into a defensive programming arrangement is again mostly inferior to expressing the property as contract code. In the first case, any developer who implements this informal Interface must re-invent the pattern. In the second one, the very declaration that some newly created classes implement the Interface with a contract enforces the property at run time; the developer can completely focus on the functionality alone. As a matter of fact, even the readers of the contract-based Interface benefit; they can trust that implementing classes enforce the property. The cost consists of the relatively small overhead from the contract system plus the calls to the update function, a price easily paid for relieving developers of the pain of re-inventing tricks.

Reading

Cameron Moy and Matthias Felleisen. Trace Contracts. *Journal of Functional Programming*, Cambridge University Press. December 2023.—Moy and the author of this book extended the higher-order contract system of Racket so that developers can express conditions that govern multiple method calls: to the same method, to several methods of the same class, or even methods across several classes. While a good part of this paper is rather academic, several sections present illustrative examples.

17 Interfaces and AI

Let us imagine for a moment that you are no longer a student or an inexperienced junior developer who is learning to cope with the design and maintenance of complex software systems. Instead you see yourself as someone who is sufficiently experienced to use AI and launch some agents to get software development work done.

What roles do Interfaces play in this context?

An Interface comes about via a use case analysis that identifies a separate concern and thus a separate component. Its technical role is to describe the public aspects of a data representations and applicable pieces of functionality. As such, it serves the developers of client components as a guide on how to use the services, how to compose it with other components into a working software system.

To the developer of the component, the Interface is a specification of the data representations and services that must be implemented. Hence it should articulate sufficiently precise data interpretations and purpose statements for methods and functions. This chapter explains that an Interface supplements the purpose statements with conditions on calls to these pieces of functionality for two aspects: (1) the method's code can assume certain properties of arguments and (2) it can make simple, but concrete promises about their return values. Finally, we have also seen Interface specifications that concern several methods of a single class and even properties concerning the methods of interrelated nests of classes.

In the era of AI, the data interpretations and purpose statements can serve as starting points for AI prompts. Since these statements merely describe *what* the methods and functions compute—not how they accomplish these goals—they come with certain degrees of freedom. In order to get the desired code, the human developer may have to refine these statements and point the AI agents into a particular direction. After going through some iterations, these prompts should yield the desired code.

What this process typically fails to account for, is the second part of the Interface: the conditions on method calls, the promises concerning return values, and the relationships among method calls. Even if it does, the resulting code might look like the old-style defensive code, which makes it difficult for humans to comprehend. At the same time, the basic law of software—all software comes with errors—applies to AI-generated code as well as human-designed pieces. The result is a problem: the software will fail, and when it fails, someone—possibly an AI agent—needs to comprehend the code.

*Defensive code was
common for decades,
and a lot of it is in
public code repos.
Hence it is sad but
unsurprising that AI
models acquired a
taste for it.*

One solution is to explicitly create a separate contract from those parts of the Interface that describe relevant properties. That is, the developer will ask a distinct AI agent to create complementary pieces of code and to compose them in the end into a coherent whole:

- Ask one AI agent to produce the functionality without paying attention to the contract-style properties of the Interface document.
- Charge a second agent with the creation of contracts for individual methods, classes, and collection of classes.
- Finally, demand from a third agent to compose these first two pieces into one coherent component.

The overall system will then consist of monitored components. “Monitoring” means an increase of the team’s confidence in the component’s working. After all, once the component is integrated into the overall software system, the contracts monitor the flow of values into and out of the component. If any of these value flows violate the specified properties, the contract part can report the problem and with it sufficient information for a debugging session. A developer can work with an AI agent to exploit this information, which will probably involve a reworking of the Interface.

17.1 Why Contracts? Why Separate?

Working software developers report that AI agents generate so much code that detailed inspections are impossible. Indeed, in some companies, people do not review these generated pieces of code; they merely check them into the code repository and hope that continuous-integration tests catch problems (if they test at all). This workflow should not come as a surprise, because AI agents either produce large numbers of changes or large pieces of code all at once.

By contrast, the typical translation of contract properties into code is concise. In many cases, the code also corresponds to the words in the Interface document in a one-to-one correspondence, or it could be arranged so that it does. Hence, *experienced* developers have much easier time to inspect this kind of code and confirm its correspondence to the native language in the Interface than the code that implements functionality.

The rationale for separating the two tasks is also straightforward. First, it yields code that can be inspected. Second, it avoids the defensive code problem. Third, if developers work in a language without a contract system, the separation enables them to tune the part of the contract code that

reports violations. Keep in mind that getting useful information about violations is one of the key objectives of existing contract systems.

When the chosen programming language comes with a contract system, the developer must instruct the AI to use it. In all likelihood, the training data for these languages contain a sufficient number of examples, so that the AI agents can easily follow such instructions.

Most programming languages, though, come with mere assertion systems. Although `assert` statements suffice to express key properties of a program's state at a certain point, they are an ingredient of contracts not a complete system. In this case, the developer has to point the AI to the program design patterns sketched out in this chapter. Keep in mind that the most important pattern composes a public method with a workhorse method, where the former performs the contract-checking work, possibly using `assert` statements, and the latter performs the computation proper.

17.2 Contract Verification via AI

Most contract systems detect violations during program execution. Microsoft's Spec# employs a hybrid approach. When programmers edit Spec# code in the VS IDE, the language implementation attempts to formulate the combination of code and contracts as mathematical theorems and to prove them by using a automated proof assistant. If this attempt succeeds, the contract disappears during translation; other the compiler generates run-time checks and the IDE informs the developer of the failure. In principle, such a failure may mean one of two things:

1. The proof assistant lacks power to verify the contract as a theorem.
2. The contract claim is false, that is, it is not a theorem.

Run-time checks help the developer decide what is the case.

While Microsoft no longer actively develops Spec#, AI makes the idea behind this language viable. A developer can teach an AI agent to work with a Spec#-like proof assistant in an interactive fashion. If the proof attempt fails, the AI agent can try to confirm that it is because the proof assistant cannot cope with the existing code. An indefatigable AI agent could repeatedly make small changes to the code proper until the proof assistant can verify the contract. If this scenario works out, software developers can deploy their product with much more confidence than is possible now.

What is programming to a software developer?

As the first chapter already indicates, programming is the creation of code that sends a message from one developer to another, possibly across long time interval. Following the preceding chapter, it is the process of realizing the planned components and implementing the desired Interfaces. While some of these plans are written down, the programming process remains a translation of a creator's thoughts into a concrete product.

This product is *the* artifact that many others (AI agents or people) will most often look at in the future. It is these “maintainers” who will have to read the code; who will have to understand how it achieves its purpose—meaning both its functionality and its basic performance characteristics—and, who, based on this understanding, will have to modify it.

Hence, the goal is to create code that allows maintainers to reconstruct the underlying thoughts quickly. Code must express this thinking as clearly and as comprehensively as possible in the chosen programming language. The preceding chapters suggest that the clarity of code depends on a developer's proper understanding of the nature of software and an awareness of the “big picture” surrounding a piece of code, but every developer knows that any modification of existing software starts with a look at code.

This chapter therefore presents code-focused principles, which range from basic ones—naming classes, methods, etc.—to large ones—creating code according to a system. The basic ideas are small and obvious, but they can go quite far. The idea of systematic program design aims at structuring containers of code (classes, modules) in such a way that even without hints from README files, maintainers can quickly find their way around. Additionally, systematic design enables a form of code inspection that helps human and AI developers recognize problems before they become a big cost factor in terms of bringing across the underlying thinking.

Over time, programming languages improve or new languages come about that allow developers to express more of their thinking directly. But this change is slow, and so we must focus on improving code per se.

18 Clarity of Code

When panelists consider some piece of code incomprehensible during an inspection, a common reaction is to promise explanatory comments. This reaction ignores the nature of comments. As is, they are just a mechanism for adding natural language text to code. These unchecked words are often out of sync with the surrounding code, hence useless or even dangerous. A cure is to use comments in a limited number of cases, where they are truly needed and where they tend to remain partially valid.

As for code itself, a range of factors affects its nature as an expression of thoughts. One truly basic factor is the naming of classes, fields, methods, parameters, and even local variables. At the large-scale level, individual developers may be able to influence the design of classes and relationships among classes or the partitioning of a sub-system into modules.

Experience also tells us that choosing a well-suited data representation is a key factor in clarifying the purpose of code. At almost every level of programming, a developer must make a choice, namely, how to represent domain information in the data sub-language of the chosen programming language. Once the choice is made, the respective code must convey how a piece of data is to be interpreted in the domain and how any piece of information is to be represented as data. Indeed, to get this part right, a comment next to data definition is almost always necessary.

*See section 14 on
encapsulation.*

The chosen data representation should induce the allocation of functionality to classes and modules. Conversely, the design of functionality should mirror the data representation. Imagine a function that consumes a union data type with n variants; if the function's body consists of a conditional with n corresponding branches, a maintainer can easily connect a branch with a variant and make changes accordingly.

Next, code benefits from some abstraction but not too much. Repetitions of code call for the creation of an abstraction and a good name that tells a future reader what the abstraction accomplishes. Then again, a developer should not abstract in anticipation of repetitions or abstract in ways that call for complex call-back protocols or deep inheritance chains.

Finally, as explained in chapter II, in-person inspections are an amazing tool for simulating the hand-over of code from one developer to another. In the spirit of social responsibility, presenting and inspecting code aren't chores but ways to simplify the lives of others—including older versions of the current team members. Getting code inspections right isn't easy; but practice makes perfect or at least good enough.

19 Comments are Okay in a Small Number of Places

Every programming language supplies a *mechanism* for adding informal, natural-language phrases to code. The idea is that comments can expose relationships among pieces of code that are difficult or impossible to express in the programming language itself. Sadly, due to their informal nature, targeted commenting isn't often practiced in university courses, and, as a result, students tend to think of them as a nuisance to be added once the coding is done. Comments have thus become the most abused language construct in the world of software, yielding over-commented, mis-commented, and under-commented software repositories.

Useless Comments The main problem with comments is that they are often out of sync with the code and thus misleading to a reader. When a comment is first added, it probably describes the code under consideration in a correct or close to correct manner. But, as others—including older versions of the original developer—modify the code, they forget to change the comments in accordance with the code changes. If this happens repeatedly, the comments become worthless and misleading.

A second well-known problem is that comments add nothing to the code. Instead they pollute it and pose an obstacle to reader. Consider the following, infamous example from the world of COBOL programming:

```
ADD 1 to X // adds 1 to x
```

It illustrates over-commenting better than any abstract argument.

Where Comments are Useful Any language mechanism, including comments, must come with a *policy* of how to use it. One useful policy for adding comments consists of the following three rules:

- to interpret a data representation, that is, how data relates to information in the problem domain and vice versa;
- to state the purpose of a unit of code, meaning *what* it computes; and
- to explain *how* complex loops or generative-recursive methods accomplish their purpose.

The following paragraphs illustrate these rules.

Data definitions need interpretations. People like to have their rooms at a certain temperature. With a heater, an air conditioner, and a thermostat, it

This policy does not cover the addition of README files, which help readers navigate a large code base.

is possible to keep the temperature in a range around this desired temperature. A thermostat consists of a thermometer, which measures the temperature, and an actuator, which calls for warm or cold air from a heater or air conditioner, respectively. Developing the software for a thermostat calls for a data representation of temperatures and differences in temperature. While temperatures in this problem domain are positive real numbers, common programming languages support only floats (double, single) and integers. In this situation, a developer may represent the crucial numbers from this domain with three integers, like thus:

```
int current;  
int heatSetting;  
int acSetting;
```

Stop! Don't proceed until you have worked through the following exercise.

Exercise 20. In this problem domain, a thermometer may measure a temperature at 17.1C. How is this temperature represented given the above data representation? Conversely, if `current` is 781, what temperature does the thermometer display?

What this exercise points out is that a developer tasked with updating this code gets no clue from reading it as to how information about temperatures is represented as data. The exercise also reminds us that some countries want thermostats that use Celsius while the United States is still using the Fahrenheit scale.

Once the data representation is equipped with comments, solving the exercise becomes possible:

```
int current;          // in 1/10 of Celsius degrees  
int heatSetting;      // call for heat when 'current' falls below  
int acSetting;        // ... for cold air when 'current' is above
```

Stop! Imagine a developer reading this code snippet. Is it now clear what is represented and how?

This simple example is illustrative of why a data definition—the result of choosing a data representation—deserves a comment. In general, representing information as data is a far more complex affair than this example indicates. See section 22.1 for some additional examples.

Classes and methods need purpose statements. Whenever the thermometer deposits a new temperature measurement, the thermostat's control software needs to decide whether it is time to call for heat or cold air. In some buildings only one or the other is available during certain months of the

year, which may depend on the hemisphere where the building is located. Assuming that the code is enclosed in a program container, the following is a sketch of what this may look like:

```
program Thermostat () {
    ...
    int current;      // in 1/10 of Celsius degrees
    int heatSetting;  // call for heat when ...
    int acSetting;    // ... for cold air when ...
    ...

    IDecision decide() {
        if ((current <= heatSetting) && heatAvailable()) {
            return new CallForHeat();
        } else if ((current >= acSetting) && acAvailable()) {
            return new CallForAC();
        } else {
            return new NoAction();
        }
    }
}
```

Now imagine a maintainer having to read this code. Even though this code looks simple, a close look suggests quite a few questions. Does this code controls all of the thermostat, that is, the thermometer, the actuator, and the external connectivity to the A/C and heater? If so, how? If not, what does this method compute?

The method's body consists of merely seven lines of code. Hence it is easy to read them all and to answer these questions. But, if a developer has to understand the purpose of *ten* methods, it means reading and comprehending *seventy* lines of code. So the real issue is that a developer should never have read entire methods to understand *what* a method computes, that is, the method's purpose.

A program, a class, or a method make up a unit of code. On rare occasions, or for extremely small units, the name of the unit brings across what the purpose of the unit is. In many more cases, though, such a unit deserves a concise purpose statement for the developer who needs just a basic idea of *what*—not *how*—the unit serves the overall objective.

For the running example, “one-liners” do suffice to articulate the purpose of both the program and the method:

Only the book's type setting makes it two lines.

```
// implements the controller software for the thermostat
program Thermostat () {
    ...
    // whether 'this' should call for heat or ac,
    // depending on the time of the year
    IDecision decide(int current) {
        ...
    }
}
```

Once again, this example is simple, and a developer with some understanding of the context may have figured out the purpose of either unit with a single glance. In reality, software is large and developers don't always have the context at their finger tips. These one-liners cost the creator of the program unit little, but create the focus needed to realize a unit properly and help readers quite a bit. Section 23.1 presents some additional examples and explains a bit more how such purpose statements help.

Recursive methods, complex loops demand explanations. While much code is straightforward, some functions or methods implement complex algorithms that rely on intricate logical invariants. Sorting algorithms, graph traversals, and context-sensitive tree traversals are some examples. A developer may then write generative-recursive methods with accumulators. Both differ from their structurally-recursive siblings because their arguments are not just pieces of the given structure. In this case, it may not be obvious how the algorithm computes its result or why it terminates. Hence, such functions deserve comments that explain their *how* and *why*.

A developer scared of recursion may avoid it in favor of a worklist algorithm. In this case, how and why comments are best placed with the looping construct (*while*, *repeat*). Academics tend to refer to such comments as loop invariants.

Finally, if any algorithms demand deep insights into the underlying domain—think Simplex for solving linear programming problems in applied mathematics—a comment pointing to a journal paper or an on-line tutorial serves as a good warning to a maintainer. Additionally, some in-lined comments should connect pieces of the implementation to the key ideas of the cited algorithm description.

In sum, here is the policy concerning comments in one line

Don't comment code except for the three situations discussed here.

To recall, comments interpret data definitions, state the purpose of every (small or large) unit of code, and elucidate complex loops.

20 The Very Basics

Making code self-explanatory starts with small things: names; the size of functions and methods; conditionals and loops. Imagine the following scenarios as someone who is tasked with modifying a method:



Style Guides

- finding the method and noticing it is some 100 lines long;
- reading the variable `x` in its body versus the variable `distance`; or
- encountering a conditional whose `then` branch is 10 lines long and comes with a one-line `else` branch.

Each of these scenarios by itself poses a small problem. But, as these small problems pile up, the time needed to eliminate this kind of friction adds up. This section makes four suggestions to address just such problems with small, preventive actions.

20.1 Names Matter

Names matter in many contexts. First, literal constants need names. Second, the names of files, classes, and modules help navigate a code base. Third, good names for methods and functions, plus their parameters, can summarize many lines of code with a “word.” Finally, introducing well-named, local variables for intermediate results helps readers comprehend the desired computational process.

Most constants need names. A name conveys meaning, usually a reference to the application domain of the software system. Think of a reader who encounters

```
100 * 200
```

versus one who looks at

```
windowWidth * windowHeight
```

where the two names stand for the two constants, respectively.

In addition to conveying meaning, names for constants also facilitate the adjustment of values. For example, nowadays end-users sit in front of significantly larger monitors than a decade ago. It’s therefore acceptable to increase the width and height of the imagined window. Doing so with named constants is easy; finding all occurrences of 100 and 200 makes such an adjustment extremely difficult.

Methods and their parameters need names that reflect their purpose. Here is a method header from a software system for an assisted-driving vehicle:

```
int moveRight(int distance) {
    .. }
```

Some programming languages support notations for units and unit-checking, which is superior to naming conventions. But most don't.

While the choice of the name `moveRight` depends on the context, the choice of `distance` is an entirely local affair. Instead of the plain word, the variable name could convey that it is the shortest distance between this car (which is running the software) and the car in the next lane. A different use might call for bringing across that it is a distance measured in centimeters or inches. Of course, it is possible that both aspects of the name are critical, which may yield a truly meaningful parameter name:

```
int moveRight(int shortestDistanceToCarOnTheLeftInCM) {
    .. }
```

And yes, it is quite long. When AI agents generate names or IDEs allow developers to complete names with a couple of keystrokes, their lengths should not matter too much. What does matter, though, is readability.

Names must satisfy a compromise that balances easy readability and precise meaning. Here “readability” means parsing the name: discerning its pieces and understanding how they hang together. The renamed parameter of `moveRight` is really a short sentence:

shortest distance to the car on the left in centimeters

The problem is that a reader must tease out, and make sense of, these distinct words on every encounter.

To avoid this effort, a compromise is needed. One approach is to put the most important aspect of the variable into its name, and document the others with an interpretation statement as spelled out in the preceding section. In this spirit, here is a variant of the `moveRight` method that emphasizes the unit of measurement (cm):

```
// 'distanceCM' denotes the shortest distance to
// an obstacle to the left of 'this' car
int moveRight(int distanceCM) {
    .. a conversion to inches for the next call .. }
```

The author of the code may think that adding the unit of measurement is more important than its precise meaning because the method calls a function that needs the same distance measured in inches.

Exercise 21. Reformulate the `moveRight` method header imagining that it is all about a reaction to a distance measurement. Ask a peer to review your reformulation and comment on it.

Names for intermediate results explain computations. Here is a function from a game program that needs to compute whether players have placed the mouse inside of some area within a surrounding context:

```
;; represents a positioned square
class SquareShape:
  Posn center; // of 'this' shape
  int width;   // in pixels of 'this' shape

;; is 'p' inside 'this' square shape
boolean onTarget(Posn p) {
  return
    this.isInside(
      Geometry.vectorFromTo(
        p,
        Geometry.shift(this.center, (width/2)))
    );
}
```

Now imagine you're assigned a "bug ticket" related to this method. Stop for a moment and try to explain *how* the function computes its result—not *what* it computes; its purpose statement properly documents the goal.

Even though the developer has nicely indented the nested expression and aligned the arguments of the static method calls, it is still a challenge to understand the steps of this computation. Breaking up such a large expression into small ones and giving names to the intermediate results greatly improves readability as the following revision shows:

```
...
;; is 'p' inside 'this' square shape
boolean onTarget(Posn p) {
  Posn nuOrigin = Geometry.shift(this.center, width / 2);
  Posn pRelative2Center = Geometry.vectorFromTo(p, nuOrigin);
  return this.isInside(pRelative2Center);
}
```

This variant does not break the expression into atomic pieces—the first definition uses two operations—but it breaks it into a sequence that explains the computation in a step-by-step fashion.

Like in the case of plain names, a sense of compromise is needed here. Some formulas are so well known that breaking them up is unnecessary.

Some operations, say division by 2, are so obvious that their results don't need a separate name. Worse, breaking up expressions inside of a function may lengthen it so much that it is no longer easy to read. And that brings us to the next important basic point about code: keeping units of code sufficiently small.

20.2 The Size of Functions and Methods

Functions and methods are the fundamental building blocks of programs, similar to the bricks and two-by-fours of houses. When developers must comprehend code with the goal of modifying it, they focus functions and methods above all. Keeping functions and methods comprehensive is thus imperative, and this usually means keeping them short.

Squeak is a great example of a large system with short, clean methods. It is a modern version of Smalltalk, the first major object-oriented programming language. A somewhat recent inspection counted between 500,000 and 600,000 lines of code. The number of methods came in at around 100,000. In other words, the developers kept the average method to about six (yes, 6) lines of code. If these methods also come with sensible and meaningful names, a developer can comprehend the essence of each with a quick glance.

The question is how developers keep methods so small. The key is to understand that there are only two kinds of functions:

- An *atomic* function depends on the language, some imported functions, and possibly itself.
- A *composite* function composes atomic and other composite functions, using mathematical function composition, ordinary sequencing, iteration, comprehension, and other composition mechanisms of the chosen programming language. The `onTarget` method of the preceding section composes two functions with a local method, and it thus represents an example of a composite method.

So, to keep methods and functions concise and still comprehensible, developers should start the design for each of them with the question of whether it is atomic or composite; conversely, they must avoid methods that consist of blocks of atomic work and compose others.

Organizing code according to this guideline rests on two ideas:

1. Don't be afraid of function or method calls.

2. Consider turning a method into a composite whenever it grows large.

The two ideas are two sides of the same coin. Jointly, they say to prefer readable code over small (if any) minor performance costs. At this point, the performance cost of calls is negligible, while the gains in comprehensibility are large when functions and methods are kept short.

Turning an existing large function into a composition of several function means recognizing blocks of code in an existing function as a separate, meaningful task. The follow-up step is to move this code into a function by itself, to give this function a name that describes the task properly, and to replace the code with a call to this function.

```

#; {World Message -> World}
;; process 'msg' from server in 'current' state of the world
(define (receive-update current msg)
  (match msg
    [(posn-msg posn dir opponents)
     (define I (fighter-update (world-I current) posn dir))
     (world CHOOSE I (map make-fighter opponents)))]
    [(fight-msg updated-fighters outcomes)
     (match-define (world _todo I opponents) current)
     (match-define (cons _I differences) updated-fighters)
     (define others (update-fighters opponents differences))
     (if (empty? outcomes)
         (continue '[] I others)
         (world [list RESULT WAIT-TIME outcomes] I others))]
    [_ (stop-with (format "can't deal with ~v yet" msg))]))

```

Figure 40: Recognizing separate functionality

Figure 40 displays a somewhat longish function. It certainly exceeds the already-mentioned six lines per method metric of Squeak. The key is to recognize the lack of balance in the three branches of the `(match)` conditional. Specifically, the second, boxed branch consists of six lines while the first one requires two and the last one is just a function call. These counts alone suggest that the method is a mix of atomic computation and a composition of helper functions.

The next step calls for factoring out pieces of code as separate functions or methods. This means identifying all those variables that are defined in the existing function and needed in the code block. These variables become

the arguments and parameters, respectively, to the factored-out functions. When a developer considers factoring out one branch of a conditional, it is often best to factor out all other branches that consist of more than a line.

```

#; {World Message -> World}
;; process 'msg' from server in 'current' state of the world
(define (receive-update current msg)
  (match msg
    [(posn-msg posn dir opponents)
     (deal-with-positioning-msg current posn dir opponents)]
    [(fight-msg updated-fighters outcomes)
     (deal-with-fight-msg current updated-fighters outcomes)]
    [_ (stop-with (format "can't deal with ~v yet" msg))]))

```

Figure 41: Breaking up the function in figure 40

In the running example, the function body consists of a pattern-match that compares `msg` with two distinct structs: `posn-msg` and `fight-msg`, respectively. This matching defines variables local to each branch. Since the code in the two branches also refers to `current`—the first parameter of `receive-update`—it becomes an argument to the factored-out functions.

Figure 41 displays the result of this re-factoring step. The re-factored variant consists of a conditional with three branches, each a single function call. Comparing the two variants should immediately clarify how the re-factoring and the choice of function names helps a developer navigate this code base. If, for example, a glitch is visible during the set-up phase of the game—when each AI player receives information about where its fighter is located and going—the name of the function in the first branch directs the developer to the correct code. Nothing else is needed. Likewise, if the result of fights is broken, the second branch calls for attention.

The factored-out `deal-with-posn-msg` function is an example of an atomic function. Atomic because it use nothing but struct operations (e.g. `world-I`, `fighter-update`) and imported functions (e.g. `map`).

20.3 A Short Note on Conditionals

Conditionals are some of the most frequently used features. They deserve some basic care. A conditional identifies distinct cases of computation, and it sets up distinct blocks of code for each case. Distinguishing cases helps

developers focus. By subjecting a block to a condition, a developer can focus on just this special one form of data and ignore all others, which also holds for the reader.

The following guidelines govern the basics of conditionals:

- all conditions and branches should use a uniform style;
- all cases of a situation are covered by a single conditional; and
- all branches should be of roughly the same size and complexity; or
- always place the shorter branches ahead of the longer ones.

Take a second look at the code in figure 40, specifically at the conditional in the function body. It distinguishes three input cases for `msg`. The conditions are formulated in a uniform style. While the first two pattern-match and deconstruct a struct into its pieces, clearly distinguishing two cases, the last one is a catch-all clause. By contrast, and as the preceding section points out, the branches themselves violate the first and third guideline with the second one standing out by size alone. The second branch also contains sophisticated pattern-matching definitions, (simple) function calls, and a nested conditional (distinguishing two different result situations). The refactoring of the preceding section creates a function whose conditional satisfies the guidelines.

As for the second guideline, consider the following code snippet:

```
/* @param 'state' the current game state
 * @param 'action' the player's chosen action
 * @return
 *   is 'target' reachable after applying 'action' to 'state'
 * @effect may modify 'state'
 */

private static boolean reachable(State state, Action action) {
    Position target = action.getTarget();
    state.applyAction(action);
    if (!state.canActivePlayerReachTile(target)) {
        state.undoAction(action);
        return false;
    };
    return true;
}
```

Although both `return` statements belong to the same situation—whether the `target` position is reachable or not—the method body does not signal this symmetry.

The violation is easily eliminated with an `else` branch. Adding it exposes that the two branches are of unequal length and the (expected) one is at the end. Removing the negation from the condition fixes this violation of the third guideline:

```
private static boolean reachable(State state, Action action) {
    Position target = action.getTarget();
    state.applyAction(action);
    if (state.canActivePlayerReachTile(target)) {
        return true;
    } else {
        state.undoAction(action);
        return false;
    }
}
```

Stop! Contrast the purpose statement of this method with the first and second organization of the method body. Which organization matches the purpose statement best?

```
/* @param 'dir' the desired direction of an action
 * @param 'ind' the row or column to be shifted
 * @return whether 'ind' is a valid index given 'dir' */
private static validIndex(Direction dir, Index ind) {
    if (dir.isVertical()) {
        if (!state.getBoard().isMovableCol(ind)) {
            return false;
        }
    } else if (!state.getBoard().isMovableRow(ind)) {
        return false;
    }
    return true;
}
```

Figure 42: Comprehending conditionals

Exercise 22. Figure 42 displays a snippet from a rule-checking class for a game whose board comes with movable rows and columns. Does the conditional satisfy all guidelines? Why? If so, explain. Why not? If not, rewrite the method. Work with a partner.

20.4 And an Even Shorter One on Loops

A typical looping construct combines a conditional with a function into a traversal of a data structure. In the case of a `for` loop, this claim is particularly obvious: (1) the (often implicitly stated) condition specifies the elements to be considered and (2) the loop body is the functionality that applies to each of these elements. Similarly, a `while` loop differs from a `for` loop only in that the components of a data structure are computed explicitly, step by step, and that the generation of components—and the termination of the loop—is governed by an explicit condition.

The key is to simplify the condition and to keep the loop body as concise as a function body. Like for an ordinary function, the loop body should not exceed an easily comprehensible size, about the same size as a function. Conversely, if the loop body gets too large, identify atomic tasks and factor them out into functions so that the loop body becomes a composite function. If necessary, turn the entire loop body into a function with a suggestive name.

Consider these two code snippets, starting with the one on the left:

```
void echo(String fn) {  
    try {  
        Reader br = open(fn);  
        int iIn;  
        while ((iIn = br.read()) != -1) {  
            char cIn = (char) iIn;  
            System.out.print(cIn);  
        }  
    }  
    catch ...  
}  
  
void echo(String fn) {  
    try {  
        Reader br = open(fn);  
        while (br.ready()) {  
            echoChar(br);  
        }  
    }  
    catch ...  
}
```

While such a short snippet is easy to understand for an experienced reader, it illustrates two problems. First, the loop condition combines an assignment statement with a comparison to a magic constant (`-1`). What does this mean? Second, the loop body consists of two lines whose purpose a code reader must also infer. Imagine a loop body of 10 or 20 lines.

Now take a look at the code on the right. It relies on a `ready` method to bring across that the termination condition is about finding the end of the readable input. The loop body is just a function call to a suitably-named function: `echoChar`. Every reader can immediately discern what this loop computes.

21 Code Inspections, the Basics

A code inspection is the best way to check whether code is comprehensible or suffers from basic coding flaws. Additionally, an inspection calls for the discovery of two other kinds of problems: errors and design mistakes.

Coding flaws make code difficult to comprehend. They are common, easily spotted, and equally easily corrected. The most basic coding flaws violate the basic suggestions of the preceding section. Others conflict with specific coding rules that a team has adopted but aren't enforced with linting tools.

Errors make code inconsistent with expected behavior. Most pieces of code come with informal specifications, from which the panelists can infer *what* the reviewed functionality is to compute. When panelists discover discrepancies between the code and the specification, everyone involved must rejoice; it is a bug not deployed. One way to search for errors is to inspect the unit tests for a piece of functionality. Panelists may notice that the tests don't cover some aspects of the desired functionality; focusing on the uncovered aspects often reveals flaws.

Design mistakes may make the code useless. In some cases, developers choose a code organization, possibly starting from a faulty data representation, that overlooks essential aspects of the problem, functionality specifications, performance constraints, or contextual peculiarities. Spotting such design flaws requires a good amount of experience, because they are about relationships of the reviewed code to other parts of the system or they call for a radically different choice of data representation and algorithm.

Panelists can discover design flaws at two stages: during an inspection of the specification architecture or during an inspection of the code itself. For the first kind, review the discussion in section 15.5; it illustrates how panelists can question a relationship between different system components, specifically the choice of one communication protocol over another between two components. This early discovery avoids the creation of code with serious design flaws, but not all design flaws manifest themselves at this stage. For the second kind, panelists must continuously question whether they are agreeing to a presenter's explanations of the code too easily. In many cases, they must insist that the presenters explicate *implicit* assumptions about the context of the presented code—and when this is done, they may see where the development went off the rails.

Using the guidelines from the preceding section, an inspection targeting coding flaws is straightforward. Simple instances of the other two are also possible to spot; for interesting instances, some more knowledge about good code design is required.

```

#; {Image [OutputPort] -> Void}
;; save 'img' in the QOI format to the 'out' port
(define (image-write-qoi img [out (current-output-port)])
  ; Write the QOI header.
  (write-bytes
    (bytes-append
      QOI-MAGIC
      ; Image width and height, unsigned 32-bit, big-endian.
      (integer->integer-bytes (image-width img) 4 #f #t)
      (integer->integer-bytes (image-height img) 4 #f #t)
      (bytes
        (image-channels img)
        (image-colorspace img)))
    out)

; Get the ARGB pixel data from the image.
(define pixels (image-pixels img))
; Allocate the index of previous pixel values.
(define pixel-index (make-vector 64 (make-bytes 4)))

(define last
  (for/fold ([pixel-prev qoi-pixel-init]
             [run-length 0])
    ; The loop will return the length of the last run.
    #:result run-length)
    ; Process pixels in raster-scan order.
    ([n (in-range 0 (bytes-length pixels) 4)])
    (define pixel (subbytes pixels n (+ 4 n)))
    ; Check whether we are continuing a run.
    (if (equal? pixel pixel-prev)
      ; If the pixel value has not changed, increment the current run length.
      (values pixel (add1 run-length))
      ; If the pixel value has changed, locate it in the index.
      (let ([pos (qoi-index-position pixel)])
        ; Write the previous runs if applicable.
        (write-qoi-op-runs run-length out)
        ; Check whether the current pixel is present in the index.
        (if (equal? pixel (vector-ref pixel-index pos))
          ; If the current pixel is present in the index, write a QOI_OP_INDEX chunk.
          (write-qoi-op-index pos out)
          ; Otherwise, compute the difference from the previous to the current pixel.
          (let-values ([([dr dg db da dr-dg db-dg] (pixel-diff pixel pixel-prev))]
            ; Add the current pixel to the index.
            (vector-set! pixel-index pos pixel)
            ; Check whether differential or luma encoding can be applied.
            (cond [(not (zero? da))
                  ; If the alpha value has changed, write the new ARGB pixel data (QOI_OP_RGBA).
                  (write-qoi-op-rgba pixel out)]
                  [(and (<= -2 dr 1) (<= -2 dg 1) (<= -2 db 1))
                  ; If the RGB deltas are small enough, encode the differences (QOI_OP_DIFF).
                  (write-qoi-op-diff dr dg db out)]
                  [(and (<= -32 dg 31) (<= -8 dr-dg 7) (<= -8 db-dg 7))
                  ; Use the green delta as a reference to encode the relative
                  ; red and blue deltas (QOI_OP_LUMA).
                  (write-qoi-op-luma dg dr-dg db-dg out)]
                  [else
                  ; As a fallback, write the RGB data (QOI_OP_RGB).
                  (write-qoi-op-rgb pixel out)])))
          ; Reset the current run length.
          (values pixel 0))))))
  ; Write the last run if applicable.
  (write-qoi-op-runs last out)
  ; Write the QOI end marker.
  (write-bytes QOI-END-MARKER out))

```

Figure 43: Inspecting for coding flaws

21.1 Inspecting Code for Coding Flaws

Thanks to Guillaume Savanton who wrote most of this code originally and who gave permission to use it for this illustration here. The imaginary dialog is loosely based on a conversation between the creator and the author.

Imagine yourself as a panelist that is about to review the code in figure 43. This function is an extract from a Quite OK Image (QOI) library. For our purposes, it does not matter what QOI is about other than that the figure displays a single function from this library. The question is where and how this code violates the basic guidelines from the preceding section. Stop! List those violations before reading on.

An actual code inspection may proceed as follows:

Here (figure 43) is the function that writes out an image to a file or a device.	This function does not fit on your laptop screen. How long is it?
It is 60 lines long. But note that it is extensively commented. A reader should have no problems with it.	Let's focus on its length for now and return to the comments later. How could you split up the function?

Notice how even the first question points to a problem and how the presenter reacts in a defensive manner. Like design inspections, code inspections are *not* about the creator. They are about improving code so that the maintainers “get it” quickly. And even the first impression suggests to the panelists that the function is too large to comprehend quickly.

But, note how the panelists also veer from the prescription for inspections. Instead of focusing on finding flaws, the second question implies suggestions of how to improve the presented code.

Is it acceptable to weave in suggestions?

Generally speaking, it is not the task of a code inspection to make suggestions on how to fix problems. The intent of the review is to understand how the function accomplishes its purpose and to uncover problems. Keep in mind that the developer of the code has studied the problem in gory detail and has figured out how to tackle it with code. Once a problem is pointed out, the developer may come up with several solutions and, if needed, can ask for advice.

One exception to this rule concerns “developers in training.” In the specific situation when a mentor reviews code with someone who is considered “in training,” weaving in potential fixes is acceptable. But even

then it is best to extract the fixes from the mentored person rather than stating them directly. This approach furthers the learning process.

Hence a restatement of the last question is in order:

...	Does the function perform distinct tasks, tasks that could be factored out? Can you just walk us through the various steps that this function takes to write the image?
The first expression writes the header of the file. It could be factored out as a function.	If you name it <code>write-header</code> , it is also clear what it computes.
Noted. And yes, a good function name might make its comment superfluous.	Also, the expression is deeply nested.
Perhaps it's best to split it into named expressions.	Do so while you create the separate function.
Noted.	Does this mean the big code block that initializes <code>last</code> is writing out the body of the image?

Now the panelist's naming suggestion continues the thinking of the presenter and drives the conversation forward. The follow-up question is natural in two ways: the variable initialization of `last` is indeed huge, and a header is usually followed by a body. Although this point isn't mentioned in the section on basics, it *is* basic to keep initializations short.

Let's continue with the inspection:

Yes, the expression is somewhat large and performs side effects.	Can you identify the tasks that this initialization expression performs?
--	--

The loop goes over the body of the image and writes it out in chunks.	But it also seems to return something.
---	--

Yes, it returns how large the last sequence of bytes is that must still be written out.	Can't this loop go into another function, one that writes the body?
---	---

No, it isn't just the loop. As the comments above suggest, the <code>pixel-index</code> vector is also needed for writing out the body.	Oh, okay.
---	-----------

Stop! What just went wrong with this dialog?

The panelist once again made a suggestion—and the presenter pushed back. Here the interaction demonstrates that presenters must know when to accept criticism and when to tell the panelist(s) why a challenge is wrong. An inspection must be a give-and-take.

The inspection could also have taken an alternative direction:

Yes, the expression is somewhat large.	Can you identify the tasks that this initialization expression performs?
--	--

The loop goes over the body of the image and writes it out in chunks.	What does it return?
---	----------------------

It returns the number of bytes that must be still written out.	The comment about <code># : return</code> says so. Could it be replaced with a better name for this variable?
--	---

True, something like <code>length-of-last-run</code> would make this comment completely superfluous.	Yes. And as a matter of fact, almost every line of code comes with a comment. Isn't that too much?
--	--

The comments are left over from when we worked out the function.	Go over them and see how many of them are still necessary.
Will do. Any other criticism?	Just a question. Now we understand that the function writes a file header and a body. Does it also write a “footer” or whatever it might be called?

For now, three major criticism is enough for 60 lines of code. Take a close look. What else might a conversation critique?

21.2 A Code Inspection Memo

The product of a code inspection is a memo that explains the panel’s criticism. It is not a set of notes about who said what. A formal code inspection allocates one person—the secretary—to this one task; the presenter’s partner, if present, may take additional notes. The person who presents must focus focus on just three points: explaining the code; understanding the questions and criticism; answering those properly or pushing back when panelists’ criticism are addressed in the code.

If you are working with an AI agent-programmer, ask a second agent to take notes and create a memo, either formal or semi-formal. For future reference, tell the AI agent to deposit the memo in the appropriate folder of a code repository. The resulting collection of memos can be used to teach the AI coding lessons.

A formal memo may come with the following information:

1. the name(s) of the presenter(s),
2. the names and roles of the panelists,
3. date and time,
4. a link (URI) to the presented code, and
5. a list of the problem descriptions that
 - enable the presenter(s) to reconstruct them all;



Taking Notes

To: Guillaume S.
 From: Matthias F.
 Date: 31 Feb. 1981
 Subject: your img-write-qoi function

An inspection of the QOI library revealed the following problems:

1. The function is too long.

SUGGESTION It might be possible to break it up into three functions: one for the file header, one for the body of the image, and one for the remaining bytes.

2. The initialization expression for 'last' is too long.

SUGGESTION Turn the loop into a function that returns how many bytes remain to be written at the end.

3. The code comes with too many comments, many of which add nothing to the code. The first one is in the loop header:

```
(for/fold ([pixel-prev qoi-pixel-init]
          [run-length 0]
          ; The loop will return the length of the last run.
          #:result run-length)
  ...)
```

This comment just repeats the #:result clause and adds nothing.

Figure 44: A code inspection memo

- are formulated in terms of the code, not of what was said.

If a suggestion comes up and the presenter accepts it, the description may include it but should clearly label it as such.

Figure 44 displays a sample memo based on the preceding snippets from a code inspection. If the setting is semi-formal, it usually suffices to send an email with just the list of problem descriptions (and suggested fixes), with a CC to the remaining people present.

```

#; {Image [OutputPort] -> Void}
;; save 'img' in the QOI format to the 'out' port
(define (image-write-qoi img [out (current-output-port)])
  (write-qoi-head img out)
  (define last-run-length (write-qoi-body img out))
  (write-qoi-op-runs last-run-length out)
  (write-bytes QOI-END-MARKER out))

#; {Image OutputPort -> Void}
(define (write-qoi-head img out)
  (define width (integer->integer-bytes (image-width img) 4 #f #t))
  (define height (integer->integer-bytes (image-height img) 4 #f #t))
  (define header (bytes (image-channels img) (image-colorspace img)))
  (write-bytes (bytes-append QOI-MAGIC width height header) out))

#; {Image OutputPort -> Void}
(define (write-qoi-body img out)
  (define pixels (image-pixels img))
  (define LENGTH (bytes-length pixels))
  (define INDEX (make-vector 64 (make-bytes 4)))
  ;; Process pixels in raster-scan order:
  (for/fold ([prev qoi-pixel-init] [run-length 0] #:result run-length) ([n (in-range 0 LENGTH 4)])
    (define pixel (subbytes pixels n (+ 4 n)))
    (cond
      [(equal? pixel prev)
       (values pixel (add1 run-length))]
      [else
       (write-qoi-op-runs run-length out)
       (write-piece pixel INDEX prev run-length out)
       (values pixel 0)])))

#; {Bytes [Vector Bytes] N OutputPort -> Void}
(define (write-piece pixel INDEX prev out)
  (define pos (qoi-index-position pixel))
  (cond
    [(equal? pixel (vector-ref INDEX pos))
     (write-qoi-op-index pos out)]
    [else
     (define-values (dr dg db da dr-dg db-dg) (pixel-diff pixel prev))
     (vector-set! INDEX pos pixel)
     (cond [(not (zero? da))
            (write-qoi-op-rgba pixel out)]
           [(and (<= -2 dr 1) (<= -2 dg 1) (<= -2 db 1))
            (write-qoi-op-diff dr dg db out)]
           [(and (<= -32 dg 31) (<= -8 dr-dg 7) (<= -8 db-dg 7))
            (write-qoi-op-luma dg dr-dg db-dg out)]
           [else (write-qoi-op-rgb pixel out)]))]))

```

Figure 45: Reacting to a code inspection memo

21.3 Reacting to a Code Inspection

Figure 45 shows how the presenter(s) could react to the criticisms from the code walk. The revised main function—`image-write-qoi`—consists of four lines of code: the first one writes the file header; the next two deal with the main body of the image file; and the last line writes an end-of-image marker.

The revised `image-write-qoi` function is an example of a composite function. It composes the functionality of four auxiliary functions:

1. `write-qoi-head`, which writes the file header;
2. `write-qoi-body`, which deals with most of the body; and
3. `write-qoi-op-runs`, which deals with the left-over bytes; and
4. `write-bytes`, a generic function that marks the end of the file.

Each of these pieces represents a natural segment of the original version of the main function.

Consider `write-qoi-head`, the function that writes the file header. The corresponding expression in the original `image-write-qoi` is deeply nested. This new auxiliary function consists of three variable definitions and one expression: `write-bytes`. The names of the variables makes it abundantly clear which role each piece plays and in which order the pieces are computed. No comments are needed to explain this computation.

Similarly, `write-qoi-body` consists of three variable definitions and the main shape of the large loop in the original `image-qoi-write` function. It contains one comment, concerning the order in which the loop traverses the image's bytes. The author clearly considered this remark critical and its content difficult to discern at one glance from the loop's body. Also note how the author split out the non-trivial, atomic step of writing pixels to the file; doing so keeps the loop to a reasonable size.

21.4 Inspecting Code for Errors and Test Coverage Problems

Spotting errors in code is challenging. To claim an error, a panelist must fully understand the specification—informal, rigorous, or formal—and the presented code. Understanding the specification means being able to imagine how the program should behave for a large part of its input spectrum or in a range of situations. Understanding the code might mean grokking one function or it might demand seeing how several methods jointly compute the desired result.

Balancing the two aspects is a challenge for every panelist. Some highly experienced developers may be able to accomplish it in some situations, perhaps because it resembles problems they have encountered before. But, for the sake of keeping the conversation productive and for the sake of egoless programming, it might be best for both to simply ask standard questions about test coverage. If the tests cover the imagined inputs or situations, there might not be a problem; otherwise, the presenter and the panelists may wish to work through such imagined scenarios together.

Let's make this concrete. Figure 46 displays a class fragment for a game called "Three in a Row." Two players place tokens on a square board of size $n \times n$. The first player to connect three tokens along a row, a column, or a diagonal wins.

```
class BoardState {
    /* each cell represents one board position:
     * "x" for player 1, "o" for player 2, "_" is free */
    String[][] board;

    public boolean winning() {
        ... same(theBoard.drDiagonalFrom(0,0)) ...;
    }
    ...

    /* @param (row,column) starting point
     * @return the first 3 elements of down-to-the-right diagonal */
    private String[] drDiagonalFrom(int row, int column) {
        String[] diagonal = new String[3];
        for(int i = 0; i < 3; i++) {
            int r = row + i;
            int c = column + i;
            diagonal[i] = board[r][c];
        };
        return diagonal;
    }

    /* @param 'three' represents a slice of length 3
     * @return whether all strings in 'three' are the same */
    private static boolean same(String[] three) {
        String t = three[0];
        ...
        return true;
    }
    ...
}
```

Figure 46: Inspecting for errors and test coverage problems

As the data-representation comment in `BoardState` says, the developer has chosen to represent the state of the game board with a two-dimensional `String` array. Each one-character `String` in this array represents an open slot or a slot taken by one of the players. To decide whether

it is a winning board state, the developer has added methods for extracting rows, columns, and diagonals, each represented with a `String` array.

Given the complete code, it is time to observe an inspection:

The most interesting methods are those that extract rows, columns, and diagonals.	Okay, why don't we look at the extraction of diagonals to get started.
It's best to highlight the method. (See boxed portion of figure 46.)	Well, the "d" and "r" part of the name are puzzling at first, but the purpose statement clarifies it.
In order to extract the diagonal, the method sets up a loop that proceeds from the given board position, one step at a time.	Is <code>diagonal</code> the loop accumulator?
Yes. As the loop takes yeah-many steps, the accumulator contains that many <code>Strings</code> .	...

So far, so good. The presenter and the panelist discuss how the code computes the diagonal, avoiding all mention of "how you, the presenter" would do it. The first interaction also illustrates how a panelist—usually the head—directs the code inspection to an interesting point, here, the diagonal-extraction method.

An experienced developer may point out the problem directly:

...	That makes sense. But could you explain how the <code>for</code> loop iterates through the two-dimensional board
-----	--

Sure. The loop takes three steps, with <code>i</code> set to 0, 1, or 2.	Doesn't the indexing into <code>board</code> with 1 or 2 raise an exception if <code>row</code> and <code>column</code> represent the corner of the board?
--	--

Then again, if an experienced panelist is dealing with a new developer, requesting a unit test might be the preferred approach so that the presenter can discover the flaw instead of being told about it:

...	That makes sense. But could you explain how the <code>for</code> loop iterates through the two-dimensional board
-----	--

Sure. The loop takes three steps, with <code>i</code> set to 0, 1, or 2.	Could you show us a test case where <code>the drDiagonalFrom</code> is called with a starting position that is only one step from the board's border?
--	---

We don't have such a test. As a matter of fact, I don't think we have any tests where the loop starts just one step away from the board's edge.	So is this a problem?
---	-----------------------

Yes, it is. We simply didn't think that a player would play it that close to the edge.	I don't think we should make such an assumption.
--	--

Correct. We don't want this code to raise an out-of-bounds exception.	...
---	-----

Note how the presenter reasons out in real time that the code would raise an exception—a true learning experience.

21.5 Pushing Back During a Code Inspection

When a panelist is wrong, the presenter must push back. The inspection of the code in figure 46 makes a good example for such an interaction. Additionally, it points out a flaw in how the code inspection proceeds:

...	That makes sense. But could you explain how the <code>for</code> loop iterates through the two-dimensional board
Sure. The loop takes three steps, with <code>i</code> set to 0, 1, or 2.	Could you show us a test case where <code>drDiagonalFrom</code> is called with a starting position that is only one step from the board's border?
We don't need such a test. The <code>winning</code> method calls <code>drDiagonalFrom</code> only if the diagonal exists.	Oh, that's surprising. You may wish to expand the purpose statement of the method to explain this pre-condition.
That's a good idea. Someone might add a call to <code>drDiagonalFrom</code> in the future without realizing that it relies on this pre-condition.	From this angle, it might even better to eliminate the code that establishes the pre-condition and to fold it into the method.
Yes!	Before we move on, let's revisit this check in <code>winning</code> .

Several aspects of this exchange are worth noting. First, a presenter should not just listen to criticism and accept it. It is perfectly reasonable to reject a criticism. Second, in this case, the calling context establishes a critical pre-condition, which implies the next two points. Third, the presenter should explain this computation in a top-down fashion, starting from the public entry points into the class, and ask the panelists to pay attention to the logical relationship with `drDiagonalFrom` when the definition of `winning` is visible. Fourth, the panelists may not rely on the word of the

presenter alone here. They must re-direct the code inspection to the `winning` method so that they can convince themselves that the pre-condition is properly established. Finally, the conversation also reminds us of the key idea behind inspections: the presenter understands that expanding the purpose statement with an appropriate remark about the pre-condition would be an act of socially responsible software development.

21.6 Inspecting Code for Design Mistakes

Generally speaking, design mistakes are the most difficult-to-spot, and yet, such mistakes may have the worst impact on the overall product. Their source varies but here are some common places to check out:

- the choice of data representation;
- the division of functionality into separate tasks;
- the choice of intermediate data structures for composing helpers;
- a mismatch between the chosen data representation and the processing functions;
- an attempt to prematurely abstract or optimize code.

The effects of such design mistakes are equally varied, usually resulting in opaque code; complex logical errors; and more.

Figure 47 presents a short excerpt from a game server. The relevant piece of the manager's specification reads as follows:

The (board game) *server* waits for sign-ups over TCP ... for a minimum number (here, 4) of remote clients to connect and send over a name. ... If a sufficient number of players sign up by the end of the waiting period, the server hands over the players to a referee, sorted in ascending order of their "sign-up age." ...

The last clause means that the server hands the clients (represented as instance of the player class) to the referee in the reverse order of connection, that is, the "youngest" player starts first, as in many board games.

Here is how the creator of a solution and a panel might interact:

Here (figure 47) is the
server class.

Can you explain its state?

```

class Server:
    """
    a server for signing up remote players for a Train game
    """

    host:          str = "0.0.0.0"
    server_socket: socket.socket
    port:          int
    index:         int = 0
    proxies:       Set[ProxyPlayer] = set()

    def __init__(self, port: int):
        self.port = port

    def sign_up_step():
        """
        EFFECT tracks signed-up player-clients and their age
        """
        ...
        self.index = self.index + 1
        self._handle_1_client(client_socket, self.index)
        ...
        ... Referee(self.sort_proxies()).run() ...
        ...

    def _handle_1_client(self, client: socket.socket, i: int):
        """
        EFFECT adds a single player-client to `proxies`
        """
        ...
        with Timeout(CLIENT_NAME_WAIT, ...):
            player_name = client.recv(RECV_SIZE).decode("utf-8")
            proxy_player = ProxyPlayer(player_name, client, i)
            self.proxies.add(proxy_player)
        ...

```

Figure 47: Inspecting for design mistakes

Its state represents its external connectivity and the players that have signed up since the call to <code>sign_up_step</code>.	Ah, most of the fields make sense. Though it's not clear why there is an <code>index</code> field.
--	---

The index represents the reverse age of the connected players.	It might be best if we check how it is used.
Look at <code>sign_up_step</code> . It is the only public method.	Okay.
It waits for a connection, increments <code>index</code> , and calls <code>_handle_1_client</code> to deal with a single connection.	What happens if this call fails to complete the sign-up step? Isn't <code>index</code> wrong in this case?
"Wrong" is the wrong word because <code>index</code> does not have to be the precise rank in the sig-up order.	Is it enough if it increases over time?
Yes. Its value is stored with the proxy player so that <code>sign_up_step</code> can sort the proxies before it hands them to the referee.	This may make sense. Let's check <code>_handle_1_client</code> next.

The code inspection starts well, with a quick look at the information the class represents and a directive from the panelists at what to look next. By the fourth interaction, the panelists seem confused about the role of `index` and its value. While the presenter can temporarily diffuse the challenge, the panelists keep questioning the code with a direction.

<code>_handle_1_client</code> waits for the name to appear on the new connection.	Okay, I see that it creates a proxy player from the <code>index</code> . Why?
A proxy player has the same interface as a local AI player. But, instead of making game decisions locally, it forwards the data via its connection.	That makes sense for a proxy pattern. But why does it need <code>index</code> ? And why is it named <code>i</code> in this method?

As I said, the index is stored in the proxy so that <code>sign_up_step</code> can sort the set of players before it hands them over to the referee.	Is there any other use of proxies in the class?
No, there are just those two: one in <code>_handle_1_client</code> for adding a proxy.	And one in <code>sign_up_step</code> to sort?
Indeed.	This sounds like a <code>set</code> is the wrong choice of data representation.
The extensive integration tests pass.	No doubt, but a <code>set</code> is an unordered data structure, and the specification calls for an ordered one.
Yes, that's true. Perhaps it would be better to keep the proxy players in a list and just add new players to the front.	Doing so would eliminate <code>index</code> and the sorting step,
Oh my, choosing a list to represent the collection of players really simplifies things a lot.	Yes. But do add to the data-interpretation how the list is ordered.
We will.	Future readers will appreciate this.

The presenter and panelists jointly figure out a code design problem. Although the code is correct, the use of `Set` signals that the proxies are collected in an unordered manner. A future reader of this code would have to reconstruct the ordering relation. The code becomes much more comprehensible with the use of an ordered data structure to represent the information about the successful connections. And the use of a list directly echoes the age-order part of the specification, which may also help future readers.

This simple design mistake causes only minor problems. It misdirects a reader, and it consumes a bit of extra time to perform the sorting step. It is easy to imagine though that such small mistakes pile up and cost maintainers extra time to wrap their head around the code. Similarly, a lot of small performance losses add up to a large one. It is therefore good to catch and eliminate even such small mistakes as early as possible.

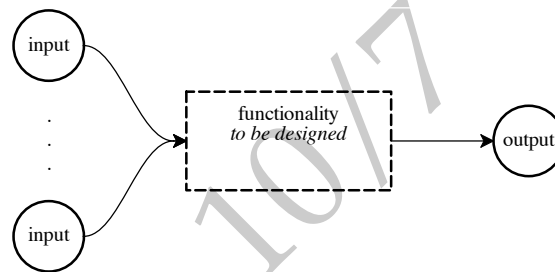
Exercise 23. Write a code inspection memo like the one in figure 44 for this code inspection. Have a partner review the memo and imagine how to respond to it.

mit: 10/7

22 Systematic Design: Data

Given that functions and methods are the fundamental building blocks, their development deserve special care. While “programming a function” might have served as a phrase in the early days of computing, here we use “designing a function” instead to bring across the importance of acting carefully and systematically at this level. The word “programming” encompasses just too many sloppy ways of creating code.

Systematically designing methods and functions starts with the proper and careful design of a data representation. After all, functions and methods consume and produce data, which is often illustrated with the most common diagram in computing:



In a sense, this kind of diagram tells the entire story of functionality design:

1. developers typically start from what kind information flows into and out of a piece of functionality; and
2. their task is to fill the box with code.

Information, however, is an idea concept; a programming language supplies only different forms of data. It follows that the first step is to choose data representation for the inputs and outputs, so that the software developer can base the creation of the functionality on those.

The question is where does data come from and what this implies for its representation. And this question brings us to the most critical insight concerning systematic program design:

Data represents the information that enters or exits the software system. Data also represents intermediate results, which flow from one component in the system to others.

So let's start with a close look at how developers systematically pick data representations for information from the outside world.

22.1 Developing Data Representations for Information

Software systems interact with their context by absorbing and injecting information. This context comes with many forms of information that the system must consume: names, distances, temperatures, pressure, touch, and so on. A device turns this information into plain bytes inside a computer, e.g., a keyboard, a mouse, a thermometer, a touch screen, and so on. These devices deliver sequences of bytes, and the software system reads these bytes. The act of reading turns information-as-bytes into data—specifically data in the chosen programming language. Conversely, a software system turns data into bytes, usually through some act of writing. The computer hardware delivers these bytes to devices, as pictures on screens, sounds on a speaker, directions to an A/C system, and so on.

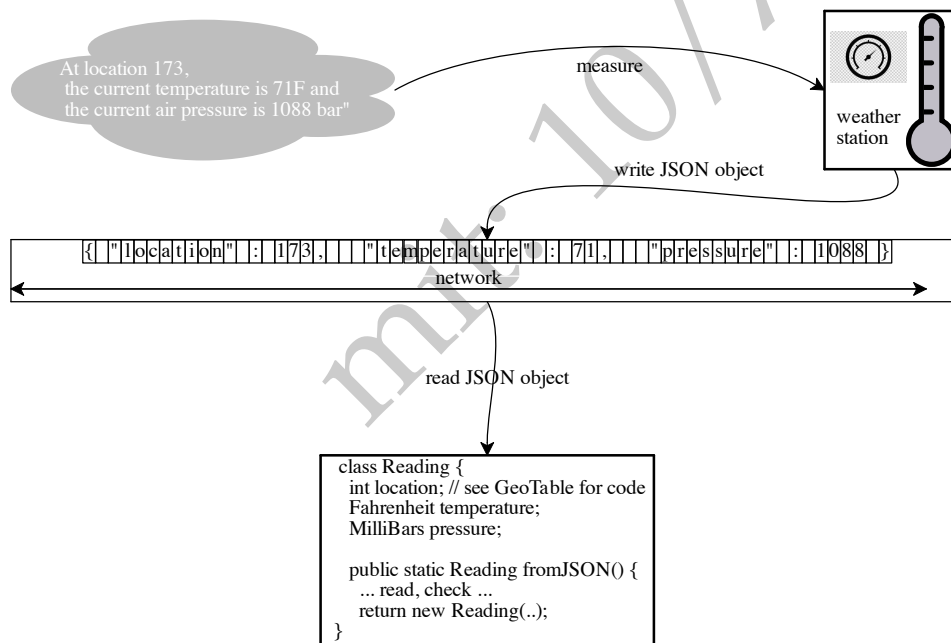


Figure 48: Information and data

A concrete example should help. Take a look at figure 48. It depicts a software system—the bottom box—for predicting the weather. The system



JSON is the currently popular choice. Tastes for such notations changes over time.

relies on weather stations to measure the current temperature and the air pressure at specific locations. These weather stations contain some simple software that capture the measurements and send them across the network to the prediction software as a sequence of bytes; these days, the bytes comprise a JSON object with appropriate attributes. The weather-prediction software reads this sequence of bytes—via some JSON library—and turns them into a data representation suitable for further processing.

In most cases, developers may ignore the byte-level layer and may instead focus on the correspondence between information and data. Turning one into the other is thought of as a direct, instantaneous step for the purpose of developing a data representation. In the context of the running example, the developer has chosen a straightforward data representation (bottom) for the information (top right): a class with three fields. What happens in between does not matter. Concretely, the `fromJSON` function in the `Readings` class translates this information into the internal data representation, and the implementation is usually just a call to some library function plus an optional check that the resulting data meets some conditions. The important choice is the `Reading` class, which for this simple example, makes objects with three relevant attributes.

Information that is Considered Atomic

Choosing a data representation for basic information is straightforward for contemporary programming languages:

information	data
single names	strings, such as "Bob"
distances	numbers, such as 1.5
keystrokes	chars, such as <code>#\a</code>
switch positions	Booleans, or symbolic values, such as "on", "off"

Arrays play the same role as strings in other contexts.

For certain kinds of compound—also known as *structured*—information developers have often made the mistake of using strings for plain text. So let's start with a negative rule.

Structured Information: Don't Use Strings to Represent It

Imagine a reader who encounters a program that represents a person's first name, middle initials, last name, titles, and suffixes as a single string. Here are six examples:

```

"Shriram Fisler"
"Shriram K. Fisler"
"Shriram K. Fisler, Jr."
"Shriram K. Fisler, Esq."
"Dr. Shriram K. Fisler, Esq."
"Prof. Dr. Shriram K. Fisler, Esq."

```

Imagine reading code based on this data representation. Over and over, you will encounter snippets that extract pieces from such strings. The accumulation of these code patterns imposes a cognitive load on the human reader. Next, imagine the processing that has to take place to extract the various pieces—the last name, the titles, the suffixes, etc.—from a plain string. In short, both human readers and the language implementation have a hard time coping with strings as a representation of this highly structured information. In short, these strings hide the structure of the information, and that’s bad.

Structured Information: Use Structs, Records, Objects

Structured information comes in many forms, but all of them share the principle of relating pieces of information to each other. Indeed, this is the definition of the word “structured” in the phrase “structured information.”

Take a second look at the examples of a person’s names with titles and suffixes. They illustrate how many pieces of basic information can show up and how they relate to the overall information: a medical doctor uses “Dr.” as a prefix title; a stuffy professor may want students to use “Prof.” and “Dr.” as prefixes; a person typically has a first and a last name; a lawyer may use “Esq.” (short for “Esquire”) as a suffix title. For this form of information, a developer can send a clear picture of the pieces of information and their relationship via a record structure:

```

(struct person
  [{titles : [Listof String]}
   {first  : String}
   {middle : Char}
   {last   : String}
   {suffix : [Listof String]})

(define example
  (person [list "Prof." "Dr."] "Shriram" #\K "Fisler" '[]))

```

In code that uses this data representation, a reader is going to see expressions such as `(person-first example)`, and it is immediately clear what they produce. Similarly, the language implementation is going to

evaluate this expression in constant time, translated into just a few machine instructions.

Here are some other examples of how complex information might be represented with data:

information	data representation
names of people,	a struct of strings
amounts of money	a struct, an exact rational
points and directions	vectors, complex numbers
a handful of game money tokens	a set of ...
sequence of temperatures	a list, for sequential access
sequence of temperatures	an array, for random access
sequences of names of people	a list of structs
hierarchically arranged decisions	a tree
the ancestors of a person	a tree
people, their social relationships	a graph

Exercise 24. Pick one case and figure out the rationale for the recommendation on the right-hand side of the table.

The above table mentions lists, trees, graphs, and so on. In many cases, these forms of data are ideas that developers inject into the conversation about information. The “graph of people” does not exist in the real world; individual pieces of information make up the collection of these links. In the world of data—in whatever programming language—this information is turned into a single piece of data so that developers can have an effective conversation about what kind of value flows from piece of code to another, and the execution of the code satisfies basic efficiency requirements.

From Information to Data Representations, And Back

Stepping back, the structure of information tends to guide the design of data representations. Basic information maps to atomic data. For structured information, contemporary languages come with an extensive collection of data-structure libraries. Often but not always, these libraries make representation choices straightforward. If a developer must represent a sequence of information—say the names of athletes in the order in which they finished a marathon—then it is natural to use a built-in list type. Even for structured information as complex as a graph, a language’s ecosystem may supply a suitable form of data structure—in this concrete case, a graph. But yes, many problems still call for bespoke data representations.

In case the structure of the information does not easily map to a supported data collection, it remains imperative to proceed systematically with the development of a data representation. A systematic development tends to be reflected in the resulting choices and code elements. It is thus likely to convey the intention of the creator.

The Representation Statement Once a developer understands the pieces of information that need a data representation, there are two key preparation steps to take. First, the developer must identify the relationships among the pieces of information. Second, the developer must articulate a focused representation statement—something akin to thesis statements for paragraphs in an essay. The goal of formulating a representation statement is to delineate what is—and what *isn't*—represented. The importance of a clear thesis grows exponentially with the number of data representations, the size of the code, and the relationships among these pieces.

Consider the following example. Suppose we wish to construct a software system that allows people to explore the biological ancestor relationships among people. Perhaps they want to make sure that a newborn gets the name of some ancestor—as is common in some Western traditions—or that the newborn's name does *not* coincide with the name of any ancestor—as is common in some Eastern traditions. This short description of information clearly identifies *a person with a name* as one kind of information and ancestor-relationship as a second one.

Given this context, a developer could start the development of a data representation with a code snippet like the following:

```
// ... represent naming information for a single person
struct Person [first : String, last : String]

const jon : Person = {first = "Jon", second = "Parson"}
const ada : Person = {first = "Ada", second = "Parson"}
const bob : Person = {first = "Bob", second = "Parson"}
```

The snippet states up front that it is a data representation for a person. A `struct` is appropriate because as the data about one person flows from one function to another, the names should stay together. Note how this data representation does not mention anything about ancestor relationships. By writing down the thesis, the developer can focus on the *one* task at hand: representing naming information of a person. Anything else is out of scope. Finally, the snippet illustrates the chosen data representation with concrete examples. Given the simplicity of information, it is obvious how the developer takes information about a parson and turns it into data.

Concisely put, a representation statement explains

how every instance of information is represented as data.

Using this statement as a guideline, a developer can implement the functionality that turns a sequence of bytes into a (useful) data representation.

The Interpretation Statement The developer-and every maintainer must also understand

how pieces of data are interpreted in the realm of information.

Using an interpretation helps implement the functionality that turns a piece of data into a sequence of bytes that affect the actual world. Without these two statements, data representations can cause serious trouble. Remember the interpretation of `distance` from section 19 and how the lack of an interpretation can cause a huge amount of damage.

Note The word “every” is missing from the above guideline. Formulating a data representation determines a set of values in the chosen programming language. Due to the nature of programming languages, it is often extremely difficult or even impossible to describe this set in such a way that every piece of data has an interpretation.

Let’s look at some examples of interpretations:

data representation	possible interpretation(s)
<code>int distance</code>	in m, cm, feet, yards, etc.
<code>List<Person></code>	an alphabetical list of members
<code>BinTree<Person></code>	a (binary) tree of ancestors
<code>struct Name { String first; String mi; String last; }</code>	does this choice need one?

Here is one more:

```
struct Coordinate {
  int x; // the distance from the left margin
  int y; // the distance from the top (in pixels)
}
```

It describes the common screen coordinate system.

Exercise 25. For each example in the table, explain why an interpretation is needed and/or make up some example data to illustrate the interpretation. How does the data definition of `Coordinate` differ from the

Cartesian coordinates you know from middle school? Does this data definition need the interpretations of the two fields?

In summary, developing a data representation calls for two comments: (1) one that explains how information is *represented* as data and (2) another one that explains how a piece of data is *interpreted* as information. In many cases, it is also good practice to come up with some examples that illustrate these two comments. A developer can re-use these data examples to work out examples of functionality and to prepare tests.

Representing Relationships Among Pieces of Information While a person and the name of the person are concrete pieces of information, most compound forms of data lack such concrete counterparts in the world of information. Recall the example of representing the ancestor relationship among some persons. Developers tend to refer to the *ancestor tree* and to use binary trees as a data representation. But, obviously, there is no such tree in the real world. It is an imagined, abstract element of our application domain.

When pieces of information are related in this imaginary way, it is best to introduce a separate data representation. Here is a sketch of an ancestor-tree representation:

```
// ... represent ancestor relationships among Persons
type Tree = Known | Unknown

struct Known [person : Person, mother : Tree, father : Tree]
struct Unknown[]

const u : Unknown =
const f : Known = {person = jon, mother = u, father = u}
const m : Known = {person = ada, mother = u, father = u}
const c : Known = {person = bob, mother = m, father = f}

// 'c' is the ancestor tree of the child of 'm' & 'f'
// neither 'm' nor 'f' have known ancestors
```

The representation statement delineates this data representation from the one for a `Person`, which is just a pair of names. That is, it is *not* about the representation of a person's names, but about the ancestor relation among persons. The examples of ancestor Trees are constructed from the examples of `Persons`. Although the interpretation of this particular data representation is apparent, the last two lines of the snippet show what a developer might want to write down to illustrate the data interpretation.

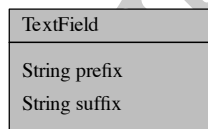
From Information to Classes via Doodling

Since the proper development of data representations lays the foundation for systematic design, let us take a close look at this first step, in the context of class-based, objected-oriented programming, which still dominates many parts of the software development field. The following list of scenarios is not exhaustive but covers many cases. It simultaneously articulates some basic principles and suggests a sketching notation for “doodling” data representations (on a napkin or a black board). You may have encountered both in a prior course on programming, so consider this list a concise summary.

Combining Several Simple Pieces into One Bundling several pieces of information into one piece of data is the most basic choice of a data representation. Doing so signals to the developer and the reader that these pieces belong together. While the class’ methods can access these pieces; the developers of client methods are forced to think of these as a single value.

The preceding subsection introduces one such example, a named person. Other examples are coordinates of a point in space or an amount of money and its denomination.

Let’s make this concrete with a virtual example of information, a text-field editor. Assuming that editing takes place at the position of the cursor, the data representation of such an editor combines two distinct pieces of information: the part of the text that is to the left of the cursor, and the part that is to the right. Representing these two pieces of text as a strings is a natural choice, and thus the appropriate graphical notation is this:



The notation says that the `TextField` representation is a class of objects, each of which comes with two strings.

Even though this particular class signals a straightforward interpretation, a diagram like the above should come with a concise comment:

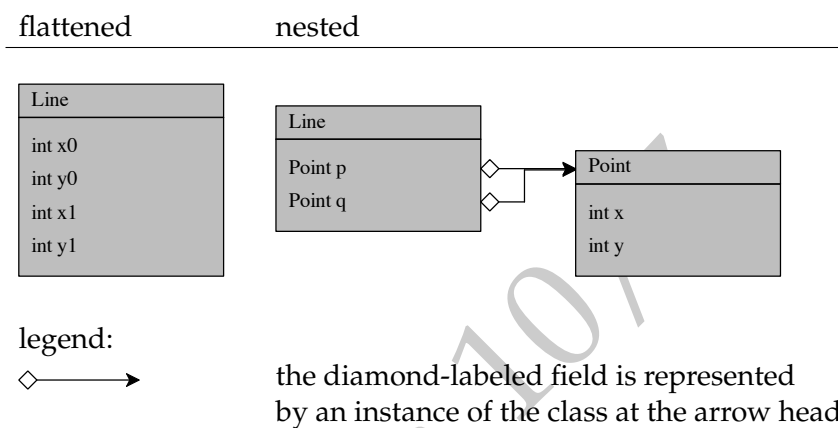
```

TextField tf = new TextField("hello ", "world");
// if the cursor is a vertical bar,
// `tf` represents "hello |world"
  
```

This one-line statement can be understood as both a representation statement and an interpretation. It definitely suffices for a client creator to continue the development from here.

Combining Simple and Complex Pieces into One As mentioned, the structure of information should drive the development of a data representation. If a piece of information refers to other forms of information, a class-based representation should consist of classes that refer to each other.

Take the example of a line. You learned in your geometry class that two points determine one line. To determine a Cartesian point in the plane, it suffices to fix two numbers, say integers. Based on this description, a developer may contemplate two different data representations of a line:



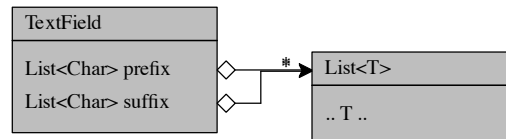
Stop! Which one would you choose and why?

The explanation in geometry books clearly identifies two distinct pieces of information: lines v. points. Given this context, it is clear that a nested representation mirrors these textbook explanations. By contrast, a flattened representation imposes a small, but non-trivial cognitive load on the reader, namely, to realize that (x_0, y_0) determines one point and (x_1, y_1) the other. Since the domain of geometry also comes with distinct operations on points and lines, which occasionally depend on each other, a separate and concept-focused representation has several advantages:

- a developer knows where to put an operation;
- a reader knows where to find an operation; and
- the arrow tells both that if an operation in the line representation needs an operation on a point, how the two methods relate.

On occasion, a class may also describe objects that contain instances of library classes. This form of containment is especially useful when the field

represents a collection of pieces of data, and a suitable form of collection container comes with the chosen programming language. For example, a developer may quickly realize that a data representation for a text-field editor should use lists of characters instead of plain strings:

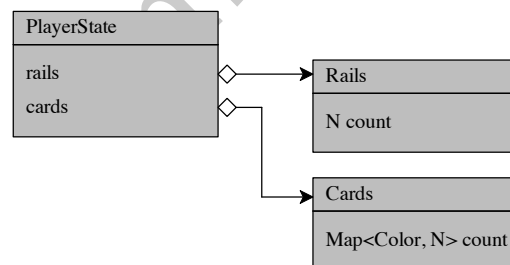


legend:

◇ —*→ each diamond-labeled fields is represented by an instance of the container class at the arrow head

This alternative data representation for text fields is inspired by the insight that editing operations address individual characters, meaning it is best to represent the two pieces via `Chars` not `Strings`.

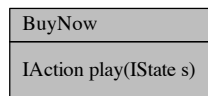
From far enough away, the two nested diagram look the same. Each comes with one class whose two fields point to the same secondary class. This is a mere coincidence, however. To make this point explicit, here is a doodling diagram for representing the state of an automated game player for the *Ticket to Ride* game mentioned in the preceding chapters:



In this diagram the main class also has two fields, but each points to two distinct classes.

Actions are Information, Too Functional programmers understand that actions *are* information and occasionally deserve to be dealt with as data. Consider the somewhat abstract and mathematical example of infinite sets, which a logician may represent with a characteristic function from elements to Booleans; if the result is true, the element is in the set and otherwise not.

An example close to our game-server examples is a game-playing strategy, which is an action in response to understanding the current game state (and some planning). With just a bit of analysis, a well-trained developer understands a strategy as a function. In a class-based language such as old-style Java, this data representation is a class with one method, often called `apply`, which realizes function application:

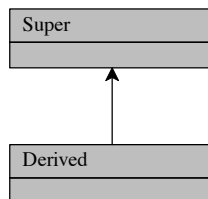


In a functional language, a strategy might just be a first-class function that is plugged into a player mechanism, which is a second-order function.

Modern Java comes with first-class functions, too.

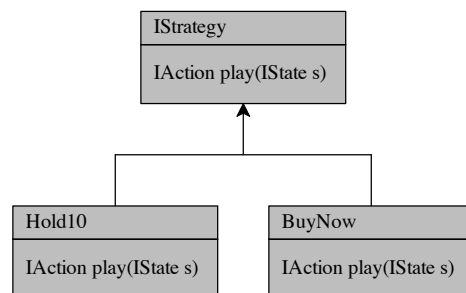
Viewing Distinct Pieces Uniformly When a form of information comes in distinct flavors, the rest of the program should use all pieces uniformly. In a class-based language that supports Java-style interfaces, we can think of this case as a *type* with several *variants*, that is, the interface and the classes that implement it to represent the various flavors. If a developer must use a language without interfaces, a class with dummy methods can serve the same purpose.

Sketching such an arrangement requires a new kind of convention:



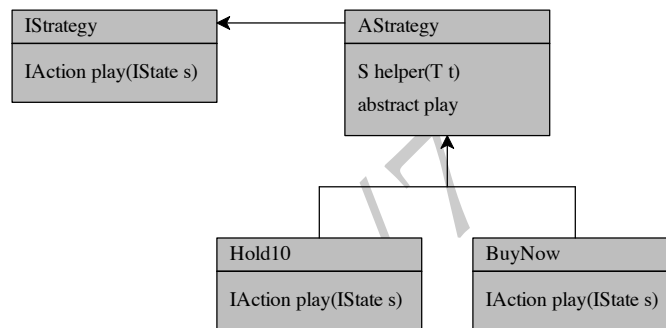
A plain arrow denotes one of two situations: (1) the `Derived` class implements the `Super` interface or (2) the class extends a `Super` class.

Let's use this new convention to doodle a data representation for an automated player that can execute one of several strategies:



The player class realizes the mechanics of performing actions that are independent of a strategy. Hence a developer is likely to program its implementation to a uniform strategy interface. Then again, the implementors of such players may wish to try out different strategies or even compositions of strategies (say chains). The above diagram sketches how an interface can uniformly describe two game-playing strategies.

Representing information with variants tends to lead to a data representation with abstraction:

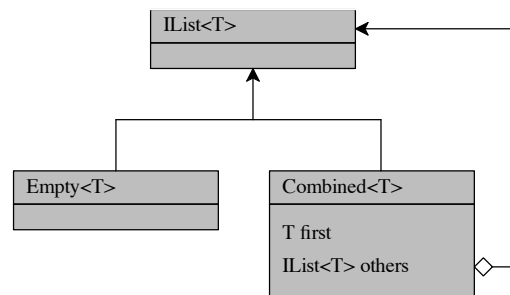


This diagram indicates that the strategies are used uniformly via an interface and that they share some common helper functions. Indeed, this arrangement may even set up a template-hook-pattern, which lays out the general play function in AStrategy with some “holes” so that the concrete implementations just fill in those “holes.” The key to either arrangement is that it is now straightforward to add more strategies so that the player’s developer can experiment some more.

Arbitrarily Large Pieces of Information The final form of information to be represented is the most interesting one: when there is no limit on the size of an individual piece. While this idea may surprise you at first, it is actually the case that all interesting computations involve such forms of unlimited information. Keep in mind that “no limit” does not mean “infinite.”

Simple examples from the real world are sequences of pieces of information: a grocery receipt; tables of a person’s stock holdings; listings of home for sales; and so on. Most such listings are of variable length, and there is no predetermined limit on the length of such sequences. You never know how many pop tarts a father has to buy for a growing family. Companies come and go; few companies from 100 years ago are still listed on the stock market yet many more companies are public now. Similarly, homes are added to sales listings and removed all the time.

Ignore for a moment that most programming languages come with a built-in form of list data, which a developer could use to represent such sequences. If a list representation had to be developed, a developer might start with a diagram like the following:



Before you ever even consider using `null` to represent empty lists, research “null billion dollar mistake.”

The interface enables uniform access to all forms of lists. At the same time, the data representation must distinguish at least two list variants: a base class that represents an empty list or a list of one item perhaps versus a class that represents a lengthening of an existing list, one that combines an item with an existing list. This last insight leads to a *self-referential* or *recursive* doodle diagram, one where arrows form a cycle.

While diagrams with cycles look complex at first glance, a second take tends to produce insights into what the data representation encodes. Consider the problem of representing arithmetic expressions, specifically expressions in what is dubbed Pressburger arithmetic. Roughly speaking, expressions in Pressburger arithmetic consists of variables, constants, and additions; practical extensions allow multiplication by constants. (Why?)

Pressburger expressions are surprisingly common in programs. They thus play a special role in compilers.

Take a look at the following three pieces of text that someone may write down in an attempt to make examples of our informal definition:

$4 + 5 * x + -1 * y$	valid Pressburger arithmetic expression
$4 + y * x + -1 * y$	valid general (Peano) arithmetic expression
$4 + 5 * x - 1 + * y$	ill-formed

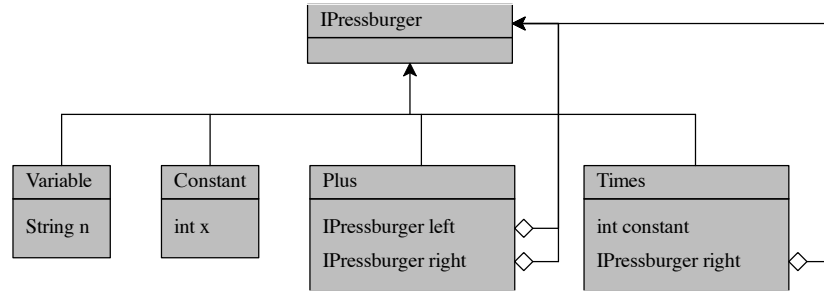
Stop! Explain the comments next to each expression.

Like all data, arithmetic expressions are likely to be sent as sequences of characters—essentially, strings. Since only some strings are valid expressions and of those only a (strict) subset are Pressburger expressions, a software system must traverse these strings and turn them into a proper data representation. Well-designed programs turn such expressions into trees, an idea that research on programming languages has confirmed time and again over five decades. Even though there is a machine-specific limit on

Turning such strings into the data representation presented here is dubbed parsing in the study of natural and artificial languages.

the size of such expressions, it is much better to choose a tree representation that does not impose any limits upfront.

Here is a cyclic doodle diagram, a first step in the development of a data representation for Pressburger arithmetic:



Stop! How does the diagram mirror the above definition?

Exercise 26. Represent the valid Pressburger expression from above as a tree, using notation from a class-based language. Why is it impossible to represent the valid Peano expression? Sketch a method that finds the set of all `Variables` in such expressions.

22.2 Choosing from Several Alternatives

On occasion, a developer can quickly see that there are several structured data representations for the same information. When this happens, the developer has to think ahead and consider possible uses of the data and has to be prepared to revise any decision as unanticipated uses appear and demand different pieces of functionality.

Here are some questions to consider when faced with such a choice:

- how easy is it to create an instance and how much does it cost?
- which operations are frequently used and should be inexpensive?
- which operations are rarely used and are acceptably slow or costly.

Above all, though, a developer must keep in mind how central the data is to the overall software system. If this data is created and used once, it may not deserve much attention; then again, if it is used in the inner loop of a central algorithm, paying attention to these questions from the very beginning makes a lot of sense.



Answers to these questions are often discovered once developers measure the end-to-end performance of the complete system. This kind of measurement may reveal that the chosen data representation is missing an access capability. For example, the measurement may clarify that client components need random access to pieces of a sequence, but the chosen data structure allows only sequential access.

At this point, developers must backtrack and revise the choice of (hopefully properly encapsulated) data representation. Developers tend to resist the idea of going back and changing a data representation—often because they know how much time it will cost to understand the existing code and to identify what impedes performance. Once again, a properly developed and encapsulated data representation—with focused purpose statements and systematically structured elements—greatly facilitates this step. That is, it helps the older version of the developer understand the younger version's choices and make changes to it. Without systematic design, modification may end up costing a lot of time and mess up unstated assumptions.

Recall the two data representations for a test-field editor's state: one just uses two strings and the other two lists of characters. A developer is likely to observe that the implementation of cursor-based editing operations demands repeated access to individual characters. Indeed, these methods always access the last character of the prefix and the first of the suffix. Hence, if the data representation uses linked lists, it is also the case that the prefix is best represented as the *reverse* of the sequence of visible characters.

A forward-looking, socially responsible developer knows that this logical invariant deserves an interpretation comment:

```
class TextField {
    private List<Char> prefix;
    private List<Char> suffix;
    // the prefix is the reverse of the visible chars,
    // so an instance represents text as follows:
    // -- prefix.reverse().toString(),
    // -- followed by the cursor, and followed by
    // -- suffix.toString()
    //
    // Example:
    // List<Char> prefix = ['o','w']
    // List<Char> suffix = ['r','l','d']
    // denotes "wo|rld" where | is the cursor
    ...
}
```

*A full text editor
demands an even more
sophisticated data
representation than
two lists.*

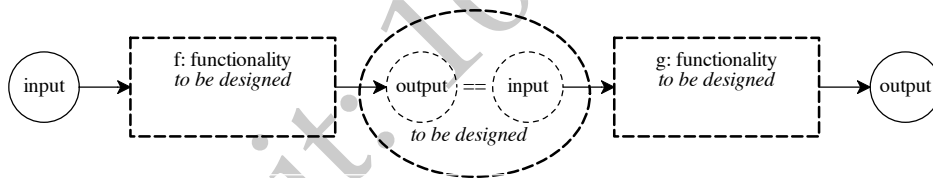
Furthermore, the interpretation is illustrated with an example and a comment that specializes the abstract interpretation statement for this example.

22.3 Developing Data Representations for Composition

No program and no software system consists of a single function, even if it acts like one from the outside. To realize its goal, an actual software system composes many functions and methods. Proper programming is thus the art of dividing large tasks into small ones, and tackling one at a time.

In mathematics, composition means that a function f produces outputs that some other function g consumes as input. To a developer using a functional approach, it means the same. For a developer still relying on old-fashioned imperative programming, the flow of data from one “function” to another is implicit; the first one is likely to deposit results in fields and variables, and the second one is supposed to retrieve the results from there as needed. Abstractly put, though, the two approaches don’t differ, because both assume a flow of data from one piece of functionality to the other—and setting up this connection properly requires systematic design.

Given this, the following diagram summarizes the design problem for a software developer:



Even a glance at the diagram suggests the difficulty. Instead of one box labeled “to be designed” in the initial design diagram (see page 184), this one contains *two* such boxes *plus* an ellipsis in the center also labeled “to be designed.” The point is that designing the two boxes and the intermediate form data are mutually dependent problems, meaning a change in one is going to change the other two.

While the structure of external information dictates the representation for the input (for f) and output (for g), the design of the intermediate form of data has no such obvious constraints. Its design is about reconciling two sets of constraints: keeping f simple and making its output a *good* input for g . It is this need for reconciliation that has to inform this part of developing a data representation.

Often the development starts from an observation about the overall functionality, wishing “if only, the data had a particular shape.” That is, a developer expresses a desire for a data representation of information that explicates implied information. By implication, the developer is asking for

a function f that consumes the given data and explicates some aspect as additional data, so that g can easily compute the desired result from this additional data.

Generally put, the request for turning some implied property into a data representation is a common way of developing intermediate result data. Additionally, dividing a large task into smaller ones is a recursive process. The design of both f and g may benefit from further divisions of labor—and when this further subdivision is considered, the idea of explicating some existing but implied property comes into play again.

Recall the trees-of-ancestors example or the graphs-of-friendships that the preceding section mentions. Both of these descriptions contain words—“tree” and “graph”—that do not really refer to things in the world, while “ancestor” and “friendship” are concepts that people accept, even if they are invisible. The point is that these relational pieces of data are often the result of a function such as f , which connects the software system to the inputs from its environment. The challenge is to think through possible future uses of these forms of data so that internal functions like g benefit from the work of the first function.

Let’s make this example even more concrete. Imagine a subway information system. Every morning, the system is booted with information about the currently operating stations and connections. Customers walk up to public information kiosks—or open an app on a mobile device—and query the system about how to get from one station to another. One possible response in Boston could be “board the red line, direction Alewife; switch at Park to the green D line, direction Reservoir; the Fenway stop is the sixth stop on this line.” Finding a route from one place to another is a graph-search problem but the input to the software system is not.

Assume that the input is something like descriptions of the operating subway stations and lines. For example, one such (greatly simplified) input might be a bunch of JSON arrays, each one for a complete line:

```
[..., Fenway, Kenmore, Hynes, Copley,  
    Arlington, Berkeley, Park, ...]
```

Functionality f would turn this linear data into a graph representation, and g might be the function that queries the graph to respond to individual customer questions.

Even though it is clear that a graph representation is needed, it is still open which one serves g best. If the developer expects that many riders need the same information, the data representation may consist of a graph and a cache. Furthermore, a subway line is not an arbitrary graph of in-

A developer may also have to consider the relationship between the function's organization and the output data.

formation but comes with additional implied structure; a developer might anticipate that representing this structure could also speed up queries. Put differently, just because the developer knows that it is necessary to turn the given information into a graph, it is still necessary to explore different alternatives. Exploring means picking an implied property that may facilitate the design; sketching the two related tasks; and weighing the pros and cons of the approach. In all cases, the consideration of “ease of designing a function” should start from the shape of the input data because it dictates the internal organization of the functionality. If this relationship isn’t obvious, a developer should look for alternatives.

mit: 10/7

23 Systematic Design: Functionality

Once a developer has developed data representations, it is time to systematically design functionality, that is, functions and/or methods. Thus far, we have mostly acted as if the two concepts are the same. Although they are closely related, the difference deserves an explanation.

Almost all programming languages support the definition of functions and methods. In a class-based, object-oriented language such as Java, a function is a static method. In a functional language such as F#, a programmer can emulate methods by storing functions in a structure (record) and following the special protocol of calling such functions with the structure itself as the first argument. The Racket language supplies a syntactic facility for this emulation and supports classes and methods in this manner.

```
interface IListN {
    // add 1 to each int on 'this'
    IListN add1()
}
```

```
class EmptyN
    implements IListN {
    IListN add1() {
        return this;
    }
}
```

```
class CombineN
    implements IListN {
    int first;
    IListN rest;
    IListN add1() {
        return new CombineN(first + 1, rest.add1())
    }
}
```

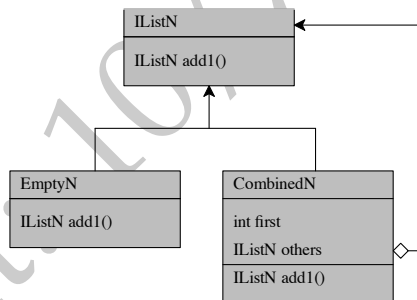


Figure 49: Integer Lists in Java, implemented by hand

It usually takes several methods to implement a complete function. This idea is best illustrated with an example. Consider the case of adding 1 to every item on a list of numbers. A linked list represents sequences of arbitrary size. In a “doodle” diagram for the corresponding classes, this relationship needs a description that points back to itself and comes with

This example is contrived to keep it concise. Experienced developers would use libraries.

(at least) two variants: empty and a combination of an existing list with another element. In Java, this arrangement is best represented with an `interface` and two implementing classes. See figure 49 for the “doodle” diagram and the methods.

Java currently undermines this form of systematic design, due to its failure to implement calls properly.

In a class-based language, a systematically proceeding designer implements this functionality by realizing the interpreter design pattern. This demands adding a method to every “box” in this class diagram. The method in the interface specifies the header of the functionality; that is, it represents the *specification* of the function. The two methods are *implementation* pieces of the functionality, i.e., the two methods implement the part of the functionality that is relevant to the data represented by the two classes, respectively: the empty list or the extended list.

```
;; data definition:
(struct empty-list [])
(struct combine [first others])
;; A [List X] is: (1) (empty-list); (2) (combine X [List X])

;; function definition:
#; {[List Number] -> [List Number]}
;; add 1 to every element of 'alist'
(define (add1 alist)
  (match alist
    [(empty-list) (empty-list)]
    [(combine first rest) (combine (+ first 1) (add1 rest))]))
```

Figure 50: Integer Lists in Racket, implemented by hand

In a functional language, a developer might use an algebraic datatype; in an untyped one such as Racket, a programmer introduces such a type with two struct type definitions plus a data-definition comment that ties them together. The “add 1 to all numbers” function follows easily; it deals with the two cases in one conditional, one clause per form of data. See figure 50. This function definition uses algebraic pattern matching to distinguish the two cases and to deconstruct the given list. Each clause in the conditional corresponds to one of the concrete methods in the Java design.

In general, the design of a function is dual to the design of the same piece of functionality with methods. A functional design uses a conditional to separate the various forms of input data from each other and to deal with

each separately. By comparison, an object-oriented design places a method with each relevant form of data; the dynamic dispatch of the language realizes the conditional of the function design implicitly.

Note An object-oriented developer should use conditionals only to distinguish distinct *result* situations; in a functional language, such conditionals are nested within the dominant conditional in a function.

Both designs assist developers with creating the functionality and readers with comprehending it. By factoring a piece of functionality into separate methods—one per data variant—the developer can focus on just one case at a time, ignoring all others; the same focus is established via a conditional for data variants, though the syntactic proximity of the other cases reduces this focus a bit. When developers read systematically designed functionality, they can again isolate cases and focus on comprehending those, one at a time, without distraction from other cases.

Unsurprisingly, these approaches have symmetric advantages and disadvantages. Roughly speaking, object-oriented program design facilitates the addition of new variants of data. A developer just adds yet another class that implements the specification interface including its methods. By contrast, functional design makes it easier to add another piece of functionality; it suffices to design a function. Most modern programming languages lead developers to favor one design over another but still accommodate the other one. Developers must keep these different styles and their implications in mind as they move from the data development stage to the functionality design stage.

Exercise 27. Use your favorite IDE to run the program from figure 49 in your preferred object-oriented programming language.

Design and add the `product` functionality, following the same object-oriented recipe used to produce the program in figure 49. The purpose of the functionality is to compute the product of all numbers. Then add `CombineN2`, another implementation of `IListN`. It is a variant that adds two `ints` to an existing list.

If you know a functional language, use it to solve the same problems.

23.1 Designing Functions, Systematically

Assuming the data representation has been chosen, it is time to design the desired functionality. Pieces of functionality come in two flavors:

- A function is *atomic* if it performs concrete tasks on the chosen input data, without demanding other pieces of functionality.

- A complex task specification calls for the design of a composite functionality, that is, a composition of several functions for the chosen data (and possibly some related ones).

The design of these functions may demand the development of an additional form of intermediate data that connects two of them. That is, the development of data representations and the design of functionality can be intertwined.



How to Design
Programs

The two flavors demand related, but different design approaches. Both design approaches demand discipline. Both benefit from proceeding according to the following steps.

The Purpose Statement The first goal is comprehend the specification of the desired computation, which come in the form of a “task ticket,” a paragraph in a book, a verbal request, or some other form. By deriving and articulating a purpose statement from the problem description, a developer demonstrates a degree of comprehension. A purpose statement answers the question *what* the function computes from the given inputs. Until this question is answered, a systematic design is impossible. When the time comes to use AI, the purpose statement is essentially the first draft of a prompt.

The answer also tends to settle whether the goal is an atomic and/or composite function:

- The design should aim for an *atomic* function, if it is straightforward to articulate what the function computes from its inputs. The word “straightforward” merely means that the function is self-contained and that it is unnecessary to design any auxiliary functions.

In case this approach still yields an overly large function, it might still be a good idea to turn it into a composite function.

- The design goal should be a *composite* function if (1) the data definition refers to other data definitions or (2) the purpose statement enumerates several tasks.

Data definitions are commonly related in a hierarchical fashion, i.e., one data definition refers to a another, which refers to a third and so on. A request to develop a function for the first data definition immediately implies the composition of several functions, one per data definition. When a function for the first data definition needs to compute something concerning the second one, it should delegate to a second function on the second datatype and so on.

For novice students and even junior developers, it is good practice to explicitly articulate the purpose statement and to write it down. Experienced developers may be able to convey the purpose with a well-chosen name, but they should also consider who will read this code in the future.

The Type Signature Articulating a purpose statement benefits from having a type signature for the function, and vice versa. Without an understanding of what kind of data flows into and out of the function, it is often difficult, if not impossible, to comprehend the desired computation. This applies to both the creator and the reader of code.

While typed languages enforce the presence of a signature, languages without a (checked) type notation leave it up to the developer to imagine a way of stating type signatures. The upside is that the signature statement may express more than a conventional type notation. The downside is that this statement is neither type-checked nor runtime-checked, meaning that it might be out of sync with the function definition just like most comments. Even though no type checking takes place, it is still important to define all the terms used in such an informal signatures—with a type-like notation.

Working Through Behavioral Examples Using the purpose statement and the signature, the developer is in a position to work through examples. Each example starts from a piece of data that belongs to the input and yields an expected output. The data definition for the inputs should suggest representative elements as well as corner cases.

The goal of “working through examples” is to gain an understanding of *how* the function might compute its results:

- If the goal is to design an atomic function, the developer should be able to imagine how every step of the work can be implemented with built-in language constructs, basic functions, or library functions. Recursive calls are also fair game.

In the uncommon case of mutually recursive data definitions, a developer will essentially create several atomic functions in parallel.

- By contrast, if the goal is a composite function, the process of working through examples may appeal to existing functions, functions already on the developer’s todo list, or functions yet to be specified.

Working through examples naturally produces (ideas for) tests: unit tests for atomic functions and simple integration tests for composite functions.

The Outline & Programming For many cases of input (or output) types, the type itself suggests how to organize the function. The analogy to writing is laying out all the ideas and organizing them into an outline.

The outline for an atomic function differs a lot from the one for a composite function:

- For an atomic function, the outline often involves distinguishing distinct forms of inputs or choosing an appropriate form of iteration over the given inputs.
- For a composite function, outlining means dividing the given data among the helper functions identified in the previous steps and figuring out how to compose the results of these function calls.

At this point, the developer has a complete outline of the function and “programming” means filling in gaps in this outline. Since each piece of the outline delivers some intermediate result, “filling the gap” means to combine these intermediate results into the final result that the function is supposed to compute.

Tests The creation of a function is not finished until it has been equipped with unit tests. As mentioned, working through examples naturally leads to tests. Running these tests exposes basic mistakes in the code; it does *not* confirm the correctness of the function(s). Conversely, fixing a bug must start with the addition of a failing test that replicates the problem. Finally, if the chosen programming language permits the placement of tests next to a function definition, a developer should do so to illustrate uses of the function, especially for functions whose calling protocol is complex. Doing so helps future readers.

23.2 Atomic vs Composite: A Tiny Case Study

Let’s take another look at the already-mentioned game program in which a *referee* component interacts with *player* components. When it is a player’s turn, the referee shares the publicly visible part of the current game state. In response, the player requests the execution of some action on this state on its behalf. Naturally, before the referee executes this action request, it must check its legality with respect to the rules of the game.

In a game such as *Ticket to Ride*, the game is played on a game map, which consists of connections between places. A player’s goal is to collect connections. To do so, the player trades assets in its possession for connections during a turn.

Here is the precise rule:

The *referee* hands *players* colored cards and rails, which they keep hidden from each other. With these assets, players may acquire and occupy available connections on a game map. If a connection of color c consists of n segments, a player must have n cards of color c and n rails to *acquire* and *occupy* the connection, respectively, during a single turn. Once a player has acquired and occupied a connection, other players cannot collect it.

From the perspective of software development, this rule description is the problem statement.

To start with, the rule description implies that a developer needs to choose data representations for game maps, connections, game states, and player states:

```

#; {type GameMap = .. Connection ..}
;; the map on which the game is played

#; {type Connection = ..}
;; ... and the connections it consists of

#; {type State = [List PlayersState]}
;; the referee's knowledge about the state of the game
;; which is just the state of all players
;; and the order in which they take turns

(struct pstate [... cards ... rails ...])
#; {type PlayersState = ... Cards .. Rails .. }
;; a referee's knowledge about a single player

```

As for the desired functionality, a developer teases out the action words in the problem description—the words in *italics*—to articulate a purpose statement, which says that the referee component must come with a function that determines whether a player may acquire and occupy a connection in the current state of the game, given a fixed game map. Since the problem description is relatively compact, a developer might wish to write an equally compact function definition. Figure 51 shows the result. The function computes the set of available connections; makes sure the requested connection is a member; and then evaluates the conditions about acquiring and occupying the connection.

Exercise 28. According to the preceding section, a developer must be able to connect each task in the problem statement with elements of the implemented function. Use a red pen to circle the pieces of legal-action?

```

#; {GameMap State Connection -> Boolean}
;; can the active player collect (acquire & occupy) the
;; given connection in this game state
(define (legal-action? gmap gs conn)
  (cond
    [(not (set-member? (available-connections gmap gs) conn))
     #false]
    [else
     (define active (state-active-player gs))
     (define seg# (connection-seg# conn))
     (and
      (<= seg# (pstate-cards active (connection-color conn)))
      (<= seg# (pstate-rails active))))))

```

Figure 51: An atomic version of `legal-action?`

in figure 51 that correspond to the act of acquiring the connection, and use a green pen to circle the pieces relevant to occupying a connection.

A look at the purpose statement of `legal-action?` in figure 51 suggests a design problem, and it implies a design alternative. The enumeration of *two* action words—“acquire” and “occupy”—suggests that `legal-action?` should be a composite function. Strictly translating the problem statement into hierarchically ordered tasks yields the following:

- see to the availability of the desired connection;
- check that a player can collect the connection, which in turn implies the execution of these two tasks:
 - one that ensures the player can acquire the connection with n cards of color c ; and
 - another one that checks that the player can occupy the connection with n rails.

That is, the desired functionality consists of two function calls, and the second one composes two more auxiliary functions.

The code of figure 52 presents this alternative. Each identified task is its own function. As a result, the functions completely align with the sentences in the rule description. Better still, each function can be understood and

```

#; {GameMap State Connection -> Boolean}
;; can the active player collect the given connection
(define (legal-action? gmap gs conn)
  (and
    (set-member? (available-connections gmap gs) conn)
    (can-acquire-and-occupy? (state-active-player gs) conn)))

#; {PlayersState Connection -> Boolean}
(define (can-acquire-and-occupy? active conn)
  (and (can-acquire? active conn)
    (can-occupy? active conn)))

#; {PlayersState Connection -> Boolean}
(define (can-acquire? active conn)
  (define color      (connection-color conn))
  (define needed     (connection-seg# conn))
  (define available (pstate-cards active color))
  (<= needed available))

#; {PlayersState Connection -> Boolean}
(define (can-occupy? active conn)
  (define needed     (connection-seg# conn))
  (define available (pstate-rails active))
  (<= needed available))

```

Figure 52: A composite view of legal-action?

unit-tested in isolation, because its code is formulated in precisely the terms of the rule description.

Exercise 29. Re-do exercise 28 for figure 52. That is, circle the pieces of functionality corresponding to acquisition and occupation, respectively, in two different colors. Are they as easy to identify in figure 51 as in figure 52?

The three auxiliary functions come without purpose statements. Do they need them?

Let's step back to look at the big picture. Working through this example drives home a key insight. Although it is possible to formulate a short, seemingly simple function in response to the problem statement, a bit of reflection reveals that this organization is not in one-to-one correspondence with the listed tasks. Even if splitting up the function means adding a few extra lines of code, it is the socially responsible action to take.

Exercise 30. Let's change the rule. A player may occupy a connection of n segments only if it has $2 * n$ rails in its possession. Modify the solutions in figures 51 and 52. Compare how you have to go about these modifications.

Even this tiny exercise indicates how much easier it is to find the relevant place if the creator of the code has organized the code so that it corresponds one-to-one to the tasks mentioned in the problem statement, especially if the functions and local variables come with suggestive names. Imagine how such small differences add up for a code base with tens of thousands of lines.

23.3 Designing Methods, Systematically



*How to Design
Classes*

The design process for a method differs from the one for functions just a bit. Since both realize a piece of functionality on data representations, a good degree of similarity should not surprise. Conversely, the wording “on data representations” tells us where the differences come from.

As seen, data representations in class-based, object-oriented languages consist of interconnected classes and interfaces. For each piece of functionality that involves a class, it comes with a method that is tailored to deal with just the information that this class represents. What all this means, is that interconnections among classes mean methods delegate computations exactly along these relationships:

- if some kind of information is represented with a single class, this statement is obvious.
- if information requires a class C that contains objects of another class D , then a method for functionality f in C delegates to a method in D for those parts of f that involve D .
- if some kind of information is a collection of disjoint sets of information, the interconnected classes represent the information as a union type. The interface is the union type itself, and all implementing classes are the variants. A piece of functionality for such information is realized with an abstract method in the interface and concrete, specialized methods in the variants.
- for information whose size is unbounded, the data representation is a union type with self-references, such as those in figure 49.

If people doodle class diagrams on a napkin or a white board to develop object-oriented data representations, they have a good foundation

for discussing the allocation of functionality. Arrows are used to indicate containment, inheritance, and implementation relationships, meaning the suggested guidelines can be turned into a one-line rule about the diagrams:

the design of methods follows the arrows in a class diagram.

Given this setup, the design of methods should follow the same essential steps spelled out in section 23.1: (1) understanding and formulating the purpose; (2) adding a type signature; (3) working through input/output examples; (4) making an outline and filling its gaps; and (5) turning the examples into tests and running the tests.

The fourth step differs a bit due to the mechanics of object-oriented languages. Recall from the beginning of this section that these languages eliminate the need for conditionals that distinguish the variants of a union representation. The dynamic dispatch of method calls automatically directs a computation to the proper method. Or, as stated there, it takes several methods to realize a single function. Hence outlining a piece of functionality with methods means outlining several methods when a union type with an interface is involved.

Favor Functions, Favor Immutability

The title of this section is a slogan, due to Josh Bloch, the designer of Java's API. Many years of experience with this API led Bloch to critique his own design as too imperative (stateful) and to propose this slogan in his book on programming effectively in Java.

Effective Java

A slogan like this one does not surprise people who know the history of object-oriented languages. Alan Kay, the architect of Smalltalk—arguably the first object-oriented programming language—writes in his recollection of the design process that “assignment statements ... express very low-level goals” and “doing encapsulation right is a commitment not just to abstraction of state, but to eliminate state oriented metaphors from programming.”

In other words, the idea of reducing the use of assignment statements to fields and even local variables goes back to the very beginning of programming language design. To be clear, the slogan does *not* say “never use assignment statement.” It merely tells developers to think twice before using them.

While the original reasoning behind this slogan may have been rather abstract, modern practice confirms it with pretty concrete arguments. The following table summarizes these arguments concisely:

immutable objects and functional methods	
positives	negatives
unit testing	algorithmic efficiency (log)
direct semantics	programming patterns

The following four paragraphs explain the four points.

Unit testing has become the most important tool for developers to confirm that methods don't suffer from basic mistakes. Over the past couple of decades, the mechanism has proven its value so clearly that some companies and some teams demand at least one unit test for each and every method in a code base. Setting up a unit test for a functional method of an immutable object is straightforward: apply the method to input values and compare the results to expected output values. By contrast, testing an imperative method properly takes several steps:

1. setting up a context, i.e. a suitable execution state;
2. applying the method to the desired arguments;
3. comparing the actual with the expected results;
4. comparing the actual with the expected side-effects;
5. making sure the method call has no undesirable effects; and
6. tearing down the context.

It is easy to see why developers should prefer immutable objects equipped with functional methods.

While ordinary English usage of “semantics” has a negative connotation, (programming) language researchers use the word to talk about the precise meaning of phrases. In this sense, *direct semantics* means that developers can understand the method's functionality in isolation and, essentially, as an application of middle-school pre-algebra. Practically put, a developer can comprehend some method M based on some basic knowledge about the fields, plus the methods that M calls.

Always programming with immutable objects may impact the (*algorithmic*) efficiency of a method. Concretely, a functional method may run logarithmically slower than an equivalent imperative method (measured according to the size of the input). The good news is that logarithmic factors are usually discounted by algorithm researchers. But developers must nevertheless be aware of this potential problem.

Finally, using immutable objects and functional methods pervasively may force programmers to occasionally use some boilerplate patterns. Such code may exhibit patterns such as the repeated decomposition of structured data and re-composition with one new piece on place. Or, functional methods may have to return an object when equivalent imperative, stateful methods return a base value. When such patterns appear frequently, developers should reconsider the use of immutable objects.

Let us briefly return to the automated player components mentioned several times in this book. As mentioned, such a component typically employs strategies to make decisions when it is granted a turn. These strategies use the current publicly visible game state as the basis for these decisions. Every game player knows that “decision” really means weighing options, comparing (at least) two different game states after applying two distinct actions to the current one.

This description clearly tells us that a player component deals with several game states at once. Starting from the given one, it creates others:

- if the game state is mutable, a turn action *changes* the state. Thus exploring several different alternative actions means (1) applying each action, (2) assessing the value of the state, (3) undoing the action. Once all actions have been explored, the best one is chosen.

Undoing can happen in several different ways. The state representation may include a `deep copy` method. In this case, the strategy method makes a copy first and discards it later. Or, the state representation may include an `undoLastAction` method, in which case the strategy component must use it before exploring the next action.

- If game states are immutable, applying a turn action to this state produces a new object. A strategy can thus explore all actions by simply applying them, assessing the value of the resulting states, and making a decision. The garbage collector of the chosen language automatically discards the generated, low-value states.

Faced with such a design decision, developers should at first heed Bloch’s slogan. If the software system exhibits performance problems *and* if profiling confirms that the problems are due to the immutable nature of objects, then—and only then—they should rework the design with mutable objects.

Readings

Alan Kay. *The Early History Of Smalltalk*. Association for Computing Machinery. 1996.—Kay is the key architect of the first major object-oriented

language, Smalltalk. In this article, he describes the design of Smalltalk, a project that started in 1972 and spanned the decade. Reading Kay’s design rationales and comparing them to contemporary teachings is also an exercise worth every student’s time.

Also see Bloch’s book, first mentioned at the end of section 15.4. In the first edition of his book, he dedicates an entire chapter with this title to the idea of “functional” programming in an object-oriented language. Although this advice may contradict many college-level and K-12 courses on object-oriented programming, Bloch’s vast experience clearly informs his writings, and he shares them to tell people of the significant advantages of this approach.

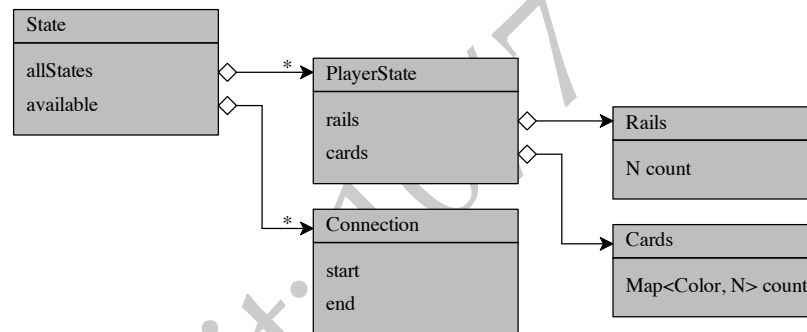


Figure 53: Designing methods

23.4 A Second Look at the Tiny Case Study

The guideline of following the arrows in a diagram has two consequences. First, the relationships of interfaces and classes essentially dictate how one method calls other methods; due to dynamic dispatch, this design keeps methods short. Second, representing information with classes tends to make for mostly easy decisions concerning atomic vs composite methods.

Let’s return to the tiny example from section 23.2. Studying and comparing the two designs will help you understand the similarities and the (few) differences.

The design of the legality check of a player’s action starts again with the development of a data representation. To begin with, take a look at the

```
// N is short for Natural (numbers)

class State:
    List<PlayerState> allStates = ..
    Set<Connection> available = ..
    Boolean legalAction(Connection c):
        PlayerState active = this.allStates.getActivePlayer()
        c in available && active.legalAction(c)

    PlayerState getActivePlayer():
        ..

class PlayerState:
    Rails rails = ..
    Cards cards = ..
    Boolean legalAction(Connection conn):
        Natural n = conn.segments()
        Color c = conn.color()
        this.rails.has(n) && this.cards.has(n,c);

class Rails:
    N count = ..
    Boolean has(N n):
        this.count >= n

class Cards:
    HashMap[Color, N] count = ...
    Boolean has(N n, Color c)
        this.count[c] >= n
```

Figure 54: A composite view of legalAction as a method

doodle diagram for the involved classes in figure 53. The diagram's entry point is the `State` class, which comes with a two fields:

- `allStates`—The * annotation on the arrow from its first field to the box labeled `PlayerState` tells us that a state consists of many instances of `PlayerState`, a class that represents the information about an individual player.
- `available`—Like the first field of this class, this one represents a multitude. Specifically, it contains all connections between points on the game map that are available for acquisition.

The `PlayerState` class also has two fields: `rails` and `cards`. They hold on to single instances of `Rails` and `Cards`, respectively. Finally, these last two classes contain fields whose type comes with the imagined language: `N` for natural number, `Map` for an association of `Color` with `N`.

The idea is of course that the referee component keeps track of the game state. When a player requests the acquisition and occupation of a particular connection on the game map, the legality-checking component (inside the referee component) delegates some of its task to the state object. The two arrows from the `State` object to `PlayerState` and `Connection`, respectively, suggest two helper methods: one for checking whether the player has what it takes to acquire and occupy a connection, and another one for checking whether the connection is available. Likewise, this pattern is repeated with the two arrows from `PlayerState` to `Rails` and `Cards`. Again, the legality-checking method in `PlayerState` delegates to two auxiliary methods and combines their results. Figure 54 displays the resulting methods. Take a close look and reflect on how much it resembles the function design and where the two designs differ.

23.5 Why Systematic Design is Socially Responsible

Designing systematically helps both the creator of code and all of its future readers. Like writing an essay, it starts with *focused thesis statements*. This applies to the development of a data representation as well as pieces of functionality. If we accept that code is a message from one developer to a future developer—that also runs—then the clarity of the message demands focus, meaning a delineating thesis.

The preceding paragraph intentionally substitutes “thesis” for “purpose statement” to emphasize the parallelism to writing essays in response to some prompt. You know that paragraphs are the basic building blocks of essays, just like functions are the brick and mortar of code. Now imagine writing an essay or even a paragraph without first determining the focus of this unit of writing. The result will be convoluted prose. One sentence will present one idea, followed by a sentence that presented an unrelated idea. It will be prose that jumps from one thought to another, prose that makes readers lose their train of thought. They will probably re-read pieces and re-read them again just to make some sense out of it—or give up. When it comes to working with AI agents, they may not give up, but their code will reflect your articulation of a prompt, and the clarity of this prompt is the clarity of the thesis statement.

So all of this is true for code. Thinking of, and even better writing down, a clear purpose statement delineates a data representation and pieces of functionality from each other. That is, coming up with this statement means answering the following questions:

*Which part of the information should a data representation express?
What should it not represent? Which part of the overall task does a
piece of functionality implement? Which part of the task is excluded?
Delegated to some other functionality?*

The thesis statement for data representations consists of two parts: a representation and an interpretation. The former conveys how a piece of information becomes a piece of data, the latter tells a developer what a piece of data means as information. Consider the following examples: `int distance`, `int weight`, and `int temperature`. All three entities are integers; for all three kinds of information, there many ways of measuring them and turning them into data; and given some integers in whatever programming language, interpreting them properly needs help.

Exercise 31. Imagine two different ways to measure a distance and turning it into an integer. Make up two different interpretations of an integer as a weight. Research how many different ways different people use to converse about temperature.

Let's next look at each step for designing functionality and how each helps readers understand code.

The *purpose statement* for pieces of functionality concisely describe *what* is computed; occasionally it describes *how* the result is computed, namely when the underlying algorithm is complex; and it should clarify to *which* task in the problem statement it is linked. Many times a well-chosen name accomplishes all three goals. But, it is also true that too often developers conflate tasks, have overlapping computations, and complicate ways to express a computation—without explanation. A reader must then wonder whether the complexity is intrinsic—or bad coding. If a creator starts the development with a clear focus—a goal of addressing just one (major) task—a lot of these problems are avoided.

In many cases, a *type signature* tells readers almost everything they need to know about calling a method or function if it comes with a good name (and possibly a purpose statement). The combination saves time because a reader can skip the actual definition during a first pass over some component. This point holds especially in settings where programmers favor immutability; an atomic method must compute its results from the values

of its inputs and the object's fields. In addition, a type signature is critical for someone who wishes to use the function for some other code; type signatures enable IDEs to make up essential hints, say, in what order arguments are passed. While adding a type signature is mandatory in statically typed programming languages, it is also important to write them down in untyped languages as comments. They play the same role in these contexts, and some IDEs look for properly shaped comments that look like type signatures.

Maintaining mental discipline is hard, which is the primary reason why many developers prefer typed languages. Types aren't necessarily checked properly, though. Worse, typed-checked programs may still go wrong in ways that are essentially type errors.

Yes, programming to a type signature in a language without type checking takes mental discipline. The word “untyped” implies that the language comes without a pre-defined notation for formulating types and, hence, it cannot check the consistency of type annotations with code. Then again, the lack of a pre-defined type notation gives the developer the freedom to superimpose a rigorous notation that fits the problem. Writing down signatures in this notation; defining all their elements; and programming to all these is then the equivalent of mentally declaring and checking types—and readers of untyped code will be grateful if both are done properly. While it is not a perfect substitute for types, it will be helpful.

Working through examples either leaves behind a trail of thoughts behind a complicated design and, at a minimum, yields *unit tests*. Both are extremely useful. Leaving behind a paper trail is imperative when a method implements a complicated algorithm. It illustrates *how* the method works with a concrete example, which gets the reader started. When combined with a look at the data representation, such worked examples also illuminate how the function takes care of other cases or corner cases.

A unit test serves a future reader and maintainer both as an illustration of *what* the function computes and what its calling protocols are. For both aspects, it is ideal if a programming languages allows developers to place short unit tests next to a function in the code. Here is an example of how this works for a Racket version of the `onTarget` method from section 20.1:

```
#; {Posn Posn SquareShape -> Boolean}
;; is 'p' inside of the square shape centered at 'c'
```

```
(module+ test
  (check-true (onTarget-range 3+4i 10+4.1i 10x10-square1)))
```

```
(define (onTarget-range? p c target)
  (define size-of-target (/ (width target) 2))
  (define new-origin      (shift c size-of-target))
  (define p-relative2-c   (vector-from new-origin p))
  (inside? p-relative2-c target))
```

The example shows how quickly such a sample call illustrates the signature. In this case, it shows that `Posn` means “complex number” and there is no need to look up the data-representation comment. Similarly, the name of the sample `square` informs the reader why the function call yields `true`.

Even in Java, where unit tests cannot be placed next to a signature in an interface, developers organize a code base into parallel directories with code in one and unit tests in a parallel one. Everybody in this ecosystem knows about this organization and can quickly find examples (if not as conveniently as in Racket). IDEs help with this task; their attached testing framework look for the tests in these places and runs them on demand. Again, it is all about systematic development, protocols that programmers follow so that their successors can benefit.

The *outline* of a function or method definition seems to become invisible once a developer fills in the gap to complete the code.—Stop! Think back to reading essays. Does the outline truly disappear? Can you tell whether the author had an outline in mind or just started writing?—While an outline might be unimportant for some simple functions, an experienced developer can actually see it as soon as it comes to data-structure traversals. Suppose that a function’s input is a list, that is, a linear data structure. Depending on the chosen language, a reader will expect to find a comprehension, a loop, or a structurally recursive function—three equivalent ways to traverse a list and process each item. Code that processes the list in a non-structural way causes cognitive dissonance in the reader; colloquially, the reader is likely to be stumped. In this case, the creator made one of two possible mistakes: the outline is done wrong or the definition is missing a concise comment that explains the workings of the algorithm with a *how* statement and worked examples.

In sum, systematic software development is about both creating and comprehending code. When developers follow such guidelines, they make themselves productive. A developer who knows about these guidelines can then easily comprehend the code weeks, months, or even years later.

24 Systematic Design: Abstraction aka DRY

*These are due to Hunt
and Thomas.*

Many text books on programming and software development present computer science as the discipline of “abstraction.” Phrases such as “programmers build, and layer, abstraction upon abstraction” or “developers program to abstractions” abound. Software gurus introduced a cool slogan and acronym for it: “don’t repeat yourself” and DRY, respectively.

What do these phrases mean, however? And do they really mean “abstraction” in the common sense of the word? A software developer needs to know because vague communication among team members or with AI agents leads to vague code, which, in the long run, costs people time and effort. We know from the preceding chapter that programmers actually program to interfaces. We also understand that interfaces encapsulate knowledge, meaning they hide details that the creator of a component should be able to change without having to inform the developers of client code.

Abstracting, by contrast, is the process of creating generalities from concrete examples. The outcome of the process is called *an abstraction*. Accordingly, every program that consumes some inputs and produces outputs is an abstraction over sample inputs and outputs. Everyone with some basic pre-algebra knowledge recognizes this idea:

Sample Problem Define the function f , which computes area of round disk that has a round hole. *Domain knowledge:* The area of a plain disk is π times radius squared.

A well-taught grade-school student works through some examples:

radius of hole	radius of disk	area of disk with hole
1	2	$\pi * (2 - 1)^2$
2	9	$\pi * (9 - 2)^2$
6	7	$\pi * (7 - 6)^2$

By erasing the differences between these examples, the student can then arrive at something like a formula with two placeholders for the differences: $\pi * (I_a - I_b)^2$. Defining the function from here is straightforward:

$$A(x,y) = \pi * (x - y)^2.$$

Once the student comes up with this function definition, a teacher considers the problem solved.

Because of such examples, many programmers, developers, and even text-book authors consider functions and methods as abstractions. Developers know that in the context of programming, these examples come

about in a different way, not as exercises but as similar pieces of code. They recognize that two (or more) pieces of code—say two statements or two expressions—are similar and that their differences give rise to an abstraction. Following the DRY slogan, a developer introduces a method or function to abstract over these pieces of code or instructs an AI agent to do so.

Here is a simplistic example, derived from the algebra exercise:

```
A = pi * power(9-7,2);    B = pi * power(2-1,2);
```

The two expressions on the right-hand side of the assignment resemble each other, and it is possible to eliminate the differences via local variables:

```
double R = 9;             double R = 2;
double r = 7;             double r = 1;
A = pi * power(R-r,2);    B = pi * power(R-r,2);
```

And now generalizing the two concrete examples is straightforward:

```
double areaOfAnnulus(double R, double r) {
    return pi * power(R-r,2);
}
```

Once `areaOfAnnulus` exists, the two pieces of code are replaced with function calls:

```
A = areaOfAnnulus(9,7);    B = areaOfAnnulus(2,1);
```

Since the variables become parameters of the function, programming language researchers refer to this process as *abstraction via parameterization*.

Naturally, developers abstract over far more complex pieces of code than such trivial expressions. Indeed, experienced developers typically don't write down such expressions, because they design software systematically and recognize the need for this kind of simple functional abstraction up front. The same is true for methods that consist of sequences of statements, conditionals, loops, and calls to other methods.

It is still possible that as a code base evolves, similar functions emerge. They may differ in just one or two places and call for further abstraction. By adding more parameters to them, their function bodies can be unified, and one of them can be eliminated. If this process is pushed to hard, however, developers end up with methods or functions that have way too many parameters. As Alan Perlis pointed out decades ago, once a method has ten parameters, the developer probably missed a few.

One way to address the problem of too many parameters is to inspect them for relationships. When two or even more parameters of a method are related, they may point to a missing data representation. Consider the following sketch of a method header:

```
int tooMany(int x, Boolean b, int y, ..., String w, int z)
```

The absence of a purpose statement makes it difficult to judge whether x , y , and z are related. But suppose that, when asked, the header's creator recognizes that they describe the coordinates of a point in 3-dimensional space. If so, the developer should use an existing data representation for points or introduce one.

One solution is to use vectors for such forms of data; another one is to use records (Java) or structures (C/C++). If an appropriate representation exists, say `Point3D`, rewriting the header yields something more manageable than a signature with large N parameters:

```
int tooMany(Point3D p, Boolean b, ..., String w)
```

Executing this change means of course that all client code must create instances of the `Point3D` data representation before calling `tooMany`.

Assuming that many parameters remain the same for most calls suggests going one step beyond simple compound data, namely the use of an object-oriented representation where `tooMany` becomes a method and parameters with default values become fields. Language researchers often refer to this solution as *abstraction via indexing*, because such forms of data support the retrieval of values via symbolic indices, i.e., names.

From a primitive perspective, an object is a container for several pieces of data combined with methods that process this data plus other values. The same perspective suggests that inheritance and overriding allows developers to selectively change just the few fields that need changing for certain calls—including methods, which indicates that proper abstraction means far more than unifying similar pieces of code.

What all this leads to, though, is that abstraction mechanisms and processes should be at every developer's finger tips. Abstracting when the need arises is nowadays a critical part of systematic and socially responsible software design, and the rest of this section therefore summarizes its most relevant aspects.

24.1 The Systematic Abstraction Process

If "abstraction is the process of creating generalities from concrete examples," executing it in the context of code demands two complementary skills. First, a developer must develop an idea for similarities of pieces of code that can be eliminated with some abstraction mechanism. Second, the developer must understand how to use the abstraction mechanisms of

the chosen programming language. After all, generalizing similar pieces of code demands that there is a mechanism that can unify them.

Once a developer recognizes the potential for abstraction and picks the mechanism with which the abstraction is to be realized, getting from the former to the latter can be done in a systematic manner. Concretely, the process starts with a comparison of the concrete examples. This comparison results in pieces of code where differences are marked in some ways. If erasing these differences yields identical pieces of code with holes, and if the targeted abstraction mechanism can express this result via filling of the holes, the developer can create the desired abstraction.

The abstraction process isn't finished, until the developer has used the abstraction to replace the original concrete examples. Unless such uses are possible, the process failed. If the pieces can be replaced, the developer still needs to run the test suite to ensure that the functionality of the software isn't affected.

In sum, the abstraction process consists of the following steps:

1. A developer recognizes that two (or more) pieces of code are similar and that some language construct could express the general idea.
2. Comparing these pieces of code and replacing the differences with holes should yield identical code fragments.
3. By using an abstraction mechanism, the developer can create a proper piece of code from these code fragments. The result of this step is the abstraction.
4. Finally, the developer replaces the pieces of similar code from step 1 with uses of the newly created abstraction and then re-runs all tests to ensure that this change to the code base does not affect its (tested) behavior.

Literal constants are an obvious corner case of this process description. They are definitely "similar pieces of code;" indeed, they are identical. And it has long been considered best practice to name constants for reasons mentioned in preceding chapters. Naming string constants has become particularly important for the internationalization of code.

A true illustration of this general sketch of the abstraction process needs a brief explanation of what kind of abstraction mechanisms (step 3) are commonly available and how developers use them.

The two functions process their inputs in an identical manner. Both traverse the given list and find the element with a maximal value. They differ in just one place. While the first one selects the value from a sales record, the other one picks the probability value in some kind of classification vector. The code displays use dots to signal that the remaining code is essentially the same for both definitions.

Any experienced software developer recognizes the similarities of the two functions and easily spots the difference. Following the functional abstraction process yields a common template. Doing so requires renaming the parameter to the same name—say, `lox`—and to erase the names of the selector functions: `sales-record-value` and `dog-prob`, respectively.

The result of generalizing these two functions is `argmax`, a function that consumes a real-valued function and a list of elements:

```
#; {[Listof X] [X -> Real] -> X}
(define (argmax f lox)
  ... (f x) ...)
```

Its result is the element `x` from the given container that maximizes `f`.

Clearly both of the initial function are instances of `argmax`:

```
#; {[Listof Sales] -> Sales}
(define (max-sales lo-sales)
  (argmax sales-record-value lo-sales))

#; {[Listof ImageClass] -> ImageClass}
(define (is-dog lo-classifiers)
  (argmax dog-prob lo-classifiers))
```

The general point is that modern functional abstraction takes advantage of functions as values. Here, `argmax` consumes a function when it is called. In other cases, an abstraction may also return a function. A classic example is a numerical differentiation function:

```
#; { (Real -> Real) -> (Real -> Real) }
(define (d/dx f)
  (define (f-prime x) ...)
  f-prime)
```

It is this flexibility that makes functional abstraction a powerful tool, now also included in class-based object-oriented languages.

Abstracting with Classes

Abstracting with Types

24.3 Using Abstractions

24.4 Abstracting is Socially Responsible

DRY

24.5 Abstraction Beyond Functions and Classes

There are abstraction mechanisms beyond functions and methods and classes. For example:

- first-class classes (Type Script?)
 - macros (from C to Racket), occasionally approximated with reflection
 - domain-specific languages as embeddings or extensions of existing languages
- Not yet in wide-spread use.

24.6 Abstracting with Functions

24.7 Abstracting with Classes

strategies: IStrategy, AStrategy, ...

In order to illustrate this abstract advice, let's return to the *Ticket to Ride* game from the preceding section. This time we will look at how an automated player chooses its actions, using one or more strategies:

The purpose of a strategy component is to make decisions on behalf of the player component, which deals with the mechanics of playing the game.

For example, the *Ticket to Ride* game requires three kinds of decisions. During setup, the player must pick destinations—pairs of cities—that it wishes to connect by acquiring and occupying connections. During a turn, it must decide whether to request additional colored cards or to collect a connection; it cannot do both. Finally, if the player decides to acquire a connection, it must choose one.

In this context,

imagine a product manager asking for two sample strategies, called `Hold10` and `BuyNow`. More specifically, the request might come with sentences like “the second strategy, `BuyNow`, shares some pieces of functionality with the first, `Hold10`.”

Like a player, a strategy is an object that bundles state variables with at least two pieces of functionality. Furthermore, as the prose suggests with “two sample strategies,” the intention is to develop many strategies. That is, a developer using should distinguish the functionality description from the concrete strategies. In terms of a class-based, object-oriented language, the developer should create

- an *interface*, to which other developers can program without committing to a concrete strategy
- *concrete classes*, which implement the common interface.

Furthermore, the task description also hints at commonalities among the two sample strategies, and in all likelihood, other strategies may also exploit these commonalities. The proper place for commonalities is in

- an *abstract class*, which can set up common concrete methods but also common abstract methods for all concrete strategies.

Figure 55 displays how a developer should turn these planning steps into a proper (module) implementation. It defines an interface with signatures for the key pieces of functionality.

To indicate how the concrete logic is organized, a developer may supplement this interface definition with a class diagram in the same file or a “read me” file. Here is an ASCII diagram from the actual implementation:

Such a diagram immediately helps a reader with the navigation of the code base, especially if the files are named appropriately. The interface is implemented by an abstract class, from which the developer has derived two concrete classes.

Exercise 32. In order to test the referee component properly, a full-fledged implementation of any game will need misbehaving player components. One way to implement such players is to equip with cheating strategies, which will exercise the referee’s legality checking during integration testing.

So, a product manager for the *Ticket to Ride* development may suggest the following:

Technically, this Racket module defines a class contract, a sophisticated form of interface. Unlike in Java, such a contract can also specify the fields that every constructor must initialize.

```

#lang racket

(provide strategy/c%)

(define strategy/c%
  (class/c
    (init-field
      (the-game-map game-map?)
      (rails#        natural?)
      (cards          (hashof color? natural?))))

    ;; pick destinations from the given set for the map
    ;; return the _rejected_ destinations
    (pick (->m (setof destination?) (setof destination?)))

    ;; choose to ask for more cards or a specific connection
    (turn (->m pstate? (or/c MORE-CARDS connection?))))))

```

Figure 55: Basic abstraction in class-based object-oriented language

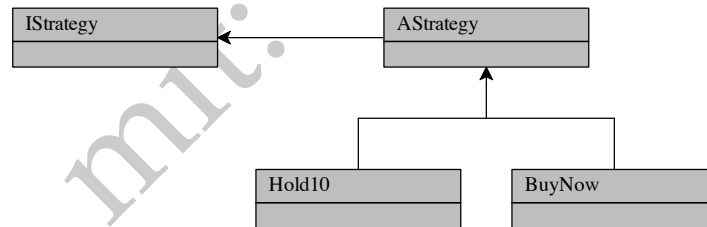


Figure 56: Abstraction via classes in Java

The `Cheat` strategy is like the `BuyNow` strategy but attempts to acquire a non-existent connection ...

Modify the diagram above to indicate how to best go about implementing this request.

mit: 10/7

25 Code Inspections, Systematically

If developers design code systematically, they can also present it using the design system. Starting from the problem statement, they can show how their data representation mirrors the concepts of information found there. The problem statement also dictates the pieces of functionality needed, so it should justify the entry-point functions and methods. When it comes to composite pieces of functionality, the developer may need to explain the design of the data that flows from one piece to another. Finally, the design steps for methods provide the scaffolding for presenting those.

Conversely, when developers review the code of their peers, they can use the principles of systematic design to understand and critique the product. They can check whether the data representation matches the description of the information in the problem statement. They can request a purpose statement if a function looks incomprehensible and its name doesn't help understanding it.

The ideas presented in the preceding section make up one particular system of design principles. This section demonstrates how these principles help with code inspections. To make this concrete, we first introduce a variant of the *Ticket to Ride* problem from section 10: the *Labyrinth* game.

25.1 The Labyrinth Game: A Problem Statement

Section 10 presents the analysis of a games-for-hackers project. A company provides game servers to which hackers can connect automated player components. The company's idea is to use variants of board games, initially *Ticket to Ride*. Another game under consideration is *Labyrinth*.

Here is an illustrative excerpt from the game description:

The Labyrinth game takes place in an ever-changing maze of twisty passages. Players navigate these passages to collect treasures from uniquely identified places.

The game comes with fifty tiles. Each tile displays one of eight connector shapes and a unique, unordered pair of gems.

The referee uses 7 x 7 tiles to set up the initial game board. The last tile—a *spare*—is kept on the side. The connector shapes on the tiles set up paths. Every even-numbered row and column of the board can slide in either direction. To move a row or column, a player inserts the current spare tile and pushes the row or column from one direction until the tile at the opposite

end comes out. This latter tile becomes the new spare. A player may not undo the sliding action of the preceding turn.

The game description implies the existence of at least two pieces of functionality that participating hackers must implement for a data representation of the board:

- the slide action for rows and columns;
- the determination of tiles reachable from a current position.

Let's see how presenters and panelists may interact once the board and its first pieces of functionality have been implemented.

25.2 Presenting Systematically Designed Code

If a milestone involves the development of a data representation, a presenter starts with an overview of the chosen data structures. For complex choices, it is best to include a justification and examples; for simple ones, a presenter may skip those.

Good morning. The goal of the current milestone is to develop a data representation for the game board and some of the obviously needed methods.	Sounds good. Tell us all about it.
---	---

The first two steps of the development were pretty much straightforward. Since the game board consists of tiles, we created two classes (figure 57): <code>Board</code> and <code>Tile</code>.	That sounds right. What are Gems?
---	--

A Gem is essentially a <code>String</code>.	Okay.
--	--------------

We also have a data representation for directions.	Directions?
---	--------------------

```
// board representation for a game of Labyrinth
public class Board {
    private Tile[][] grid; // row major

    public Board(Tile[][] grid) {
        this.checkDimensions(grid);
        this.checkGemUniqueness(grid);
        this.grid = grid;
    }

    // raise an exn if the dimensions don't satisfy the spec
    private void checkDimensions(Tile[][] grid) {
        ...
    }
    ...
}

// tile representation: gems plus path segments
public class Tile {
    private UnorderedPair<Gems> gems;
    private Char segments;
}
```

Figure 57: A Java data representation for the game of Labyrinth

The directions for the sliding actions.

Really? Shouldn't an enumeration of the four possibilities suffice?

That's a choice we explored. It forced us to replicate code, so we came up with an alternative representation.

I guess we will explore it when we look at the functionality. Let's focus on the board representation now.

There are three things to observe. First, the presenter enumerates all classes involved in the development of the board representation. It doesn't consist of just one single class but several. Second, the panelist questions one aspect of the design choice, and the presenter explains that the design of the requested functionality calls for the extra form of data—*without going into*

details. The presentation stays at the overview level. Finally, the (head) panelist controls the presentation and re-directs the presentation to what the panel expects to focus its energy on: the board representation.

The <code>Board</code> class is essentially just a wrapper around a 2-dimensional array of tiles.	Where does the array store rows and columns?
At first we struggled with this decision. In the end we decided it doesn't matter.	Yes, from the perspective of the game it doesn't matter, but we need to know.
That's why we documented it on line 2 of the class definition.	Thanks for pointing this out. Why does the constructor start with two method calls?
The purpose of these calls is to ensure that every new <code>Board</code> object satisfies the game specifications.	Shouldn't this invariant be established by the code that calls the constructor?
We don't think so but if we decided to re-arrange things like this, this would be easy.	True. We may make these checking methods public. But why do you disagree?
Our course on designing classes taught us that a constructor should enforce such an invariant.	Yeah, you never know whether the calling code checks properly and, if the constructor does it, it's guaranteed.
Yes!	Do the remaining methods preserve the game specification for boards?

The presenter and the panel again discuss a design choice. This time the presenter pushes back against the panelist's implied suggestion, because at this moment, it is just that.

```

... in the Board class ...

// raise an exn if any two tile display the same pair of gems
private void checkGemUniqueness(Tile[][] grid) {
    Set<UnorderedPair<Gem, Gem>> seenGems = new HashSet<>();
    for (Tile[] tileRow : grid) {
        for (Tile cell : tileRow) {
            if (seenGems.contains(cell.getGems()))
                throw new IllegalArgumentException("duplicate gems");
            seenGems.add(cell.getGems());
        }
    }
}

```

Figure 58: A Java constructor “contract” for the game of Labyrinth

Once the data representation is understood, the presenter can move on to pieces of functionality.

Perhaps we should look at the constructor’s contracts first.	As you wish. But I bet checking dimensions is easy.
--	---

I was thinking of the uniqueness check for gems (figure 58).	Go ahead.
--	-----------

The purpose of this method is to raise an exception if it finds a problem.	Okay.
--	-------

It is an atomic method that traverses the given array with two nested <code>for</code> loops.	Doesn’t it have to track the gems it has seen?
---	--

Yes, it uses an accumulator, a set of gems.	Why does it track them in a set?
---	----------------------------------

It is the fastest way to check whether one tile comes with the same gems as another.

Oh yes, that's correct. Why does your code use "cell" as the name for each tile?

We are thinking of the elements of an array as mutable cells.

Fine, let's move on to actual functionality.

The presenter proceeds step by step, starting with a purpose statement—to raise an exception. Next the presentation provides an overview of a basic atomic method. And the rest dives into some basic details concerning the loop. In a setting with experienced developers, the head panelist might re-direct the conversation after the overview has been delivered.

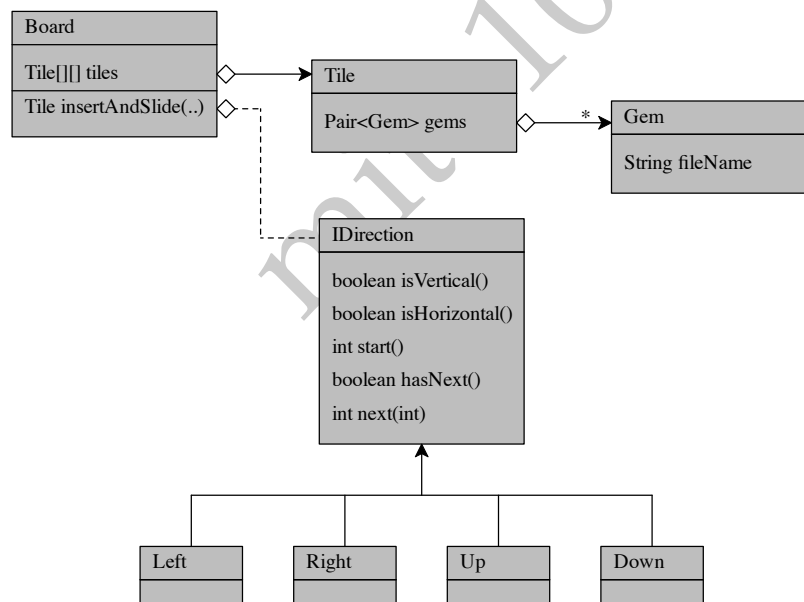


Figure 59: Doodling for the development of the board representation

It might be best if I start with the doodle diagram (figure 59) that my partner and I drew as we revised the first implementation of <code>insertAndSlide</code> .	Sure. What does the dashed line mean? I have never seen this before.
That's the key to this diagram. It says the method needs a data representation for directions.	Okay. But why?
As we designed the functionality, we realized that the code for all four variants looked nearly the same.	Do you mean the code for sliding a row left or right and a column up or down?
Precisely. And the differences were isolated in how the algorithm traverses just one sequence of tiles.	Interesting.
With <code>IDirection</code> , there is just one major method for all four.	This sounds really cool. Let's see
Before I dive in, I just want to point out the three critical methods that <code>IDirection</code> specifies: <code>start</code> , <code>hasNext</code> , and <code>next</code> .	These make it look like an iterator.
Almost. But an iterator isn't quite right.	Got it. Let's take a close look.

The presenter could have shown this diagram at the very beginning of the code inspection but injecting it into the inspection now is also a good choice. For the first part of the inspection, `Board` and `Tile` are the most

relevant classes; the diagram for their relationship is trivial. By delaying the diagram to this point, it provides a good overview of the data needed to design the requested functionality.

```

class Board {
    ...
    // effect: slide column 'n' of this board if 'd' is vertical;
    // otherwise slide row 'n' in direction 'd'---by inserting 'in'
    // return the tile that's pushed out
    public Tile insertAndSlide(Tile in, int n, Direction d) {
        this.checkConsistency(n,d);
        Tile newSpare;
        if (d.isVertical()) {
            Tile[] seq = this.extractColumn(n);
            newSpare = slide(seq,in,d);
            replaceColumn(n,seq);
        }
        else {
            Tile[] seq = this.extractRow(n);
            newSpare = slide(seq,in,d);
            replaceRow(n,seq);
        }
        return newSpare;
    }

    // raise an exn if 'n' and 'd' are incompatible
    private void checkConsistency(int n, Direction d) {
        ... this.grid ...
    }
    ...
}

```

Figure 60: Java method for sliding a row or column

Here is the implementation of `Okay`.
the slide action (figure 60).

Our `insertAndSlide`
method changes the board's
`tiles` array and returns the
tile that is pushed out.

Why does it return this tile?

The <code>Board</code> class is a data representation of the game board. The spare tile is separate from the game board; it is not a part of the board.	True enough. Even the game description keeps the spare tile separate from the board. It is a part of the game state and when we design its data representation, we need to include the spare tile there.
That was our thinking.	Make sure to double check the component descriptions.

Just stating the purpose statement of the method reminds of the principle of top-down planning, bottom-up programming. The second is going to rely on the component design of the first. Or, it is going to add to it. In this spirit, this last remark of the reviewer is directed at the secretary to make sure the data representation for the spare tile isn't forgotten.

The method is given the tile to insert, a row or column index, and a direction.	That sounds reasonable.
But not all arguments make sense, and not all combinations of arguments make sense. That's why the method runs a contract check.	Is this like the contract check in the constructor?
Yes, it is.	Too bad Java doesn't have a proper contract system.
Indeed.—The method composes a workhorse, <code>slide</code> , with two pairs of helper methods.	Okay.

Depending on whether the goal is to slide a row or a column, the method uses a helper method to extract the appropriate slice from the <code>tiles</code> array. When <code>slide</code> has done its work, the method replaces the original row or column in the <code>tiles</code> array with the new sequence of tiles.	This looks quite concise and elegant.
--	---------------------------------------

When a presentation covers a composite method, it is important to give the panel an idea how many methods are involved. Doing so prepares the panelists for how much code is going to be covered to explain a single piece of functionality. Unprepared panelists asks questions at bad moments.

The <code>slide</code> method is atomic. Its purpose is to insert a tile into a one-dimensional array of tiles, shift the tiles in a given direction, and return the tile that is pushed out.	This sounds almost like the purpose of <code>slideAndInsert</code> .
---	--

It differs in that this method is given a one-dimensional array. It does not work on <code>grid</code> .	Indeed.
--	---------

The method iterates over the array in the order specified via <code>direction</code> . At each step, it swaps out the tile in the current cell for the current value of <code>in</code> . When the loop is done <code>out</code> contains the tile that is pushed out through the sliding action.	Now the idea of a separate <code>IDirection</code> type makes complete sense. Each direction represents whether to start at one end or the other, how to proceed, and when to end. But for that, it must know the dimensions of the grid.
---	---

```

class Board {
    ...
    // slide 'seq' of this board in direction 'd' by inserting 'in'
    // return the tile that's pushed 'out'
    private Tile slide(ArrayList<Tile> seq, Tile in, Direction d) {
        Tile out = null;
        for(int i = d.start(); d.hasNext(i); i = d.next(i)) {
            out = seq.get(i);
            rowN.set(i, in);
            in = out;
        }
        return out;
    }

    private ArrayList<Tile> extractRow(int n) {
        ... this.grid ...
    }

    private void replaceRow(int n, Tile[] seq) {
        ... this.grid ...
    }

    private ArrayList<Tile> extractColumn(int n) {
        ... this.grid ...
    }

    private void replaceColumn(int n, Tile[] seq) {
        ... this.grid ...
    }
}

```

Figure 61: Auxiliary Java methods for sliding a row or column

True! We assume that some global Java interface will supply system-wide constants such as the grid size.

We need to double-check or update the component design to make sure this is true.

We already added such an interface to the code base. We needed it for our unit tests.

Good but this should be reflected in the design README. One last question. Why does the method initialize `newSpare` to `null`?

The <code>for</code> loop is going to replace it with the first tile in the array.	What if the array is empty?
In that case, the method returns <code>null</code> unfortunately.	It could return <code>in</code> , which would avoid the billion-dollar mistake.

The code presentation proceeds systematically, covering each design step, and ends in the discovery of a minor flaw. At this point a panelist might ask to see the unit tests because the presenters mentioned them. Or the head panelist may direct the presenter to cover the auxiliary methods.

25.3 Inspecting Systematically Designed Code

On occasion, a developer gets lost in details while presenting code, reading every line to the panel. In that case, the head panelist must step in and re-direct the code walk. This situation can come about for many different reasons:

- a panelist's question causes the presenter to veer from a basic plan;
- the developer is stressed;
- the design does not follow the basic principles; and
- the presenter does not follow a top-down plan, possibly because the chosen programming language suggests a bottom-up organization.

All are equally common, but the first two differ in nature from the last two. While the first two often just reflect a lack of practice and routine—and inspections improve over time—the last two are logical problems and subject to guidance using the principles presented here.

So let's take a look at how these problems may show up in a code walk and how a (head) panelist can use the design principles to re-direct the presenter and make the inspection a success. To keep things familiar and yet inject some variety, the inspection covers the board design combined with the reachability functionality for the Labyrinth game; ssee page 233.

```

class Board:
    """ Represents the game Board, a matrix of Tiles

    Args:
        rows (int): its height
        cols (int): its width
    """

    def __init__(self, rows: int, cols: int, spare: Tile):
        self.rows = rows
        self.cols = cols
        self.spare: Tile = spare
        self.board: List[List[Tile]] = self.__randomized_board()
        self.last_slide_move: Optional[Slide] = None

```

Figure 62: A Python data representation for the game of Labyrinth

Good morning. Here (figure 62) is the class we developed for milestone 2.

Why don't you remind us of the goals?

The first goal of the milestone is to create a data representation for the board.

And the second one?

The second one is to develop some methods that might be useful to plan a trip through the maze.

Let's focus on the data representation, since it is up on the screen already.

Our board is just a matrix. The constructor takes in the number of rows and columns and generates the board.

Slow down. There seems to be a conflict between the method header and the class's purpose statement.

Oh yes. The constructor also takes in the spare tile.

So is the spare tile a part of the board class?

We figured it might as well be.	But the purpose statement mentions only a matrix. The spare doesn't show up. It is actually questionable whether the spare tile is a part of the board.
---------------------------------	---

Even if we assume that everyone is familiar with what the code walk is about, it is always a good idea to remind an audience of the purpose of a meeting, here, the review of a specific data representation and functionality. Next, notice how the panel immediately observes a gap between the class's purpose statement—its thesis—and the implementation. The game description cleanly distinguishes the board from the spare tile, yet these developers combined the two *without* stating so up front—which spells trouble for a clean design. Also, the presenter's response to the panelist's question does not provide a design rationale. While a panel may wish to question this decision, we skip this part of the conversation.

When panelists ask "why" questions, the goal should be to expose gaps between the plan and the code—and only then:

The fourth line of the constructor is the key. It makes a random board of the given size.	Why is the board always random?
---	---------------------------------

We figured games are played on new board configurations.	Unit tests will need fixed inputs so that the output is predictable. Can you show us the unit tests for your reachability method?
--	---

Ah, we don't have any. We had not considered unit tests.	Oh.
--	-----

In this case, the developers overlooked the obvious use case for a data representation: unit testing.

The last exchange exposes yet another contradiction, once again contrasting the purpose statement with the actual constructor:

We will address this gap soon.	We're not done yet with the constructor. Please explain the last line of the constructor.
Our reading of the rules suggests that the board must represent the last sliding action.	Even if, this line introduces a new form of data.
How so?	What is <code>Slide</code> ?
Oh yes, it is our data representation of a sliding action.	So it is another form of data. But what might be worse is that the purpose statement doesn't mention this aspect.
We will reword the purpose statement.	I think you might be better off reconsidering your data representation. The focused representation statement might be a good starting point. In the meantime, let's move on to the requested functionality.

In sum, this conversation shows how a panelist can use the planning and the representation statement to inspect the actual code. The planning process separates the representation of the game state from the representation of a game board; the latter should really just mirror the physical reality of the board. While the representation statement seems to suggest that the code implements this mirroring, going through the five lines of the constructor reveals that this is not the case. Finally, by keeping the remaining design principles in mind—especially unit testing for the requested functionality—the panelists exposed another weakness of the code base.

Once the review panel understands the data representation, it can turn its attention to the reachability functionality:

```
import collections

class Board:
    ...

    def __horizontal(self, side, row, column, accessible_tiles): ...

    def __vertical(self, side, row, column, accessible_tiles): ...

    def __adjacent(self, row, column):
        tile = self.grid[row][column]
        accessible_tiles = []
        for s in tile.accessible_sides:
            if s == "left" or s == "right":
                self.__horizontal(s, row, column, accessible_tiles)
            else:
                self.__vertical(s, row, column, accessible_tiles)
        return accessible_tiles

    def reachable_tiles(self, row, column):
        """
        returns list of all accessible locations
        """
        queue = collections.deque()
        queue.append((row, column))
        accessible_tiles = set()
        path = []
        while queue:
            row, column = queue.popleft()
            if (row, column) in accessible_tiles
                or self.grid[row][column] == 0:
                continue

            accessible_tiles.add((row, column))
            path.append((row, column))

            for neighbor in self.__adjacent(row, column):
                queue.append(neighbor)
            path.remove((row, column))
        return path
```

Figure 63: Python board functionality for the game of Labyrinth

Here is the first piece of the requested functionality (figure 63).	Okay.
The first two methods are <code>__horizontal</code> and <code>__vertical</code> .	Stop! These are private methods. They can't possibly be the starting point.
No, they are not.	Which of the two pieces are you presenting?
I am trying to present the functionality that computes the tiles that an avatar can reach from its current spot	Good. Why don't we start with this method then. We will look at the private methods if and when needed.
Our code follows the on-line suggestion of putting magic methods first and public methods last. In this case, we want to focus on <code>reachable_tiles</code> .	Go for it. The purpose statement is clear. Consider adding an informal type signature, too.
Noted. So the first four lines initialize the necessary data structures.	Stop! Please provide an overview. We already know that this method is a composite. So explain the general idea first.

The directions from the panelist directly follow the design principles concerning composite pieces of functionality. An understanding of a composite methods demands guidance, meaning, an explanation of its overall purpose, the number of methods involved, and perhaps even the depth of the dependency chain.

Okay. The <code>reachable_tile</code> method consists of a loop that needs three helper methods.	What is the basic purpose of these helpers?
---	--

The important one is <code>__adjacent</code>. Given a grid position, it returns the list of immediately reachable neighbors.	That makes sense. As a matter of fact, that would be truly descriptive name for this method. It then wouldn't matter that it comes without purpose statement.
---	--

The panelist's remark points to the first principle of function design: articulate a clear purpose statement or give the method an equally clear name.

Noted. This method uses conditional to add vertical or horizontal neighbors to the list it is constructing.	Now the names of these two private methods make some sense. Let's go back to <code>reachable_tiles</code>.
--	---

As I said before, the method itself starts with four lines that set up basic data structures.	It might be best to describe the organization of the method body first.
--	--

The method uses a worklist algorithm. Each element of the list is a grid position. The loop removes one of these positions, determines its immediately reachable neighbors, and sticks those back into the worklist.	This description makes sense. It is a common approach to implement recursive task descriptions. How does it avoid following cycles in this labyrinth?
---	--

A conditional checks whether the newly removed position is already on the list of reachable tiles.	Good. Let's take a look at the details now.
---	--

The key to note here is how the panelist requests an overview of how the method computes the result. Even a quick glance at the code suggests that the composition is non-trivial, so an explanation of the “how” is appropriate and it may even deserve a refinement of the purpose statement.

The first two lines initialize the worklist.	Why is it called <code>queue</code> ?
Yeah, <code>worklist</code> might be a better name. The third line creates the set of positions that the method has already visited.	Is it used for cycle detection?
Indeed. And the last line sets up the list that the method eventually returns.	Why is this variable called <code>path</code> ? In what sense is it a path?
Come to think of it, the content does not at all describe a path.	What is the content?
It's the list of reachable coordinates.	This almost sounds like the last two variables represent the same information, one as a set and the other as a list.
That's quite possible.	Why does the method return a list? Could it return a set instead?
We thought that the ordering might matter.	How?

At this point, it is clear that the panel has discovered two naming problems and at least one design issue. If the two pieces of data represent the same information, only one is needed. And if the clients of the `Board` class don't need to rely on an ordering of the reachable positions, returning the set works just as well as returning the list.

Since we don't have any clients yet, we should be able to change the signature of the method so it returns the set.	Let's keep this in mind. Explain the loop.
The loop runs as long as there is a position in the <code>deque</code> . The first line of the loop body removes the leftmost position. And the conditional checks whether this position has been visited before.	Why is there a zero condition?
I forgot. We might have used 0 to mark empty grid positions.	Are there any?
Not really. We should remove this part of the condition. Anyways if the condition succeeds, the rest of the loop body is skipped.	Okay.
The next two lines update the set of visited positions and the path of accessible tiles.	The two lines add the exact same data to the containers. Since they contain the same data beforehand, they really are in sync all the time.
Yes, we will reconsider just using one of them.	What does this nested <code>for</code> loop do?
The call to <code>__adjacent</code> returns a list of grid positions that are immediate and reachable neighbors of the currently processed position. And the <code>for</code> loop adds them to the worklist.	Isn't <code>queue</code> a container?

It is.	So <code>reachable_tiles</code> could hand it to <code>__adjacent</code> and get it to update the structure?
--------	--

Yes. As a matter of fact <code>__adjacent</code> asks its helpers to do just that.	This might be worth considering then.
--	---------------------------------------

This last part of the inspection is a deep-dive into the details of the method, ending in a lightweight suggestion. Such line-by-line inspections are useful when the code looks more complex than the purpose statement—or a comment concerning the algorithm—suggest.

```
class Board:
    ...
    def reachable_tiles(self, row, column) -> Set[Tuple[int,int]]:
        """
        returns the set of of locations reachable from (row,column)
        uses a worklist algorithm of positions-still-to-visit
        """
        reachables = set()
        worklist = collections.deque()
        worklist.append((row,column))
        while worklist:
            row, column = worklist.popleft()
            if (row, column) in reachables:
                continue
            reachables.add((row, column))
            self.__add_adjacents(row, column, worklist)
        return reachables
```

Figure 64: Reacting to the code walk

25.4 A General Guide to Code Inspections

As the preceding subsections show, presenting and inspecting code make up two sides of the same coin. The same principles apply to both: the presenter uses them to guide a reader through a code (re)design; the panelists

uses them to question the design of the code (base). While the preceding subsections illustrate this idea with examples, this section presents a general guide to both sides of code inspections. The key recommendation is to create README files that serve both as outlines for a code inspection and as guides for future readers of the code base.

README Files

Section 12.1 suggests that software developers should place the design ideas into a README file at the top-level of a code repository. Once developers turn to the creation of code, they should update this README file with comments about the code. That way it can serve as a living explanation of the design rationale and a navigation document for future maintainers. As the code base grows, the README file may grow too large for a structured document. The simplest way to break it up into manageable chunks is to equip each folder with a README file and to link these files with each other like the chapters of a book.

A top-level README file may contain a brief overview of the specification, with links to other documents and readings; download and installation instructions; links to a user guide if needed; instructions for running the program(s), the integration tests, or the complete set of unit tests; a table of contents pointing to the READMEs in various folder; and diagrams that explain the most important static and dynamic relationships among the components of the code base.

The README files in subdirectories have a local focus: the code in the files and its purpose. Assuming each subdirectory corresponds to a distinct component of the project, its README starts with a purpose statement for the component and comes with a table of contents that lists all the files in the folder and their purposes. If the code uses a class-based language, the files may contain classes, and the explanations in the README file clarify how they relate to each other statically and dynamically.

Let's look at some of these elements in some detail, using the example of a game-referee component that comes with a game observer:

- Given the context of the top-level README, a *purpose statement* for the referee component may consist of a single line:

```
this component implements the Ticket referee
```

If the top-level file does not supply enough context, a developer may wish to add a short paragraph:

The referee is organized as a mechanism that sets up an initial state; consults external players for requests to transition from one state to the next; applies a rule checker to each request; transitions to the next state until a final state is reached.

- The *table of contents* lists four files, each containing one data representation or one piece of functionality:

```
referee:: mediates between players and the referee state
state:    a representation of the referee's game knowledge
obs-ref:  an interface for a game observer
observer: a primitive observer;
          allows developers to view games
```

To ensure that developers write down focused purpose statements per class, it is best to generate this table from the files instead of maintaining it by hand. Doing so guarantees that (1) the purpose statements exist in the various files and (2) the README file's table of content isn't out of sync with the actual files. After all, a README file is a form of comment, and as observed before, comments are easily out of sync with the code.

- When a component exists of several classes (modules, files, data representations, pieces of functionality), navigation is greatly facilitated with a doodle *diagram of the static relationships*, meaning whether one piece of the component refers to another:

```
+-----+ +-----+ +-----+
| Referee | -----> | Obs-Ref | | Observer |
+-----+ +-----+ +-----+
| State s | ----> | State | ---|| (knows)
| Rules r | +-----+
+-----+ |../Common |
          +-----+
```

This ASCII diagram brings across a number of basic facts of how the four pieces in the folder relate to each other. The referee contains a `State` object; its content consists of pieces from the `Common` ontology (for players and the referee) and other pieces of data. It relates to the observer interface but not the observer itself. The former knows about the referee's state, presumably to render it.

- The diagram of static relationships leaves open how the `Referee` gets to know about an actual `Observer` object. A *diagram of the dynamic relationships* settles this point. See figure 65 for the dynamic interactions among these two pieces.

```

Context
|
| new
o = | ----> Observer
|
|
|
|
| new (O)
| -----> Referee
|
| update(State s)
| <-----
|
| .
| .
%% before each legal turn .
| .
| .
| .
|
| update(State s)
| <-----
|
|
%% until the game ends:
|
| game_over(State S) (last State)
| <-----
|
| ---- the observer shuts down
|
|

```

Figure 65: An example of a README diagram of dynamic relationships

While tools may exist to generate such diagrams from code, the results tend to be uninformative, containing too much or too little detail. Devel-

opers are better off drawing such diagrams on boards and placing scanned copies into the folders. With mark-down formats, it is easy to include such diagrams in README files.

Presenting

If the README files are kept up-to-date every time, a developer can use them to get a code presentation started. As a matter of fact, with such READMEs the presentation almost runs itself. To start with, the developer should point to the problem statement (link) in the README and remind the panelists of the role of the component in this context. Even if a developer presents just a change to a component, it is good practice to begin the code walk with such a reminder.

Before going into any details, the developer next provides an overview. If the READMEs include diagrams for the static and dynamic relationships, the developer should use them, focusing on those parts that are relevant to the goal of the code walk.

As a reminder, the general goals of a code presentation are to discover design flaws, errors, and comprehensibility problems. Given these goals and the general constraints on time for inspections, the developer must carefully choose what to present. That is, the presenter should explicitly state the goal(s) of the code walk, meaning those pieces of code that will be the focus of the presentation—and how to get there from the global overview:

- If the code walk involves an entire new component, the presenting developer is keenly aware of which pieces of code deserve attention and which ones don't.
- If it is about a large change, act as if it is the review of new component.
- If the presentation concerns a small change, explain the path to the modified pieces of code.
- If help is needed with a known bug, explain the failing test case and, if useful, run it to show the symptoms of the failure.

Comprehensibility issues will come up as a result of presenting the path

An explanation of a path has two components: the static program text and the dynamic call chain. The first is needed because a future reader must start with text; the dynamic call chain(s) is needed to understand whether

the calls are legitimate and the function works for all calls. For a concrete example of where both are needed, see section 21.5, especially the sample dialog on see page 178.

To repeat this point with a simple example, consider the goal of a developer to present the complex code of function *h* and its only call site in the one of these two composite methods:

<pre>Integer f(String s): String t = lowercase_of(s); Integer i = h(t); return i;</pre>	<pre>Integer f(String s): lower_the_case_of(s); Integer i = h(s); return i;</pre>
---	---

Even if there is no manifest flow of values from call to another, a developer must understand and expose imperative data flow, too.

Both establish a similar calling context for *h*, but in two different ways. The one on the left uses a primitive on string values that produces a version of the string with all lowercase letters. In contrast, the one on the right *modifies* the given string, lowering the case of all letters. A presenter must explain this much of the context before diving into the definition of *h*.

As a developer presents a path from the entry point(s) of a component—for example, the public methods of a class; the calls from an integration test script—each unit of code deserves a basic high-level explanation. Once again, if the table of content of a corresponding README file can be generated, the developer knows that each module or class-level unit comes with a purpose statement and can leverage it for the presentation.

For each function or method, a developer first states their purpose. If the names are self-explanatory, the developer should be able to formulate a coherent sentence using this name; otherwise, the developer can read out the purpose statement. Explaining the pieces of functionality then depends on their nature:

- For a *composite* method, a developer begins with an enumeration of the tasks it composes, how it composes them, and the flow of intermediate data among the pieces. See *f* above on this page for a composite method that uses plain sequential composition; the data flow for the left variant is explicit and the one for the right is implicit.

The key is to explain the entire function before diving into any of the tasks. Doing so should establish what kind of data flows *into* the function to be inspected and what happens to the data that results *from* the call. Once the panelists understand this context, it is time to dive into the function or method that needs close inspection—*h* in the above example.

- For an *atomic* method, a developer first explains the *what* and *how* of the computation holistically. After that, it is appropriate to explain

the functionality on a line-by-line basis. If methods and functions are kept to a reasonable size, the functionality may explain itself mostly but in any case, a developer will have an easy time explaining how the lines accomplish the desired goal.

In sum, a presentation proceeds in a present top-down manner. The “top” is the big picture; the “down” is the path from the entry point(s) of a component to those pieces of code that deserve special attention. The “bottom” are those pieces of functionality with which the developer(s) had a hard time and that just need more pairs of eyes on them.

As for the interactions with the panelists, a presenter is usually confronted with three kinds of questions. The first and most basic kind is about the comprehensibility of what is written down. Since panelists are potential future readers, a presenter’s task is to recognize when such an issue requires a change and when not; when in doubt, take a note and improve. Perhaps a simple rule of thumb is that once a second presentation attempt has failed, it is time to think of a different way of writing this code.

The second kind of question is about bugs. In this case, the conversation proceeds until the developer has understood the mismatch between the specification and the code *or* has justified why there is no mismatch. For this latter case, the panel may wish to analyze the problem as a potential comprehensibility problem.

Finally, the third kind of conversation is about raise design concerns. In all likelihood, a developer has already considered design alternatives. Hence, presenting the chosen design as the best understood trade-off may pacify the panel; if not, the focus must be on understanding why the panel considers the design flawed so that the developer can go back and come up with an alternative or an improvement.

Inspecting

From the perspective of the code reader, inspecting code is the mirror image of presenting code. To start with, a panelist must insist on seeing the relevant README files and the expected pieces in those files:

- If the README files don’t exist, have the presenter state so and have the secretary take a note.
- If the README files miss a statement of purpose for the documented component, request it.

- If the README file lacks an overview of the component's pieces and their static/dynamic relationships, ask whether those aren't needed.

Once the overview is completed, the panelists must keep the content of the READMEs in mind; it is now their task to check whether the code is consistent with the overview based on the README files. Let's state this as a distinct guideline:

panelists check whether the code reflects the README file's information.

Before a developer moves from the overview to the presentation of the code itself, it is important to ensure that all participants understand which pieces of the component deserve special attention. The head reader must make sure that the presenter stays on the path from the overview to these attention-worthy pieces of code. In other words, if the presenting developer spends time on obvious or irrelevant code, the head can re-direct.

Once the presentation arrives at an interesting piece of code—a function, a method, or a collection of those—the panel needs to understand for each piece whether it is to be atomic or composite:

- For a *composite* method, the panelists must understand how data flows from each separate task to the next. It is particularly important to understand the logical state in which potentially problematic helper methods are called.

If the developer does not present the entire method before jumping to the implementation of a sub-task, the head reader must prune this dynamic jump. It should suffice to use the purpose statement of each task to explain the flow of data.

A panel may wish to see unit tests that check whether the integration of all these tasks satisfies expected properties.

- For an *atomic* method, the panel must insist on a precise explanation of the purpose, which is either explicitly written down or apparent from a well-chosen name. If only a name exists and the name isn't clear enough, the panel should say so. This shortcoming may impose a cost on future readers.

A panel may request to see unit tests to clarify the purpose of a method. This may happen at any time during the discussion of an atomic method. It is most often useful before the presenter dives into a line-by-line explanation or after this explanation is completed.

Once the purpose of the method is understood, the panel should read the lines of code. The head panelist may even read those aloud. At the end of the presentation of an atomic method, the panel should be convinced that it achieves the stated purpose.

While the preceding items explain the mechanics of how to follow a presentation of a software component, they leave open when to raise design questions, how to expose bugs (mismatches with the specification), or where to point out readability questions. Let's tackle these questions, one at a time.

Developers design at several levels. They design components and interactions between components, after getting at least a preliminary understanding of a software system's requirements. Part III focuses on this level. This high-level design spells out coarse specifications for components. For example, it may come with a request for a referee component with a game observer. The details of this component—how to represent the referee's game knowledge, how to interact with the observer—are left to the designer(s) of the component, which brings us to the second level of design, the one likely to be presented at the beginning of a code inspection. Finally, as the introduction to this part spells out, developers should also bring a "design attitude" to the creation of data representations and functionality.

Based on this summary of design levels, the answer to the first question is straightforward. Assuming that the specifications have been inspected (see section 15.5), panelists encounter the refinement of a component specification into a design near the beginning of a well-organized presentation. If they see something unusual or spot a gap in the explanation of purpose statements for pieces (classes, modules) and static or dynamic relationships, they must question the presenter there and then while the relevant information is on the screen. In contrast, questions concerning the systematic design of data representations and pieces of functionality are much more naturally discussed while the actual code is on the screen.

When some piece of code is visible, it is also the correct time to pose correctness questions. Following the idea of systematic design, this kind of conversation can always start with a request to see unit tests, specifically a unit test that covers a particular case. If the developer presents such a test, the panelists have two tasks:

- One of the panelists' task is to check the expected output. When the expected output is challenged, the team could re-read the specification together. The goal is to decide whether the expected output lives

up to the specification. If not, the issue gets noted; otherwise, the panel proceeds to the second task.

- The other task is to make sure the inputs represent the case under consideration and drive the execution to the questionable code. When in doubt, the panelists may request a step-by-step justification. As they follow the presentation, the panelist should question what is visible, not some code off-screen.

Once the panelists and the developer agree that a bug exists, they move on; it is the task of the developer, not the panel, to fix mistakes.

Lastly, panelists represent future readers of code, meaning they should be able to comprehend code in a reasonable amount of time. If any piece of code is difficult to read, the panelists stop the code walk, state the code's purpose, and read it aloud. Doing so, slows down the presentation and may eliminate the comprehension obstacle. If it doesn't, the presenter can request a description of what the obstacle is so that any corrective action addresses it directly.

In all cases, the conversation should not continue until the presenter and the panelists have agreed on an issue, be that a design issue, a correctness problem, or a comprehensibility obstacle. It is a good practice for the head panelists to have the presenter re-state what the issue is as the last step before the presentation continues. Also, in no case should a panelist push a re-resolution of any issue—unless the entire team agrees that the severity of the issue demands it. Inspection time is too restricted and too valuable to try to resolve problems on the spot.

Postlude

You cannot acquire the mindset of a socially responsible programmer over the course of one semester. The process has to continue for a long time; indeed it should never end. At this point, you may wonder how you can make this work. Conversely, if you are in the business of recruiting developers, you may wish to figure out up front whether a candidate already has the proper mindset. In turn, the question is how a short interview can identify a mindset. Finally, AI affects both of these aspects: personal development and recruiting. Here are some ideas.

How to Continue the Learning Process, with AI

Imagine yourself having landed a well-paying software developer job. You successfully completed a course based on *SRSD*, and now reality hits. You are put on a project. You begin to add code that matters. You become a part of a second task, a third, and more. You get busy. And suddenly you realize that you slipped, you no longer practice the principles of socially responsible software development.

This realization is a *good thing*. It is the first step toward correcting your approach, because developing in a socially responsible manner means to

stop, drop, and reflect

Here is how this mantra is to be understood:

1. Every so often, *stop* doing whatever you are doing.
2. *Drop* everything and take a deep breath.
3. *Reflect* on what went well and what went badly recently.

And then

draw a lesson.

One approach to getting this right is to keep a work log, a systematic diary of “lessons learned.” For the reflection step, revisit the key entries. If one of the recorded lessons shows up again as something you should do, promote it to a slogan that you look at on a daily basis.

If your primary partner is an AI, ask it to conduct such “stop, drop, and reflect” actions, too. Concretely, observe what the AI delivers. Check

whether it lives up to the specified skill set. Any failure to do so suggests it is time for a reflective conversation. Challenge the AI to articulate the key lesson drawn from this conversation; don't write it down yourself. Once you agree with the lesson, ask the AI to record it in the repository. Every so often, you want to request a comparison of these lessons in the various directories of your repository (or repositories) and suggest integrating repeated or similar lessons into its skills specification.

Finally, you may wish to identify those of your human colleagues who are interested in being socially responsible developers. Find out whether they want to meet, reflect, and exchange ideas. Such meetings can even serve as inspection sessions, involving

- best practice for development with a (n AI) partner;
- comparisons of skills and lessons drawn; and
- problems with AI-generated products (specifications, plans, code).

Critically such meetings put the "social" back into the development process that was so common, and so rewarding, when the software discipline was in its infancy.

How to Identify Socially Responsible Candidates

Imagine yourself as an experienced software developer whose manager asks you to recruit your first new colleague from a pool of recent graduates. You know about *SRSD*, you even practice its principles, and you turn them into lessons applicable to your particular situation. You consider *SRSD* as the right approach to developing software, especially in the age of AI, and you want a colleague who comes with a similar mindset.

What characteristics should you look for?

The most important question is whether the candidate truly understands the difference between programming and software development. Here are some sample questions that set the proper tone:

1. What is the largest project you have worked on in a course? How many weeks did it last? How many milestones did it involve?
2. How many partners, including AI agents, did it involve? Did it involve switching to a different code base?
3. How did the partners guide each other? How did you instruct the AI to learn from its mistakes (if applicable)?

4. Did you have to revisit existing code to fix it or add features?

You may wish to get a description of the most difficult bug or the most complex feature that the candidate ever dealt with.

5. How did you go about the bug fix? The feature addition?

The answer should involve comprehension of existing code, (the request to an AI to make) a plan, tests, and the actual work on the code.

And then you must find out what the candidate considers

the most important lesson to draw from these experiences.

Some of your candidates will get close to the idea that software is a message across time and from one person to another, possibly an older version of the original creator. If so, ask the candidate what matters to this “message sending” process most.

Instead of asking candidates to solve problems with a few dozen lines of code, possibly via AI, confront them with snippets from your software base. Pick examples that are illustrative of good practices, and create one that needs improvements. Ask the candidates to inspect the product and to identify its good and bad attributes. The goal is to see whether the candidate can spot problematic pieces in specifications, plans, code, or tests. Have the candidate explain why these pieces cause problems and how they affect future recipients. If there is sufficient time, request suggestions on how to improve code that suffers from inequities. This step may include instructions to the AI on how to spot the problem, how to go about fixing it, and how to articulate a lesson for the future.

Onward!

As the evolution of AI continues to change the profession of software developers, the relevance of socially responsible software development will only increase. We, as professionals and as members of society, cannot let AI take over without guiding its work and double-checking it somehow. This book provides some basic principles on how to work with others and AI as partners. It is the application of these principles that will change, and it will demand that all of us take the time to stop our work, drop what we’re doing, and reflect on how to go forward. In short, that all of us continue to adapt the practice of socially responsible software development.

mit: 10/7

Acknowledgments

I gratefully acknowledge the many students who worked through, and suffered from, my Software Development courses. Quite a few have offered overwhelmingly positive feedback years and even decades after the experience. Thanks to every one of them, the creation of this book benefited greatly from your efforts and feedback, both positive and negative.

A good number of people have adopted the course at various institutions or worked through early drafts of the book at different times. A fair number of my PhD students served as lead teaching assistants and pushed back on many of the flawed parts of my various attempts. Thanks to all of you: Michael Ballantyne, Sam Caldwell, Richard Cobbe, Ryan Culpepper, Christos Dimoulas, Robby Findler, Matthew Flatt, Ben Greenman, Jason Heman, Niousha Jafari, Tony Garnock-Jones, Shriram Krishnamurthi, Ben Lerner, Cameron Moy, Afsaneh Rahbar, and Sam Tobin-Hochstadt. I could not have done it without you.